

Flutter cho người mới bắt đầu

Xây dựng ứng dụng mobile đa nền tảng

Bản dịch tiếng Việt của *Beginning App Development with Flutter*

Rap Payne · Apress, 2019

Bản dịch phục vụ mục đích tự học cá nhân.

Mục lục

Về tác giả

Về người review kỹ thuật

Cuốn sách này dành cho ai?

Sách bao gồm những gì?

Sách không bao gồm những gì, và tìm chúng ở đâu?

PHẦN I — Nhập môn Flutter

Chương 1: Hello Flutter

Flutter là gì?

Tại sao lại là Flutter?

Những lựa chọn khác

Các giải pháp native

Tổng kết

Chương 2: Phát triển ứng dụng với Flutter

Bộ công cụ (toolchain) của Flutter

Giữ cho công cụ luôn được cập nhật

Quy trình phát triển với Flutter

Tổng kết

PHẦN II — Nền tảng Flutter

Chương 3: Mọi thứ đều là Widget

UI as code

Các widget dựng sẵn của Flutter

Cách tự tạo stateless widget của riêng bạn

Stateless và Stateful widget

Tổng kết

Chương 4: Value Widget

Widget Text

Widget Icon

Widget Image

Input widget

Ghép các widget form lại với nhau

Một ví dụ Form lớn

Tổng kết

Chương 5: Phản hồi cử chỉ người dùng

Làm quen với họ nhà button

Dismissible

Gesture tùy chỉnh cho widget tùy chỉnh của bạn

Ví dụ 1: Phản hồi cú nhấn giữ

Ví dụ 2: Chạm ngón tay để thêm mục mới

Ví dụ 3: Vuốt sang trái hoặc phải

Nếu có từ hai gesture xảy ra cùng lúc thì sao?

Tổng kết

Chương 6: Sắp xếp bố cục cho Widget

Bố trí toàn bộ scene

Flutter quyết định kích thước widget như thế nào

Đặt widget cạnh nhau hoặc dưới nhau

Widget của bạn sẽ không bao giờ vừa khít!

Nếu còn thừa chỗ trống thì sao?

Nếu không đủ chỗ thì sao?

Widget Container và box model

Các widget bố cục đặc biệt

Tổng kết

Chương 7: Navigation và Routing

Stack navigation

Drawer navigation

Tab navigation

Widget Dialog

Có thể kết hợp các phương pháp navigation

Chương 8: Tạo kiểu cho Widget

Tư duy theo kiểu Flutter

Vài lời về màu sắc

Tạo kiểu cho Text

Trang trí cho Container

Xếp chồng widget

Widget Card

Theme

Tổng kết

Chương 9: Quản lý State

State là gì?

Một StatefulWidget gồm những gì?

Quy tắc quan trọng nhất về state!

Truyền state xuống dưới

Đưa state ngược lên trên (lifting state up)

Một ví dụ về quản lý state

Khi nào nên dùng state?

Quản lý state nâng cao

Tổng kết

PHẦN III — Đi xa hơn

Chương 10: App Flutter làm việc với file

Đưa thư viện vào app Flutter

Future, async và await

Đóng gói file kèm theo app

Ghi file

Sử dụng JSON

Shared preferences

Tổng kết

Chương 11: Gọi RESTful API bằng HTTP

API call là gì?

Thực hiện HTTP GET hoặc DELETE request

Thực hiện HTTP PUT, POST hoặc PATCH request

Đưa HTTP response vào widget

Class có kiểu chặt chẽ

Một ví dụ lớn

Tổng kết

Chương 12: Sử dụng Firebase với Flutter

Giới thiệu Firebase

Thiết lập Firebase

Sử dụng Firestore

Đi tiếp từ đây

Phụ lục A: Tổng quan ngôn ngữ Dart

Dart là gì?

Những tính năng quen thuộc — Dart Cheatsheet

Những điều bất ngờ về Dart

Bổ sung: những tính năng Dart ra đời sau cuốn sách

Bảng thuật ngữ Anh – Việt

A – C

D – G

H – N

O – S

T – Z

Quy ước ký hiệu trong bản dịch

Về tác giả

Rap Payne tập trung vào mảng phát triển mobile từ khi thành lập Agile Gadgets — một công ty phát triển ứng dụng di động — vào năm 2003. Ông là chuyên gia tư vấn, giảng viên và doanh nhân; đã viết ứng dụng, hướng dẫn developer và giảng dạy các lớp phát triển phần mềm cho những tập đoàn trong nhóm Fortune 500 như Boeing, Walmart, Coca-Cola, Wells Fargo, Honda, CVS, GE, Chase, HP, Lockheed, ExxonMobil, Lowe's, Nike, J.C. Penney, USAA và Walgreens; cho các cơ quan chính phủ như NSA, Không quân, Hải quân, Lục quân Mỹ, NASA, GCHQ của Anh và cơ quan bưu chính Canada; cùng nhiều chính quyền cấp bang khác.



Với vai trò người hướng dẫn và giảng viên chuyên nghiệp, Rap có biệt tài truyền đạt những ý tưởng cực kỳ phức tạp theo cách dễ hiểu. Và với tư cách một developer thực chiến, ông hiểu rằng những chủ đề này cần được dạy bằng các ví dụ và bài tập thực tế.

Về người review kỹ thuật

Massimo Nardone có hơn 22 năm kinh nghiệm trong lĩnh vực Security, phát triển Web/Mobile, Cloud và kiến trúc CNTT. Hai niềm đam mê thực sự của ông trong ngành là Security và Android.

Ông đã lập trình và dạy lập trình với Android, Perl, PHP, Java, VB, Python, C/C++ và MySQL trong hơn 20 năm.

Ông có bằng Thạc sĩ Khoa học Máy tính tại Đại học Salerno, Ý.

Ông từng làm Project Manager, Software Engineer, Research Engineer, Chief Security Architect, Information Security Manager, PCI/SCADA Auditor, và Senior Lead IT Security/Cloud/SCADA Architect trong nhiều năm.

Kỹ năng kỹ thuật của ông bao gồm Security, Android, Cloud, Java, MySQL, Drupal, Cobol, Perl, phát triển Web/Mobile, MongoDB, D3, Joomla, Couchbase, C/C++, WebGL, Python, Pro Rails, django CMS, Jekyll, Scratch, và nhiều thứ khác.



Ông hiện là Chief Information Security Officer (CISO) tại Cargotec Oyj.

Ông từng là giảng viên thỉnh giảng và người hướng dẫn thực hành tại Networking Laboratory của Đại học Công nghệ Helsinki (Đại học Aalto). Ông sở hữu bốn bằng sáng chế quốc tế trong các lĩnh vực PKI, SIP, SAML và Proxy.

Cuốn sách này dành cho ai?

Nếu bạn là một developer đã có kinh nghiệm với một ngôn ngữ hướng đối tượng nào đó như Java, C#, C++ hay Objective-C, và bạn muốn tạo ra app Android, app iOS hoặc web app bằng Flutter, thì cuốn sách này dành cho bạn. Nó đặc biệt quan trọng nếu bạn muốn viết một app chạy được trên nhiều nền tảng và bạn hoàn toàn mới với Flutter.

Nếu bạn đã có chút kinh nghiệm với Flutter, chắc chắn bạn vẫn học được điều gì đó, nhưng chúng tôi không kỳ vọng bạn phải có sẵn kiến thức hay kinh nghiệm nào về Flutter. Mọi chương đều được viết với giả định rằng mọi thứ trong Flutter đều hoàn toàn mới với bạn.

Nếu bạn biết chút gì đó về phát triển iOS, Android hay Web, điều đó chắc chắn sẽ giúp bạn hiểu bài nhanh hơn, vì có rất nhiều điểm tương đồng với Flutter. Bạn biết càng nhiều về những thứ đó thì càng tốt, đặc biệt là JavaScript và React. Nhưng nếu bạn chẳng biết gì trong số đó thì cũng đừng lo — chúng hoàn toàn không bắt buộc.

Biết ngôn ngữ Dart cũng sẽ giúp ích. Chúng tôi nhận thấy Dart chắc chắn có những đặc điểm riêng, nhưng nó cực kỳ dễ nắm bắt nếu bạn hiểu các khái niệm hướng đối tượng. Thậm chí nếu bạn biết Java hoặc C#, phần lớn các đoạn code trong sách đều đọc hiểu được mà không cần giải thích gì về ngôn ngữ. Đọc vài đoạn là bạn sẽ tự viết được ngay.

Đồng thời, Dart có một số tính năng độc đáo và rất hay mà chúng tôi coi là best practice. Lẽ ra chúng tôi có thể "đơn giản hóa" code cho dân Java bằng cách không dùng những best practice này, nhưng về lâu dài điều đó chẳng có lợi gì cho bạn. Vì vậy chúng tôi vẫn dùng chúng, nhưng có giải thích trong "**Phụ lục A: Tổng quan ngôn ngữ Dart**". Ở đó chúng tôi cung cấp một bản tra cứu nhanh với vừa đủ chi tiết để viết được code, tiếp theo là phần giải thích sâu hơn về những tính năng mà developer đến từ ngôn ngữ khác sẽ thấy bất ngờ. Hãy chú ý đặc biệt tới mục có tên "**Những điều bất ngờ về Dart**".

Sách bao gồm những gì?

Cuốn sách này dạy bạn cách tạo ra những app đầy đủ chức năng, giàu tính năng, chạy được trên iOS, Android và Web. Chúng tôi chia thành ba phần.

Phần I: Nhập môn Flutter

1. **Hello Flutter** — Đặt nền cho cả cuốn sách. Cho bạn cảm nhận vì sao bạn ở đây. Flutter giải quyết vấn đề gì? Vì sao sếp lại chọn Flutter thay vì một giải pháp khác.
2. **Phát triển ứng dụng với Flutter** — Flutter có một bộ công cụ riêng, nhưng không phải lúc nào cũng rõ ràng công cụ nào làm gì và dùng ra sao. Chương này dẫn bạn qua quy trình viết – debug – test – chạy. Chúng ta sẽ hiểu về bộ công cụ, bao gồm cả việc cài đặt và bảo trì.

Phần II: Nền tảng Flutter

3. **Mọi thứ đều là Widget** — Widget cực kỳ quan trọng với Flutter vì chúng là những viên gạch xây nên mọi app Flutter. Chúng tôi sẽ chỉ ra lý do, đồng thời cung cấp động lực và công cụ cơ bản để tạo widget. Các chủ đề gồm composition, UI as code, các loại widget, key, và stateless so với stateful widget.
4. **Value Widget** — Đi sâu vào những widget chứa một giá trị, đặc biệt là các trường nhập liệu của người dùng. Các chủ đề gồm file `pubspec.yaml`; widget Text, Image và Icon; và cách tạo form trong Flutter.
5. **Phản hồi cử chỉ người dùng** — Cách làm cho chương trình phản ứng lại các hành động của người dùng như chạm, vuốt, chạm ngón tay và tương tự. Chúng tôi sẽ giới thiệu họ nhà button và widget GestureDetector.
6. **Sắp xếp bố cục cho Widget** — Học cách bố trí một màn hình: điều khiển việc đặt widget cạnh nhau và/hoặc trên dưới nhau, định nghĩa khoảng cách giữa các widget, và căn chỉnh chúng theo chiều dọc lẫn chiều ngang.

7. **Navigation và Routing** — Navigation là việc app ẩn một widget đi và hiện một widget khác lên để phản hồi hành động của người dùng, tạo cảm giác họ đang di chuyển từ màn hình này sang màn hình khác. Chúng ta sẽ tìm hiểu stack navigation, tab navigation và drawer navigation.
8. **Tạo kiểu cho Widget** — Tiếp đó chúng ta xem cách điều khiển màu sắc, đường viền, trang trí, hình dạng và các đặc điểm hiển thị khác của từng widget. Chúng tôi đã đề cập sơ về styling khi giới thiệu từng widget ở các chương trước, nhưng đây mới là nơi giải đáp mọi câu hỏi cần thiết để một app thực tế trông đẹp và nhất quán xuyên suốt nhờ theme.
9. **Quản lý State** — Cách đưa dữ liệu từ widget này sang widget khác và cách thay đổi dữ liệu đó. Chúng ta sẽ học cách tạo StatefulWidget và thiết kế chúng theo cách tốt nhất. Chúng tôi cũng cung cấp một cái nhìn tổng quan về các công cụ xử lý bài toán quản lý state phức tạp trong thực tế.

Phần III: Đi xa hơn

10. **App Flutter làm việc với file** — Sử dụng thư viện. Future, async, await. Đóng gói file kèm theo app. Đọc và ghi file. Serialize JSON.
11. **Gọi RESTful API bằng HTTP** — Cách đọc và ghi dữ liệu tới một HTTP API server. Đây là nơi chúng tôi chỉ bạn cách thực hiện các request GET, POST, PUT, DELETE và PATCH.
12. **Sử dụng Firebase với Flutter** — Chúng tôi sẽ giới thiệu một giải pháp cloud thực tế, mạnh mẽ, hoạt động cực kỳ ấn ý với Flutter. Không có gì bất ngờ khi nó cũng là một sản phẩm của Google.

Sách không bao gồm những gì, và tìm chúng ở đâu?

Cũng quan trọng không kém, bạn nên biết những gì **không** có trong cuốn sách này. Chúng tôi sẽ không dạy bạn ngôn ngữ Dart từ đầu, ngoài phần phụ lục đã nói ở trên. Đơn giản là chúng tôi thấy đó không phải cách dùng thời gian hiệu quả nhất của bạn, và chúng tôi muốn lao thẳng vào Flutter. Nếu sau này bạn thấy cần học Dart bài bản, hãy vào <https://dart.dev/guides/language/language-tour>, sau đó là <https://dart.dev/tutorials>.

Chúng tôi cũng chọn không bàn về việc deploy lên các app store. Bản thân các store đã hướng dẫn rất tốt cách nộp app rồi. Hơn nữa, quy trình đó thay đổi liên tục nên nguồn tài liệu chuẩn xác nhất phải là chính các store. Bạn sẽ tìm thấy hướng dẫn tại <https://developer.apple.com/ios/submit/> và <https://play.google.com/apps/publish>.

Và chúng tôi sẽ không đề cập một số chủ đề nâng cao như phát triển đặc thù cho từng thiết bị trên iOS và Android, hay việc thêm Flutter vào một dự án iOS/Android có sẵn. Đây là sách cho người mới bắt đầu và

chúng tôi không muốn làm bạn quá tải. Những chủ đề này cùng rất nhiều chủ đề khác có thể tìm thấy trên mạng, hoặc qua một số nguồn tài liệu mà chúng tôi sẽ giới thiệu ở chương cuối sách.

 **Ghi chú cập nhật:** Đường dẫn `dart.dev/guides/language/language-tour` nay đã chuyển thành `https://dart.dev/language`. Trang tổng hợp tài liệu Flutter chính thức hiện là `https://docs.flutter.dev`.

PHẦN I — Nhập môn Flutter

Chương 1: Hello Flutter

Hãy tưởng tượng cảnh này. Bạn là một CEO siêu thông minh và cực kỳ có năng lực của một doanh nghiệp mới thành lập. Nhiệm vụ của bạn thì đã rõ: tối đa hóa doanh thu, tối thiểu hóa chi phí. "Hừmmm...", bạn nghĩ. "Mình có thể tăng doanh số kha khá nếu đưa sản phẩm lên Web." Thế là bạn đi hỏi mấy người bạn xem làm một web app thì cần những gì, và họ trả lời...

"Cậu cần thuê một web developer. Người đó phải biết HTML, CSS, JavaScript, và có lẽ là một framework nào đó như React, Vue hay Angular."

Đắt đấy, nhưng bạn vẫn làm — và canh bạc đó có lãi. Doanh số tăng thấy rõ. Để bắt kịp nhu cầu, bạn theo dõi mạng xã hội và trò chuyện với khách hàng. Bạn nghe họ nói rằng web app này hay thật đấy, nhưng "Bọn tôi đã đến với các bạn sớm hơn rồi nếu các bạn có app trên App Store."

Thế là bạn hỏi đội của mình — những người vốn là chuyên gia về Web, chứ không phải iOS developer. Họ bảo bạn...

"Cậu cần thuê một chuyên gia iOS. Người đó phải biết iOS, Swift hoặc Objective-C, Xcode, macOS, và CocoaPods để phát triển ứng dụng."

Bạn tìm hiểu và thấy rằng người này còn chuyên biệt hơn — và do đó đắt hơn — mấy web dev của bạn. Nhưng một lần nữa, đó có vẻ vẫn là việc nên làm, nên bạn chấp nhận thuê họ. Có điều, ngay khi app này còn đang được phát triển, bạn nhận ra rằng những phản hồi kia không chỉ nói riêng về app iOS, mà là về **tất cả** thiết bị di động. Và 85% thiết bị trên toàn thế giới chạy Android chứ không phải iOS. Bạn tự hỏi liệu mình có kham nổi cái giá của việc bỏ qua 85% khách hàng tiềm năng hay không. Các cố vấn của bạn nói...

"Cậu cần thuê một chuyên gia Android. Người đó phải biết Android OS, Gradle, Android SDK, XML, Android Studio, và Java hoặc Kotlin."

"Thật hả?!? Lại thêm một developer nữa à?", bạn thốt lên. "Đúng vậy. Và đặt ngang ông iOS developer kia luôn", họ đáp.

Chẳng lẽ không có lấy một người nào làm được cả ba việc đó sao? Không có cách nào để dùng chung code giữa từng ấy môi trường à? Nếu có, bạn chỉ cần thuê đúng một người. Thậm chí họ có thể viết code một lần rồi deploy lên Web, lên App Store, và lên Google Play Store. Một codebase duy nhất để bảo trì. Một chỗ duy nhất để cải tiến và nâng cấp. Một chỗ duy nhất để sửa lỗi.

Thưa quý vị, cho phép tôi giới thiệu: **Flutter!**

Flutter là gì?

Flutter là một bộ công cụ cho phép chúng ta tạo ra những ứng dụng đẹp mắt, chạy được trên iOS, Android, Web và cả desktop.¹

Flutter là...

- **Miễn phí** — không tốn tiền để sử dụng
- **Open source** — mã nguồn được công khai
- Được Google hậu thuẫn, và cũng khởi nguồn từ Google
- Đang được phát triển và bảo trì bởi một đội ngũ developer của Google cùng hàng trăm người đóng góp bên ngoài Google trên khắp thế giới
- Hiện đang được hàng nghìn developer tại các tổ chức trên toàn cầu dùng cho những app chạy production
- **Nhanh**, bởi vì nó biên dịch ra app native thực thụ, không phải phụ thuộc vào các lớp trung gian như WebView hay JavaScript bridge
- Viết ở một nơi rồi biên dịch ra: một web app cho hàng tỉ trình duyệt, một app iOS cho iPhone và iPad, và một app Android cho toàn bộ số điện thoại, máy tính bảng còn lại ngoài kia

Tại sao lại là Flutter?

Sứ mệnh mà Google đặt ra cho Flutter là...

Xây dựng một cách tốt hơn để phát triển ứng dụng cho mobile.

Hãy để ý xem **cái gì không có** trong sứ mệnh đó. Không hề nhắc tới Android (thứ mà Google cũng sở hữu), cũng không nhắc tới iOS, cũng chẳng nhắc tới Web. Mục tiêu của Flutter là tạo ra một cách tốt hơn để phát triển ứng dụng cho *mọi* thiết bị. Nói cách khác, viết app iOS bằng Flutter phải tốt hơn viết bằng Swift. Viết app Android bằng Flutter phải tốt hơn viết bằng Kotlin. Viết web app bằng Flutter phải tốt hơn viết bằng HTML/JavaScript. Và nếu bạn có được tất cả những điều đó cùng lúc chỉ với một codebase duy nhất, thì càng tốt.

Đội ngũ Flutter đã hoàn thành sứ mệnh này một cách xuất sắc.

Bằng chứng: Eric Seidel đưa ra ví dụ này² — đội Google CRM đã dùng Flutter để xây một app Android nội bộ, và họ làm xong **nhANH GẤP ba lần** so với khi dùng bộ toolchain Android truyền thống.

Tuy nhiên, Flutter không phải là lựa chọn cross-platform duy nhất. Bạn còn có những phương án khác.

Những lựa chọn khác

Việc phát triển ứng dụng cross-platform nhìn chung có ba nhóm giải pháp, liệt kê trong Bảng 1-1.

Bảng 1-1. Các nhóm giải pháp phát triển cross-platform

	Một số công nghệ	Nhược điểm	Ưu điểm
Progressive Web Apps (PWA)	HTML/CSS, React, Angular, Vue	Không phải app thật. Chạy trong trình duyệt web. Không có mặt trên các app store. Khó tạo shortcut ngoài màn hình chính. Không truy cập được nhiều tài nguyên của thiết bị như cảm biến gia tốc, la bàn, cảm biến tiệm cận, Bluetooth, NFC và nhiều thứ khác	Dễ viết
Hybrid	PhoneGap, Cordova, Sencha, Ionic	Chạy trong một WebView nên có thể chậm. Gần như không thể dùng chung code với web app	Dễ học hơn với web dev, vì nó dùng chính HTML và JavaScript làm ngôn ngữ và cấu trúc
Compile-to-native	React Native, NativeScript, Flutter, Xamarin	Học một framework có thể sẽ khó. Còn làm chủ được cả toolchain thì chắc chắn là khó	App thật, có mặt trên các store và chạy nhanh

Nếu bạn có sẵn một nhóm người dùng trung thành — những người coi trọng app của bạn đến mức chấp nhận một trải nghiệm kém hơn — thì giải pháp rẻ nhất là làm một PWA. Nếu app của bạn cực kỳ đơn giản và tốc độ không phải vấn đề, một giải pháp hybrid có thể là phù hợp. Nhưng nếu tốc độ, độ mượt và khả năng xử lý phức tạp là quan trọng, bạn sẽ phải chọn giải pháp native.

Các giải pháp native

Tính đến thời điểm sách được viết, có bốn giải pháp compile-to-native khá phổ biến (Bảng 1-2).

Bảng 1-2. Các framework cross-platform biên dịch ra native

	 Xamarin	 NativeScript	 React Native	 Flutter
Năm ra mắt	2011	2014	2015	2018
Hậu thuẫn bởi	Microsoft	Telerik	Facebook	Google
Ngôn ngữ dựng giao diện	XAML và/hoặc xamarin.forms	Của riêng nó, nhưng trông giống XML	Của riêng nó, nhưng trông giống JSX	Dart
Ngôn ngữ xử lý logic	C#	JavaScript	JavaScript	Dart

 **Ghi chú cập nhật:** Bảng này phản ánh tình hình năm 2019. Xamarin đã được Microsoft thay thế bằng **.NET MAUI** (Xamarin ngừng hỗ trợ từ tháng 5/2024). NativeScript nay do cộng đồng duy trì chứ không còn thuộc Telerik. React Native hiện thuộc Meta và đã chuyển sang kiến trúc mới (New Architecture) bỏ dần JavaScript bridge. Flutter vẫn do Google phát triển, ngôn ngữ vẫn là Dart.

Đây đều là những lựa chọn tử tế cả. Tất cả đều miễn phí để phát triển và đã được kiểm chứng kỹ, với rất nhiều ứng dụng production đã ra đời. Tất cả đều đã được dùng trong các tổ chức lớn.

Nhưng chỉ có duy nhất một cái cho phép bạn tạo ra một web application **bên cạnh** những app iOS và Android sẽ được deploy lên các app store — đó là Flutter.

Flutter là framework ra mắt muộn nhất trong nhóm này. Chính vì vậy nó có một lợi thế rõ rệt: được quan sát và rút kinh nghiệm từ những nền tảng đi trước. Đội ngũ Flutter đã ghi nhận xem những framework khác

làm tốt ở chỗ nào và thất bại ở chỗ nào. Ngoài ra, Flutter còn bổ sung những cải tiến và ý tưởng mới — tất cả đều được thiết kế sẵn ngay từ đầu, chứ không phải chắp vá thêm vào sau này qua từng bản cải tiến.

Nhưng nếu bạn đã tìm đến cuốn sách này thì hẳn bạn cũng không cần tôi thuyết phục thêm, nên tôi dừng ở đây. Chỉ cần nói thế này là đủ: Flutter rất tốt. Nó dễ viết, gọn gàng, được thiết kế tốt — code bằng nó thực sự là một niềm vui.³

Tổng kết

Bây giờ, nếu bạn đúng là kiểu developer mà tôi hy vọng, thì hẳn bạn đang nôn nóng muốn bắt tay vào viết code rồi. Vậy thì bắt đầu thôi. Chúng ta sẽ khởi động bằng việc cài đặt và tìm hiểu bộ toolchain phát triển của Flutter.

-
1. Bản gốc viết "desktop sắp ra mắt", với dự kiến Flutter sẽ chạy trên Windows, macOS, Chromebook và Linux.  **Cập nhật:** điều này đã thành hiện thực. Flutter hiện hỗ trợ chính thức và ổn định cho Windows, macOS, Linux và Web, bên cạnh iOS và Android. 
 2. http://bit.ly/eric_seidel_flutter_keynote_video — xem từ phút 21:47. 
 3. Nếu bạn vẫn muốn đọc thêm, đây là một bài phân tích sâu hơn so sánh Flutter với một số framework khác: <http://bit.ly/2HC9Khm> 

Chương 2: Phát triển ứng dụng với Flutter

Như đã thấy ở chương trước, Flutter cho phép chúng ta tạo ra những app chạy trên Web, trên máy tính để bàn, và trên thiết bị di động (mảng này có vẻ là điểm hấp dẫn chính). Nhưng khoan đã — chính xác thì chúng ta tạo ra những app đó bằng cách nào? Nên dùng editor gì? Một project Flutter cần có những gì? Biên dịch mã nguồn Dart ra sao? Có cần thêm công cụ nào khác để hỗ trợ project không? Làm sao đưa app lên trình duyệt hoặc lên thiết bị để chạy thử? Toàn những câu hỏi hay, đúng không?

Hãy cùng trả lời những câu hỏi đó và nhiều hơn nữa trong chương này. Chúng ta sẽ bàn hai chủ đề lớn:

1. **Những công cụ cần thiết** — cách cài đặt và bảo trì chúng
2. **Quy trình phát triển** — cách tạo app, chạy nó, và debug nó

Cẩn trọng

Về bản chất, bộ công cụ phát triển app cross-platform bao gồm rất nhiều thành phần rời rạc đến từ nhiều tổ chức khác nhau, và hiếm khi tổ chức nào hỏi ý kiến tổ chức nào trước khi thay đổi. Hơn nữa, vì chúng ta đang làm việc với một công nghệ còn non trẻ và liên tục phá vỡ giới hạn, thay đổi diễn ra rất thường xuyên. Trong chương này chúng tôi đã cố gắng chỉ trình bày những thông tin ít bị lỗi thời, nhưng ngay cả chúng rồi cũng sẽ cũ đi. Hãy kiểm tra với tác giả của từng công cụ để có thông tin mới nhất.

Bộ công cụ (toolchain) của Flutter

Danh sách những công cụ hữu ích mà cộng đồng developer tạo ra là vô tận. Nhiều đến mức choáng ngợp. Chúng tôi không có ý định liệt kê hết. Chúng tôi chỉ muốn cung cấp vừa đủ để bạn làm việc thành thạo, chứ không nhiều đến mức khiến bạn quá tải. Xin lượng thứ nếu tôi bỏ sót công cụ ưa thích của bạn.

Flutter SDK

Flutter SDK là công cụ duy nhất không thể thiếu. Nó bao gồm trình biên dịch Flutter, công cụ tạo project, trình quản lý thiết bị, trình chạy test, và các công cụ chẩn đoán — thậm chí sửa chữa — các vấn đề trong cấu hình Flutter.

Cài đặt Flutter SDK

Hướng dẫn cài đặt nằm ở đây: <https://flutter.dev/docs/get-started/install>. Nói ngắn gọn, việc này gồm tải về file zip công cụ mới nhất và thiết lập biến `PATH` trỏ tới thư mục bạn vừa giải nén. Các bước có khác nhau tùy hệ điều hành, nhưng trang web đó trình bày rất rõ ràng.

Mẹo

Bước này trông có vẻ rất "thấp tầng" và hơi đáng sợ, nhưng sau bước này mọi thứ sẽ dễ hơn và ít lỗi hơn nhiều. Đừng để nó làm bạn nản.

 **Ghi chú cập nhật:** Đường dẫn cài đặt hiện nay là <https://docs.flutter.dev/get-started/install>. Trên macOS và Linux bạn cũng có thể cài qua trình quản lý gói. Cách được khuyến nghị hiện nay là dùng **FVM** (Flutter Version Management) nếu bạn cần làm việc với nhiều phiên bản Flutter cùng lúc.

IDE

Về lý thuyết thì IDE không thực sự cần thiết. Bạn có thể viết Flutter bằng bất kỳ editor nào, rồi biên dịch và chạy bằng Flutter SDK đã cài trước đó. Nhưng trên thực tế gần như chẳng ai làm vậy. Việc gì phải thế? Những IDE dưới đây đã tích hợp sẵn hỗ trợ Flutter.

VS Code của Microsoft

VS Code là sản phẩm của Microsoft. Tên chính thức của nó là "Microsoft Visual Studio Code", nhưng đa số chúng ta chỉ gọi là VS Code. Dù gọi thế nào, xin đừng nhầm nó với một sản phẩm khác của Microsoft tên là "Microsoft Visual Studio". Hai thứ này hoàn toàn khác nhau, bất chấp cái tên na ná.

Bạn tải VS Code tại <https://code.visualstudio.com>.

Android Studio / IntelliJ của JetBrains

Android Studio và IntelliJ về cơ bản là một. Chúng được xây từ cùng một codebase và có cùng bộ tính năng.

Bạn tải Android Studio tại <https://developer.android.com/studio> và IntelliJ IDEA tại <https://www.jetbrains.com/idea/download>.

Nên dùng IDE nào?

Cả VS Code lẫn Android Studio/IntelliJ đều miễn phí và open source. Cả hai đều chạy được trên Windows, Mac và Linux. Cả hai đều phổ biến gần như ngang nhau trong giới developer Flutter,¹ không cái nào chiếm ưu thế rõ rệt. Bạn chọn cái nào cũng không sai.

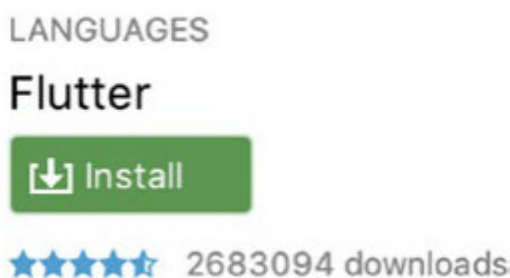
Nhưng nếu buộc phải chọn một, chúng tôi nhận thấy xuất thân của bạn có thể ảnh hưởng tới việc bạn hợp với công cụ nào. Những developer đến từ thế giới web — những người dùng HTML, CSS, JavaScript, NodeJS, React, Angular hay Vue — rất thích VS Code. Ngược lại, những developer đến từ thế giới Java, đặc biệt là Android developer, có xu hướng nghiêng về Android Studio/IntelliJ.

Tin tốt là lựa chọn này không hề áp lực. Việc đổi editor là chuyện nhỏ — kể cả khi đang làm dở một project. Hãy bắt đầu với một cái và xem bạn thấy thế nào. Nếu không hợp, dùng thử cái kia một thời gian. Cứ qua lại vài lần cho tới khi bạn có lựa chọn rõ ràng. Đổi qua đổi lại thực sự chẳng có gì to tát.

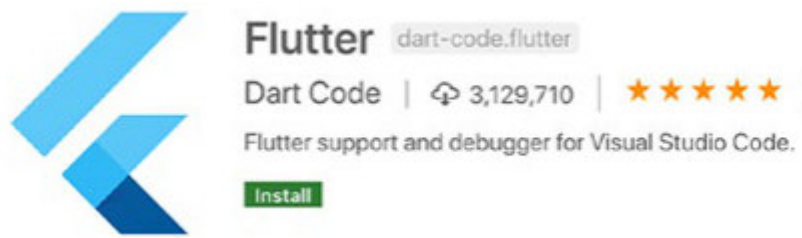
DevTools cho IDE

Các IDE trên rất tốt, nhưng chúng không được thiết kế riêng cho Flutter; chúng còn được dùng để phát triển với nhiều ngôn ngữ và framework khác. Vì vậy, để cải thiện luồng làm việc với Flutter, chúng ta nên cài Flutter DevTools. Nó bổ sung hỗ trợ debugger, cho phép xem log, kết nối liền mạch với emulator, và một vài thứ khác.

Việc cài DevTools được thực hiện ngay trong từng IDE. Trong Android Studio/IntelliJ, vào **"Preferences → Plugins"** từ menu chính (Hình 2-1). Trong VS Code, vào **"View → Extensions"** (Hình 2-2). Bộ Flutter devtools đơn giản chỉ có tên là "Flutter", tìm kiếm là ra ngay. Ở cả hai nền tảng, bấm nút "Install" màu xanh lá.



Hình 2-1. Cài DevTools trong Android Studio



Hình 2-2. Cài DevTools trong VS Code

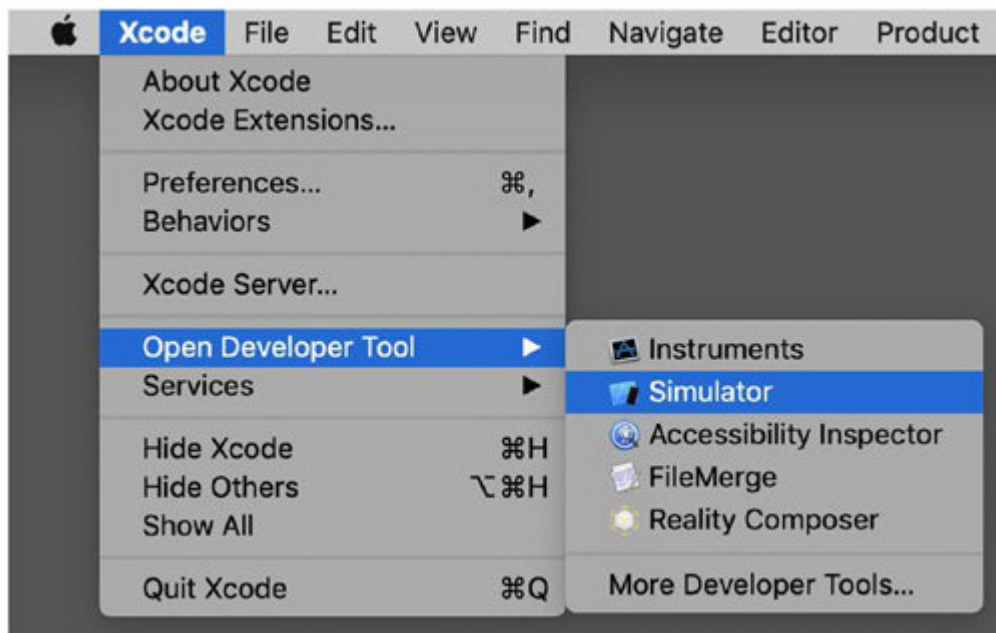
Bạn có thể cần khởi động lại IDE sau khi cài.

Emulator

Khi đã cài xong IDE và DevTools, bạn đã sẵn sàng biên dịch app. Nhưng để chạy được nó, bạn cần đưa nó lên một thiết bị. Một emulator — thiết bị ảo chạy ngay trên máy laptop/desktop của bạn — giúp việc chạy, test, debug và trình diễn app trở nên cực kỳ dễ dàng. Bạn hẳn sẽ muốn test trên cả iOS lẫn Android, nên bạn cần emulator cho từng bên. Có khá nhiều emulator, nhưng tôi chỉ nhắc tới hai cái: iOS simulator của Xcode và Android emulator của AVD.

iOS simulator

Nếu bạn không sở hữu máy Mac, bạn sẽ không chạy được iOS emulator, thậm chí cũng không biên dịch được cho iOS.² Nhưng nếu bạn có Mac và đã cài Xcode thì bạn may mắn rồi: bạn đã có sẵn iOS simulator. Để chạy nó, mở Xcode, rồi vào **Xcode** → **Open Developer Tool** → **Simulator** (Hình 2-3). Simulator sẽ khởi động, và từ trong đó bạn có thể chọn bất kỳ thiết bị iOS nào, kể cả iPhone lẫn iPad.

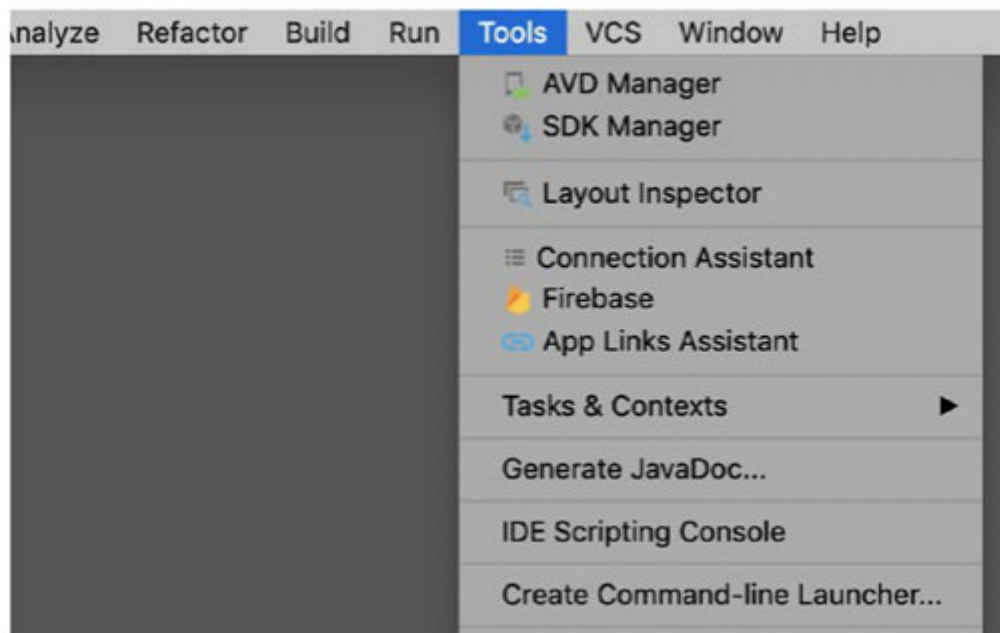


Hình 2-3. Mở iOS Simulator từ Xcode

Android emulator

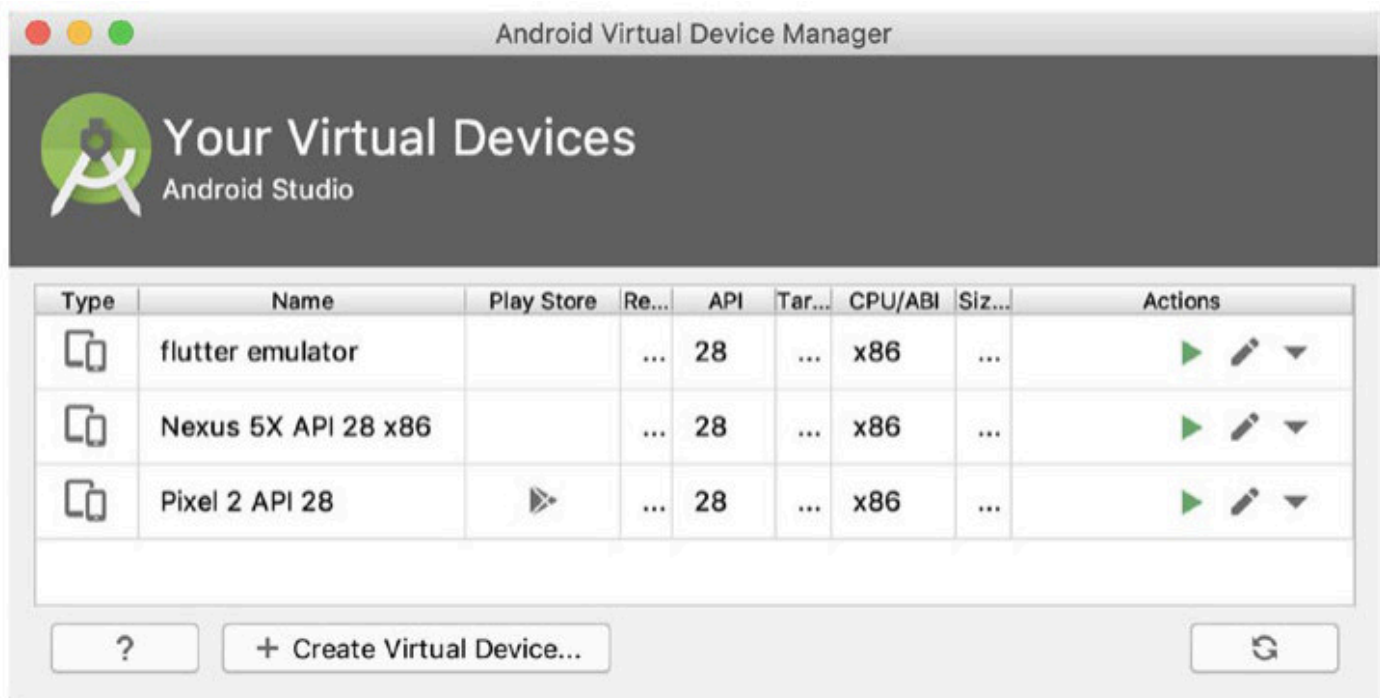
Cũng như có vô số mẫu máy Android, có vô số Android emulator, nhưng chỉ có hai cách phổ biến để làm việc với chúng: Genymotion và AVD Manager. Genymotion là một công ty kinh doanh, nên khi bạn ghé thăm website của họ, họ sẽ tìm mọi cách hướng bạn sang bản trả phí. Điều đó cũng dễ hiểu. Chúng ta sẽ tập trung vào AVD Manager vì nó hoàn toàn miễn phí và phổ biến hơn với dân Flutter.

AVD là viết tắt của "Android Virtual Device". AVD Manager nằm trong Android Studio, ở mục **Tools** (Hình 2-4).



Hình 2-4. Tìm AVD Manager trong Android Studio

Sau khi mở, bạn sẽ thấy danh sách các emulator hiện đã cài. Ban đầu danh sách này sẽ trống. Bạn có thể cài một hoặc nhiều trong số hàng trăm emulator thiết bị Android bằng cách bấm nút **" + Create Virtual Device..."** ở phía dưới (Hình 2-5).



Hình 2-5. AVD Manager hiển thị danh sách thiết bị hiện có. Bấm "+" để thêm

Bấm vào đó và bạn có thể chọn đủ loại thiết bị, hoặc tự tạo thiết bị của riêng mình. Mỗi thiết bị chỉ cần cài một lần. Sau khi cài xong, thiết bị ảo đó dùng được từ bất kỳ IDE nào, dù là IntelliJ/Android Studio hay VS Code. Không cần thiết lập riêng cho VS Code.

Giữ cho công cụ luôn được cập nhật

Thời kỳ đầu, việc phát triển cross-platform với các công cụ như Xamarin và React Native cực kỳ khó khăn vì số lượng công cụ tham gia quá lớn và chúng phụ thuộc lẫn nhau chẳng chịt.

Nhưng vì Flutter xuất hiện muộn hơn nên nó học được từ sai lầm của những người đi trước. Đội ngũ Flutter nhận ra những điểm khó chịu này và cho chúng ta một công cụ rất hay để quản lý toàn bộ phần còn lại của toolchain. Nó sẽ kiểm tra máy phát triển của bạn, tìm tất cả công cụ cần thiết để làm app Flutter, xem bạn đang có phiên bản nào, phiên bản nào đang sẵn có, chúng phụ thuộc nhau ra sao, rồi đưa ra chẩn đoán về các vấn đề. Nó thậm chí còn kê đơn cách khắc phục. Nghe cứ như bác sĩ vậy. Và đúng thế — xin giới thiệu với bạn: **flutter doctor**.

flutter doctor

Bạn chạy **flutter doctor** từ dòng lệnh. Nó kiểm tra toàn bộ công cụ trong toolchain và báo cáo lại mọi vấn đề gặp phải. Dưới đây là một ví dụ trong đó Xcode cần được xử lý thêm:

```
$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):

[✓] Flutter (Channel beta, vX.Y.Z, on Mac OS X X.Y.Z, locale en-US)
[✓] Android toolchain - develop for Android devices (SDK version X.Y.Z)
[!] Xcode - develop for iOS and macOS (Xcode X.Y)
    Xcode requires additional components to be installed in
    order to run.
    Launch Xcode and install additional required components
    when prompted.
[✓] Android Studio (version X.Y)
[✓] VS Code (version X.Y.Z)
[!] Connected device
    ! No devices available

! Doctor found issues in 2 categories.
$
```

Lỗi "No devices available" rất hay gặp, và thường thì bạn có thể bỏ qua nó. Nó chỉ có nghĩa là tại thời điểm đó không có emulator nào đang chạy.

Còn đây là kết quả mà chúng ta mong muốn thấy — mọi thứ đều ổn:

```
$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):

[✓] Flutter (Channel beta, vX.Y.Z, on Mac OS X X.Y.Z, locale en-US)
[✓] Android toolchain - develop for Android devices (SDK version X.Y.Z)
[✓] Xcode - develop for iOS and macOS (Xcode X.Y)
[✓] Android Studio (version X.Y)
[✓] VS Code (version X.Y.Z)
[✓] Connected device (1 available)

• No issues found!
```

`flutter doctor` không chỉ phát hiện và báo cáo vấn đề, mà thường còn kê luôn cách khắc phục cho từng vấn đề. Nó thậm chí báo cho bạn biết khi nào cần nâng cấp chính nó bằng lệnh `flutter upgrade`.

flutter upgrade

Đúng là việc cài đặt Flutter SDK lần đầu hơi đáng ngại, nhưng việc nâng cấp thì nhẹ nhàng. Bạn chỉ cần gõ đúng hai từ, `flutter upgrade`:

```

$ flutter upgrade
Upgrading Flutter from /usr/local/bin/flutter...

From https://github.com/flutter/flutter
    2d2a1ff..a72edc2  beta      -> origin/beta
    3932ffb..cc3ca9a  dev       -> origin/dev
    5a3a46a..a085635  master    -> origin/master
* [new branch]      refactor -> origin/refactor
<snip>
* [new tag]          v1.10.5   -> v1.10.5
Updating c382b8e..a72edc2
    11 files changed, 413 insertions(+), 302 deletions(-)
Building flutter tool...

Upgrading engine...
Downloading ios-deploy...                                0.3s

Flutter X.Y.Z · channel beta · https://github.com/flutter/flutter.git
Framework · revision a72e06 (23 hours ago) · 20XX-YY-ZZ 15:41:01 -0700
Engine · revision b863200c37
Tools · Dart X.Y.Z

Running flutter doctor...
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel beta, vX.Y.Z, on Mac OS X X.Y.Z, locale en-US)
[✓] Android toolchain - develop for Android devices (SDK version X.Y.Z)
[✓] Xcode - develop for iOS and macOS (Xcode X.Y)
[✓] Android Studio (version X.Y)
[✓] VS Code (version X.Y.Z)
[✓] Connected device (1 available)

• No issues found!

```

Lưu ý rằng `flutter doctor` được tự động chạy ở bước cuối cùng, xác nhận rằng mọi thứ đều ổn. Việc nâng cấp rất đơn giản.

 **Ghi chú cập nhật:** Channel `beta` trong ví dụ nay đã đổi — channel mặc định và được khuyến nghị hiện là `stable`. Channel `dev` đã bị khai tử. Nếu bạn quản lý Flutter bằng FVM hoặc cài qua trình quản lý gói, hãy nâng cấp bằng công cụ đó thay vì `flutter upgrade`.

Quy trình phát triển với Flutter

Giờ khi đã cài đủ công cụ và cập nhật chúng, hãy tạo một app và chạy nó qua debugger.

Tạo bộ khung app và các file

Tạo mới hoàn toàn một app Flutter bằng cách chạy...

```
$ flutter create my_app
```

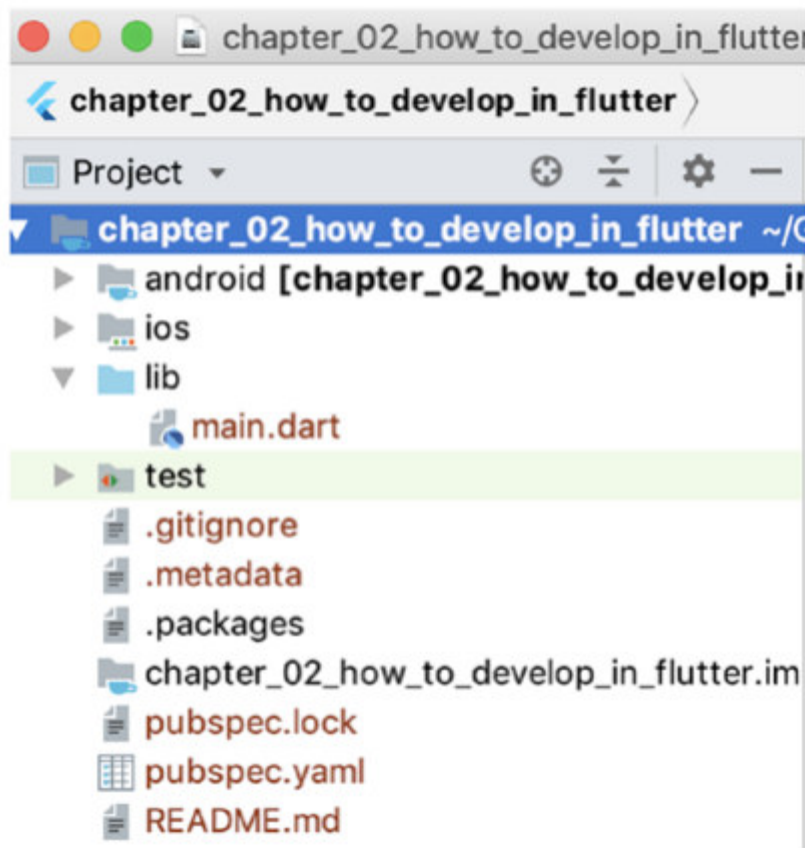
Lệnh này sẽ tạo một thư mục con tên `my_app` bên trong thư mục hiện tại. Nó sẽ chứa đầy code Dart sẵn sàng chạy.

Mẹo

Tên app không phân biệt hoa thường, nên bạn hãy viết thường toàn bộ. Dấu gạch ngang là ký tự không hợp lệ, nên bạn không thể dùng kebab-case. Cách đặt tên được khuyến nghị là `lowercase_with_underscores`.

Giải phẫu một project Flutter

Không nhất thiết phải biết hết mọi file và thư mục trong project vừa tạo. Nhưng nếu bạn tò mò, hãy cùng dạo qua nhanh một project Flutter mới tạo như trong Hình 2-6.



Hình 2-6. Một project Flutter mới toanh do `flutter create` sinh ra

Bạn sẽ có những thư mục sau:

- **android và ios** — Đây là những phần đặc thù cho từng nền tảng của project. Phần lớn thời gian bạn sẽ không cần đụng tới chúng.
- **lib** — Đây là nhà của toàn bộ mã nguồn Dart của bạn. Bạn sẽ xây dựng cây phân cấp của app ở đây. Đây là nơi bạn dành gần như toàn bộ thời gian và sự chú ý.
- **test** — Nếu bạn có unit test (và sớm muộn gì bạn cũng nên có), hãy đặt chúng ở đây.

Và bạn sẽ có những file sau:

- **pubspec.yaml** — Về cơ bản đây là file project của các dự án Dart. Đây là nơi chúng ta đặt tên project, mô tả, các dependency và nhiều thứ khác. Hãy đọc kỹ các dòng comment trong đó để hình dung rõ hơn những gì được khuyến nghị và những gì có thể làm.
- **.gitignore và README.md** — Rất quen thuộc với các developer dùng git và GitHub làm nơi lưu mã nguồn. Những người khác thì không cần quan tâm.

- **.metadata** và **.packages** — Đây là những file cấu hình quan trọng mà bạn sẽ không bao giờ phải mở. Nhưng Flutter cần chúng.

Mẹo

Còn một file nữa bạn nên biết: `analysis_options.yaml`. File này không bắt buộc, nhưng nếu có thì bạn sẽ viết code tốt hơn. File này báo cho IDE biết cần thực hiện **linting** (còn gọi là phân tích tĩnh) trên code ngay khi bạn viết. Với `analysis_options.yaml`, IDE sẽ cảnh báo khi bạn không tuân theo best practice.

Thay vì tự viết từ đầu, tôi khuyên bạn nên bắt đầu từ file của người khác. Đây là một file rất phổ biến trong cộng đồng Flutter: https://github.com/flutter/plugins/blob/master/analysis_options.yaml.

Nó khá khắt khe. Nếu muốn tắt bớt một số quy tắc, chỉ cần xóa dòng đó đi hoặc comment lại. Giải thích về toàn bộ các quy tắc linting có ở đây: <http://dart-lang.github.io/linter/lints/>.

 **Ghi chú cập nhật:** Repo `flutter/plugins` đã bị lưu trữ (archived). Hiện nay `flutter create` **tự động sinh sẵn** file `analysis_options.yaml` cho bạn, trong đó tham chiếu tới bộ quy tắc chính thức qua package `flutter_lints`. Danh sách quy tắc linting hiện nằm tại <https://dart.dev/tools/linter-rules>. Ngoài ra, file `.packages` đã bị loại bỏ và thay bằng `.dart_tool/package_config.json`.

Chạy app

Giờ bạn đã có một app Flutter. Hãy chạy nó. Có nhiều cách để chạy app. Cách phổ biến nhất là bấm nút "Play" màu xanh lá trong Android Studio/IntelliJ hoặc VS Code. Bạn cũng có thể chạy từ dòng lệnh bằng `flutter run`:

```
$ flutter run
Running "flutter pub get" in chapter_02_how_to_develop_in_flutter... 0.5s
Launching lib/main.dart on iPhone X in debug mode...
Running Xcode build...
  Assembling Flutter resources... 6.1s
  Compiling, linking and signing... 5.9s
Xcode build done. 13.8s
Syncing files to device iPhone X... 1,852ms

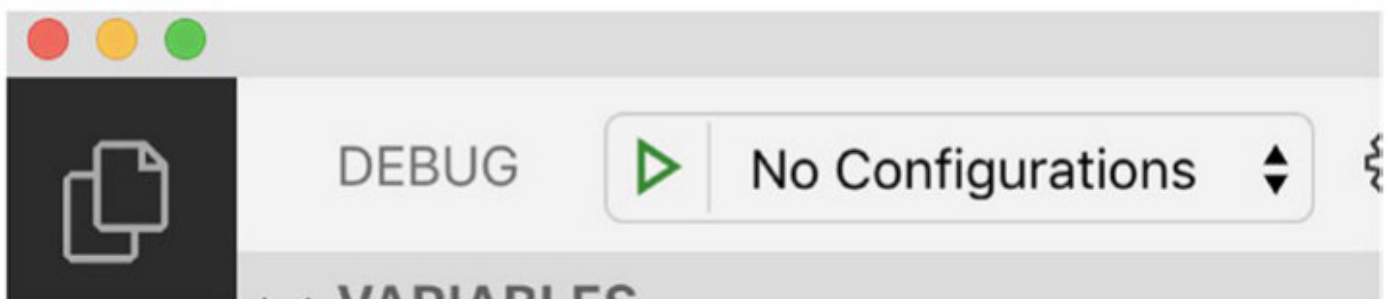
To hot reload changes while running, press "r". To hot
restart (and rebuild state), press "R".

An Observatory debugger and profiler on iPhone X is available
at: http://127.0.0.1:52550/8m0h8zacV58=/
For a more detailed help message, press "h". To detach, press
"d"; to quit, press "q".
```

Nhưng nếu bạn bấm nút Play/Debug màu xanh lá trong IDE (Hình 2-7), bạn sẽ có thêm lựa chọn debug app bằng cách đặt breakpoint và chạy từng bước qua code bằng developer tools (Hình 2-8).



Hình 2-7. Nút Play và Debug nằm ở phía trên trong Android Studio



Hình 2-8. Nút Play nằm ở góc trên bên trái trong VS Code

Rõ ràng bạn cần chạy app trên một thiết bị nào đó. Có vài lựa chọn: trình duyệt Chrome cho web app, emulator, hoặc một thiết bị vật lý được nối vào máy phát triển bằng cáp. Khi bạn bấm nút Play/Debug, bạn được chọn thiết bị muốn chạy tại thời điểm đó. Để ý trong ảnh chụp Android Studio ở trên, có một menu thả xuống liệt kê các thiết bị khả dụng. Trong VS Code, bấm nút Play và một menu sẽ hiện ra ngay với các lựa chọn của bạn. Với cả hai IDE, quyền kiểm soát nằm trong tay bạn.

Mẹo

Bạn có thể kiểm tra những thiết bị nào đang khả dụng bằng cách chạy `flutter devices` từ dòng lệnh.

```
$ flutter devices
3 connected devices:

Vivo XL3      • 55S...KF • android-arm64 • Android 8.0.0 (API 26)
Android SDK   • emul...4 • android-x86    • Android 9 (API 28) (emulator)
iPhone X      • E6...39A • ios            • com.apple...OS-12-1 (simulator)
```

Kết quả mẫu ở trên cho biết chúng ta có ba thiết bị. Thiết bị thứ nhất và thứ hai chạy Android, thiết bị thứ ba chạy iOS. Thiết bị đầu tiên là máy thật nối cáp. Thiết bị thứ hai và thứ ba là emulator.

Lưu ý lệnh này khác với lệnh `flutter emulators` — lệnh đó liệt kê tất cả emulator mà bạn *có thể* chọn. Còn `flutter devices` cho biết những thiết bị nào *đang* sẵn sàng để chạy app của bạn.

Chạy app dưới dạng web app

Flutter coi trình duyệt của bạn là một thiết bị khi bạn chạy dưới dạng web app. Vì vậy tất cả những gì cần làm là bật trình duyệt Google Chrome lên như một thiết bị. Bạn bật nó bằng lệnh chỉ cần chạy một lần:

```
$ flutter config --enable-web
Setting "enable-web" value to "true".
```

Từ đó về sau, khi bạn xem danh sách thiết bị để chạy app, "Chrome" sẽ xuất hiện như một lựa chọn. Chỉ cần chọn chạy app trong Chrome và IDE sẽ nạp web app của bạn vào đó.

 **Ghi chú cập nhật:** Lệnh `flutter config --enable-web` **không còn cần thiết**. Từ Flutter 2 trở đi, hỗ trợ web đã được bật sẵn trên channel stable. Chrome sẽ tự động xuất hiện trong danh sách thiết bị.

Chạy app trên thiết bị nối cáp

Có những lúc bạn cần chạy app trên máy thật. Ví dụ, tôi từng làm một project liên quan tới việc in nhãn ra một máy in vật lý kết nối qua Bluetooth. Emulator không ghép nối được qua Bluetooth. Để test việc in, tôi cần một thiết bị thật đã được ghép nối sẵn với máy in Bluetooth của mình.

Để nối một thiết bị thật vào máy phát triển, bạn dùng cáp USB với phần lớn máy Android và cáp Lightning với phần lớn iPhone.

Mẹo

#1 Khi kết nối một thiết bị Android, ban đầu nó sẽ tưởng bạn đang muốn sạc pin hoặc chuyển ảnh. Để báo cho nó biết bạn đang muốn debug, hãy mở màn hình Developer Options trên thiết bị và chọn "**Enable USB debugging**".

#2 Rất nhiều lỗi kết nối bắt nguồn từ cáp USB kém chất lượng. Nghe có vẻ khó tin, nhưng không phải cáp USB nào cũng như nhau. Hãy đổi sang cáp chất lượng cao hơn nếu bạn vẫn không kết nối được sau khi đã chỉnh đúng thiết lập.

 **Ghi chú cập nhật:** iPhone từ đời 15 trở đi dùng cổng USB-C thay cho Lightning.

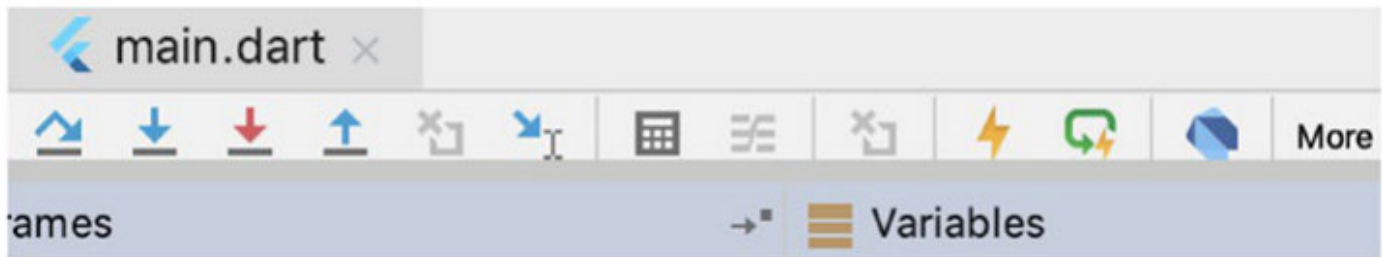
Hot reload

Khi app đã chạy trong emulator/trình duyệt/máy thật, bạn sẽ muốn sửa mã nguồn rồi chạy lại. Và đây là điều thực sự hay ho: mỗi khi bạn lưu một thay đổi trong mã nguồn, nó được biên dịch lại và phiên bản mới được nạp vào ngay lập tức. App của bạn tiếp tục từ đúng chỗ bạn đang dừng — cùng màn hình, cùng state, cùng dữ liệu. Chúng ta gọi đó là **hot reload**, và nó khiến chu kỳ phát triển nhanh và mượt đến khó tin.

Debug

Cả hai IDE về cơ bản đều có những công cụ debug mà bạn đã quen thuộc ở mọi IDE khác. Khi bạn khởi chạy project, các công cụ debug sẽ hiện ra.

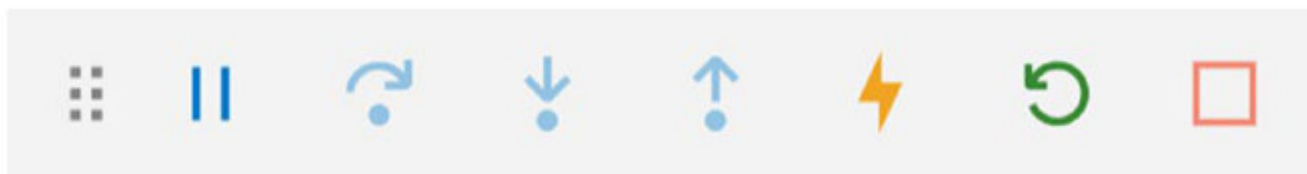
Trong Android Studio, cửa sổ debug mở ra, thường ở phía dưới IDE. Nó có một thanh công cụ nhỏ trông như Hình 2-9.



Hình 2-9. Thanh công cụ debug trong Android Studio

Các tùy chọn lần lượt từ trái sang phải là "step over", "step into", "force step into" và "step out".

Trong VS Code, thanh công cụ hiện lên nổi phía trên mã nguồn của bạn (Hình 2-10).



Hình 2-10. Thanh công cụ debug trong VS Code

Các tùy chọn của nó là "play/pause", "step over", "step into", "step out", "hot reload", "restart" và "stop debugging".

📌 Lưu ý

Flutter khó tính hơn khi bạn debug so với khi chạy thật trên thiết bị. Đây là điều tốt, vì trong lúc debug nó làm lộ ra những lỗi mà bạn nên sửa, dù chúng không nhất thiết gây chết chương trình. Ở bản release, nó nuốt luôn những lỗi đó và (hy vọng là) cho phép người dùng tiếp tục chạy app.

Một nhóm lỗi như vậy là **runtime assertion**. Bạn biết mình đang gặp loại lỗi này khi debugger đưa ra thông báo kiểu:

```
== Exception caught by gesture ==  
The following assertion was thrown while handling a gesture:  
setState() callback argument returned a Future.  
The setState() method on _FooState#236 was called with a  
closure or method that returned a Future.  
Maybe it is marked as "async".  
v.v.
```

Điều bạn cần nhớ là: khi thấy một lỗi như thế, hãy sửa nó. Đó là việc nên làm. Nhưng đừng bối rối nếu bạn không thấy lại đúng lỗi đó sau khi đã deploy.

Tổng kết

Tôi biết đây là một lượng kiến thức khá lớn cần tiếp thu. Bản chất của phát triển cross-platform khiến bộ công cụ trở nên rối rắm. Nhưng phần khó nhất đã qua rồi. Khi bạn đã cài xong Flutter SDK và một IDE (VS Code / Android Studio / IntelliJ IDEA), thế là đủ những gì bạn thực sự cần. Tất nhiên, DevTools và một hai emulator sẽ giúp ích nhiều. Việc còn lại chỉ là luyện tập cho quen tay. Bạn sẽ làm tốt thôi!

Giờ khi đã xem qua toolchain của Flutter, hãy bắt đầu tạo widget nào!

1. Một khảo sát gần đây với các developer Flutter do Andrew Brogdon (@redbrogdon) thuộc đội Flutter thực hiện cho thấy 53% dùng VS Code, 30% dùng Android Studio, và 15% dùng IntelliJ. Xem http://bit.ly/flutter_devtools_poll ↩
2. Tác giả châm biếm: "Cảm ơn nhé, Apple." — ý nói việc Apple bắt buộc phải có máy Mac mới build được app iOS là điều gây khó chịu cho developer. ↩

PHẦN II — Nền tảng Flutter

Chương 3: Mọi thứ đều là Widget

Hãy giả sử bạn là một tín đồ Lego cực kỳ tài năng và vừa được nhận một trong số ít vị trí đáng mơ ước: Lego Master Builder. Xin chúc mừng! Và giả sử nhiệm vụ đầu tiên của bạn là dựng một tượng Thor cao gần 2 mét từ 26.000 mảnh Lego (Hình 3-1).



Hình 3-1. Tượng Thor bằng Lego. Tác giả chụp bức ảnh này tại một rạp chiếu phim

Bạn sẽ làm việc đó như thế nào? Hãy suy nghĩ một phút. Cứ từ từ, chúng tôi chờ.

Bạn có nhặt đại từng viên gạch rồi ghép lại với nhau không? Chắc là không. Bạn có bắt đầu từ lòng bàn chân Thor rồi xây dần lên không? Cũng không nốt. Đây là dự đoán của tôi về chiến lược hợp lý của bạn:

1. Bạn hình dung ra thứ mình sắp dựng. Nhìn được toàn cảnh.
2. Nhận ra rằng cả dự án quá phức tạp để dựng một lần.
3. Chia dự án thành các phần (chân, tay trái, tay phải, thân, kiếm trái, kiếm phải, mũ, áo choàng, đầu).
4. Nhận ra rằng mỗi phần vẫn còn quá phức tạp.
5. Với mỗi phần, bạn chia nhỏ tiếp thành các phần con.
6. Lập lại bước 4 và 5 cho tới khi bạn có được những thành phần đủ đơn giản để mỗi cái đều dễ hiểu, dễ dựng và dễ bảo trì — cho bạn và cho bất kỳ đồng đội nào của bạn.
7. Tạo ra từng thành phần đơn giản đó.
8. Ghép các thành phần đơn giản lại để tạo thành những thành phần lớn hơn, phức tạp hơn.
9. Lập lại bước 7 và 8 cho tới khi toàn bộ dự án hoàn thành.

Quy trình này có một cái tên: **componentization** (thành phần hóa), và đó chính xác là cách tư duy chúng ta sẽ áp dụng với các project Flutter.

Componentization không phải là điều gì mới. Thực tế nó đã được đề xuất từ tận năm 1968.¹ Nhưng kỹ thuật này gần đây bùng nổ về độ phổ biến nhờ các web framework như Angular, React, Vue, Polymer và native web components. Có vẻ như thời nay ai cũng làm software component.

Ý tưởng chia nhỏ để quy những phần phức tạp thành những phần đơn giản hơn được gọi là **decomposition** (phân rã). Còn hành động ghép các mảnh đã viết trở lại thành những thành phần lớn hơn được gọi là **composition** (kết hợp).

Trong thế giới Flutter, những thành phần này được gọi là **widget**. Dân Flutter hay nói "everything is widgets" (mọi thứ đều là widget), nghĩa là bạn và tôi sẽ dùng những widget do Google cung cấp — những cái đi kèm sẵn với Flutter. Chúng ta sẽ kết hợp chúng lại để tạo ra widget tùy chỉnh của riêng mình. Rồi các widget tùy chỉnh đó lại được kết hợp với nhau để tạo ra những widget tùy chỉnh ngày càng phức tạp hơn. Cứ thế cho tới khi bạn có một app hoàn chỉnh.

Trong thế giới Flutter, component được gọi là **widget**.

Mọi app đều có thể được nhìn nhận theo hai phần:

1. **Behavior** (hành vi) — Phần mềm *làm* gì. Toàn bộ business logic nằm ở đây: đọc, ghi và xử lý dữ liệu.
2. **Presentation** (trình bày) — Phần mềm *trông* như thế nào. Giao diện người dùng. Các nút bấm, ô nhập liệu, nhãn.

Chỉ có Flutter gộp hai phần này vào **một** ngôn ngữ thay vì hai.

UI as code

Các framework khác đã chứng minh componentization là hướng đi đúng. Đội ngũ Flutter công khai thừa nhận rằng họ chịu ảnh hưởng rất lớn từ React² — vốn dựa trên componentization. Thực tế thì các nhà làm framework đều học hỏi lẫn nhau rất nhiều. Nhưng Flutter độc đáo ở cách biểu đạt giao diện người dùng. Developer dùng **cùng một** ngôn ngữ Dart để biểu đạt cả giao diện đồ họa lẫn hành vi của app (Bảng 3-1). Chúng ta gọi đây là "**UI as code**".

Bảng 3-1. Chỉ Flutter dùng cùng một ngôn ngữ cho cả presentation và behavior

Framework	Behavior viết bằng...	UI viết bằng...
Xamarin	C#	XAML
React Native	JavaScript	JSX
NativeScript	JavaScript	XML
Flutter	Dart	Dart

Vậy giao diện này được tạo ra như thế nào? Giống như nhiều framework và ngôn ngữ khác, một app Flutter bắt đầu bằng hàm `main`. Trong Flutter, `main` sẽ gọi một hàm tên là `runApp()`. Hàm `runApp()` này nhận vào một widget — widget gốc (root widget), có thể đặt tên bất kỳ, nhưng nó phải là một class kế thừa từ `StatelessWidget` của Flutter. Nó trông như thế này:

```
// import package Dart cần thiết cho mọi app Flutter
import 'package:flutter/material.dart';

// Đây là main gọi runApp
void main() => runApp(RootWidget());

// Và đây là root widget của bạn
class RootWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text("Hello world");
  }
}
```

Chỉ cần vậy là bạn đã có một "Hello world" trong Flutter.

Nhưng khoan... cái `Text()` này là gì vậy? Đó là một widget dựng sẵn của Flutter. Vì những widget dựng sẵn này rất quan trọng, chúng ta cần xem qua chúng.

Các widget dựng sẵn của Flutter

Những widget nền tảng của Flutter chính là viên gạch xây nên mọi thứ chúng ta tạo ra, và chúng có rất nhiều — khoảng 160 cái theo lần đếm gần nhất.³ Đây là con số khá lớn để bạn và tôi ghi nhớ. Nhưng nếu bạn sắp xếp chúng lại trong đầu, mọi thứ sẽ dễ quản lý hơn nhiều.

Chúng rơi vào những nhóm chính sau:

- Value widget
- Layout widget
- Navigation widget
- Các widget khác

Lưu ý

Đây không phải danh sách phân loại chính thức của Flutter. 14 nhóm chính thức của họ được liệt kê tại <https://flutter.dev/docs/development/ui/widgets>. Chúng tôi chỉ thấy rằng việc sắp xếp lại như thế này giúp dễ nhớ hơn.

Chúng ta sẽ lướt nhanh qua từng nhóm kèm một hai ví dụ, rồi đào sâu ở các chương sau. Bắt đầu với value widget.

Value widget

Một số widget chứa một giá trị — có thể là giá trị đến từ bộ nhớ cục bộ, từ một service trên Internet, hoặc từ chính người dùng. Chúng được dùng để hiển thị giá trị cho người dùng xem và để lấy giá trị từ người dùng vào app. Ví dụ kinh điển là widget `Text`, hiển thị một đoạn chữ. Một ví dụ khác là widget `Image`, hiển thị ảnh `.jpg`, `.png` hoặc định dạng khác.

Đây là một số value widget khác:

Checkbox	FormField	RefreshIndicator
CircularProgressIndicator	Icon	RichText
Date & Time Pickers	IconButton	Slider
DataTable	Image	Switch
DropDownButton	LinearProgressIndicator	Text
FlatButton	PopupMenuButton	TextField
FloatingActionButton	Radio	Tooltip
FlutterLogo	RaisedButton	
Form	RawImage	

 **Ghi chú cập nhật:** FlatButton và RaisedButton trong bảng trên **đã bị gỡ khỏi Flutter**. Chúng được thay thế bằng TextButton và ElevatedButton. Code dùng hai widget cũ sẽ không biên dịch được trên các phiên bản Flutter hiện nay.

Chúng ta sẽ khám phá value widget chi tiết hơn ở chương tiếp theo.

Layout widget

Layout widget cho chúng ta rất nhiều quyền kiểm soát để bố trí màn hình đúng ý — đặt widget cạnh nhau hay trên dưới nhau, làm cho chúng cuộn được, cho chúng xuống dòng, quy định khoảng trống xung quanh widget để chúng không bị chật chội, v.v.:

Align	FittedBox	Padding
AppBar	Flow	PageView
AspectRatio	FractionallySizedBox	Placeholder
Baseline	GridView	Row
BottomSheet	IndexedStack	Scaffold
ButtonBar	IntrinsicHeight	Scrollable
Card	IntrinsicWidth	Scrollbar
Center	LayoutBuilder	SingleChildScrollView
Column	LimitedBox	SizedBox
ConstrainedBox	ListBody	SizedOverflowBox
Container	ListTile	SliverAppBar
CustomMultiChildLayout	ListView	SnackBar
Divider	MediaQuery	Stack
Expanded	NestedScrollView	Table
ExpansionPanel	OverflowBox	Wrap

Đây là một chủ đề lớn nên chúng tôi dành hẳn một chương cho nó: Chương 6, "Sắp xếp bố cục cho Widget".

Navigation widget

Khi app của bạn có nhiều scene ("màn hình", "trang", hay bạn muốn gọi là gì cũng được), bạn cần một cách nào đó để di chuyển giữa chúng. Đó là lúc navigation widget phát huy tác dụng. Chúng điều khiển việc người dùng đang xem một scene rồi chuyển sang scene tiếp theo. Thông thường việc này xảy ra khi người dùng chạm vào một nút. Và đôi khi nút điều hướng nằm trên một tab bar hoặc trong một drawer trượt ra từ cạnh trái màn hình. Đây là một số navigation widget:

AlertDialog	MaterialApp	TabBar
BottomNavigationBar	Navigator	TabBarView
Drawer	SimpleDialog	

Chúng ta sẽ học cách chúng hoạt động ở Chương 7, "Navigation và Routing".

Các widget khác

Và không, không phải widget nào cũng rơi gọn vào những nhóm trên. Hãy gom phần còn lại vào một nhóm hỗn hợp. Đây là một số widget thuộc nhóm này:

GestureDetector	Cupertino	Transitions
Dismissible	Theme	Transforms

Nhiều widget trong nhóm hỗn hợp này sẽ được đề cập rải rác trong sách ở những chỗ phù hợp. Riêng GestureDetector quan trọng đến mức nó có hẳn một chương riêng: Chương 5, "Phản hồi cử chỉ người dùng".

Cách tự tạo stateless widget của riêng bạn

Vậy là chúng ta đã biết mình sẽ kết hợp những widget dựng sẵn này để tạo ra widget tùy chỉnh của riêng mình, rồi những widget đó lại được kết hợp với các widget dựng sẵn khác để cuối cùng tạo thành một app.

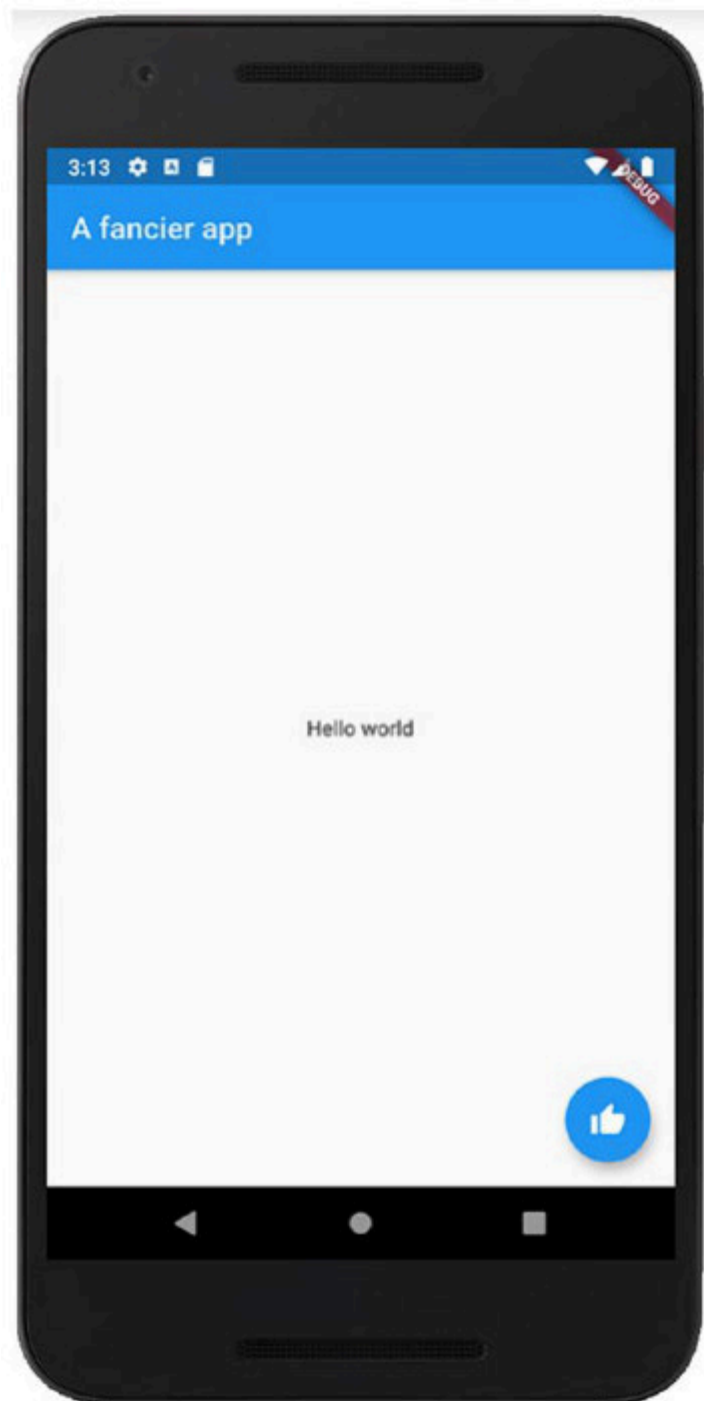
Widget được thiết kế rất khéo, bởi mỗi widget đều dễ hiểu và do đó dễ bảo trì. Nhìn từ bên ngoài thì widget là trừu tượng, còn bên trong thì hợp lý và dễ đoán. Làm việc với chúng rất dễ chịu.

Mỗi widget là một class có thể có property và method. Mỗi widget có thể có một constructor với không hoặc nhiều tham số. Và quan trọng nhất, mỗi widget đều có một method `build` — nhận vào một `BuildContext`⁴ và trả về **một** widget Flutter duy nhất. Nếu bạn thắc mắc vì sao một widget lại trông như vậy, hãy tìm tới method `build` của nó:

```
class RootWidget extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Text('Hello world');  
  }  
}
```

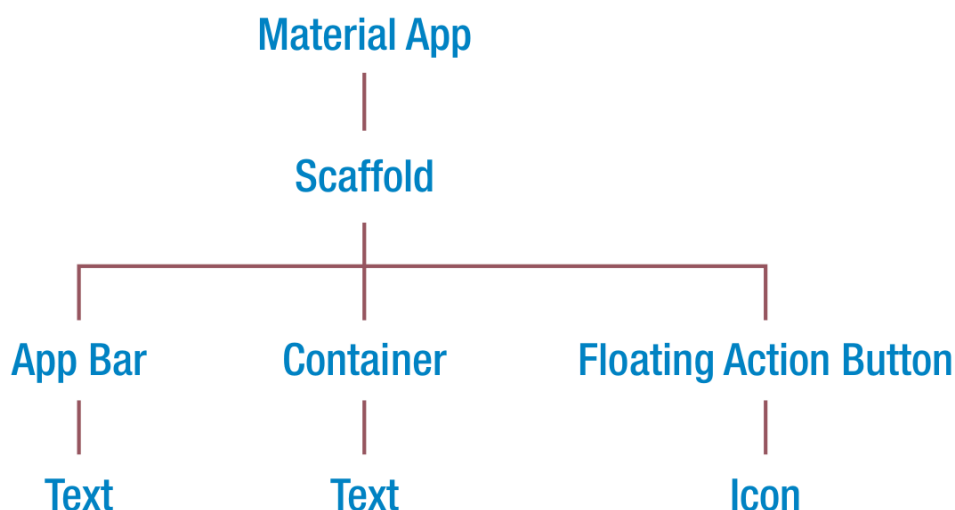
Trong ví dụ hello world này — lặp lại từ đầu chương — chúng ta đang hiển thị một widget `Text` (Hình 3-2). Một widget con duy nhất thì chạy được, nhưng app thực tế sẽ phức tạp hơn nhiều. Root widget có thể được cấu thành từ rất nhiều widget con khác:


```
class FancyHelloWidget extends StatelessWidget {  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: Text("A fancier app"),  
        ),  
        body: Container(  
          alignment: Alignment.center,  
          child: Text("Hello world"),  
        ),  
        floatingActionButton: FloatingActionButton(  
          child: Icon(Icons.thumb_up),  
          onPressed: () => {},  
        ),  
      ),  
    );  
  }  
}
```



Hình 3-2. App được tạo ra bởi widget đơn giản này

Như bạn thấy, method `build` trả về **một** widget duy nhất là `MaterialApp`, nhưng bên trong nó chứa một `Scaffold`, và `Scaffold` chứa ba widget con: `AppBar`, `Container` và `FloatingActionButton` (Hình 3-3). Mỗi widget đó lại chứa các widget con của riêng nó.



Hình 3-3. Cây widget của app ví dụ ở trên

Method `build` của bạn sẽ luôn hoạt động như vậy. Nó trả về một biểu thức duy nhất, đồ sộ và lồng nhau. Chính việc widget lồng trong widget lồng trong widget cho phép bạn tạo ra widget tùy chỉnh phức tạp của riêng mình.

Widget có key

Bạn có thể nghe các developer khác nhắc tới **virtual DOM** khi nói về Flutter. Khái niệm này đến từ thế giới React. (Nhớ rằng Flutter học hỏi rất nhiều từ kiến trúc xuất sắc của React.) Nói chính xác thì Flutter không có DOM, nhưng nó có duy trì một thứ tương tự — **element tree**. Element tree là một bản sao nhỏ gọn của toàn bộ widget đang hiển thị trên màn hình. Flutter duy trì một element tree hiện tại và một bản đã được áp dụng các thay đổi theo lô.

Lý do là thế này: Flutter sẽ rất chậm nếu nó áp dụng từng thay đổi nhỏ xíu lên màn hình rồi cố vẽ lại hàng trăm lần mỗi giây. Thay vào đó, Flutter áp dụng tất cả thay đổi đó lên một bản sao của element tree. Sau đó nó định kỳ "diff" element tree hiện tại với bản đã sửa và quyết định phần nào thực sự cần vẽ lại. Nó chỉ vẽ lại đúng những phần cần thiết. Cách này nhanh hơn rất, rất nhiều.

Nhưng đôi khi Flutter bị nhầm lẫn khi khớp các widget giữa hai element tree. Bạn sẽ biết là mình cần gán **key** bằng code khi: dữ liệu thay đổi nhưng widget lại được vẽ sai vị trí, dữ liệu không được cập nhật trên màn hình, hoặc vị trí cuộn không được giữ nguyên.

Phần lớn thời gian bạn không cần bận tâm tới key. Nó cần thiết hiếm đến mức chúng tôi thấy chỉ cần bạn hiểu rằng...

1. Key tồn tại, và vì sao Flutter có thể cần tới chúng.

2. Nếu widget của bạn không được vẽ lại như mong đợi khi dữ liệu thay đổi, key có thể là lời giải.
3. Bạn có quyền gán key cho một số widget nhất định.

Nếu bấy nhiêu vẫn chưa đủ với bạn lúc này, Emily Fortuna có một video mười phút rất hay về key.⁵

Truyền giá trị vào widget của bạn

Bạn có biết công thức này nghĩa là gì không?

$$y = f(x)$$

Dân toán sẽ đọc nó là "Y là một hàm của X". Nó diễn đạt ngắn gọn rằng khi X (biến độc lập) thay đổi, Y (biến phụ thuộc) sẽ thay đổi theo một cách có thể dự đoán được. Flutter sống dựa trên ý tưởng này, nhưng trong Flutter công thức đó đọc là:

$$\text{Scene} = f(\text{Data})$$

Nói cách khác, khi dữ liệu trong app thay đổi, màn hình sẽ thay đổi tương ứng. Và bạn — developer — là người quyết định dữ liệu đó được trình bày ra sao khi bạn viết method `build` trong widget của mình. Đây là một khái niệm nền tảng của Flutter.

Vậy dữ liệu đó thay đổi bằng cách nào? Có hai cách:

1. Widget có thể được vẽ lại với dữ liệu mới được truyền từ bên ngoài vào.
2. Dữ liệu có thể được duy trì bên trong một số widget nhất định.

Hãy nói về cách thứ nhất. Để truyền dữ liệu vào một widget, bạn gửi nó vào dưới dạng tham số constructor như sau:

```
Widget build(BuildContext context) {  
  return Person("Sarah"); // Truyền "Sarah" vào một widget  
}
```

Nếu một widget đại diện cho cách hiển thị một `Person`, thì việc truyền vào một `firstName` là hoàn toàn bình thường, giống như chúng ta vừa làm với "Sarah". Nếu làm vậy, bạn cần viết constructor của widget để nhận giá trị đó:

```
class Person extends StatelessWidget {
  final String firstName;
  Person(this.firstName) {}
  Widget build(BuildContext context) {
    return Text('$firstName');
  }
}
```

Đây là cú pháp Dart. Chú ý ba điểm. **Thứ nhất**, bạn liệt kê tham số đầu vào trong constructor (`this.firstName` trong ví dụ trên). **Thứ hai**, nhớ đặt `this.` phía trước nó. Chữ `this.` khớp tham số đó với một property ở cấp class, thay vì một tham số cục bộ của hàm constructor. **Thứ ba**, đánh dấu property tương ứng của class là `final`.

Bạn có thể muốn truyền vào hai property trở lên như sau:

```
Widget build(BuildContext context) {
  return Person("Sarah", "Ali");
}
```

Tất nhiên truyền vào hai giá trị nghĩa là phải tạo hai biến `final` và hai tham số constructor để xử lý chúng:

```
class Person extends StatelessWidget {
  final String firstName;
  final String lastName;
  Person(this.firstName, this.lastName) {}
  Widget build(BuildContext context) {
    return Text('$firstName $lastName');
  }
}
```

Như bạn đoán được, chúng được khớp theo **vị trí** — điều này rất dễ nhầm và không linh hoạt lắm. Cách làm tốt hơn là dùng tham số có tên (**named parameter**):

```
Widget build(BuildContext context) {  
  return Person(firstName: "Sarah", lastName: "Ali");  
}
```

Cách này giảm nhầm lẫn cho những developer khác sử dụng widget của bạn. Đây là cách viết widget để nhận giá trị đó:

```
class Person extends StatelessWidget {  
  final String firstName;  
  final String lastName;  
  Person({this.firstName, this.lastName}) {}  
  Widget build(BuildContext context) {  
    return Container(child: Text('$firstName $lastName'));  
  }  
}
```

Bạn thấy khác biệt chưa? Khá tinh tế. Giờ đây có cặp dấu ngoặc nhọn bao quanh các tham số constructor. Điều này khiến chúng trở thành tham số tùy chọn và có tên.

 **Ghi chú cập nhật:** Từ khi Dart có **null safety** (Dart 2.12 trở đi), đoạn code trên sẽ báo lỗi biên dịch vì `firstName` và `lastName` là kiểu `String` không cho phép null nhưng lại là tham số tùy chọn. Cách viết đúng hiện nay là dùng `required`:

```
class Person extends StatelessWidget {  
  final String firstName;  
  final String lastName;  
  const Person({super.key, required this.firstName, required this.lastName});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(child: Text('$firstName $lastName'));  
  }  
}
```

Nếu bạn thực sự muốn tham số là tùy chọn, hãy khai báo kiểu là `String?` hoặc cung cấp giá trị mặc định.

Mẹo

Lưu ý rằng trong cả ba ví dụ trên, chúng ta dùng class `Person` có thể được định nghĩa ngay trong cùng file Dart nơi bạn sử dụng nó. Nhưng cách làm tốt hơn là tạo mỗi class trong một file Dart riêng rồi import vào những file Dart khác cần dùng.

```
import 'Person.dart';
```

Stateless và Stateful widget

Từ đầu tới giờ chúng ta cố ý chỉ tạo stateless widget. Nên chắc bạn cũng đoán được là còn có stateful widget. Bạn đoán đúng. **Stateless widget** là widget không duy trì state của riêng nó. **Stateful widget** thì có.

"State" trong ngữ cảnh này chỉ dữ liệu bên trong widget có thể thay đổi trong suốt vòng đời của nó. Hãy nghĩ về widget `Person` ở trên. Nếu đó là widget chỉ hiển thị thông tin của một người, nó nên là stateless. Nhưng nếu đó là widget quản lý thông tin người dùng, nơi ta cho phép người dùng thay đổi dữ liệu bằng cách gõ vào một `TextField`, thì ta sẽ cần một `StatefulWidget`.

Có hẳn một chương về stateful widget ở phần sau. Nếu bạn nóng lòng muốn biết thêm ngay, hãy đọc Chương 9, "Quản lý State", rồi quay lại đây.

Vậy tôi nên tạo loại nào?

Câu trả lời ngắn gọn: hãy tạo stateless widget. Đừng bao giờ dùng stateful widget cho tới khi buộc phải dùng. Hãy mặc định rằng mọi widget bạn tạo đều sẽ là stateless và bắt đầu theo cách đó. Chỉ refactor chúng thành stateful widget khi bạn chắc chắn mình thực sự cần state. Nhưng hãy nhận ra rằng state có thể tránh được thường xuyên hơn nhiều so với suy nghĩ của các developer. Hãy tránh nó khi có thể, để widget đơn giản hơn và do đó dễ viết, dễ bảo trì, dễ mở rộng hơn. Đồng đội của bạn sẽ biết ơn bạn vì điều đó.

Lưu ý

Thực ra còn có loại widget thứ ba: `InheritedWidget`. Bạn đặt một giá trị vào `InheritedWidget` và bất kỳ widget con cháu nào cũng có thể ngược lên trên cây để hỏi trực tiếp dữ liệu đó. Đây là chủ đề hơi nâng cao, nhưng Rémi Rousselet chắc sẽ không tha cho tôi nếu tôi không nhắc tới nó. Bạn có thể đọc thêm ở Chương 9, "Quản lý State", hoặc xem phần tổng quan ngắn gọn về `InheritedWidget` của Emily Fortuna tại <http://bit.ly/inheritedWidget>.

Tổng kết

Vậy là giờ chúng ta đã biết app Flutter xoay quanh widget. Bạn sẽ tự kết hợp nên các Stateless hoặc Stateful widget tùy chỉnh của riêng mình, mỗi cái có một method `build` để render ra một cây các widget dựng sẵn của Flutter. Rõ ràng chúng ta cần hiểu về những widget dựng sẵn đó — và chúng ta sẽ bắt đầu học từ chương tiếp theo.

1. <http://bit.ly/componentHistory> ↩

2. Nguồn: <https://flutter.dev/docs/resources/faq#does-flutter-come-with-a-framework> ↩

3. Bạn có thể xem danh sách đầy đủ tại <https://flutter.dev/docs/reference/widgets> ↩

4. Đừng để `BuildContext` làm bạn phân tâm. Nó được framework sử dụng và thình thoảng chúng ta cũng có nhắc tới, nhưng những ví dụ đó để dành cho phần sau của sách. Còn bây giờ, cứ coi nó như một phần trong công thức để viết một widget tùy chỉnh. ↩

5. Bạn có thể xem video của Emily tại <http://bit.ly/FlutterKeys> ↩

Chương 4: Value Widget

Chương trước chúng ta đã học rằng mọi thứ đều là widget. Mọi thứ bạn tạo ra đều là widget và mọi thứ Flutter cung cấp cũng là widget. Tất nhiên có ngoại lệ, nhưng nghĩ theo hướng này không bao giờ có hại, nhất là khi bạn mới bắt đầu với Flutter.

Trong chương này chúng ta sẽ đào sâu vào nhóm widget cơ bản nhất mà Flutter cung cấp — những widget chứa một giá trị. Chúng ta sẽ nói về widget `Text`, widget `Icon` và widget `Image`, tất cả đều hiển thị đúng như tên gọi. Sau đó chúng ta đi vào các input widget — những widget được thiết kế để lấy dữ liệu từ người dùng.

Widget Text

Nếu bạn muốn hiển thị một chuỗi ra màn hình, `Text` chính là widget bạn cần.

```
Text('Hello world'),
```

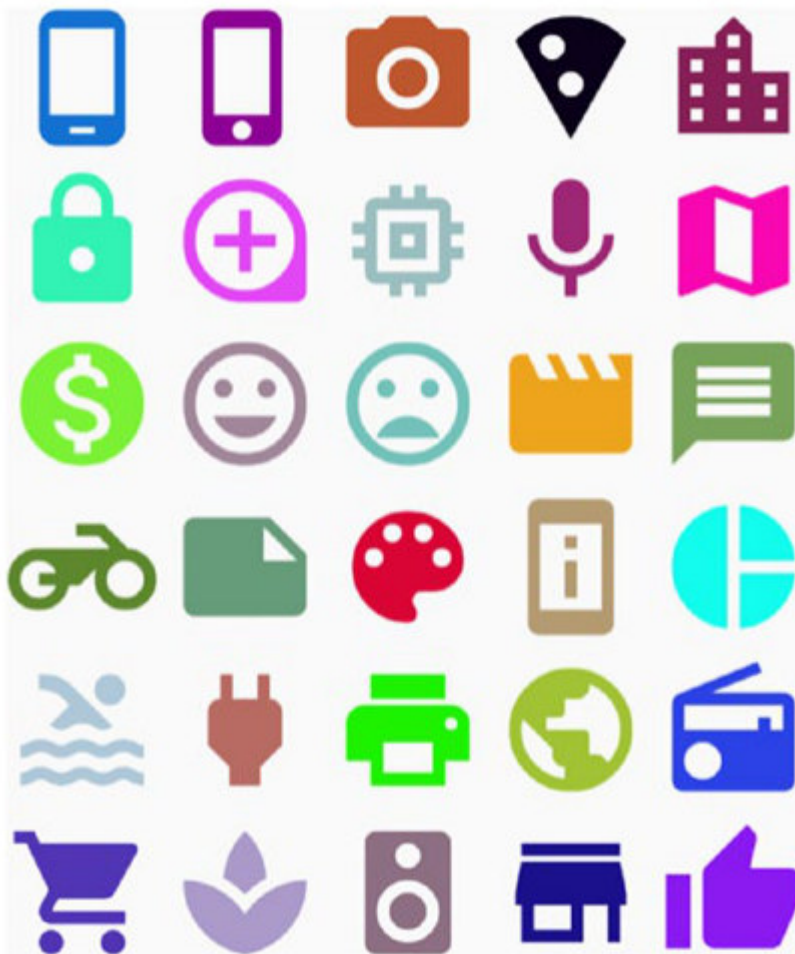
Mẹo

Nếu `Text` của bạn là một giá trị hằng, hãy đặt từ khóa `const` phía trước, widget sẽ được tạo ở thời điểm biên dịch thay vì lúc chạy. File apk/ipa của bạn sẽ lớn hơn một chút nhưng chạy nhanh hơn trên thiết bị. Rất đáng đánh đổi.

Bạn có toàn quyền điều khiển kích thước, font, độ đậm, màu sắc và nhiều thứ khác của `Text` thông qua property `style`. Nhưng phần đó sẽ được trình bày ở Chương 8, "Tạo kiểu cho Widget".

Widget Icon

Flutter đi kèm một bộ icon dựng sẵn rất phong phú (Hình 4-1), từ máy ảnh, con người, thẻ bài, phương tiện, mũi tên, pin cho tới thiết bị Android/iOS. Danh sách đầy đủ có tại <https://api.flutter.dev/flutter/material/Icons-class.html>.



Hình 4-1. Một số icon dựng sẵn của Flutter với màu ngẫu nhiên

Để đặt một icon, bạn dùng widget `Icon`. Không có gì bất ngờ. Bạn dùng class `Icons` để chỉ định icon nào. Class này có hàng trăm giá trị static như `Icons.phone_android`, `Icons.phone_iphone` và `Icons.cake`. Mỗi giá trị trở tới một icon khác nhau như trong hình trên. Đây là cách bạn đặt một chiếc bánh sinh nhật đỏ to đùng (Hình 4-2) vào app:

```
Icon(  
  Icons.cake,  
  color: Colors.red,  
  size: 200,  
)
```



Hình 4-2. Icon bánh kem màu đỏ

Widget Image

Hiển thị ảnh trong Flutter phức tạp hơn một chút so với `Text` hay `Icon`. Nó gồm vài việc:

1. **Lấy nguồn ảnh** — Nguồn này có thể là ảnh nhúng sẵn trong app hoặc tải trực tiếp từ Internet. Nếu ảnh không bao giờ thay đổi trong suốt vòng đời app, ví dụ logo hay hình trang trí, thì nên dùng ảnh nhúng.
2. **Định kích thước** — Phóng to hoặc thu nhỏ về đúng kích thước và hình dạng mong muốn.

Ảnh nhúng (embedded image)

Ảnh nhúng nhanh hơn nhiều nhưng sẽ làm tăng dung lượng cài đặt của app. Để nhúng ảnh, hãy đặt file ảnh vào thư mục project, tốt nhất là trong một thư mục con tên `images` cho gọn gàng. Ví dụ `assets/images` là hợp lý.

Sau đó sửa file `pubspec.yaml`. Thêm vào nội dung này:

```
flutter:  
  assets:  
    - assets/images/photo1.png  
    - assets/images/photo2.jpg
```

Lưu file lại và chạy `flutter pub get` từ dòng lệnh để project xử lý file đó.

💡 Mẹo

File `pubspec.yaml` chứa đủ loại thông tin quan trọng về project của bạn. Nó chứa metadata của project như tên, mô tả, địa chỉ repository và số phiên bản. Nó liệt kê các dependency thư viện và font chữ. Đây là nơi đầu tiên các developer mới tham gia project sẽ tìm đến. Với những bạn làm JavaScript, nó chính là file `package.json` của project Dart.

Sau đó bạn đưa ảnh vào widget tùy chỉnh của mình bằng cách gọi constructor `asset()` như sau:

```
Image.asset('assets/images/photo1.jpg'),
```

Ảnh từ mạng (network image)

Ảnh từ mạng gần gũi hơn với những gì web developer đã quen. Đơn giản là lấy ảnh qua Internet bằng HTTP. Bạn dùng constructor `network` và truyền vào một URL dạng chuỗi.

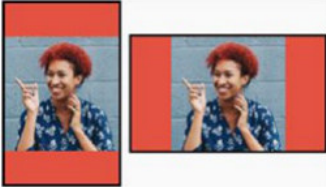
```
Image.network(imageUrl),
```

Như bạn đoán được, cách này chậm hơn ảnh nhúng vì có độ trễ khi request được gửi tới server qua Internet và ảnh được thiết bị tải về. Ưu điểm là những ảnh này "sống": bất kỳ ảnh nào cũng có thể được nạp động chỉ bằng cách đổi URL.

Định kích thước ảnh

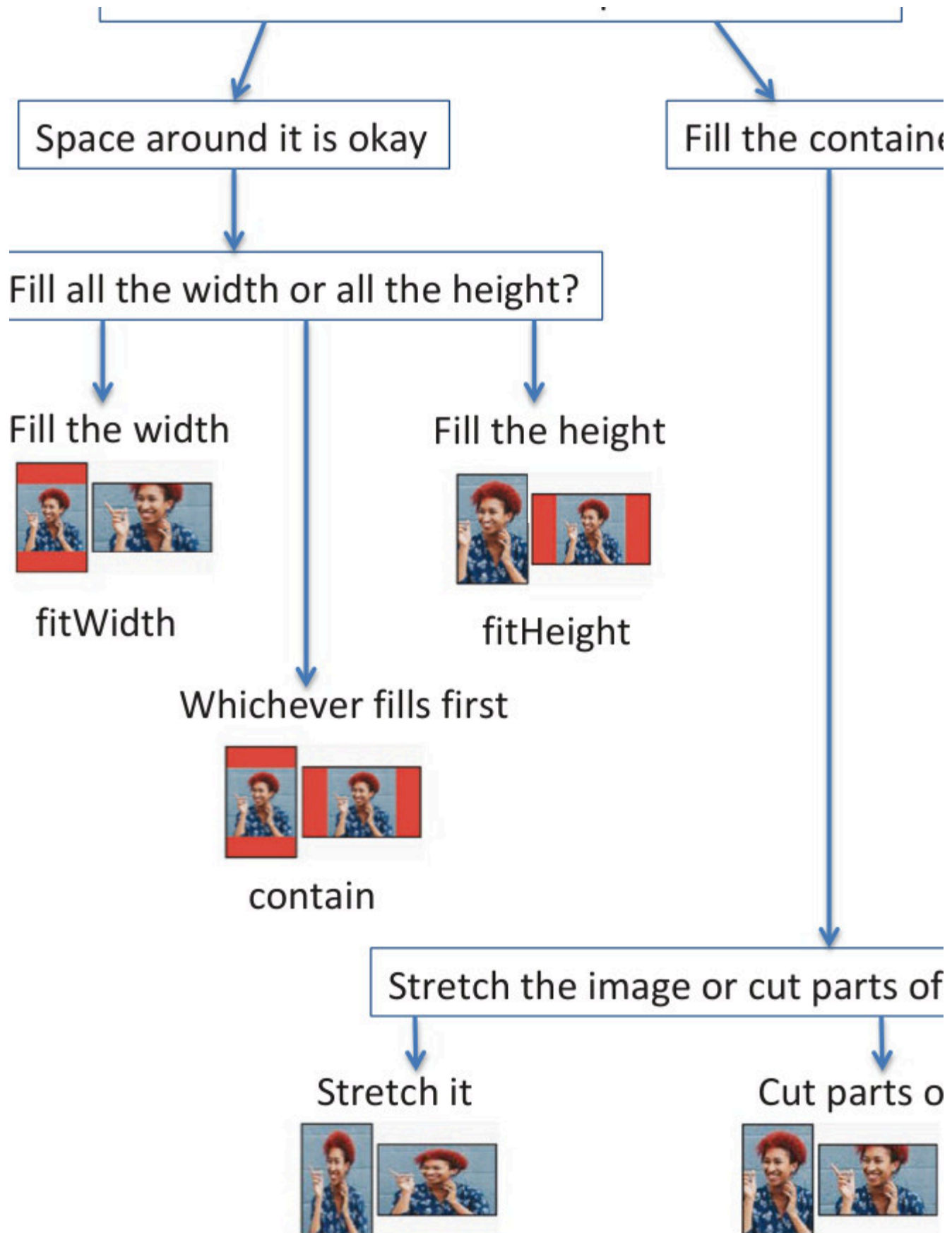
Ảnh gần như luôn được đặt trong một container. Không phải vì đó là yêu cầu bắt buộc, chỉ là tôi không hình dung nổi một tình huống thực tế nào mà ảnh lại không nằm bên trong một widget khác. Container có tiếng nói trong việc ảnh được vẽ ở kích thước nào. Sẽ là một trùng hợp kỳ diệu nếu kích thước tự nhiên của ảnh vừa khít kích thước container. Thay vào đó, layout engine của Flutter sẽ thu nhỏ ảnh cho vừa container, nhưng không phóng to. Kiểu fit này gọi là `BoxFit.scaleDown`, và nó hợp lý khi làm hành vi mặc định. Nhưng còn những lựa chọn nào khác và làm sao quyết định dùng cái nào? Bảng 4-1 trình bày các lựa chọn `BoxFit` của bạn.

Bảng 4-1. Các lựa chọn `BoxFit`

Giá trị	Ảnh gốc → kết quả	Mô tả
fill		Kéo giãn sao cho cả chiều rộng lẫn chiều cao vừa khít. Ảnh bị méo
cover		Thu nhỏ hoặc phóng to cho tới khi lấp đầy khoảng trống. Phần trên/dưới hoặc hai bên sẽ bị cắt
fitHeight		Làm cho chiều cao vừa khít. Cắt bớt chiều rộng hoặc thêm khoảng trống nếu cần
fitWidth		Làm cho chiều rộng vừa khít. Cắt bớt chiều cao hoặc thêm khoảng trống nếu cần
contain		Thu nhỏ cho tới khi cả chiều cao lẫn chiều rộng đều vừa. Sẽ có khoảng trống thừa ở trên/dưới hoặc hai bên

Ảnh: Eye for Ebony trên Unsplash

Đó là các lựa chọn của bạn, nhưng chọn thế nào? Hình 4-3 có thể giúp bạn quyết định dùng kiểu fit nào trong từng tình huống.



Hình 4-3. Cách quyết định kiểu fit cho ảnh

Để chỉ định kiểu fit, bạn đặt property `fit` :

```
Image.asset('assets/images/woman.jpg',  
  fit: BoxFit.contain),
```

Input widget

Nhiều người trong chúng ta đến từ nền tảng web, nơi ngay từ đầu đã có `<form>` với `<input>` và `<select>` trong HTML. Tất cả những thứ đó tồn tại để người dùng đưa dữ liệu vào web app — một hoạt động mà app mobile cũng không thể thiếu. Flutter cung cấp các widget để nhập liệu giống như trên Web, nhưng chúng không hoạt động theo cùng cách. Chúng tốn công hơn nhiều để tạo và sử dụng. Xin lỗi về điều đó. Nhưng đổi lại chúng an toàn hơn và cho chúng ta nhiều quyền kiểm soát hơn.

Một phần của sự phức tạp là những widget này không tự duy trì state của chúng; bạn phải làm việc đó thủ công.

Một phần khác là các input widget không biết tới sự tồn tại của nhau. Nói cách khác, chúng không phối hợp tốt với nhau cho tới khi bạn gom chúng lại bằng một widget `Form`. Cuối cùng chúng ta sẽ cần tập trung vào widget `Form`. Nhưng trước đó, hãy tìm hiểu cách tạo text field, checkbox, radio button, slider và dropdown.

⚠️ Cẩn trọng

Input widget rất khó làm việc trừ khi chúng được dùng bên trong một `StatefulWidget`, bởi vì về bản chất chúng thay đổi state. Nhớ rằng chúng tôi đã nhắc sơ về `StatefulWidget` ở chương trước và sẽ nói sâu về chúng ở Chương 9, "Quản lý State". Còn tới lúc đó, hãy cứ tin lời chúng tôi và đặt chúng trong một stateful widget.

Text field

Nếu bạn chỉ cần một ô nhập văn bản đơn lẻ, có lẽ bạn muốn dùng widget `TextField`. Đây là một ví dụ đơn giản về `TextField` với một nhãn `Text` phía trên:


```
const Text('Search terms'),  
TextField(  
  onChanged: (String val) => _searchTerm = val,  
)
```

Property `onChanged` là một event handler được kích hoạt sau mỗi lần gõ phím. Nó nhận một giá trị duy nhất — một `String`. Đây chính là giá trị người dùng đang gõ. Trong ví dụ trên, chúng ta gán biến cục bộ `_searchTerm` bằng bất cứ thứ gì người dùng gõ.

Để cung cấp giá trị khởi tạo cho `TextField`, bạn cần tới `TextEditingController` — một thứ phức tạp không cần thiết:

```
TextEditingController _controller =  
  TextEditingController(text: "Initial value here");
```

Sau đó báo cho `TextField` biết về controller đó:

```
const Text('Search terms'),  
TextField(  
  controller: _controller,  
  onChanged: (String val) => _searchTerm = val,  
)
```

Bạn cũng có thể dùng property `_controller.text` để lấy ra giá trị mà người dùng đang gõ vào ô đó.

Bạn có để ý dòng `Text('Search terms')` không? Đó là nỗ lực vụng về của chúng ta trong việc đặt một nhãn phía trên `TextField`. Có một cách tốt hơn nhiều. Xem đây...

Làm cho `TextField` đẹp hơn

Có rất nhiều tùy chọn để `TextField` trở nên hữu ích hơn — không phải vô hạn, nhưng rất nhiều. Và tất cả đều có sẵn thông qua widget `InputDecoration` (Hình 4-4):

```
return TextField(  
  controller: _emailController,  
  decoration: InputDecoration(  
    labelText: 'Email',  
    hintText: 'you@email.com',  
    icon: Icon(Icons.contact_mail),  
  ),  
),
```



Hình 4-4. Một `TextField` với `InputDecoration`

Bảng 4-2 trình bày thêm một số tùy chọn của `InputDecoration`.

Bảng 4-2. Các tùy chọn của `InputDecoration`

Property	Mô tả
<code>labelText</code>	Xuất hiện phía trên <code>TextField</code> . Cho người dùng biết ô này dùng để làm gì
<code>hintText</code>	Chữ mờ bên trong <code>TextField</code> . Biến mất khi người dùng bắt đầu gõ
<code>errorText</code>	Thông báo lỗi hiện bên dưới <code>TextField</code> . Thường có màu đỏ. Nó được đặt tự động bởi validation (nói sau), nhưng bạn có thể tự đặt nếu cần
<code>prefixText</code>	Chữ nằm trong <code>TextField</code> , phía bên trái phần người dùng gõ
<code>suffixText</code>	Giống <code>prefixText</code> nhưng ở phía ngoài cùng bên phải
<code>icon</code>	Vẽ một icon ở bên trái toàn bộ <code>TextField</code>
<code>prefixIcon</code>	Vẽ một icon bên trong <code>TextField</code> , phía bên trái
<code>suffixIcon</code>	Giống <code>prefixIcon</code> nhưng ở phía ngoài cùng bên phải

 Mẹo

Để biến nó thành ô nhập mật khẩu (Hình 4-5), hãy đặt property `obscureText` thành `true`. Khi người dùng gõ, mỗi ký tự hiện ra trong một giây rồi được thay bằng một dấu chấm.

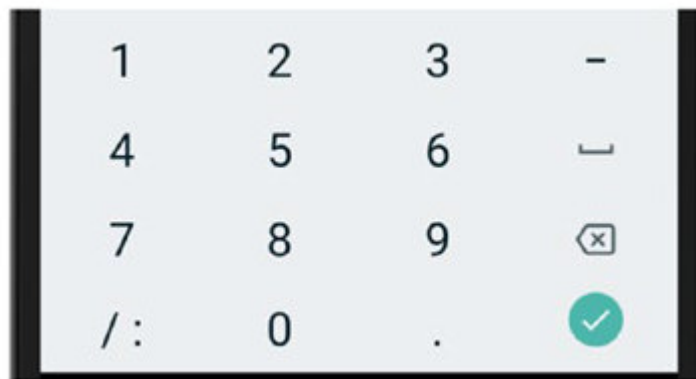
```
return TextField(  
  obscureText: true,  
  decoration: InputDecoration(  
    labelText: 'Password',  
  ),  
);
```



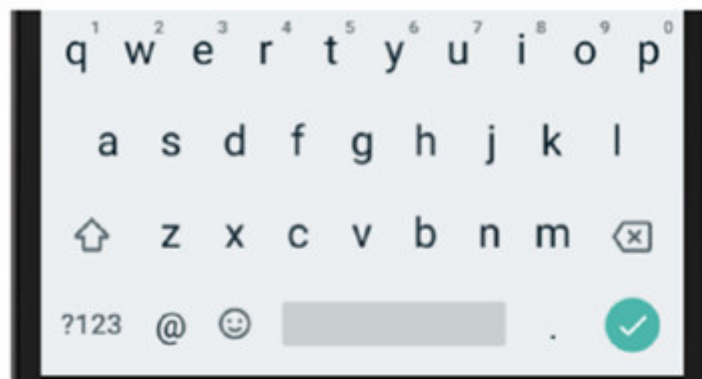
Hình 4-5. Ô nhập mật khẩu với `obscureText`

Muốn một bàn phím ảo đặc biệt? Không vấn đề gì. Chỉ cần dùng property `keyboardType`. Kết quả được minh họa từ Hình 4-6 đến Hình 4-9.

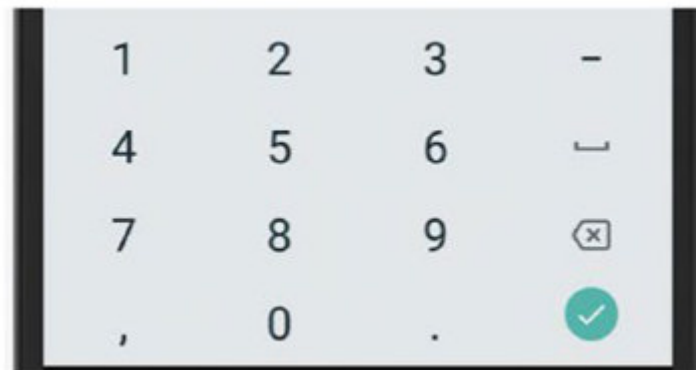
```
return TextField(  
  keyboardType: TextInputType.number,  
);
```



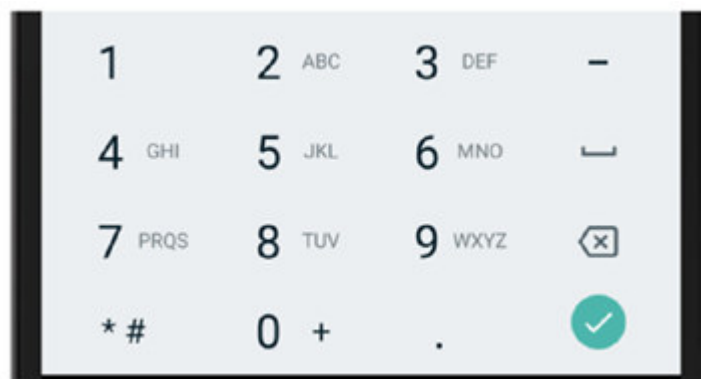
Hình 4-6. `TextInputType.datetime`



Hình 4-7. `TextInputType.email` — để ý dấu @



Hình 4-8. `TextInputType.number`



Hình 4-9. `TextInputType.phone`

 Mẹo

Nếu bạn muốn giới hạn loại ký tự được phép nhập, bạn có thể làm điều đó bằng property `inputFormatters` của `TextField`. Nó thực chất là một mảng nên bạn có thể kết hợp một hoặc nhiều trong số...

- `BlacklistingTextInputFormatter` — Cấm nhập một số ký tự nhất định. Chúng đơn giản là không xuất hiện khi người dùng gõ.
- `WhitelistingTextInputFormatter` — Chỉ cho phép nhập những ký tự này. Bất cứ gì ngoài danh sách đều không hiện ra.
- `LengthLimitingTextInputFormatter` — Không cho gõ quá X ký tự.

Hai cái đầu cho phép bạn dùng biểu thức chính quy (regular expression) để mô tả mẫu ký tự bạn muốn (white list) hoặc không muốn (black list). Ví dụ:

```
return TextField(  
  inputFormatters: [  
    WhitelistingTextInputFormatter(RegExp('[0-9 -]')),  
    LengthLimitingTextInputFormatter(16)  
  ],  
  decoration: InputDecoration(  
    labelText: 'Credit Card',  
  ),  
);
```

Trong `WhitelistingTextInputFormatter`, chúng ta chỉ cho phép số từ 0–9, dấu cách hoặc dấu gạch ngang. Còn `LengthLimitingTextInputFormatter` giới hạn tối đa 16 ký tự.

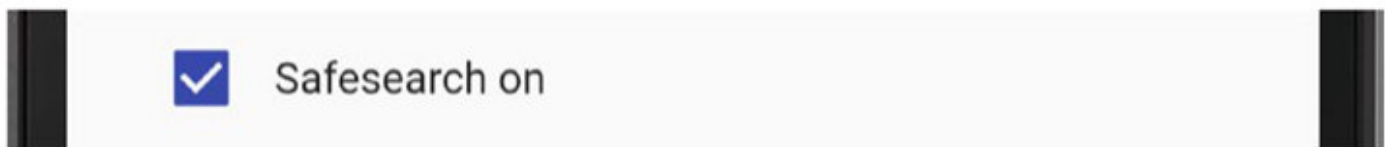
Ghi chú cập nhật: `BlacklistingTextInputFormatter` và `WhitelistingTextInputFormatter` **đã bị gỡ bỏ**. Hãy dùng `FilteringTextInputFormatter` thay thế:

```
inputFormatters: [
  FilteringTextInputFormatter.allow(RegExp('[0-9 -]')), // thay cho Whitelisting
  // FilteringTextInputFormatter.deny(RegExp('[abc]')), // thay cho Blacklisting
  LengthLimitingTextInputFormatter(16),
],
```

Checkbox

Checkbox trong Flutter (Hình 4-10) có property `value` kiểu boolean và method `onChanged` được kích hoạt sau mỗi lần thay đổi. Giống mọi input widget khác, method `onChanged` nhận vào giá trị mà người dùng vừa đặt. Với Checkbox, giá trị đó là kiểu `bool`.

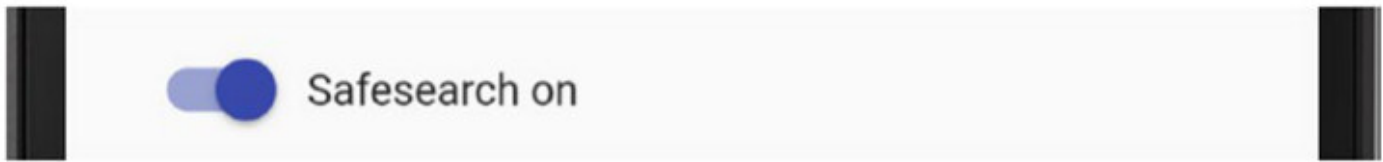
```
Checkbox(
  value: true,
  onChanged: (bool val) => print(val)),
```



Hình 4-10. Widget *Checkbox* của Flutter

Mẹo

Widget `Switch` của Flutter (Hình 4-11) phục vụ cùng mục đích với `Checkbox` — bật hoặc tắt. Vì vậy `Switch` có cùng bộ tùy chọn và hoạt động theo cùng cách. Nó chỉ trông khác đi thôi.



Hình 4-11. *Widget Switch của Flutter*

Ghi chú cập nhật: Với null safety, `onChanged` của `Checkbox` nhận kiểu `bool?` (có thể null), không phải `bool`. Hãy viết `onChanged: (bool? val) => ...`.

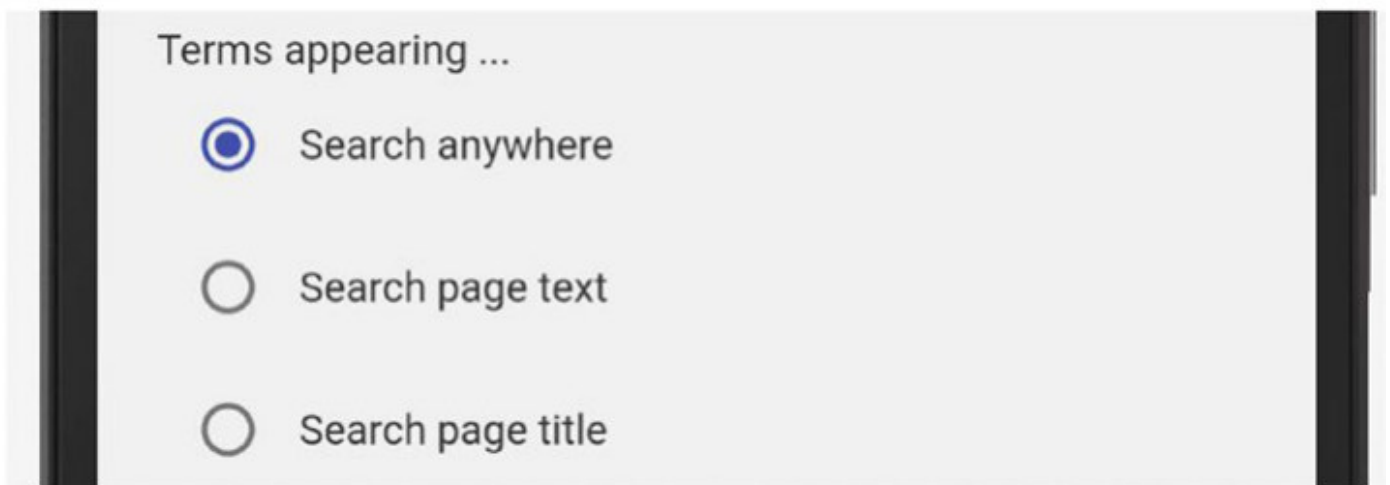
Radio button

Điều đặc biệt ở radio button là khi bạn chọn một cái, những cái khác trong cùng nhóm sẽ bị bỏ chọn. Nên rõ ràng chúng ta cần nhóm chúng lại bằng cách nào đó. Trong Flutter, các widget `Radio` được nhóm lại khi bạn đặt property `groupValue` trở tới cùng một biến cục bộ. Biến này giữ giá trị của cái `Radio` đang được bật.

Mỗi `Radio` cũng có property `value` riêng — giá trị gắn với widget đó, bất kể nó có được chọn hay không. Trong method `onChanged`, bạn gán biến `groupValue` bằng giá trị `value` của radio đó:


```
SearchType _searchType;
// Code khác nằm ở đây
Radio<SearchType>(
  groupValue: _searchType,
  value: SearchType.anywhere,
  onChanged: (SearchType val) => _searchType = val),
const Text('Search anywhere'),
Radio<SearchType>(
  groupValue: _searchType,
  value: SearchType.text,
  onChanged: (SearchType val) => _searchType = val),
const Text('Search page text'),
Radio<SearchType>(
  groupValue: _searchType,
  value: SearchType.title,
  onChanged: (SearchType val) => _searchType = val),
const Text('Search page title'),
```

Đoạn code rút gọn này sẽ tạo ra thứ gì đó như Hình 4-12.



Hình 4-12. Các widget *Radio* của Flutter

Slider

Slider là một điều khiển tiện lợi khi bạn muốn người dùng chọn một giá trị số nằm giữa một giới hạn dưới và một giới hạn trên (Hình 4-13).



Hình 4-13. Một slider với giá trị 25

Để có một cái trong Flutter, bạn dùng widget `Slider`, vốn cần một sự kiện `onChanged` và một property `value` kiểu `double`. Nó cũng có `min` mặc định là `0.0` và `max` mặc định là `1.0`. Khoảng từ 0 tới 1 hiếm khi hữu ích, nên thường bạn sẽ đổi giá trị đó. Nó còn có property `label` — một chỉ báo cho người dùng biết họ đang chọn giá trị nào.

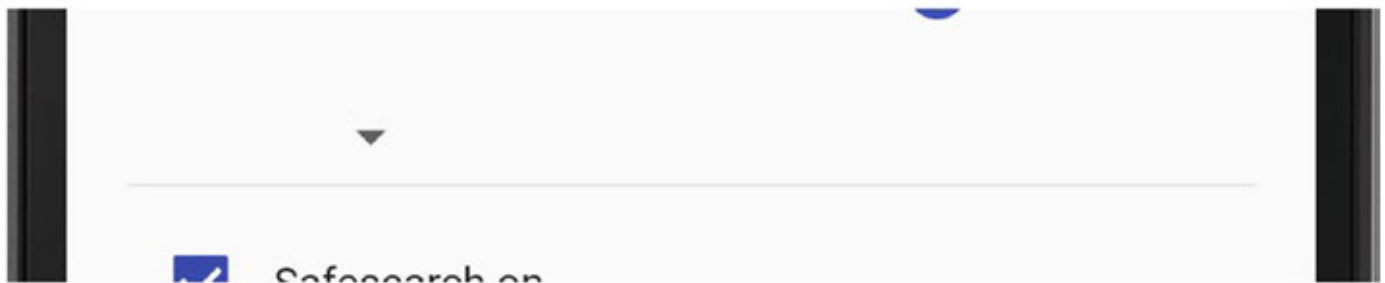
```
Slider(  
  label: _value.toString(),  
  min: 0, max: 100,  
  divisions: 100,  
  value: _value,  
  onChanged: (double val) => _value = val,  
),
```

Dropdown

Dropdown rất hợp để chọn một trong số ít lựa chọn, ví dụ như trong một enum. Giả sử chúng ta có một enum như sau:

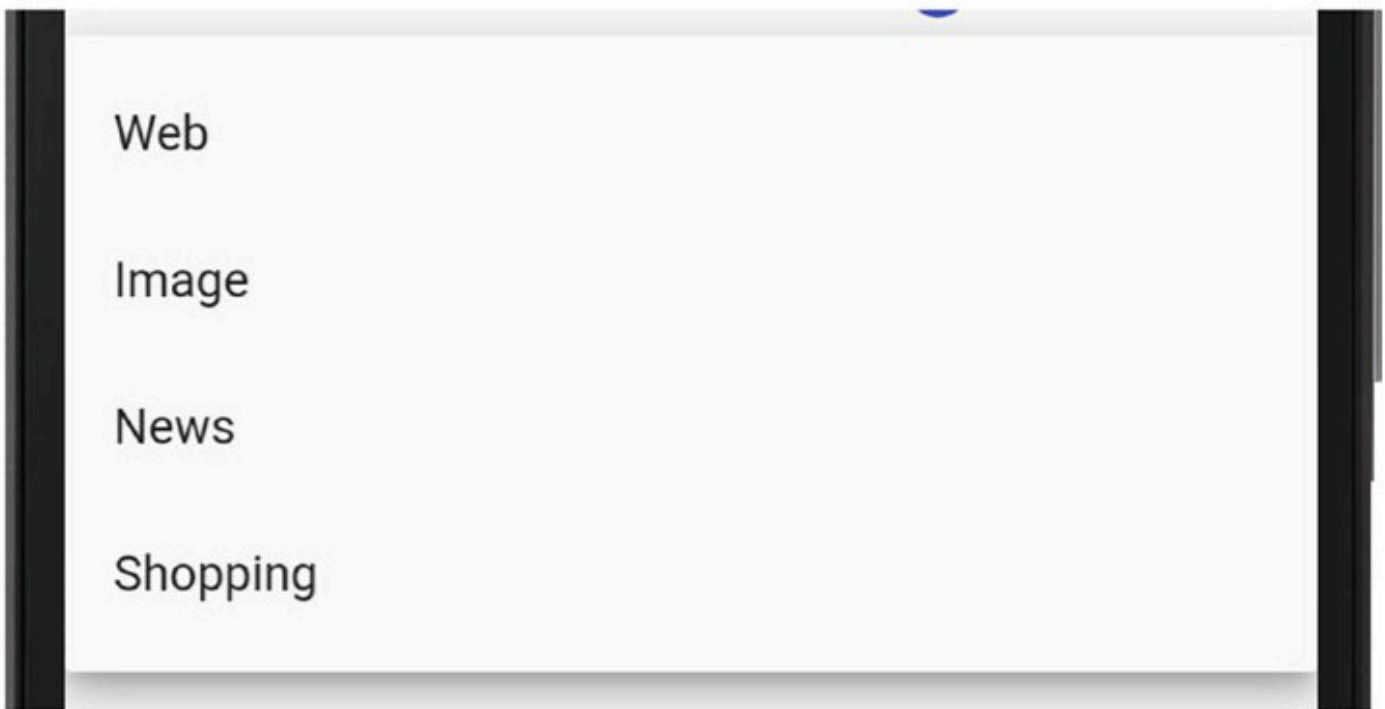
```
enum SearchType { web, image, news, shopping }
```

Rõ ràng ở đây chúng ta định nghĩa `SearchType` là một trong bốn giá trị "web", "image", "news" hoặc "shopping". Nếu muốn người dùng chọn một trong số đó, chúng ta có thể trình bày cho họ một widget `DropDownButton`, ban đầu trông như Hình 4-14.



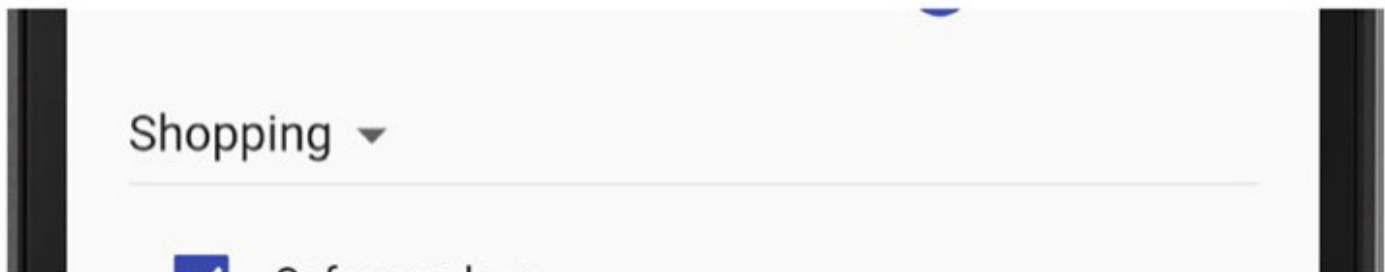
Hình 4-14. *DropdownButton* khi chưa chọn gì

Rồi khi họ chạm vào dropdown, nó trông như Hình 4-15.



Hình 4-15. *DropdownButton* mở ra để hiện các lựa chọn

Và khi họ chạm vào một trong các lựa chọn, nó được chọn (Hình 4-16).



Hình 4-16. *DropdownButton* sau khi đã chọn một mục

Để tạo `DropDownButton` đó, code Flutter của chúng ta có thể trông như sau:

```
SearchType _searchType = SearchType.web;
// Code khác nằm ở đây
DropDownButton<SearchType>(
  value: _searchType,
  items: const <DropDownMenuItem<SearchType>>[
    DropdownMenuItem<SearchType>(
      child: Text('Web'),
      value: SearchType.web,
    ),
    DropdownMenuItem<SearchType>(
      child: Text('Image'),
      value: SearchType.image,
    ),
    DropdownMenuItem<SearchType>(
      child: Text('News'),
      value: SearchType.news,
    ),
    DropdownMenuItem<SearchType>(
      child: Text('Shopping'),
      value: SearchType.shopping,
    ),
  ],
  onChanged: (SearchType val) => _searchType = val,
),
```

Ghép các widget form lại với nhau

Có đủ loại trường nhập liệu trông đẹp và chạy tốt thì hay đấy. Nhưng bạn sẽ thường muốn nhóm chúng lại để có thể điều khiển cả nhóm cùng lúc. Bạn làm điều này bằng widget `Form`.

Widget Form

Giống như trong HTML, bạn hoàn toàn sống ổn mà không cần widget `Form`. Nó là một widget tiện ích và không có thành phần hiển thị nào. Nghĩa là bạn không bao giờ thực sự nhìn thấy nó trên thiết bị. Mục đích

duy nhất của nó là bọc tất cả các input bên trong, qua đó nhóm chúng — và dữ liệu của chúng — thành một khối. Nó làm điều này bằng một **key**.

Nhớ rằng chúng ta đã giới thiệu key ở chương trước và nói rằng ngoài vài trường hợp đặc biệt, bạn có thể bỏ qua key. Đây chính là một trong những chỗ cần key. Nếu bạn quyết định dùng `Form`, bạn cần một `GlobalKey` kiểu `FormState`:

```
GlobalKey<FormState> _key = GlobalKey<FormState>();
```

Bạn đặt key đó làm property cho form:

```
@override
Widget build(BuildContext context) {
  return Form(
    key: _key,
    autovalidate: true,
    child: // Toàn bộ các trường của form sẽ nằm ở đây
  );
}
```

Thoạt nhìn, `Form` có vẻ chẳng thay đổi gì. Nhưng nhìn kỹ hơn sẽ thấy giờ chúng ta có quyền truy cập tới:

- **autovalidate**: kiểu `bool`. `true` nghĩa là chạy validation ngay khi có bất kỳ trường nào thay đổi. `false` nghĩa là bạn sẽ tự chạy thủ công. (Chúng ta sẽ nói về validation ở vài trang tới.)
- **Chính cái key** mà chúng ta đặt tên là `_key` trong ví dụ trên.

Cái `_key` đó có một property `currentState`, và property này lại có những method sau:

1. **save()** — Lưu toàn bộ các trường bên trong form bằng cách gọi `onSaved` của từng trường
2. **validate()** — Chạy hàm `validator` của từng trường
3. **reset()** — Đặt lại từng trường trong form về `initialValue` của nó

 **Ghi chú cập nhật:** Property `autovalidate` **đã bị gỡ bỏ**. Hãy dùng `autovalidateMode` thay thế:

```
Form(  
  key: _key,  
  autovalidateMode: AutovalidateMode.onUserInteraction, // hoặc .always / .disabled  
  child: ...,  
)
```

`AutovalidateMode.onUserInteraction` thường là lựa chọn tốt nhất: nó chỉ bắt đầu báo lỗi sau khi người dùng đã tương tác với trường đó, thay vì hiện lỗi đỏ ngay khi form vừa mở.

Ngoài ra, với null safety, `_key.currentState` có kiểu nullable nên bạn phải viết `_key.currentState!.save()` hoặc kiểm tra null trước.

Với những thứ này trong tay, bạn có thể đoán được cách `Form` nhóm các trường nằm bên trong nó. Khi bạn gọi một trong ba method này trên `FormState`, nó duyệt qua các trường bên trong và gọi method tương ứng trên từng cái. Một lời gọi ở cấp `Form` kích hoạt tất cả.

Nhưng khoan đã! Nếu `_key.currentState.save()` gọi `onSaved()` của một trường, thì chúng ta phải cung cấp method `onSaved`. Tương tự với `validate()` gọi `validator`. Nhưng bản thân các widget `TextField`, `Dropdown`, `Radio`, `Checkbox` và `Slider` lại **không có** những method đó. Giờ làm sao? Chúng ta bọc mỗi trường trong một widget `FormField` — widget này thì có những method đó.

Widget FormField

Mục đích tồn tại duy nhất của widget này là cung cấp các event handler `save`, `reset` và `validator` cho một widget bên trong. Widget `FormField` có thể bọc bất kỳ widget nào thông qua property `builder`:

```
FormField<String>(  
  builder: (FormFieldState<String> state) {  
    return TextField(); // Bất kỳ widget nhập liệu nào: DropdownButton,  
                        // Radio, Checkbox, hay Slider.  
  },  
  onSave: (String initialValue) {  
    // Đẩy giá trị vào repository hoặc làm gì đó ở đây.  
  },  
  validator: (String val) {  
    // Đặt logic validation ở đây (giải thích thêm bên dưới).  
  },  
),
```

Vậy trước tiên chúng ta bọc một widget `FormField` quanh mỗi input widget, và làm việc đó trong một method tên là `builder`. Sau đó chúng ta có thể thêm các method `onSaved` và `validator`.

 Mẹo

Hãy đối xử với `TextField` theo cách khác. Thay vì bọc nó lại, hãy **thay** nó bằng widget `TextFormField` nếu bạn dùng nó bên trong một `Form`. Widget mới này rất dễ nhầm với `TextField` nhưng chúng khác nhau. Về cơ bản...

`TextFormField = TextField + FormField`

Đội ngũ Flutter biết rằng chúng ta sẽ thường xuyên cần một `TextField` kết hợp với một `FormField`, nên họ tạo ra `TextFormField` — có đầy đủ property của `TextField` nhưng bổ sung thêm `onSaved`, `validator` và `reset`:

```
TextFormField(  
  onSaved: (String val) {  
    print('Search Term TextField: form saved $val');  
  },  
  validator: (String val) {  
    // Đặt logic validation của bạn ở đây  
  },  
),
```

Vậy chẳng phải dễ chịu hơn sao? Cuối cùng cũng có một chỗ được làm cho đơn giản hơn. `Checkbox` không có tính năng này. `Radio` cũng không. `Dropdown` cũng không. Chỉ có `TextField` mà thôi.

Best practice: Input văn bản nằm **ngoài** `Form` thì luôn dùng `TextField`. Input văn bản nằm **trong** `Form` thì luôn dùng `TextFormField`.

onSaved

Nhớ rằng `Form` của bạn có một `key`, `key` này có `currentState`, và `currentState` có method `save()`. Nắm hết chưa? Chưa rõ lắm? Hãy thử cách này: khi người dùng bấm nút "Save", bạn sẽ viết code gọi...

```
_key.currentState.save();
```

...và đến lượt nó, nó gọi method `onSaved` của từng `FormField` nào có method đó.

validator

Tương tự, chắc bạn đã đoán ra rằng bạn có thể gọi...

```
_key.currentState.validate();
```

...và Flutter sẽ gọi method `validator` của từng `FormField`. Nhưng còn hơn thế! Nếu bạn đặt property `autovalidate` của `Form` thành `true`, Flutter sẽ validate ngay lập tức khi người dùng thay đổi dữ liệu.

Mỗi hàm `validator` sẽ nhận vào một giá trị — giá trị cần kiểm tra — và trả về một chuỗi. Bạn viết nó sao cho trả về `null` nếu giá trị nhập vào hợp lệ, và trả về một chuỗi thật nếu không hợp lệ. Chuỗi được trả về đó chính là thông báo lỗi mà Flutter sẽ hiển thị cho người dùng.

Validate ngay khi đang gõ

Nhớ rằng cách để validate tức thời là đặt `Form.autovalidate` thành `true` và viết một `validator` cho `TextFormField` của bạn:

```
return Form(  
  autovalidate: true,  
  child: Container(  
    TextFormField(  
      validator: (String val) {  
        // Giả sử giá trị rỗng là không hợp lệ.  
        if (val.isEmpty)  
          return 'We need something to search for';  
        return null;  
      },  
    ),  
  ),  
);
```

Rõ ràng chẳng có ý nghĩa gì khi validate một `DropDownButton`, `Radio`, `Checkbox`, `Switch` hay `Slider` trong lúc gõ, vì bạn có gõ vào chúng đâu. Nhưng có một điểm ít rõ ràng hơn: cách này **không** hoạt động với một `TextField` nằm bên trong một `FormField`. Nó chỉ hoạt động với `TextFormField`. Lạ nhỉ?

 Mẹo

Nhắc lại, best practice là dùng `TextFormField`. Nhưng nếu bạn nhất định muốn dùng `TextField` bên trong `FormField`, bạn có thể tự đặt `errorText` một cách thủ công như sau:

```
FormField<String>(
  builder: (FormFieldState<String> state) {
    return TextField(
      controller: _emailController,
      decoration: InputDecoration(
        // Đoạn này nói rằng: nếu giá trị trông giống email thì đặt
        // errorText thành null. Nếu không, hiển thị thông báo lỗi.
        errorText:
          RegExp(r'^[a-zA-Z0-9.]+@[a-zA-Z0-9]+\.[a-zA-Z]+' )
            .hasMatch(_emailController.text)
            ? null
            : "That's not an email address",
      ),
    );
  },
);
```

Chỉ validate sau khi bấm submit

Có những lúc bạn không muốn code validate cho tới khi người dùng nhập xong dữ liệu. Trước tiên bạn đặt `autovalidate` thành `false`. Sau đó gọi `validate()` trong sự kiện `onPressed` của nút:

```
RaisedButton(  
  child: const Text('Submit'),  
  onPressed: () {  
    // Nếu mọi trường đều qua được validation, chạy method save của chúng.  
    if (_key.currentState.validate()) {  
      _key.currentState.save();  
      print('Successfully saved the state.');    }  
  },  
)
```

 **Ghi chú cập nhật:** `RaisedButton` **đã bị gỡ bỏ**. Hãy dùng `ElevatedButton` — cách dùng gần như y hệt:

```
ElevatedButton(  
  child: const Text('Submit'),  
  onPressed: () {  
    if (_key.currentState!.validate()) {  
      _key.currentState!.save();  
    }  
  },  
)
```

Một ví dụ Form lớn

Tôi biết, tôi biết. Đây là phần khá phức tạp. Có thể sẽ dễ hiểu hơn nếu bạn thấy chúng trong ngữ cảnh — cách tất cả ghép lại với nhau. Dưới đây là một ví dụ đầy đủ kèm chú thích... một ví dụ lớn. Nhưng dù lớn thế này, bản gốc của nó còn lớn hơn nhiều. Hãy xem repository mã nguồn trực tuyến của chúng tôi để có ví dụ đầy đủ. Hy vọng nó giúp bạn hiểu các trường trong `Form` liên hệ với nhau ra sao.

Giả sử chúng ta muốn tạo một màn hình để người dùng gửi một truy vấn tìm kiếm web kiểu Google. Chúng ta sẽ cho họ một `TextFormField` để nhập chuỗi tìm kiếm, một `DropDownButton` để chọn loại tìm kiếm, một checkbox để bật/tắt `safeSearch`, và một nút để gửi đi:

```
enum SearchType { web, image, news, shopping }

// Đây là một stateful widget. Đừng lo về cách nó hoạt động hay cách
// setState() hoạt động cho tới Chương 9.
// Bây giờ, chỉ cần tập trung vào bản thân cái Form.
class ProperForm extends StatefulWidget {
  @override
  _ProperFormState createState() => _ProperFormState();
}

class _ProperFormState extends State<ProperForm> {
  // Một Map (còn gọi là hash) để giữ dữ liệu từ Form.
  final Map<String, dynamic> _searchForm = <String, dynamic>{
    'searchTerm': '',
    'searchType': SearchType.web,
    'safeSearchOn': true,
  };

  // Cái key của Flutter để trỏ tới Form
  final GlobalKey<FormState> _key = GlobalKey();

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _key,
      // Đặt autovalidate là true để validate sau mỗi lần gõ phím.
      // Trong trường hợp này ta chỉ muốn validate khi submit.
      //autovalidate: true,
      child: Container(
        child: ListView(
          children: <Widget>[
            TextFormField(
              initialValue: _searchForm['searchTerm'],
              decoration: InputDecoration(
                labelText: 'Search terms',
```

```

    ),
    // Sau mỗi lần gõ phím, bạn có thể làm gì đó.
    onChanged: (String val) {
      setState(() => _searchForm['searchTerm'] = val);
    },
    // Khi người dùng submit, bạn có thể làm gì đó
    // cho trường này
    onSave: (String val) {},
    // Được gọi khi ta gọi "validate()". val là chuỗi
    // trong ô nhập liệu.
    // Lưu ý nó trả về một String; null nếu validation qua
    // và một thông báo lỗi nếu vì lý do nào đó không qua.
    validator: (String val) {
      if (val.isEmpty) {
        return 'We need something to search for';
      }
      return null;
    },
  ),

  FormField<SearchType>(
    builder: (FormFieldState<SearchType> state) {
      return DropdownButton<SearchType>(
        value: _searchForm['searchType'],
        items: const <DropdownMenuItem<SearchType>>[
          DropdownMenuItem<SearchType>(
            child: Text('Web'),
            value: SearchType.web,
          ),
          DropdownMenuItem<SearchType>(
            child: Text('Image'),
            value: SearchType.image,
          ),
          DropdownMenuItem<SearchType>(
            child: Text('News'),

```

```

        value: SearchType.news,
      ),
      DropdownMenuItem<SearchType>(
        child: Text('Shopping'),
        value: SearchType.shopping,
      ),
    ],
    onChanged: (SearchType val) {
      setState(() => _searchForm['searchType'] = val);
    },
  );
},
onSaved: (SearchType initialValue) {},
),

// Bọc Checkbox trong một FormField để có được
// onSaved và validator
FormField<bool>(
  //initialValue: false, // Bị bỏ qua với Checkbox
  builder: (FormFieldState<bool> state) {
    return Row(
      children: <Widget>[
        Checkbox(
          value: _searchForm['safeSearchOn'],
          // Mỗi lần nó thay đổi, bạn có thể làm gì đó.
          onChanged: (bool val) {
            setState(() => _searchForm['safeSearchOn'] = val);
          },
        ),
        const Text('Safesearch on'),
      ],
    );
  },
  // Khi người dùng lưu, đoạn này được chạy
  onSaved: (bool initialValue) {},

```

```
// Không cần validation vì đây là checkbox. Nhưng
// nếu muốn, hãy đặt một hàm validator ở đây.
),

// Đây là nút 'Submit'
RaisedButton(
  child: const Text('Submit'),
  onPressed: () {
    // Nếu mọi trường qua được validation thì cho đi tiếp.
    // Nhớ rằng lệnh này gọi validator trên tất cả các
    // trường trong form.
    if (_key.currentState.validate()) {
      // Tương tự, lệnh này gọi onSave() cho tất cả các trường
      _key.currentState.save();
      // Bạn sẽ lưu dữ liệu vào database hay gì đó ở đây
      print('Successfully saved the state.');
```

 **Ghi chú cập nhật:** Để chạy được trên Flutter hiện nay, ví dụ trên cần các thay đổi sau:

- `RaisedButton` → `ElevatedButton`
- `autovalidate: true` → `autovalidateMode: AutovalidateMode.always`
- `_key.currentState.validate()` → `_key.currentState!.validate()`
- `validator: (String val) → validator: (String? val)` và kiểm tra `val == null || val.isEmpty`
- `onChanged: (bool val)` của `Checkbox` → `onChanged: (bool? val)`
- `onChanged: (SearchType val)` của `DropDownButton` → `onChanged: (SearchType? val)`

Tổng kết

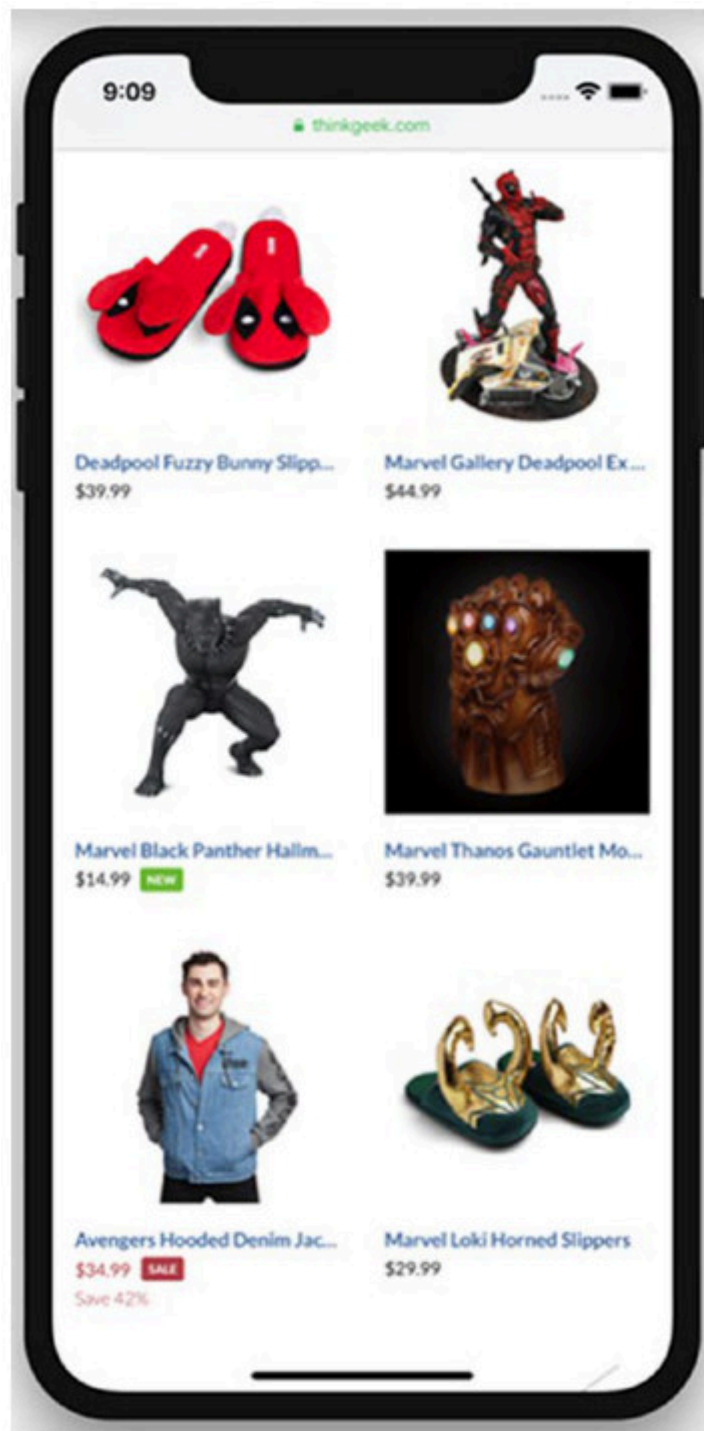
Phải mất một thời gian mới hiểu được form trong Flutter. Đừng nản lòng. Hãy xem lại ví dụ trên vài lần nữa và viết thử một ít code. Mọi thứ sẽ sáng tỏ rất nhanh. Và tuy chủ đề Form có thể hơi đáng ngại, thì `Image`, `Icon` và `Text` lại rất dễ, đúng không?

Ở chương tiếp theo, chúng ta sẽ bắt đầu thấy app của mình sống động lên, vì chúng ta sẽ học cách tạo đủ loại nút bấm và làm cho chúng — hay bất kỳ widget nào khác — phản hồi lại cú chạm và các cử chỉ khác!

Chương 5: Phản hồi cử chỉ người dùng

Chúng ta đã tiến khá xa rồi! Giờ bạn đã biết Flutter là gì. Bạn nắm được quy trình phát triển và debug hoạt động ra sao. Bạn hiểu vì sao chúng ta dùng widget và đã khá quen thuộc với các value widget từ chương trước. Thậm chí bạn còn tự tạo được stateless widget của riêng mình. Nhưng chúng ta vẫn còn thiếu một tính năng nền tảng quan trọng: **xử lý sự kiện**.

Giả sử bạn có một màn hình nơi người dùng chọn sản phẩm và bỏ vào giỏ hàng. Họ sẽ phải cuộn lên xuống qua một danh sách sản phẩm (Hình 5-1). Việc vuốt lên xuống để cuộn là một **cử chỉ** (gesture). Để chọn một sản phẩm, họ sẽ chạm vào nó. Đó cũng là một gesture. Rồi để bỏ vào giỏ, có thể chúng ta cho họ vuốt sang phải. Đó lại là một gesture khác.



Hình 5-1. Một app mua sắm

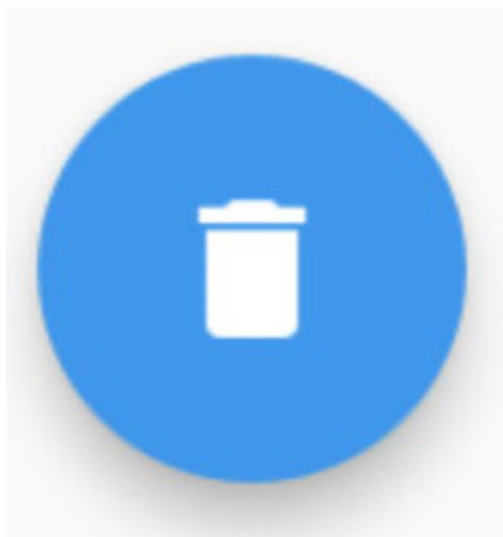
Chương này bàn về việc xử lý những gesture đó. Chúng ta sẽ chia gesture thành hai nhóm: gesture trên widget dựng sẵn và gesture trên widget tùy chỉnh của bạn. Bắt đầu với gesture trên widget dựng sẵn.

Làm quen với họ nhà button

Một số gesture cực kỳ dễ vì chúng đã được tích hợp sẵn vào một số widget nhất định. Ví dụ, người tạo ra các widget button biết rằng mục đích sống duy nhất của chúng là để được bấm và làm gì đó sau khi bấm. Vì vậy mọi button đều có một property tên là `onPressed`. Để dùng nó, bạn chỉ cần trỏ nó tới một hàm sẽ chạy khi người dùng bấm:

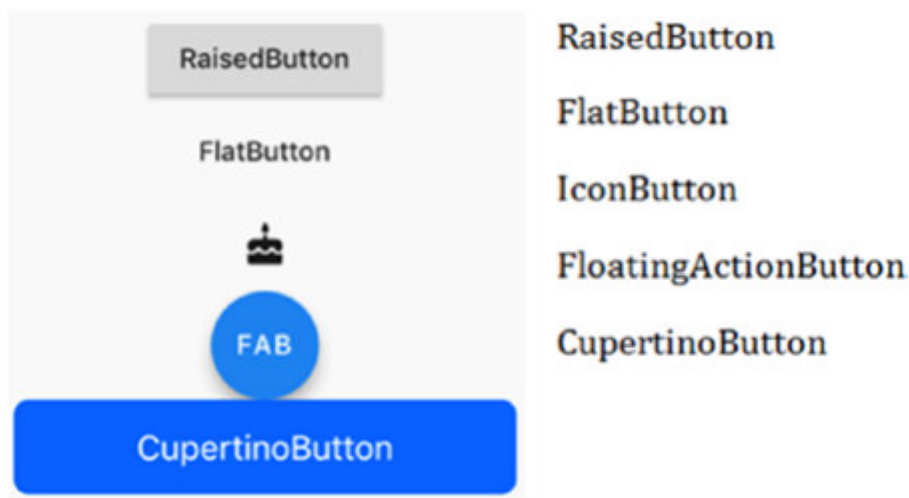
```
Product _product;
// Code khác ở đây
Widget foo() {
  return IconButton(
    icon: Icon(Icons.delete),
    onPressed: removeProduct // Callback phải trả về void
  );
}
// Code khác ở đây
void removeProduct() {
  // Làm gì đó để xóa _product
}
```

Hình 5-2 cho thấy kết quả.



Hình 5-2. Một *IconButton*

Bạn có thể coi `Button` như class cơ sở cho tất cả các button khác. Thực ra không phải vậy, nhưng nghĩ về những cái còn lại như một `Button` có thêm đặc điểm riêng cũng không hại gì. Ví dụ, tất cả những widget sau đều là các dạng button chuyên biệt (Hình 5-3).



Hình 5-3. Họ nhà button

Ghi chú cập nhật quan trọng cho cả chương này: Ba widget button mà sách gọi tên — `RaisedButton`, `FlatButton` và `OutlineButton` — **đã bị gỡ bỏ hoàn toàn khỏi Flutter**. Code trong chương này dùng chúng sẽ **không biên dịch được**. Bảng thay thế:

Widget trong sách (2019)	Widget hiện nay
<code>RaisedButton</code>	<code>ElevatedButton</code>
<code>FlatButton</code>	<code>TextButton</code>
<code>OutlineButton</code>	<code>OutlinedButton</code>
<code>IconButton</code>	<code>IconButton</code> (không đổi)
<code>FloatingActionButton</code>	<code>FloatingActionButton</code> (không đổi)
<code>CupertinoButton</code>	<code>CupertinoButton</code> (không đổi)

Cách tạo kiểu cũng thay đổi: thay vì các property rời rạc như `color`, `textColor`, `elevation` đặt thẳng trên button, các button mới nhận một object `style` thông qua `ElevatedButton.styleFrom(...)` hoặc `ButtonStyle(...)`.

RaisedButton

Widget này đơn giản là một `Button` nhưng trông như đang nổi lên trên trang. Nó có property `elevation` để tăng độ cao mô phỏng so với mặt phẳng trang (Hình 5-4):

```
RaisedButton(  
  child: Text(text: "Go"),  
  onPressed: () => print("You swiped the raised button"),  
  elevation: 5.0  
)
```



Hình 5-4. Các mức *elevation*

Cách viết hiện nay:

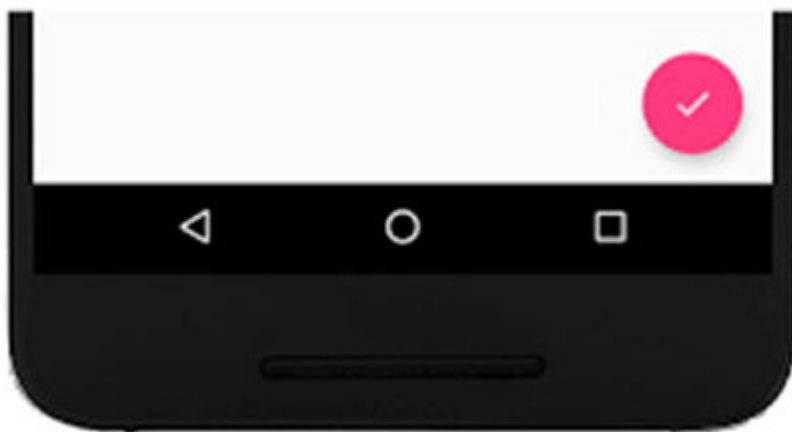
```
ElevatedButton(  
  child: const Text("Go"),  
  onPressed: () => print("You pressed the elevated button"),  
  style: ElevatedButton.styleFrom(elevation: 5.0),  
)
```

FlatButton và IconButton

Đây là kiểu ngược lại với `RaisedButton`. Chúng trông hoàn toàn phẳng. Chúng kín đáo, chỉ có chữ đơn giản hoặc một icon không hề "gào lên" đòi được bấm — kiểu như nút UNDO hay nút BACK.

FloatingActionButton

Đây là nút bạn hay thấy ở góc dưới bên phải màn hình. Nó thường có dạng tròn và là gợi ý không thể nhầm lẫn cho người dùng về cách đi tiếp bước tiếp theo trong quy trình (Hình 5-5).



Hình 5-5. *Floating action button*

Trong Flutter, FAB là một trong ba thành phần chính của một `Scaffold`. Bạn thường thấy nó được đưa vào kiểu như thế này:

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: Text(_title)),  
    body: OtherWidgetsHere(),  
    floatingActionButton: FloatingActionButton(  
      child: Icon(Icons.check),  
      onPressed: () => {},  
    ),  
  );  
}
```

CupertinoButton

Một nút theo phong cách iOS. Trông rất đẹp trên iPhone, nhưng hơi kỳ khi có cảm giác iOS trên một thiết bị Android. Nếu dùng nó, bạn phải nhớ thêm dòng này vào đầu file Dart:

```
import 'package:flutter/cupertino.dart';
```

Dismissible

Các button đều được tạo ra cho một mục đích: phản hồi một cú chạm. Tương tự, `Dismissible` được tạo ra cho một mục đích: phản hồi một cú vuốt. Để dùng nó, thường bạn sẽ dựng một danh sách widget và bọc mỗi cái bằng một `Dismissible`. Khi đó, mỗi widget trong danh sách đều có thể phản hồi cử chỉ vuốt:

```
Dismissible(  
  // Nền màu xanh nếu vuốt sang phải và  
  // nền màu đỏ nếu vuốt sang trái  
  background: Container(color: Colors.blue),  
  secondaryBackground: Container(color: Colors.red),  
  onDismissed: (direction) {  
    print("You swiped $direction");  
  },  
  child: SomeWidget(),  
);
```

Lưu ý rằng đúng như tên gọi, widget này dùng để **loại bỏ** một widget, gỡ nó khỏi màn hình khi bạn "vuốt nó đi". Nhưng nếu tôi muốn vuốt mà không loại bỏ thì sao? Lúc đó cần đến gesture tùy chỉnh.

✦ **Lưu ý:** `Dismissible` bắt buộc phải có một `key` duy nhất cho mỗi phần tử, nếu không Flutter sẽ báo lỗi khi phần tử bị xóa. Ví dụ `key: ValueKey(person['id'])`.

Gesture tùy chỉnh cho widget tùy chỉnh của bạn

Vì sao `Dismissible` hiểu được cử chỉ vuốt? Vì sao các button hiểu được gesture `onPressed`? Bởi vì developer đã lập trình sẵn cho chúng. Widget tùy chỉnh của bạn cũng cần được lập trình gesture. Nhưng vì bạn là người viết chúng, bạn được tự tạo gesture của riêng mình. Và bạn có thể tạo ra những gesture thú vị hơn nhiều so với một cú bấm đơn giản. Bạn có thể cho widget phản hồi cú vuốt, nhấn giữ, chạm đúp và chạm-để-phóng-to.

Gesture	Mô tả
Tap	còn gọi là press (bấm). Bao gồm cả chạm đúp (tap-tap)
LongPress	Nhấn giữ trên màn hình lâu hơn — kiểu một hai giây
Scale	còn gọi là chạm hoặc mở ngón tay ra
Drag	còn gọi là vuốt (swipe)

Lưu ý: Còn có `Pan`, đủ giống `Drag` đến mức chúng tôi bỏ qua cho đơn giản.

Để phản hồi gesture tùy chỉnh, bạn cần các bước sau:

1. Quyết định gesture và hành vi tương ứng.
2. Tạo widget tùy chỉnh như bình thường.
3. Thêm một widget `GestureDetector`.
4. Gắn gesture với hành vi của nó.

Bước 1: Quyết định gesture và hành vi

Bước này đơn giản. Chuyên gia UX của bạn có thể đã làm xong việc này trước khi bản thiết kế tới tay bạn. Bạn chỉ cần liệt kê những gesture muốn phản hồi và việc chúng nên làm khi gesture đó được phát hiện.

Chúng ta sẽ làm qua một ví dụ. Giả sử người dùng nhìn thấy một danh sách người và phải chọn ra những người họ thích và những người họ không thích. Hãy để người dùng vuốt sang phải với những người họ thích và vuốt sang trái với những người họ không thích. Và giả sử thỉnh thoảng người dùng muốn chen thêm một người mới vào giữa hai người khác. Chúng ta sẽ cho họ tách hai người đó ra bằng hai ngón tay — kiểu như tạo khoảng trống giữa họ cho mục mới. Cuối cùng, có lẽ chúng ta cho người dùng nhấn giữ để xóa một người.

Gesture	Hành động
Vuốt sang phải	Thêm vào danh sách "ngoa"
Vuốt sang trái	Thêm vào danh sách "hư"
Chạm ngón tay (thực ra là mở ngón tay ra)	Chèn một người mới
Nhấn giữ	Xóa người đó

Bước 2: Tạo widget tùy chỉnh của bạn

Viết code Dart như đã học ở các chương trước. Đây là danh sách người:

```
class ManagePeople extends StatelessWidget {  
  List<Map> fetchPeople() {  
    return [  
      {"first": "Jim", "last": "Halpert"},  
      {"first": "Kelly", "last": "Kapoor"},  
      {"first": "Creed", "last": "Bratton"},  
      {"first": "Dwight", "last": "Schrute"},  
      {"first": "Andy", "last": "Bernard"},  
      {"first": "Pam", "last": "Beasley"},  
      {"first": "Jim", "last": "Halpert"},  
      {"first": "Robert", "last": "California"},  
      {"first": "David", "last": "Wallace"},  
      {"first": "Ryan", "last": "Howard"},  
    ];  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    var _peopleObjects = fetchPeople();  
    return ListView(  
      children: _peopleObjects.map((person) =>  
        Person(person: person)).toList(),  
    );  
  }  
}
```

Bước 3: Thêm widget GestureDetector

Widget `GestureDetector` khác với hầu hết widget UX — bạn không nhìn thấy nó. Bạn hoặc bọc một `GestureDetector` quanh widget nào đó, hoặc lồng widget đó vào property `child`; nó rất linh hoạt. Dù cách nào, nó cũng phát hiện và xử lý gesture cho widget kia. Vì bạn không nhìn thấy nó nên nó không bị phình to bởi property nào ngoài `child`, cũng không có method nào ngoài `build` — đúng như bạn mong đợi. Chính các sự kiện mới là nơi mọi thứ diễn ra!

```
return ListView(
  children: _peopleObjects
    .map((person) =>
      GestureDetector(child: Person(person: person))
    ).toList(),
);
```

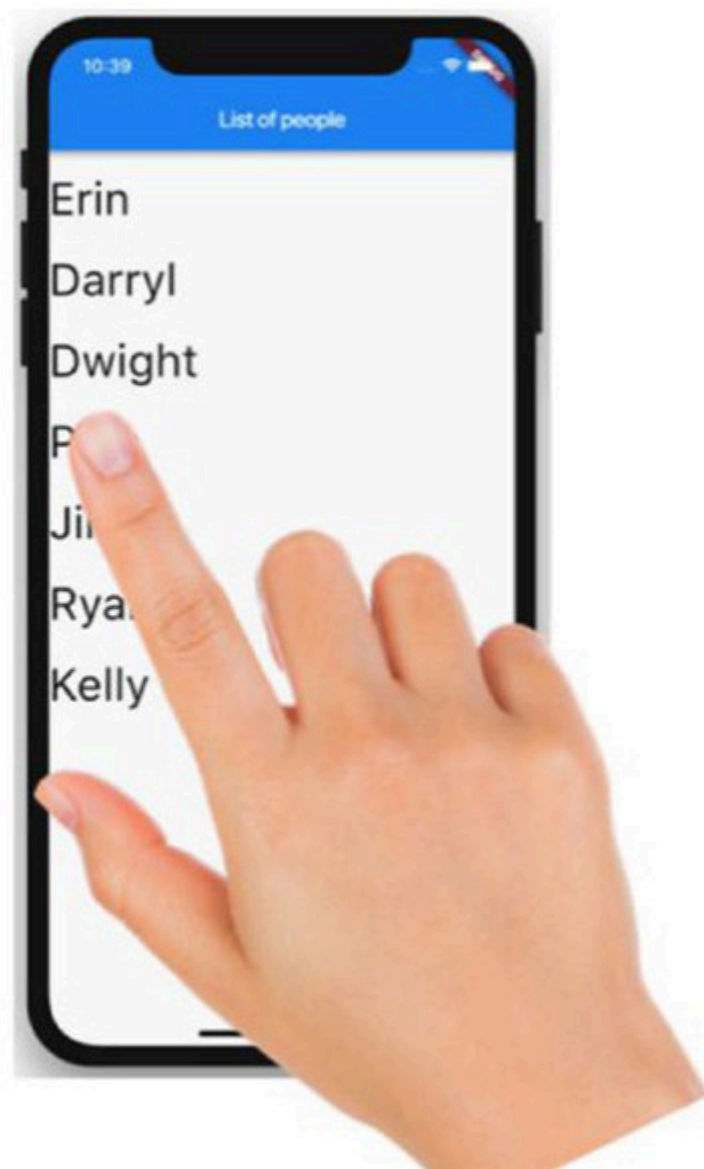
Bước 4: Gắn gesture với hành vi

Bước cuối cùng: với mỗi sự kiện bạn đã thiết kế ở bước 1, hãy gán một method. `GestureDetector` hỗ trợ rất nhiều sự kiện¹ nên chúng dễ gây rối. Chúng tôi đã chốt lọc lại những cái hữu ích nhất ở đây.

Gesture	Sự kiện cần dùng
Tap (bấm)	<code>onTap</code>
Chạm đúp	<code>onDoubleTap</code>
Nhấn giữ	<code>onLongPress</code>
Vuốt ngang	<code>onHorizontalDragUpdate</code> , <code>Start</code> , <code>End</code>
Vuốt dọc	<code>onVerticalDragUpdate</code> , <code>Start</code> , <code>End</code>
Vuốt chéo	<code>onPanUpdate</code> , <code>Start</code> , <code>End</code>
Chạm ngón tay	<code>onScaleUpdate</code> , <code>Start</code> , <code>End</code>

Ví dụ 1: Phản hồi cú nhấn giữ

Nhấn giữ (Hình 5-6) sẽ bỏ qua những cú chạm thông thường nhưng được kích hoạt khi người dùng nhấn trong thời gian dài — kiểu một hai giây. Giả sử đội UX của chúng ta quyết định rằng nhấn giữ báo hiệu người dùng muốn xóa một người.



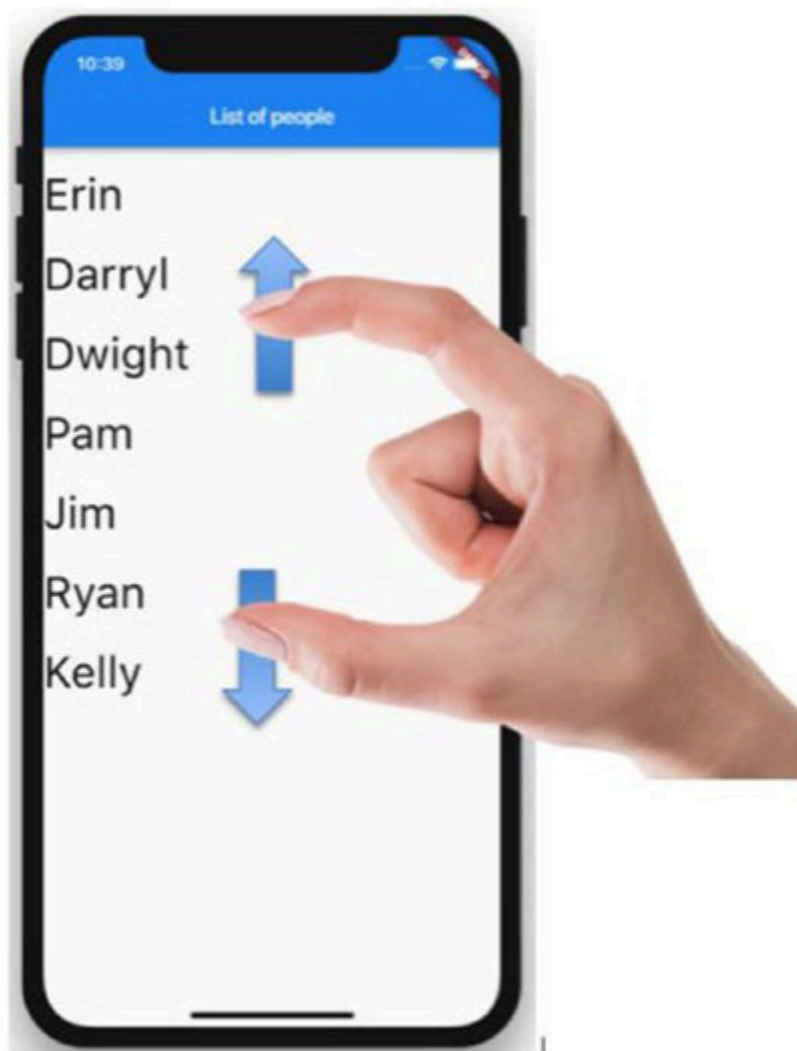
Hình 5-6. Nhấn giữ

Để làm được điều này, chúng ta thêm event handler `onLongPress` :

```
GestureDetector(  
  child: Person(person: person),  
  onLongPress: () {  
    _people.remove(person);  
    print("Deleted ${person['first']}");  
  },  
);
```

Ví dụ 2: Chạm ngón tay để thêm mục mới

Giả sử chuyên gia UX gợi ý rằng người dùng sẽ muốn thêm mục vào danh sách và chỉ định vị trí chèn. Để diễn đạt điều đó, họ sẽ "mở" danh sách ra bằng cách tách hai mục ra bằng hai ngón tay (Hình 5-7).



Hình 5-7. Chạm ngón tay

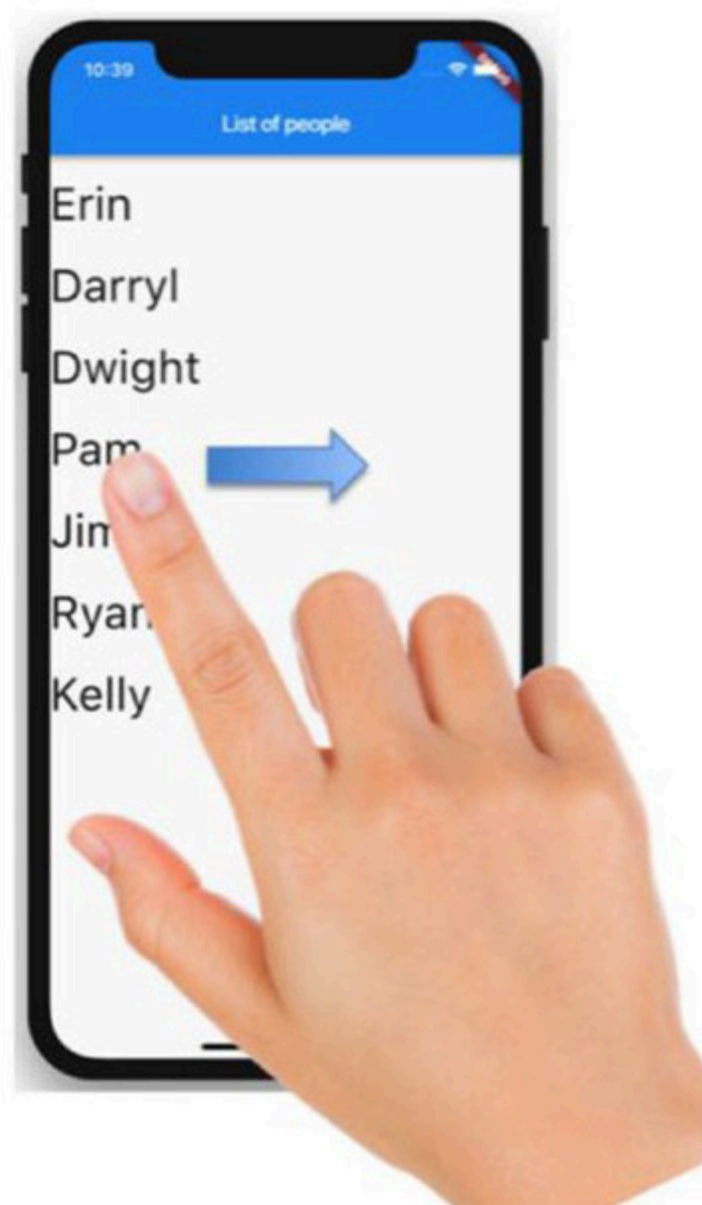
Chúng ta muốn phát hiện xem người dùng đang chạm vào hay mở ra. Chạm vào bình thường thì nên bỏ qua. Nhưng mở ra — khi họ tách hai ngón tay — nghĩa là chúng ta sẽ thêm một người mới.

Lưu ý rằng một số event handler nhận vào một object sự kiện. Object này chứa thông tin về sự kiện cụ thể đó. Trong trường hợp scale/pinch, nó có property tên là `scale`. Nếu `scale` lớn hơn `1.0`, đó là một cú mở ra. Giả sử nếu người dùng mở ra gấp đôi tỉ lệ bình thường, chúng ta sẽ hiểu là họ muốn thêm một người mới vào danh sách:

```
onScaleUpdate: (e) {  
  if (e.scale > 2.0)  
    addPerson(context);  
},
```

Ví dụ 3: Vuốt sang trái hoặc phải

Giờ đội UX quyết định rằng nếu người dùng vuốt sang phải trên một người trong danh sách, chúng ta thêm họ vào danh sách "ngoan", còn nếu vuốt sang trái thì thêm vào danh sách "hư" (Hình 5-8).



Hình 5-8. Vuốt

Để phát hiện cú vuốt, chúng ta tìm tới drag hoặc pan. Pan là cần thiết khi ta mong đợi người dùng có thể vuốt chéo. `HorizontalDrag` chỉ dành cho trái và phải; nó bỏ qua hướng Y. `VerticalDrag` chỉ dành cho lên và xuống; nó bỏ qua mọi thay đổi theo hướng X. Vì chúng ta chỉ quan tâm tới vuốt trái hoặc vuốt phải, ta sẽ tập trung vào gesture `HorizontalDrag`.

App của chúng ta có thể phản hồi bất kỳ cú vuốt nào bằng sự kiện `onHorizontalDragEnd`. Trong trường hợp này chúng ta còn quan tâm tới **hướng** vuốt: từ trái sang phải hay từ phải sang trái? Nên ta phải xem object sự kiện trong từng trường hợp. Khi bắt đầu kéo, ta lưu lại vị trí X nơi ngón tay người dùng đang đặt. Sau đó với mỗi pixel di chuyển, sự kiện drag update ghi lại vị trí X hiện tại. Cuối cùng khi kết thúc kéo, ta

làm một phép tính đơn giản: nếu vị trí X cuối lớn hơn, ta biết đó là vuốt sang phải. Ngược lại là vuốt sang trái:

```
double _swipeStartX;
String _swipeDirection;
return GestureDetector(
  child: Person(person: person),
  onHorizontalDragStart: (e) {
    _swipeStartX = e.globalPosition.dx;
  },
  onHorizontalDragUpdate: (e) {
    _swipeDirection =
      (e.globalPosition.dx > _swipeStartX) ? "Right" : "Left";
  },
  onHorizontalDragEnd: (e) {
    if (_swipeDirection == "Right")
      updatePerson(person, status: "nice");
    else
      updatePerson(person, status: "naughty");
  },
);
```

Nếu có từ hai gesture xảy ra cùng lúc thì sao?

Lấy một ví dụ. Người dùng đang chạm-để-phóng-to. Họ đang chạm vào màn hình, nên đó là Tap. Nhưng họ cũng chạm trong thời gian dài, nên đó là LongPress. Nhưng mỗi điểm chạm lại đang di chuyển, nên đó là Pan. Nhưng có tận hai điểm chạm, nên đó là Scale. Vậy chúng ta phản hồi cái nào?

`GestureDetector` đưa tất cả những cái này vào một **GestureArena** (đấu trường gesture). Tôi không bịa ra đâu! Gesture nào trụ lại cuối cùng trong đấu trường thì thắng. Nếu không có listener nào cho một gesture, nó bị loại khỏi đấu trường. Ở bất kỳ thời điểm nào, một gesture đều có thể tự rút lui. Vậy nên trong trường hợp chạm-để-phóng-to, vì có hai điểm chạm và chúng đang di chuyển, mọi gesture khác đều rút lui và rời đấu trường. Cái trụ lại cuối cùng là gesture Scale. Vậy nó là scale.

Một ví dụ khác: giả sử bạn định nghĩa cả handler `onVerticalDragEnd` lẫn `onHorizontalDragEnd`. Khi người dùng vuốt xong theo hướng bất kỳ, số pixel được đếm lại, và nếu họ di chuyển theo chiều ngang nhiều

pixel hơn chiều dọc thì `onVerticalDragEnd` rời khỏi `GestureArena` và handler `onHorizontalDragEnd` được chạy.

Tổng kết

Điểm mấu chốt là gesture trong Flutter rất trực quan. Chúng hoạt động đúng như một developer bình thường mong đợi, khiến việc code của chúng ta dễ dàng và việc sử dụng của người dùng cũng dễ dàng. Khi được kích hoạt, mọi sự kiện đều chạy trên một luồng riêng nên hoàn toàn ổn nếu chúng trả về một object `Async<>`. Vì vậy, cứ thoải mái đánh dấu các hàm xử lý sự kiện là `async` và dùng thật nhiều `await` bên trong.²

-
1. Ngoài các sự kiện đã liệt kê, nhiều gesture còn có các sự kiện nâng cao cho `Start`, `End`, `Cancel`, `Up` và/hoặc `Down`.
Khá nhiều thứ để tiếp thu nhưng có thể hữu ích, nên hãy vào đây để đọc về tất cả: <http://bit.ly/FlutterGestures> ↩
 2. Để biết thêm về `Future`, `async` và `await`, hãy xem Chương 10: "App Flutter làm việc với file". ↩

Chương 6: Sắp xếp bố cục cho Widget

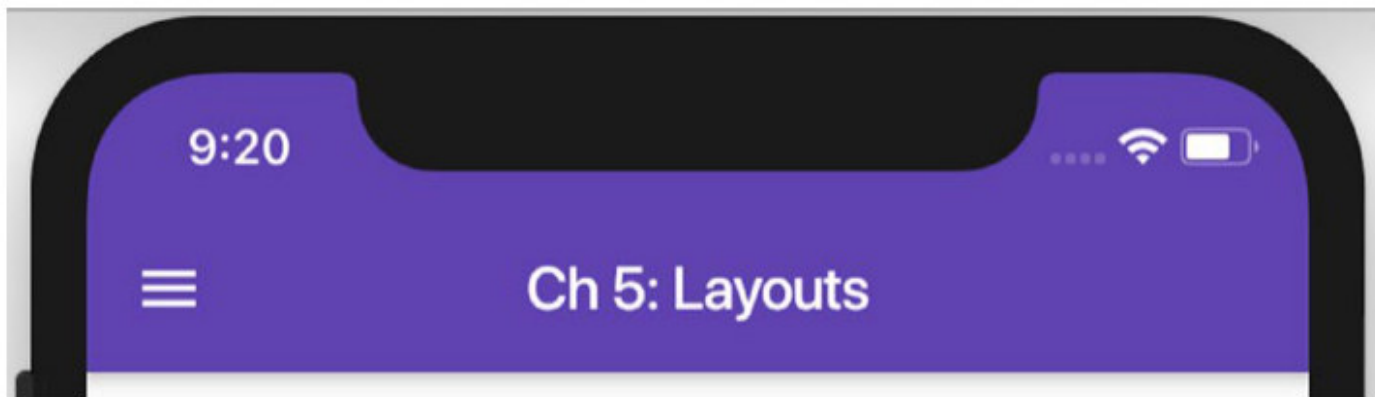
Giờ khi đã quen với một số widget chứa giá trị và cách làm cho chúng phản hồi cử chỉ, chúng ta đã sẵn sàng sắp xếp chúng cho đúng chỗ. Trong chương này, chúng ta sẽ xử lý những kỹ thuật chính để đưa widget lên màn hình theo các quan hệ vị trí khác nhau và quản lý khoảng cách giữa chúng.

Đề ý là tôi nói "những kỹ thuật chính" chứ không phải "tất cả kỹ thuật". Lý do là Flutter có tới hàng chục widget để bố trí, nhiều cái chồng lẫn chức năng với nhau. Điều này rất tuyệt nếu bạn thích có nhiều lựa chọn, nhưng càng nhiều lựa chọn thì chủ đề càng phức tạp.¹ Vì vậy, để bạn khỏi rối, chúng tôi sẽ không trình bày 100% widget hay tùy chọn. Thay vào đó chúng ta tập trung vào những cái đủ để giải quyết công việc thực tế mà không làm bạn quá tải. Chúng tôi gợi ý bạn học các kỹ thuật trong chương này để có được 90% những gì bạn sẽ cần cho việc bố trí. Sau đó, khi gặp tình huống không giải quyết được bằng những kỹ thuật này, bạn có thể tự tìm hiểu thêm hoặc nhờ trợ giúp.

Để đạt được mục tiêu, chúng ta thực sự cần biết làm năm việc:

1. Bố trí toàn bộ màn hình (còn gọi là scene)

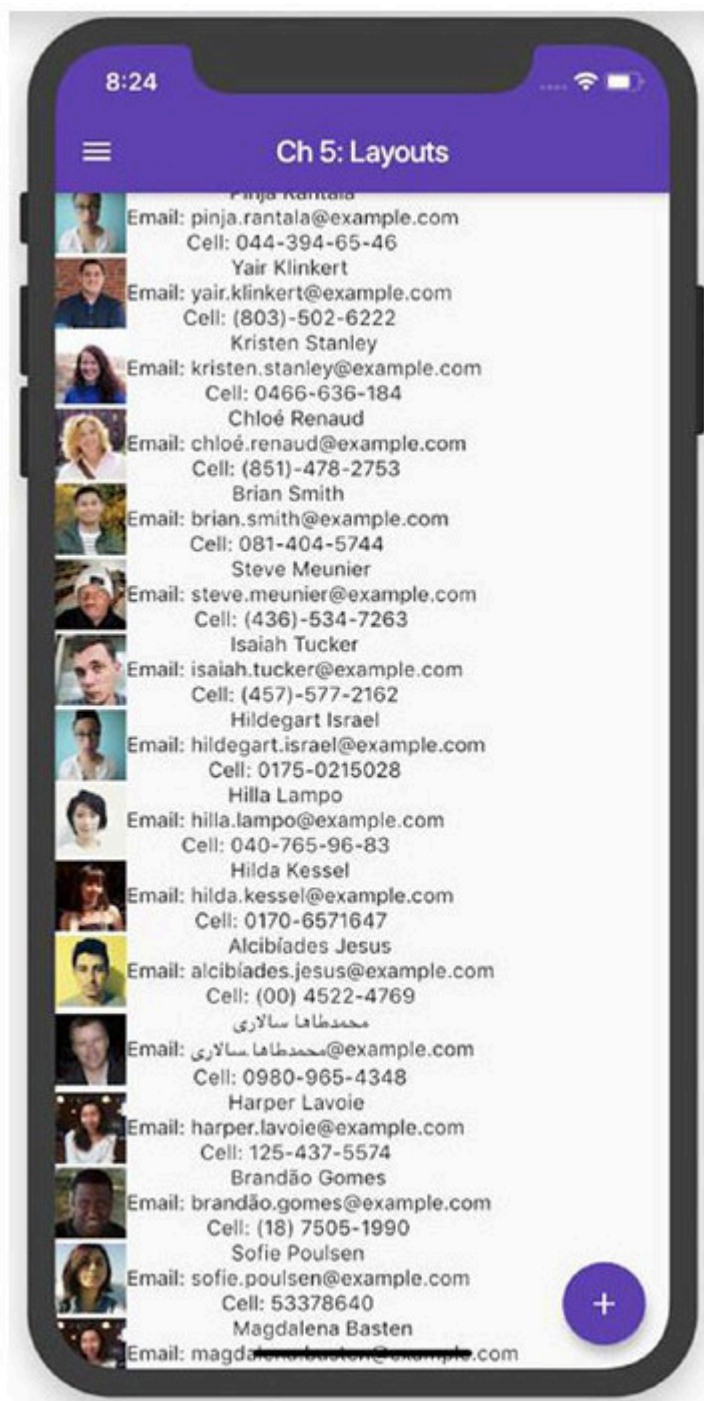
Đây là nơi chúng ta thiết lập diện mạo tổng thể cho app và tạo cấu trúc của scene như tiêu đề, nút hành động và menu (Hình 6-1).



Hình 6-1. Tiêu đề và menu xuất hiện ở phía trên cùng với những thứ khác như nút hành động

2. Đặt widget trên dưới nhau hoặc cạnh nhau

Khi thiết kế bất kỳ scene nào, chúng ta chia nó thành các widget và đặt chúng lên màn hình. Ví dụ, scene sau (Hình 6-2) phải được chia thành các widget. Vì đó là một danh sách người có thể cuộn, có lẽ chúng ta muốn một loạt widget `PersonCard` (Hình 6-3) trên scene, mỗi cái nằm trên dưới cái khác. Chúng ta làm điều đó bằng `ListView`.



Hình 6-2. Một `ListView` có thể đặt các widget trên dưới nhau



Hình 6-3. Chúng ta có thể tạo một widget `PersonCard`

Rồi đến lượt mình, mỗi widget `PersonCard` nên có một ảnh nằm cạnh phần chữ (Hình 6-4). Làm sao đặt chữ bên cạnh ảnh? Chúng ta dùng widget `Row`. Cũng để ý rằng phần chữ là một loạt dữ liệu về người đó. Làm sao đặt chữ trên dưới nhau? Chúng ta dùng widget `Column`.



Hình 6-4. Widget `Row` và `Column` có thể dùng để sắp xếp các thành phần

3. Xử lý khoảng trống thừa trong scene

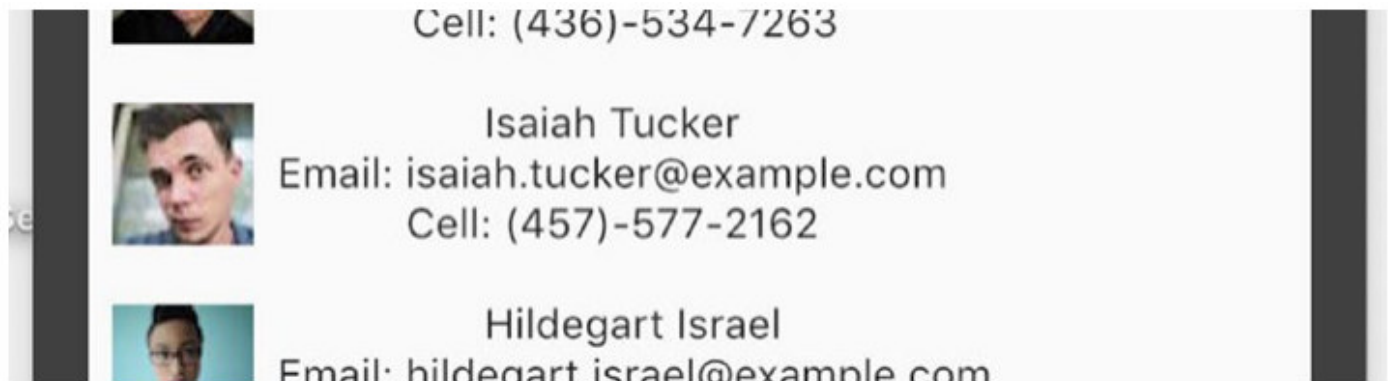
Này, có khoảng trống thừa ở bên phải mỗi `Person`. Nếu chúng ta muốn khoảng trống đó nằm bên trái thì sao? Hay muốn đặt một phần khoảng trống đó vào bên trái ảnh?

4. Xử lý tình huống hết chỗ và tràn ra khỏi scene

Trong scene chứa toàn bộ `PersonCard`, chúng ta có nhiều người hơn diện tích màn hình nên đã bị tràn. Thông thường điều này ném ra lỗi, nhưng có vài cách khắc phục. Chúng ta sẽ xem cách tốt nhất.

5. Tinh chỉnh vị trí chi tiết hơn

Scene hiện tại có cảm giác chật chội. Chúng ta có thể làm gì để tạo thêm chút không gian thở (Hình 6-5)? Hãy làm cho nó giống như trong hình:



Hình 6-5. Khoảng cách đã được tinh chỉnh

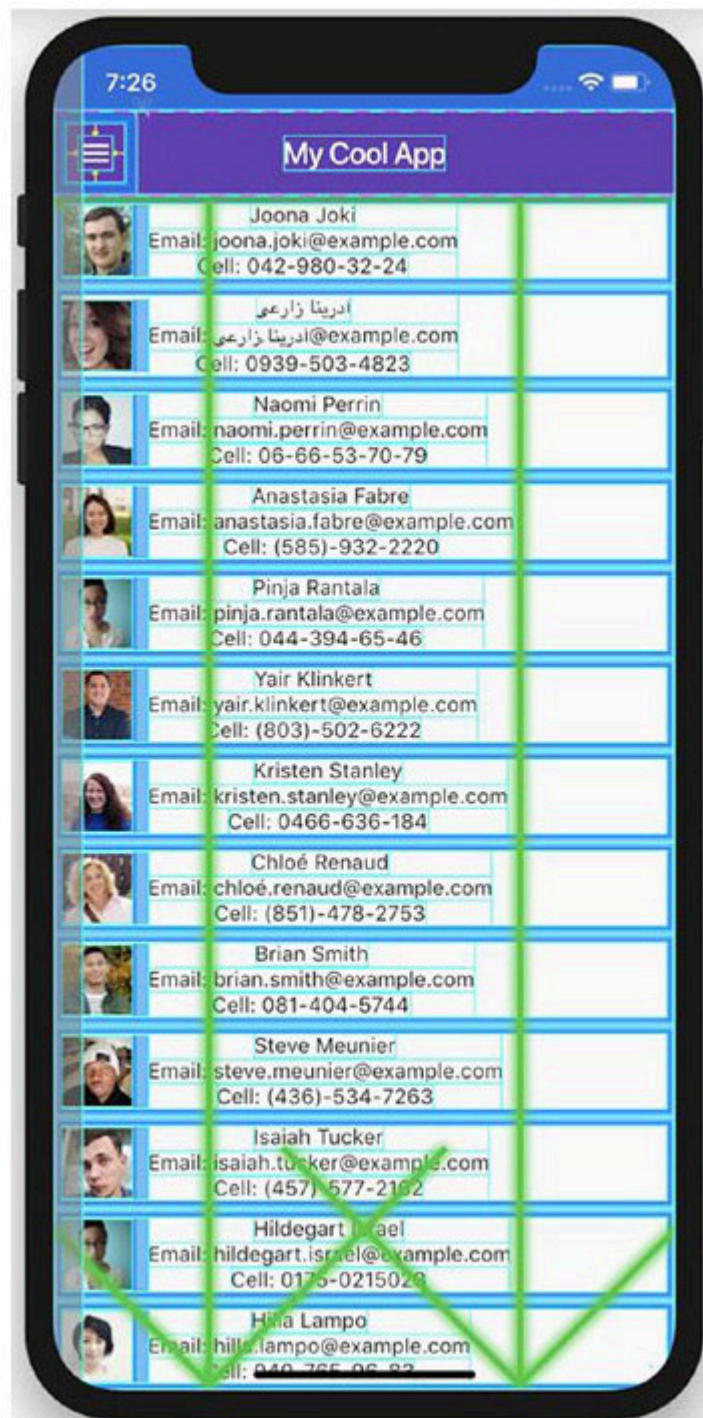
Được rồi, đó là kế hoạch cho chương này. Chúng ta sẽ đào sâu vào từng bước trong năm bước. Nhưng trước đó, hãy dành một chút thời gian xem cách debug bố cục trực quan.

Mẹo

Hãy dùng visual debugging để thấy Flutter đưa ra quyết định bố cục như thế nào. Hình 6-6 là màn hình bình thường; khi bạn bật debug painting, bạn sẽ thấy Hình 6-7.



Hình 6-6. Khi chưa bật visual debugging



Hình 6-7. Khi đã bật *visual debugging*

Mọi khung hiển thị đều có viền màu xanh ngọc. Padding, margin và border được tô màu xanh dương. Việc căn chỉnh/định vị được làm rõ bằng mũi tên vàng. Các mũi tên xanh lá lớn cho thấy những widget có thể cuộn. Khi đã quen với chúng, những gợi ý trực quan này sẽ giúp bạn thấy được cách Flutter "nghĩ" để bạn tinh chỉnh bố cục.

Để bật tính năng này:

- Trong VS Code, mở command palette (`Cmd+Shift+P` hoặc `Ctrl+Shift+P`) và gõ "Toggle debug painting".
- Trong Android Studio/IntelliJ, vào **View** → **Tool Windows** → **Flutter Inspector** và bấm nút "Show debug paint" trên thanh công cụ.

Bố trí toàn bộ scene

Đây là một lời khuyên cho bạn: **App không bao giờ nên làm người dùng bất ngờ.**² Khi app làm mọi thứ theo cách người dùng mong đợi, họ thấy app thân thiện, đơn giản và dễ dùng. Người dùng đã được "huấn luyện" để thấy một thanh trạng thái ở trên cùng, tiếp theo là thanh tiêu đề. Và tuy các thành phần khác trên màn hình thay đổi tùy nhu cầu, vẫn có những quy ước rõ ràng. Flutter có sẵn widget để bố cục của bạn cho cảm giác... bình thường.

Widget MaterialApp

Widget `MaterialApp` tạo ra khung ngoài cùng cho app của bạn. Dù quan trọng đến vậy, người dùng không bao giờ nhìn thấy `MaterialApp` vì về mặt kỹ thuật không phần nào của nó hiển thị. Nó bọc toàn bộ app, cho bạn cơ hội đặt tiêu đề để khi hệ điều hành đưa app xuống chạy nền, app có một cái tên. Đây cũng là nơi bạn áp dụng theme mặc định cho app — font chữ, kích thước, màu sắc, v.v. Chúng ta sẽ đi sâu hơn nhiều về theme ở chương styling. `MaterialApp` cũng là nơi chỉ định các **route**, thứ chúng ta sẽ bàn nhiều hơn ở chương routing.

Lưu ý

Chữ "Material" trong `MaterialApp` đúng là chỉ Material Design, vốn là thứ của Google/Android. Nhưng có lẽ đó là cái tên gây hiểu lầm, vì **mọi** app — kể cả app hướng iOS — đều có một `MaterialApp` ở gốc. Nó không làm app của bạn mang cảm giác Android hơn hay iOS ít đi.


```
Widget build(BuildContext context) {
  return MaterialApp(
    home: MainWidget(),
    title: "Ch 6 Layouts",
    theme: ThemeData(primarySwatch: Colors.deepPurple),
    routes: <String, WidgetBuilder>{
      '/scene1': (BuildContext ctx) => MyWidget1(),
      '/scene2': (BuildContext ctx) => MyWidget2(),
      '/scene3': (BuildContext ctx) => MyWidget3(),
    },
    debugShowCheckedModeBanner: false,
  );
}
```

 **Ghi chú cập nhật:** Từ Flutter 3.16, **Material 3** là mặc định. `primarySwatch` vẫn dùng được nhưng cách viết được khuyến nghị hiện nay là tạo bảng màu từ một màu hạt giống:

```
theme: ThemeData(
  colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
  useMaterial3: true,
),
```

Ngoài ra, với các app có nhiều màn hình, cách định tuyến được khuyến nghị hiện nay là `go_router` thay cho `routes` map, vì nó xử lý deep link và URL trên web tốt hơn.

Cuối cùng, `MaterialApp` có property `home`. Nhớ rằng project của bạn sẽ có rất nhiều widget tùy chỉnh. Bạn chỉ định widget nào là widget khởi động bằng cách đặt property `home` của `MaterialApp`. Widget này sẽ là gốc của scene chính và do đó nhiều khả năng sẽ bắt đầu bằng một widget `Scaffold`. "`Scaffold` là gì?", bạn hỏi. Rất mừng là bạn đã hỏi...

Widget Scaffold

Trong khi `MaterialApp` tạo ra khung vô hình bên ngoài, thì `Scaffold` tạo ra khung hữu hình bên trong.

`Scaffold` có một mục đích sống: bố trí cấu trúc hiển thị cho app, mang lại bố cục dễ đoán — và do đó dễ dùng — mà rất nhiều app khác cũng có. Trong số những thứ nó tạo ra có:

- Một `AppBar` cho tiêu đề
- Một khu vực cho phần thân (`body`)
- Một thanh điều hướng ở dưới cùng, hoặc một navigation drawer ở bên trái
- Một floating action button
- Một **bottom sheet** — khu vực thường bị thu gọn nhưng có thể trượt lên để hiện thông tin theo ngữ cảnh của scene mà người dùng đang xem

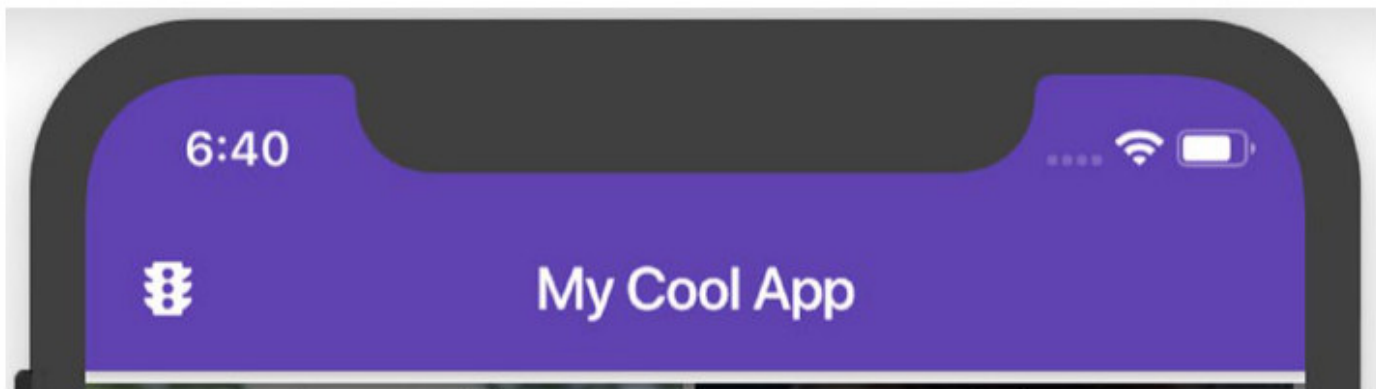
```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: MyAppBar(),
    drawer: MyNavigationDrawer(),
    body: TheRealContentOfThisPartOfTheApp(),
    floatingActionButton: FloatingActionButton(
      child: Icon(Icons.add),
      onPressed: () { /* Làm gì đó ở đây */ },
    ),
    bottomNavigationBar: MyBottomNavBar,
  );
}
```

Mọi thành phần của `Scaffold` đều là tùy chọn. Điều đó hợp lý vì không phải lúc nào bạn cũng muốn có `floatingActionButton`, `drawer` hay `bottomNavigationBar`. Thiết kế màn hình sẽ quyết định phần nào cần và phần nào không.

Widget AppBar

Để tạo thanh header ở đầu màn hình, hãy dùng widget `AppBar` (Hình 6-8). Đây hoàn toàn là tùy chọn. Nhưng người dùng chắc chắn sẽ mong đợi có `AppBar` ở gần như mọi app không phải game. Bạn hầu như luôn có một tiêu đề. Và bạn có thể muốn thêm một `Icon` ở đầu. Icon đó là property `leading`:

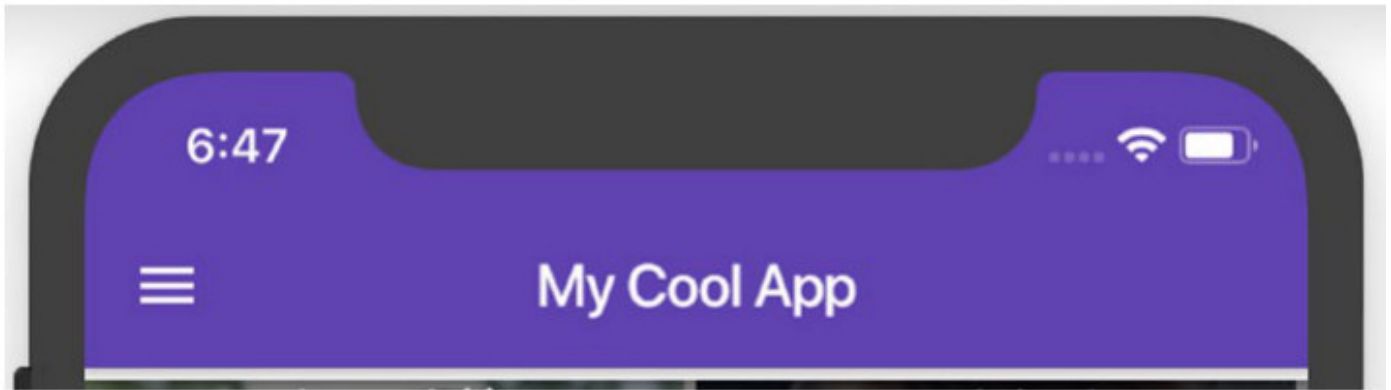
```
return Scaffold(  
  appBar: AppBar(  
    leading: Icon(Icons.traffic),  
    title: Text("My Cool App"),  
  ),  
  /* Những thứ khác ở đây: FAB, body, drawer, v.v. */  
);
```



Hình 6-8. Widget `AppBar` với một icon `leading` và một tiêu đề

Nhưng có một vấn đề. Nếu bạn vừa có icon `leading` vừa có navigation drawer, thì Flutter không còn chỗ để hiển thị nút menu hamburger (Hình 6-9):

```
return Scaffold(  
  appBar: AppBar(  
    /* Lần này không có leading. */  
    title: Text("My Cool App"),  
  ),  
  /* Những thứ khác ở đây: FAB, body, drawer, v.v. */  
);
```



Hình 6-9. AppBar không có icon leading thì hiển thị được icon menu hamburger

Nếu bạn có navigation drawer, có lẽ bạn nên bỏ icon `leading` đi.

Widget SafeArea

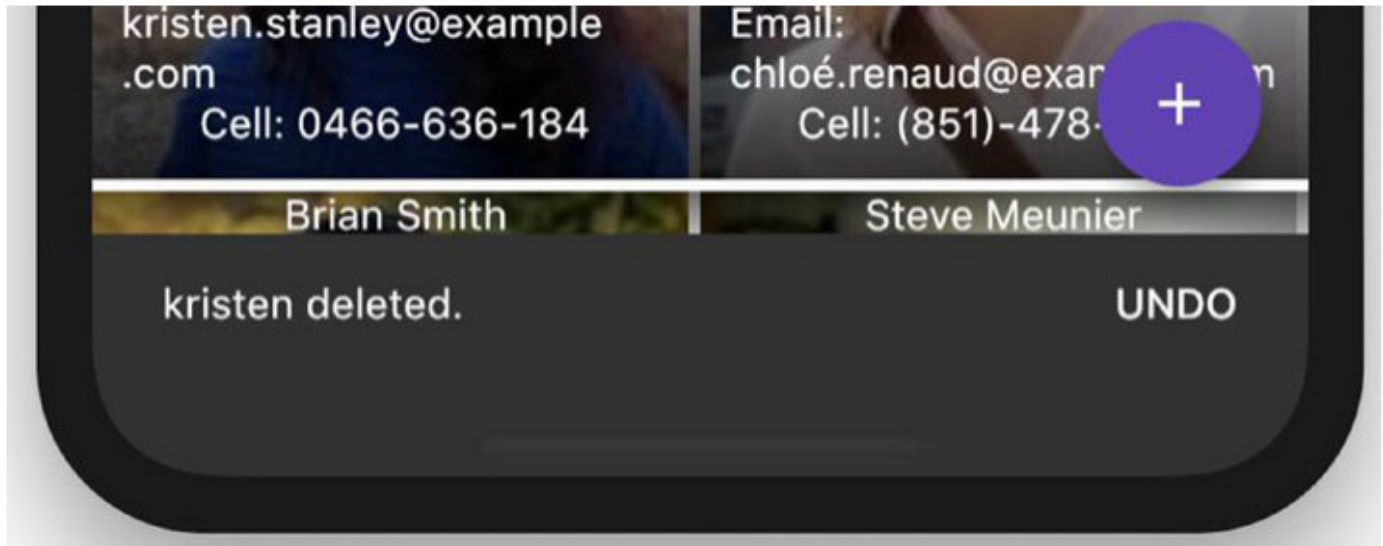
Màn hình thiết bị hiếm khi là hình chữ nhật gọn gàng. Chúng có góc bo tròn, có tai thỏ và thanh trạng thái ở trên. Nếu bỏ qua những thứ đó, một số phần của app sẽ bị cắt mất hoặc bị che. Không muốn thế? Bạn có hai lựa chọn: giữ một cơ sở dữ liệu khổng lồ về mọi thiết bị cùng vùng hiển thị của chúng và viết cả đống code điều kiện rối rắm — thật kinh khủng! Hoặc dùng widget `SafeArea`, về bản chất nó làm việc đó thay bạn.

Chỉ cần bọc `SafeArea` quanh toàn bộ nội dung `body` và để nó lo phần nặng nhọc. Đặt nó bên trong `Scaffold` nhưng bao quanh `body` là một vị trí rất tốt:

```
return Scaffold(  
  drawer: LayoutDrawer(),  
  body: SafeArea(  
    child: MyNormalBody(),  
  ),  
  floatingActionButton: FloatingActionButton(  
    child: Icon(Icons.next),  
    onPressed: () {},  
  ),  
);
```

Widget SnackBar

Cái tên nghe lạ, tôi biết. Nghe như món ăn vặt gì đó, nhưng widget này thực chất là cách chuẩn để thông báo điều gì đó cho người dùng. Một `SnackBar` (Hình 6-10) sẽ xuất hiện ở dưới cùng màn hình, che đi bất cứ thứ gì đang ở đó, rồi biến mất sau một lúc ngắn. Bạn quyết định `SnackBar` nói gì, và bạn thậm chí có thể đặt một nút lên đó để người dùng hành động.



Hình 6-10. *SnackBar hiển thị một thông báo và hành động tùy chọn*

Bạn có thể hiển thị `SnackBar` ở bất kỳ scene nào, miễn là bạn làm điều đó trong một widget nằm bên trong một `Scaffold` :

```
GestureDetector(
  child: PersonCard(person),
  onTap: () {
    String msg = '${person['name']['first']} deleted.';
    final SnackBar sb = SnackBar(
      content: Text(msg),
      duration: Duration(seconds: 5),
      action: SnackBarAction(
        textColor: Colors.white,
        label: "UNDO",
        onPressed: () {},
      ),
    );
    Scaffold.of(context).showSnackBar(sb);
  },
)
```

Lưu ý rằng bạn chạy method `showSnackBar()` để đưa `SnackBar` lên. Bạn kiểm soát thời lượng nó hiển thị. Cuối cùng, bạn có thể thêm một `action` vào `SnackBar` nếu muốn. Tất nhiên bạn cũng có thể chỉ hiện thông báo mà không có hành động nào. Tùy bạn.

 **Ghi chú cập nhật:** `Scaffold.of(context).showSnackBar(...)` **đã bị gỡ bỏ**. Hãy dùng `ScaffoldMessenger` :

```
ScaffoldMessenger.of(context).showSnackBar(sb);
```

Thay đổi này khắc phục lỗi kinh điển "Scaffold.of() called with a context that does not contain a Scaffold" và cho phép `SnackBar` tồn tại qua các lần chuyển màn hình.

Flutter quyết định kích thước widget như thế nào

Ai trong đời cũng có ràng buộc — những quy tắc, luật lệ và giới hạn phải tuân theo. Nếu không tuân thủ, sẽ có hậu quả. Widget của Flutter cũng có ràng buộc và cũng có hậu quả. Giống như ngoài đời, mọi thứ sẽ dễ dàng hơn nếu bạn học được luật chơi và hiểu các ràng buộc đó hoạt động ra sao.

Trong Flutter, mọi widget trên màn hình thiết bị cuối cùng đều có một chiều cao và một chiều rộng, gọi là **RenderBox**. Mỗi widget cũng có các ràng buộc: `minHeight`, `minWidth`, `maxHeight` và `maxWidth`, gọi chung là **BoxConstraints**.

📌 Lưu ý

Tất cả các số đo này đều tính bằng đơn vị pixel, và rõ ràng là phụ thuộc thiết bị. Dân iOS gọi chúng là *point*, còn dân Android gọi là *density-independent pixel*.

Miễn là `RenderBox` của widget nằm trọn trong `BoxConstraints` của nó thì mọi thứ đều ổn. Nói cách khác, chiều cao của nó phải nằm giữa `minHeight` và `maxHeight`, chiều rộng phải nằm giữa `minWidth` và `maxWidth`. Nhưng ngay khi một widget đòi được vẽ ra ngoài phạm vi ràng buộc, chuyện xấu sẽ xảy ra. Đôi khi Flutter ném ra exception, đôi khi nó cố hết sức rồi cắt bớt hoặc thu nhỏ widget lại.

Lỗi "unbounded height" đáng sợ

Tôi đảm bảo rằng đến một lúc nào đó trong sự nghiệp, bạn sẽ thấy Flutter ném ra lỗi này:

== EXCEPTION CAUGHT BY RENDERING LIBRARY ==

The following assertion was thrown during `performLayout()`:

`RenderFlex` children have non-zero flex but incoming width constraints are unbounded. When a row is in a parent that does not provide a finite width constraint, for example if it is in a horizontal scrollable, it will try to shrink-wrap its children along the horizontal axis. Setting a flex on a child (e.g. using `Expanded`) indicates that the child is to expand to fill the remaining space in the horizontal direction. These two directives are mutually exclusive. If a parent is to shrink-wrap its child, the child cannot simultaneously expand to fit its parent.

Không phải thông báo lỗi thân thiện với developer lắm, đúng không? Đa số chúng ta chẳng có hy vọng gì hiểu được vấn đề trong code của mình từ thông báo đó. Những thông báo tương tự có thể là "Vertical viewport was given unbounded height" hoặc "RenderViewport does not support returning intrinsic dimensions". Chẳng cái nào hữu ích cả. Nếu tử tế hơn, lẽ ra chúng nên nói kiểu như:

== BẠN ĐANG LÀM SAI RỒI ==

Cái ListView bạn đang vẽ muốn cao vô hạn và nó cần một widget cha giữ cho nó ngắn ở mức hợp lý. Hay là bảo nó nhỏ lại bằng cách bọc nó trong một widget LimitedBox?

Chẳng phải như thế thì rõ ràng hơn sao? Bạn sẽ hiểu vấn đề và biết rõ cách khắc phục.

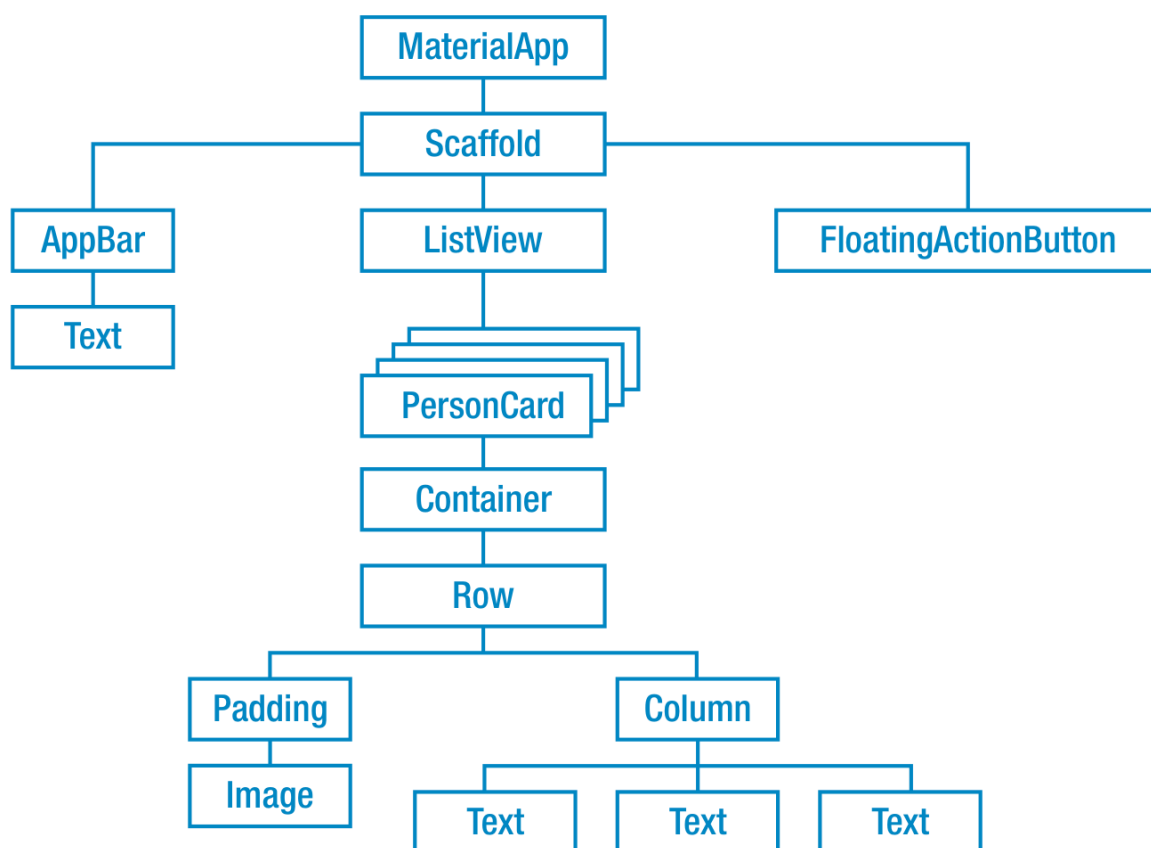
Để tôi giúp bạn diễn giải điều Flutter đang cố nói: một số widget muốn lấp đầy toàn bộ không gian có thể. Nói cách khác, chúng tham lam. Chúng cần một widget cha để ràng buộc lại. Nếu chúng nằm trong một widget cha từ chối cung cấp ràng buộc đó, Flutter hoảng loạn vì không hiểu chúng ta — những developer — đang định làm gì. Nói thẳng ra, đây là triệu chứng của việc developer chưa thực sự hiểu Flutter xử lý bố cục ra sao. Vậy nên hãy để tôi giải thích thuật toán bố cục của Flutter, với hy vọng bạn có thể dự đoán và do đó tránh được những rắc rối như ví dụ trên.

📌 Lưu ý

Nếu bạn không hiểu trọn vẹn thuật toán bố cục của Flutter thì cũng không phải tận thế. Bạn vẫn làm việc được với Flutter mà không cần thuộc lòng phần này. Nhưng bạn hiểu khái niệm này càng rõ thì càng ít bực bội khi gặp vấn đề bố cục trong thực tế. Nên hãy cố gắng.

Thuật toán bố cục của Flutter

Trong widget tùy chỉnh của bạn, có một widget gốc ở đầu method chính. Nó có các nhánh, rồi nhánh của nhánh, cứ thế tiếp diễn. Hãy gọi đó là **widget tree** (Hình 6-11). Flutter phải quyết định vẽ mỗi widget trong cây to đến đâu. Nó làm điều đó bằng cách hỏi mỗi widget muốn to cỡ nào và hỏi widget cha xem như vậy có được không.



Hình 6-11. Mọi scene đều có một widget tree

Flutter đi từ gốc xuống dưới cây. Nó đọc ràng buộc của widget gốc: "Anh có thể cao nhất là bao nhiêu? Rộng nhất là bao nhiêu?" Nó ghi nhớ rồi tìm các widget con. Với mỗi con, nó truyền BoxConstraints xuống rồi đi tiếp tới các cháu. Nó cứ làm vậy tới tận cùng mọi nhánh. Chúng ta gọi đó là **cấp lá** (leaf level).

Sau đó nó hỏi mỗi lá muốn to cỡ nào: "Chiều cao yêu thích của anh là bao nhiêu? Chiều rộng yêu thích là bao nhiêu?" Nó cho phép lá được vẽ ở kích thước ưa thích, trong phạm vi ràng buộc của toàn bộ tổ tiên. Nếu kích thước ưa thích quá lớn, Flutter cắt bớt nó lúc chạy — điều mà chúng ta rất cố tránh! Nếu kích thước ưa thích quá nhỏ, Flutter chèn thêm khoảng trống cho vừa.

Rồi nó quay ngược lên một cấp và cố xếp các nhánh đó vào bên trong widget cha chung, vốn cũng có ràng buộc riêng. Và cứ thế ngược lên tới đỉnh.

Kết quả là mỗi widget con được ở đúng chiều cao và chiều rộng nó thích — miễn là widget cha cho phép. Và không widget cha nào có kích thước cuối cùng cho tới khi mọi widget con của nó đã có.

💡 Mẹo

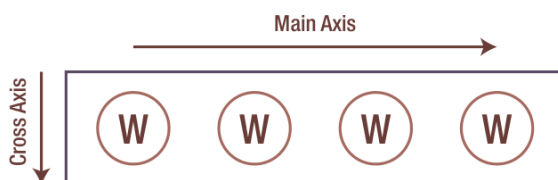
Một tình huống khác bạn sẽ gặp là khi `RenderBox` của một widget lớn hơn `BoxConstraints` của nó. Nói cách khác, riêng widget này không vừa với widget cha. Giải pháp cho vấn đề đó đôi khi là `FittedBox`³ — một widget thu nhỏ widget con cho vừa. Mặc định bạn sẽ có một widget được căn giữa và thu nhỏ lại cho tới khi vừa khít cả chiều ngang lẫn chiều dọc, nhưng bạn vẫn có các tùy chọn để căn dọc/ngang, kéo giãn, hoặc cắt bớt trên/dưới hay trái/phải.

Vậy là bạn thấy vì sao chúng ta gặp lỗi "unbounded height". Nếu có một widget con cố hết sức phình to mà lại không có widget cha bảo nó dừng lại, Flutter hoảng vì giờ nó cao vô hạn. Để khắc phục, widget con đó chỉ cần một widget cha bảo nó ngừng lớn. Đặc điểm chính của widget `LimitedBox` chính là làm việc đó: nó nói cho widget con biết nó được phép lớn tới đâu, trong trường hợp widget cha từ chối làm việc đó.

Flutter có rất nhiều widget để kiểm soát kích thước và vị trí. Phần còn lại của chương này, chúng ta sẽ nghiên cứu những widget bố cục quan trọng nhất — những cái bạn nhất định phải biết. Bắt đầu với `Row` và `Column`.

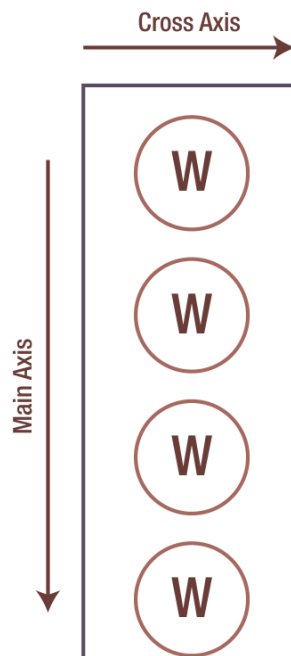
Đặt widget cạnh nhau hoặc dưới nhau

`Row` và `Column`, đúng như tên gọi, được tạo ra để xếp widget cạnh nhau (`Row`, Hình 6-12) hoặc trên dưới nhau (`Column`, Hình 6-13). Ngoài cách xếp widget con ra, chúng gần như giống hệt nhau.



Hình 6-12. Widget `Row` xếp các phần tử cạnh nhau

```
Row(  
  children: <Widget>[  
    Widget(),  
    Widget(),  
    Widget(),  
    Widget(),  
  ],  
)
```



Hình 6-13. *Widget Column xếp các phần tử trên dưới nhau*

```
Column(  
  children: <Widget>[  
    Widget(),  
    Widget(),  
    Widget(),  
    Widget(),  
  ],  
)
```

✦ **Về Main Axis và Cross Axis trong hai hình trên:** **Main Axis** (trục chính) của `Row` là trục ngang, còn **Cross Axis** (trục phụ) là trục dọc. Với `Column` thì ngược lại: **Main Axis** là trục dọc, **Cross Axis** là trục ngang. Hai khái niệm này sẽ quay lại liên tục ở phần sau, nên hãy ghi nhớ.

Để ý rằng cả hai đều có property `children` — một mảng các `Widget`. Mọi widget trong mảng `children` sẽ được hiển thị theo đúng thứ tự bạn thêm vào. Bạn thậm chí có thể đặt row bên trong column và ngược lại, lồng bao nhiêu cấp tùy thích. Bằng cách này bạn có thể tạo ra gần như mọi bố cục có thể tưởng tượng trong bất kỳ app nào.

`Row` và `Column` sẽ là những widget bố cục bạn dùng đầu tiên. Đúng là còn những cái khác, nhưng hai cái này là lựa chọn số một.

✦ Lưu ý

Thỉnh thoảng bạn sẽ muốn hai thứ nằm trên dưới nhau khi thiết bị nằm ngang và cạnh nhau khi thiết bị dựng đứng. Tức là lúc thì bạn muốn chúng trong một row, lúc lại trong một column. Đó là khi bạn dùng widget `Flex`, vốn làm được cả hai. Nó có property `direction` có thể đặt theo điều kiện:

```
Flex(  
  direction:  
    MediaQuery.of(context).orientation == Orientation.landscape  
      ? Axis.horizontal  
      : Axis.vertical,  
  children: <Widget>[  
    SomeWidget(),  
    SomeWidget(),  
    SomeWidget(),  
  ],  
)
```

Chuyện này không xảy ra thường xuyên như bạn nghĩ đâu. Hãy dùng tiết chế.

Widget của bạn sẽ không bao giờ vừa khít!

Sẽ là một trùng hợp cực kỳ hiếm nếu các thành phần vừa khít hoàn hảo trong một scene. Và nếu chúng có vừa khít đi nữa, thì ngay khi app chạy trên màn hình kích thước khác hoặc xoay từ dọc sang ngang, mọi thứ sẽ thay đổi. Vì vậy chúng ta cần xử lý hai tình huống:

1. Nếu còn thừa chỗ trống thì sao? (màn hình rộng hơn tổng số pixel mà các widget chiếm)
2. Nếu không đủ chỗ thì sao? (quá nhiều widget trong một không gian nhất định)

Cả hai đều có khả năng xảy ra đồng thời ở những phần khác nhau của scene. Hãy xử lý chỗ trống thừa trước.

Nếu còn thừa chỗ trống thì sao?

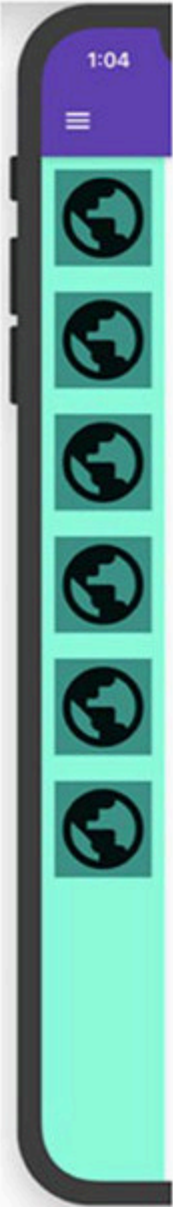
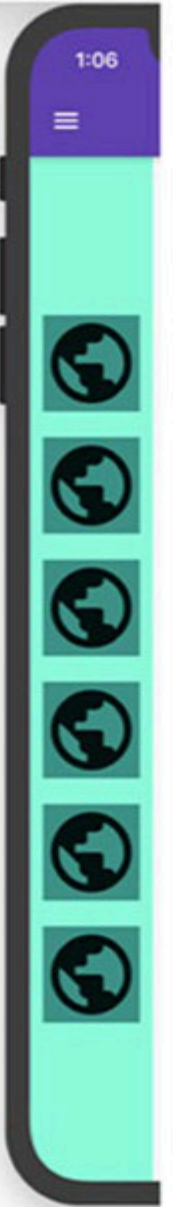
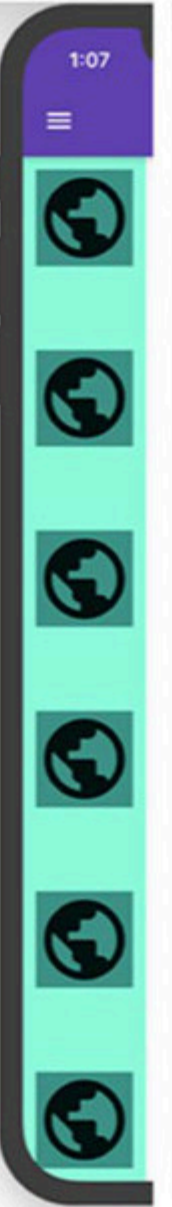
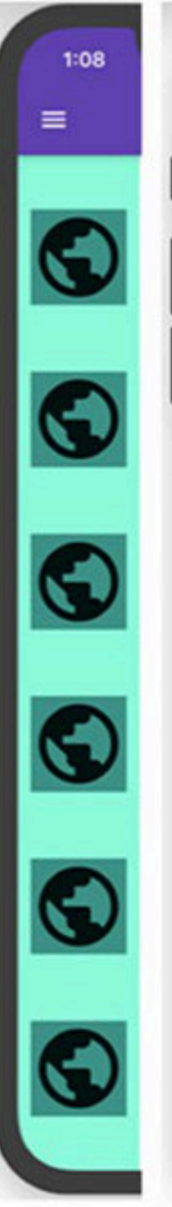
Đây là vấn đề dễ giải quyết. Câu hỏi duy nhất bạn thực sự cần trả lời là phân bổ chỗ trống thừa ra sao. Bạn muốn dành bao nhiêu khoảng trống quanh từng widget? Bạn có vài lựa chọn. Cách dễ và nhanh nhất là dùng `mainAxisAlignment` và `crossAxisAlignment`.

`mainAxisAlignment`

`mainAxisAlignment` là một property của `Row` hoặc `Column` (Hình 6-14). Với nó bạn kiểm soát cách phân bổ chỗ trống thừa dọc theo **trục chính** — trục dọc với column và trục ngang với row:

```
child: Column(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
  children: <Widget>[  
    SubWidget(),  
    SubWidget(),  
    // ...  
  ],  
)
```

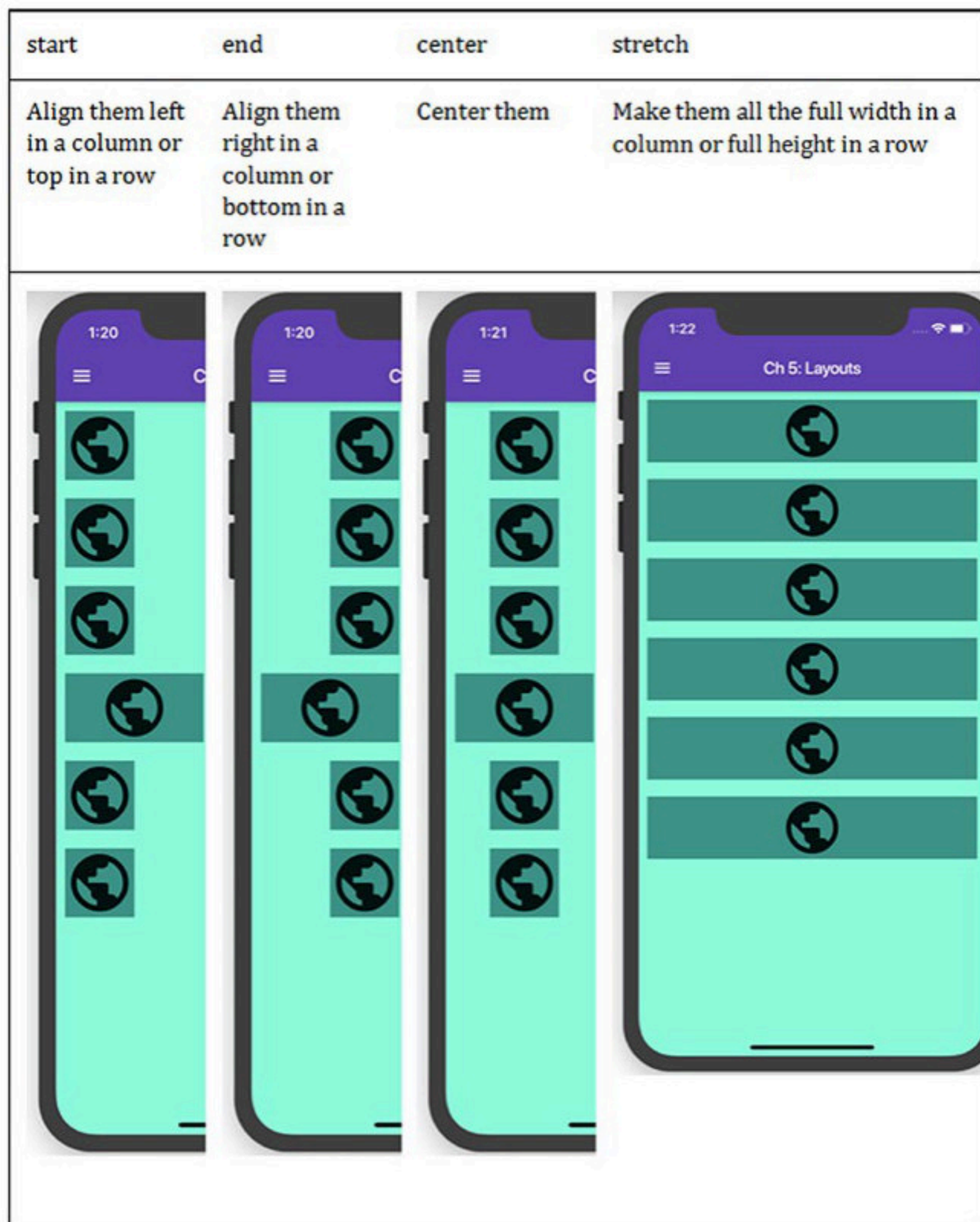
Bạn có vài lựa chọn:

start	end	center	spaceBetween	spaceEvenly	spaceAround
No space between them. All bunched up at the start. The default.	Same but at the end.	Same but we put space before the first and after the last	All remaining space is divided between each child widget	Same but some is saved for before the first and after the last	Same but the the spaces at the ends get half as much as the spaces between
					

Hình 6-14. `mainAxisAlignment` quy định cách phân bổ chỗ trống thừa dọc theo trục chính

`crossAxisAlignment`

`crossAxisAlignment` cũng là property của `Row` hoặc `Column`; nó quyết định đặt chỗ trống thừa ở đâu khi các widget có chiều cao khác nhau trong một row, hoặc chiều rộng khác nhau trong một column (Hình 6-15). Các lựa chọn của bạn là:



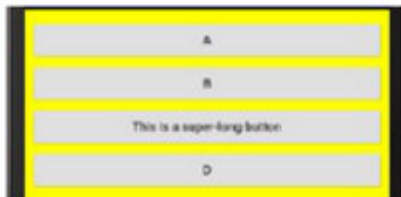
Hình 6-15. `crossAxisAlignment` quy định cách phân bổ chỗ trống thừa dọc theo trục phụ

Còn một giá trị nữa: `baseline`. Nhưng nó chỉ có ý nghĩa trong một row, và ít được dùng hơn nhiều.

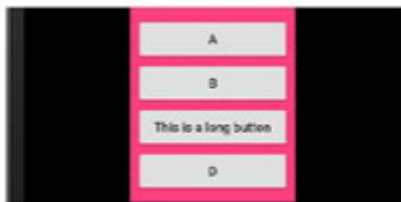
💡 Mẹo

Nếu bạn muốn tất cả widget con của một `Column` có cùng chiều rộng nhưng không nhất thiết bằng toàn bộ chiều rộng màn hình, hãy dùng widget `IntrinsicWidth`. Với `crossAxisAlignment.stretch`, tất cả sẽ giãn ra tới chiều rộng tối đa (Hình 6-16), nhưng khi được bọc trong `IntrinsicWidth`, tất cả sẽ có cùng kích thước với widget rộng nhất (Hình 6-17 và 6-18).

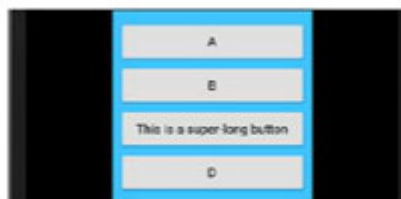
```
child: IntrinsicWidth(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: <Widget>[ /* ... */ ],
  ),
),
```



Hình 6-16. Không có `IntrinsicWidth`, mọi phần tử sẽ giãn ra hết chiều rộng



Hình 6-17. Với `IntrinsicWidth`, chúng chỉ rộng bằng phần tử rộng nhất



Hình 6-18. Với `IntrinsicWidth` và một phần tử rộng hơn, tất cả đều rộng ra theo

Vậy bạn thấy đấy: khi chiều rộng của nút dài nhất tăng lên, tất cả cũng tăng theo.

Widget Expanded

`mainAxisAlignment` rất tuyệt nếu việc chia khoảng cách là rõ ràng — bạn muốn khoảng cách đều nhau theo cách nào đó. Nhưng nếu bạn không muốn có khoảng cách gì cả thì sao? Nếu bạn muốn các widget giãn ra để lấp đầy chỗ trống còn lại thì sao? Widget `Expanded` sẽ ra tay (Hình 6-19).

Lấy đoạn code này làm ví dụ:

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceAround,  
  children: <Widget>[  
    SubWidget(),  
    SubWidget(),  
    SubWidget(),  
    SubWidget(),  
    SubWidget(),  
    SubWidget(),  
  ],  
)
```

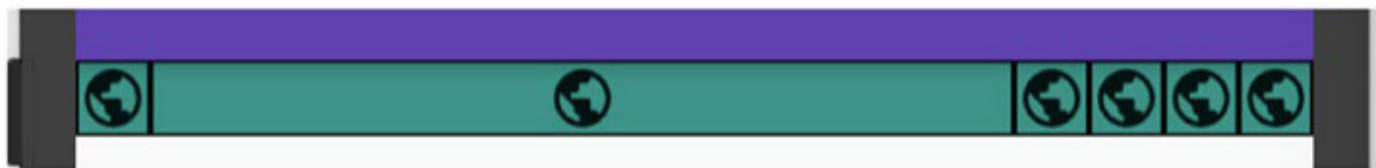


Hình 6-19. Widget Row này còn rất nhiều chỗ trống thừa

Khi bạn bọc một widget con của `Row / Column` trong một `Expanded` (Hình 6-20), nó khiến widget con đó trở nên linh hoạt, nghĩa là nếu còn chỗ trống thừa, nó sẽ giãn ra dọc theo trục chính để lấp đầy chỗ đó.

Đây vẫn là ví dụ trên nhưng có `Expanded()` bọc quanh widget thứ hai:

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceAround,
  children: <Widget>[
    SubWidget(),
    Expanded(child: SubWidget()),
    SubWidget(),
    SubWidget(),
    SubWidget(),
    SubWidget(),
  ],
)
```

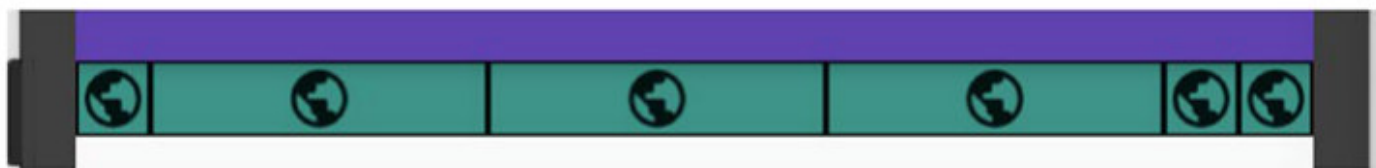


Hình 6-20. Widget thứ hai được bọc trong một Expanded

Lưu ý rằng `mainAxisAlignment` giờ không còn tạo ra khác biệt nào nữa, vì không còn chỗ trống thừa. Tất cả đã bị `Expanded` "ăn" hết.

Nếu chúng ta thêm một `Expanded` nữa thì sao? Hãy bọc cả widget thứ ba và thứ tư (Hình 6-21):

```
Row(
  children: <Widget>[
    SubWidget(),
    Expanded(child: SubWidget()),
    Expanded(child: SubWidget()),
    Expanded(child: SubWidget()),
    SubWidget(),
    SubWidget(),
  ],
)
```

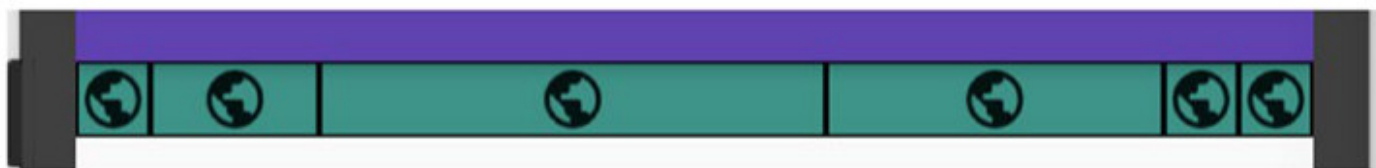


Hình 6-21. Các *Expanded* chia đều chỗ trống cho nhau

Lưu ý rằng cái thứ hai giờ nhỏ hơn vì chỗ trống thừa được chia sẻ với widget thứ ba và thứ tư, chia đều cho cả ba.

Nhưng khoan! Còn nữa! Chúng ta có thể kiểm soát mỗi *Expanded* được bao nhiêu chỗ. *Expanded* có một property gọi là **flex factor**, kiểu số nguyên. Khi *Row* / *Column* được bố trí, các phần tử cứng được tính kích thước trước. Sau đó các phần tử linh hoạt được giãn ra theo flex factor của chúng (Hình 6-22). Trong các ví dụ trước, các *Expanded* có flex factor mặc định là 1 nên chúng nhận được lượng chỗ trống bằng nhau. Nhưng nếu ta cho chúng flex factor khác nhau, chúng sẽ giãn ra với tỉ lệ khác nhau:

```
Row(
  children: <Widget>[
    SubWidget(),
    Expanded(flex: 1, child: SubWidget()),
    Expanded(flex: 3, child: SubWidget()),
    Expanded(flex: 2, child: SubWidget()),
    SubWidget(),
    SubWidget(),
  ],
)
```



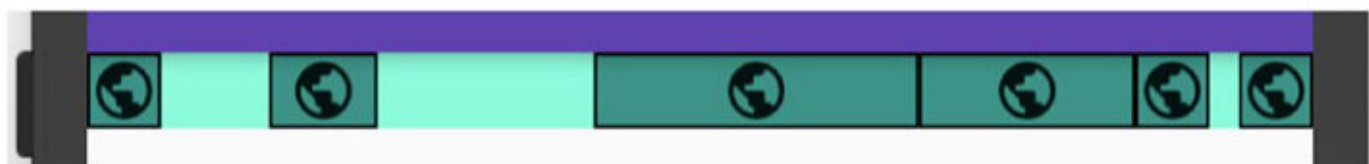
Hình 6-22. *Expanded* có property *flex* để kiểm soát mỗi cái nhận bao nhiêu chỗ trống thừa

Để ý rằng chỗ trống vẫn được phân bổ cho các *Expanded*, nhưng theo tỉ lệ 1, 3 và 2 thay vì chia đều. Vậy cái có flex factor là 3 nhận được lượng chỗ gấp ba lần cái có flex factor là 1.

✚ Lưu ý

`Expanded` ăn hết chỗ trống. Nhưng nếu bạn muốn dùng `Expanded` mà vẫn muốn có khoảng cách giữa một số widget con, hãy dùng widget `Spacer` hoặc `SizedBox` (Hình 6-23). `Spacer` có flex factor phối hợp tốt với mọi flex factor khác trên cùng trục. `SizedBox` có property `height` và `width` cho khi bạn muốn diễn đạt kích thước bằng pixel:

```
Row(
  children: <Widget>[
    SubWidget(),
    Spacer(),
    Expanded(flex: 1, child: SubWidget()),
    Spacer(flex: 2),
    Expanded(flex: 3, child: SubWidget()),
    Expanded(flex: 2, child: SubWidget()),
    SubWidget(),
    SizedBox(width: 10),
    SubWidget(),
  ],
)
```



Hình 6-23. `Spacer()` và `SizedBox()` trả lại chỗ trống nhưng cho bạn quyền quyết định đặt ở đâu và bao nhiêu

Nếu không đủ chỗ thì sao?

Chúng ta đã xử lý tình huống thừa chỗ và cách kiểm soát nơi phân bổ chỗ thừa. Nhưng nếu quá ít chỗ thì sao? Kiểu như ta đang cố nhét quá nhiều widget vào một row hay column quá nhỏ? Nếu bạn không làm gì cả, Flutter sẽ cắt bớt widget — trông rất tệ, và tệ hơn là có thể che mất một số widget khỏi mắt người dùng.

Vậy bạn làm gì? Bạn cho phép cuộn!

Tuy có thể cuộn theo cả hai chiều, việc đó tạo ra vấn đề nghiêm trọng về trải nghiệm sử dụng. Vì vậy chúng tôi khuyên bạn chỉ nên cuộn theo một chiều, và thường là cuộn dọc. Cách cuộn dễ nhất là dùng `ListView`.

Widget ListView

`ListView` thực ra có bốn cách dùng khác nhau:

1. `ListView` **thường** — Dùng bình thường. Nó có property `children` nhận một tập hợp các widget tĩnh.
2. `ListView.builder` — Để tạo widget con động từ một danh sách các mục.
3. `ListView.separated` — Giống `builder` nhưng còn chèn thêm một widget giữa mỗi mục. Rất hợp để chèn quảng cáo định kỳ vào danh sách. Đọc thêm tại http://bit.ly/flutter_listview_separated.
4. `ListView.custom` — Để tự xây dựng listview nâng cao của riêng bạn. Đọc thêm tại http://bit.ly/flutter_listview_custom.

Hãy xem hai lựa chọn đầu tiên, bắt đầu với `ListView` thường.

ListView thường: khi bạn có vài widget cần hiển thị

Nói chung, `ListView` nhận một số lượng nhỏ widget khác và làm cho chúng cuộn được. Vì sao lại là "số lượng nhỏ"? Vì nó được thiết kế cho danh sách tĩnh — loại mà bạn, developer, gõ thẳng bằng tay vào method `build()`. Thực tế, thường thì bạn phát hiện mình cần một `ListView` thường khi `Column` của bạn bị tràn. Cách khắc phục là bỏ bớt widget con, đổi kích thước widget con, hoặc đơn giản là đổi `Column` thành `ListView`. Cả `Column` lẫn `ListView` đều có property `children`:

```
Widget _build(BuildContext context) {  
  return ListView(  
    children: <Widget>[  
      FirstWidget(),  
      SecondWidget(),  
      ThirdWidget(),  
    ],  
  );  
}
```

Phiên bản `ListView` này rất tốt cho số lượng widget nhỏ, nhưng `ListView` thực sự tỏa sáng khi bạn muốn hiển thị một danh sách các thứ — người, sản phẩm, cửa hàng — bất cứ gì bạn lấy về từ database hay từ một service. Để hiển thị số lượng mục cuộn không xác định trước, chúng ta dùng constructor `ListView.builder`.

ListView.builder: khi bạn dựng widget từ một danh sách object

Constructor thay thế của `ListView` là `ListView.builder`, nhận vào hai tham số: `itemCount` và một property `itemBuilder` vốn là một hàm. Điều này khiến `ListView` được **lazy-load**. Hàm `itemBuilder` tạo widget con một cách động theo nhu cầu. Khi người dùng cuộn gần tới cuối danh sách, `itemBuilder` tạo mục mới để cuộn vào tầm nhìn. Và khi ta cuộn thứ gì đó ra khỏi màn hình đủ xa, nó bị đẩy ra khỏi bộ nhớ và hủy đi. Khá hay.

```
Widget _build(BuildContext context) {  
  return ListView.builder(  
    scrollDirection: Axis.vertical,  
    itemCount: _people.length,  
    itemBuilder: (BuildContext context, int i) {  
      return PersonCard(_peopleList[i]);  
    },  
  );  
}
```

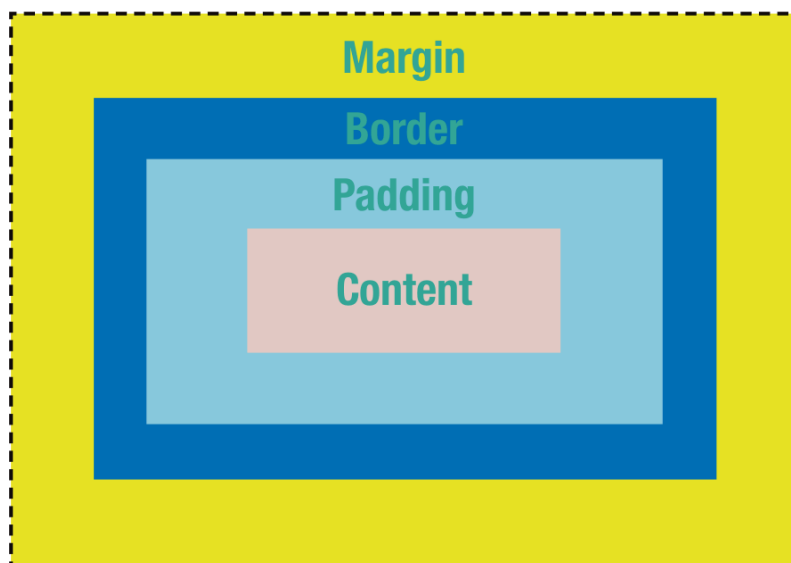
Property `itemCount` là một số nguyên cho biết chúng ta sẽ vẽ bao nhiêu thứ, nên thường ta đặt nó bằng độ dài của mảng/tập hợp đang duyệt. Hàm `itemBuilder` nhận hai tham số: `context` và một số nguyên bắt đầu từ 0 cho mục đầu tiên và tăng dần sau mỗi lần chạy.

Chúng ta đã bàn về việc bố trí scene, kể cả khi có chỗ trống thừa hay khi không đủ chỗ. Vậy hãy chuyển sang chủ đề cuối cùng trong năm chủ đề: cách tinh chỉnh khoảng cách và vị trí của widget. Chúng ta làm điều này bằng cách khám phá box model.

Widget Container và box model

Flutter vay mượn rất nhiều từ các công nghệ khác, trong đó có HTML và Web — nơi có khái niệm border, padding và margin. Chúng được gọi chung là **box model**. Chúng được dùng để tạo khoảng cách để nhìn

quanh và giữa các thành phần trên màn hình. Đây là một khái niệm đã được kiểm chứng qua thời gian và hoạt động rất tốt cho Web, vậy tại sao không mượn nó cho Flutter?



Hình 6-24. Box model định nghĩa padding, border và margin

Giả sử chúng ta có một ảnh đã được định kích thước và muốn "đóng khung" nó với padding là 8, margin là 10 và border là 1. Người mới với Flutter có thể thử cách này trước:

```
Image.network(  
  _peopleList[i]['picture']['thumbnail'],  
  padding: 8.0,  
  margin: 10.0,  
  border: 1.0,  
)
```

Cách này sẽ không chạy vì widget `Image` không có `padding`, `margin` hay `border`. Nhưng bạn biết cái gì có không? `Container` !

Web developer thường áp dụng những thứ này bằng cách bọc phần tử trong một container chung gọi là `<div>` rồi áp style lên đó để tạo khoảng cách để nhìn cho trang web.

Flutter không có `<div>`, nhưng nó có một widget giống `<div>` gọi là `Container` — vốn... chứa những thứ khác. Thực tế, mục đích sống của nó là áp dụng bố cục và style cho những thứ bên trong. Một `<div>` trong HTML có thể chứa nhiều thứ, nhưng `Container` của Flutter chỉ chứa **một** widget con. Nó có các

property `padding`, `margin` và `decoration`. Chúng ta để `decoration` cho chương styling, nhưng `padding` và `margin` thì cực kỳ tiện để tạo khoảng cách đẹp mắt:

```
Container(
  padding: EdgeInsets.all(8.0),
  margin: EdgeInsets.all(10.0),
  decoration: BoxDecoration(border: Border.all(width: 1.0)),
  child: Image.network(thePicture),
  // Container còn *rất nhiều* property khác, phần lớn
  // sẽ được trình bày ở chương Styles.
),
```

💡 Mẹo

Margin và padding hẳn sẽ dễ học hơn nếu họ chỉ cho phép ta liệt kê bốn số đại diện cho bốn cạnh. (Nhưng họ không làm cho nó dễ thế đâu.) Thay vào đó, chúng ta dùng một widget hỗ trợ tên là `EdgeInsets`.

- `EdgeInsets.all(8.0)` — Cùng một giá trị áp cho cả bốn cạnh.
- `EdgeInsets.symmetric(horizontal: 7.0, vertical: 5.0)` — Trên và dưới giống nhau. Trái và phải giống nhau.
- `EdgeInsets.only(top: 20.0, bottom: 40.0, left: 10.0, right: 30.0)` — Trái, trên, phải, dưới đều có thể khác nhau.
- `EdgeInsets.fromLTRB(10.0, 20.0, 30.0, 40.0)` — Giống cái trên nhưng gọn ít hơn.

Cũng lưu ý rằng nếu bạn chỉ muốn padding — không định dạng gì khác — thì widget `Padding` là cách viết tắt.

```
Container(
  padding: EdgeInsets.all(5),
  child: Text("foo"),
),
```

```
Padding(
  padding: EdgeInsets.all(5),
  child: Text("foo"),
),
```

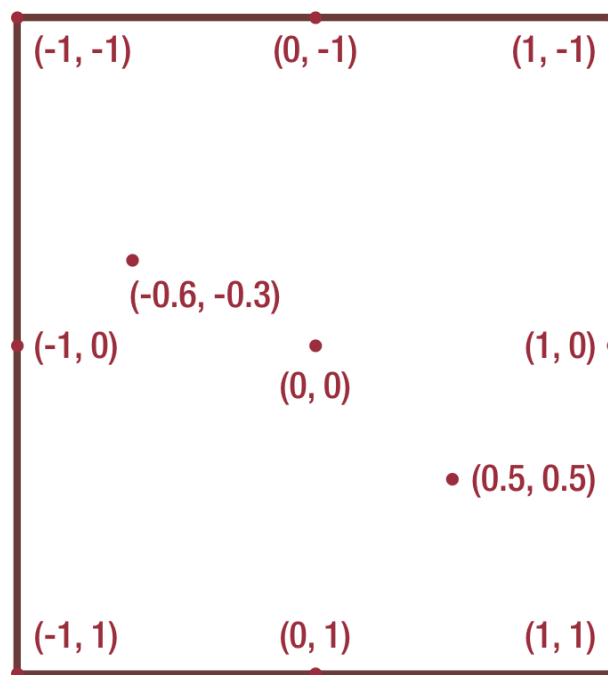
Hai cái này tương đương nhau.

Căn chỉnh và định vị bên trong Container

Khi bạn đặt một widget con nhỏ vào một `Container` lớn, sẽ có nhiều chỗ trống trong `Container` hơn mức widget con cần. Widget con đó mặc định sẽ nằm ở góc trên bên trái. Bạn có lựa chọn định vị nó bằng property `alignment`:

```
Container(
  width: 150, height: 150,
  alignment: Alignment(1.0, -1.0),
  child: Image.network(
    _peopleList[i]['picture']['thumbnail'],
  ),
),
```

Những con số alignment đó đại diện cho căn chỉnh ngang (`-1.0` là ngoài cùng bên trái, `0.0` là giữa, `1.0` là ngoài cùng bên phải) và căn chỉnh dọc (`-1.0` là trên cùng, `0.0` là giữa, `1.0` là dưới cùng). Xem Hình 6-25.



Hình 6-25. Hệ tọa độ alignment với điểm 0,0 nằm ở chính giữa

Nhưng có lẽ bạn sẽ thích dùng chữ hơn là số khi có thể:

```
Container(
  width: 150, height: 150,
  alignment: Alignment.centerLeft,
  child: Image.network(
    _peopleList[i]['picture']['thumbnail'],
  ),
),
```

`Alignment` có thể nhận bất kỳ giá trị nào trong số: `topLeft`, `topCenter`, `topRight`, `centerLeft`, `center`, `centerRight`, `bottomLeft`, `bottomCenter` và `bottomRight`. Chẳng phải như vậy dễ viết hơn và dễ đọc hơn cho đồng nghiệp của bạn sao?

💡 Mẹo

Widget `Align` là cách viết tắt khi bạn chỉ cần chỉ định alignment mà không cần property nào khác. Còn widget `Center` chỉ là cách viết tắt để căn giữa.

```
Container(
  alignment: Alignment.center,
  child: Text("foo"),
),
```

```
Align(
  alignment: Alignment.center,
  child: Text("foo"),
),
```

```
Center(
  child: Text("foo"),
),
```

Ba cái này tương đương nhau.

Vậy xác định kích thước của `Container` thế nào?

Có thể bạn đã để ý rằng tôi đã lên đưa `width` và `height` vào phần trước. Đúng vậy, bạn có thể bảo `Container` rằng bạn muốn nó có một chiều rộng và chiều cao cụ thể, và nó sẽ tuân theo **khi có thể**. Cả `width` lẫn `height` đều nhận một số đơn giản trong khoảng từ 0 tới `double.infinity`. Giá trị `double.infinity` **gợi ý** rằng nó muốn to hết mức widget cha cho phép.

Giờ tôi biết bạn đang nghĩ gì. "Rap này, 'khi có thể' và 'gợi ý' nghĩa là sao? Không có quy tắc cứng nào à? Tôi muốn kích thước `Container` phải dự đoán được chứ!" Và tôi hoàn toàn đồng ý. Kích thước của `Container` khó dự đoán cho tới khi bạn biết luật của nó. Vậy nó quyết định như thế nào?

Hãy nhớ hai điều. **Thứ nhất**, `Container` được tạo ra để chứa một widget con, nhưng việc có widget con là tùy chọn. 99% thời gian nó sẽ có widget con. 1% còn lại, chúng ta dùng `Container` để cung cấp màu nền hoặc tạo khoảng cách cho các widget hàng xóm/anh em. **Thứ hai**, nhớ rằng Flutter xác định bố cục qua hai pha: đi xuống cây render để xác định `BoxConstraints`, rồi đi ngược lên để xác định `RenderBox` (còn gọi là "kích thước", nhớ chứ?).

Đi từ trên xuống:

- Flutter giới hạn kích thước tối đa bằng cách truyền `BoxConstraints` từ widget cha xuống `Container`.
- `Container` khá dễ tính, nó nói với widget cha: "Nếu hàng xóm của tôi cần chỗ, cứ lấy đi. Tôi sẽ nhỏ đúng mức anh cần."
- Nếu `height` và/hoặc `width` được đặt, nó tôn trọng những giá trị đó cho tới kích thước tối đa do `BoxConstraints` quy định. Lưu ý rằng việc bạn khai một kích thước lớn hơn `BoxConstraints` **không phải là lỗi**, chỉ là nó sẽ không lớn hơn nữa. Đây là lý do bạn có thể dùng `double.infinity` mà không gặp lỗi.

Mẹo

Đặt `height` và `width` khiến `Container` trở nên cực kỳ cứng nhắc; nó khóa chặt một kích thước. Tuy điều này tiện khi bạn muốn tinh chỉnh bố cục, best practice là tránh dùng chúng trừ khi bạn có lý do thật chính đáng. Nói chung bạn nên để widget tự quyết định kích thước của mình.

Rồi đi từ dưới lên:

- Trong 1% trường hợp không có widget con, nó chiếm toàn bộ chỗ trống còn lại cho tới `BoxConstraint` tối đa.
- Nhưng phần lớn thời gian nó có widget con, nên layout engine nhìn vào `RenderBox` của widget con.
- Nếu `RenderBox` của widget con lớn hơn `BoxConstraints` của tôi, nó cắt bớt widget con — đây là một vấn đề lớn. Về mặt kỹ thuật thì không phải lỗi, nhưng trông rất tệ. Nên hãy tránh. Ở chế độ debug, Flutter sẽ vẽ các sọc vàng đen ở chỗ bị tràn để developer không bỏ sót.
- Nếu `RenderBox` của widget con nằm trong `BoxConstraints` của tôi, sẽ còn chỗ thừa nên ta nhìn vào property `alignment`. Nếu `alignment` không được đặt, ta đặt widget con ở góc trên bên trái và làm `Container` co lại vừa khít widget con. Chỗ thừa chỉ là khoảng trống. Nếu `alignment` được đặt, `Container` trở nên tham lam. Điều này khá hợp lý nếu bạn nghĩ kỹ, bởi làm sao nó căn được trên/dưới/trái/phải nếu nó không phình ra để tạo thêm chỗ?
- Sau tất cả những điều đó, nó co lại nếu cần để tôn trọng margin.

Các widget bố cục đặc biệt

Như đã nói ở đầu chương, chúng ta đã bàn xong những công cụ đủ cho 90% nhu cầu bố cục, nhưng vẫn còn nữa. Có vài cái đáng liếc qua để bạn biết mà tìm khi tình huống phát sinh. Những widget này được thiết kế cho các tình huống bố cục rất đặc thù — tuy phổ biến nhưng không phải ngày nào cũng gặp, và cần công cụ chuyên biệt mới làm được.

Widget Stack

Đây là widget dành cho khi bạn muốn xếp lớp các widget chồng lên nhau. Bạn muốn xếp chúng theo chiều Z. Với `Stack`, bạn liệt kê một số widget và chúng sẽ được hiển thị theo thứ tự đó, cái nọ chồng lên cái kia. Cái cuối cùng sẽ che cái trước nếu chúng chồng lấn, cái đó lại che cái trước nữa, và cứ thế.

Tôi khá phân vân không biết nên trình bày `Stack` ở đâu. Một mặt, nó liên quan tới việc bố trí màn hình nên hợp với chương này hơn. Mặt khác, `Stack` tỏa sáng khi tạo card — vốn rõ ràng là khái niệm styling nên hợp với chương sau hơn. Chúng tôi quyết định nhắc tới nó ở đây nhưng sẽ tập trung vào nó ở phần sau.

Widget GridView

Lại một thứ nữa mượn từ HTML và Web. `GridView` dùng để hiển thị một danh sách widget khi bạn muốn chúng xuất hiện theo hàng và cột nhưng không đặc biệt quan tâm cái nào ở hàng nào, cột nào — bạn chỉ muốn chúng hiện ra dưới dạng lưới.

Để dùng `GridView`, bạn đặt property `children` bằng danh sách widget muốn hiển thị, và nó sẽ tạo lưới, điền theo chiều ngang rồi xuống dòng sang hàng tiếp theo, tự điều chỉnh kích thước cho tới khi vừa khít. Và phần hay nhất: nó tự động cuộn!

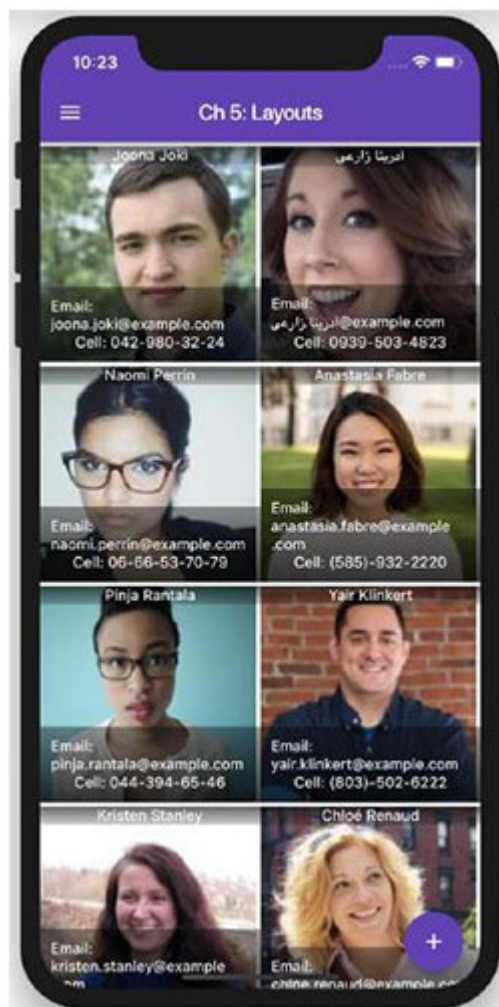
`GridView` có hai constructor: `GridView.extent()` và `GridView.count()`.

GridView.extent()

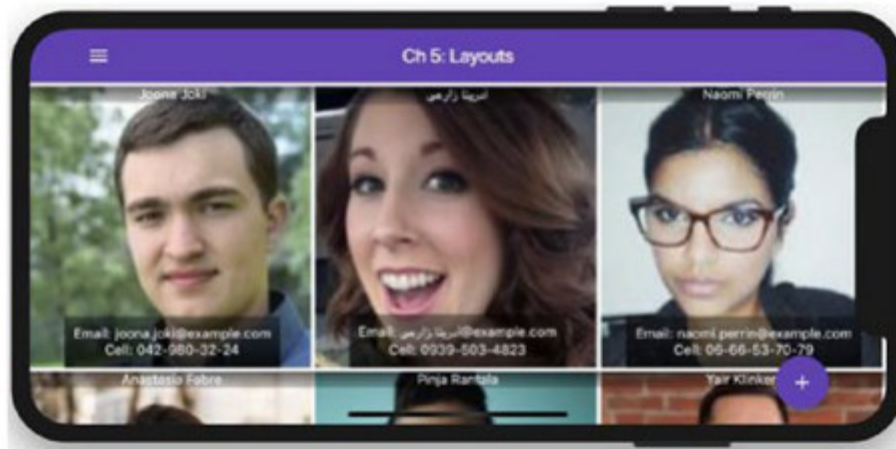
"Extent" ở đây chỉ chiều rộng tối đa của widget con. `GridView` chỉ cho phép các widget con lớn tới kích thước đó. Nếu chúng cố lớn hơn, nó đặt thêm một phần tử vào hàng đó và thu nhỏ tất cả lại cho tới khi vừa khít toàn bộ chiều rộng. Hãy xem ví dụ này:

```
Widget build(BuildContext context) {
  return GridView.extent(
    maxCrossAxisExtent: 300.0,
    children: people
      .map<Widget>((dynamic person) => PersonCard(person))
      .toList(),
  );
}
```

Để ý trong Hình 6-26 và 6-27 cách các container tự đổi kích thước xuống dưới 300. `GridView` quyết định rằng nó vừa được hai cái theo chiều ngang ở hướng dọc màn hình. Nhưng khi xoay ngang, hai cái đó sẽ phải to hơn 300 nên nó đặt ba cái mỗi hàng.



Hình 6-26. `GridView.extent()` ở hướng dọc

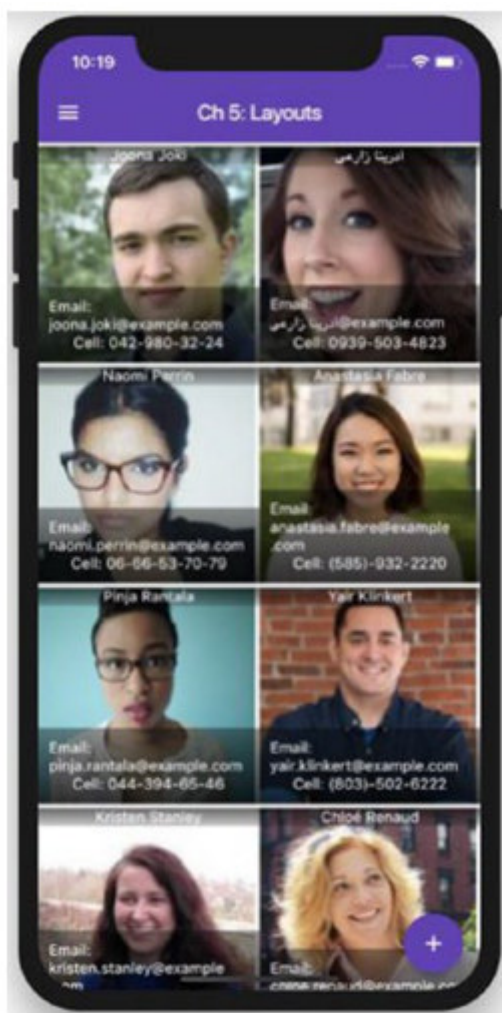


Hình 6-27. Vẫn `GridView.extent()` đó ở hướng ngang

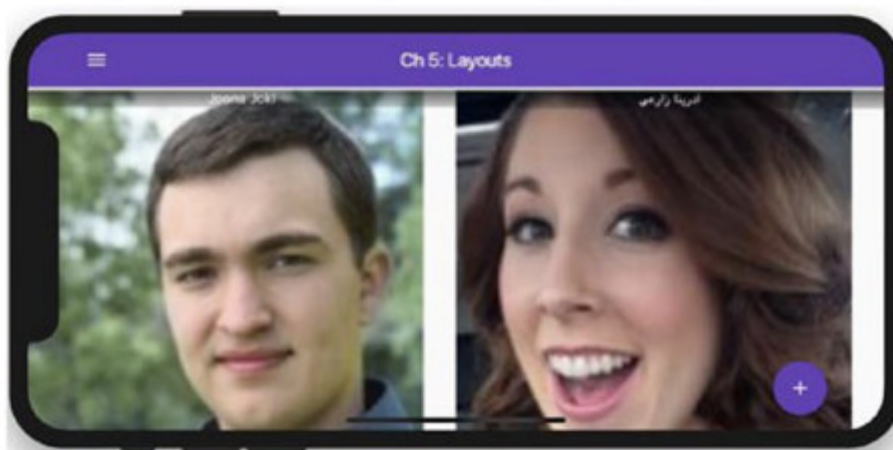
GridView.count()

Với constructor `count()`, bạn chỉ định muốn bao nhiêu cột bất kể hướng màn hình. `GridView` lo việc đổi kích thước nội dung cho vừa. Trong ví dụ sau, chúng ta bảo `GridView.count()` rằng ta muốn hai cột bất kể hướng nào, và `GridView` sẽ định kích thước widget con để vừa đúng hai cái theo chiều ngang (Hình 6-28 và 6-29):

```
Widget build(BuildContext context) {
  return GridView.count(
    crossAxisCount: 2,
    children: people
      .map<Widget>((dynamic person) => PersonCard(person))
      .toList(),
  );
}
```

Hình 6-28. `GridView.count()` ở hướng dọc



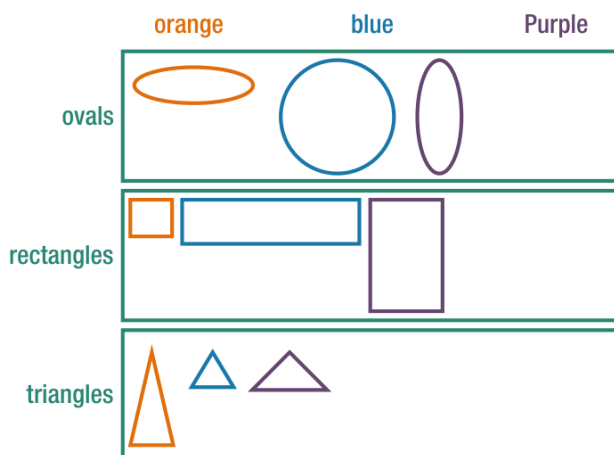
Hình 6-29. Vẫn `GridView.count()` đó ở hướng ngang

`GridView.extent()` có lẽ hữu ích hơn, vì khi thiết bị ở hướng dọc thì có thể bạn có hai cột, nhưng khi xoay ngang thì bạn nhét được ba cột mà nội dung vẫn vừa.

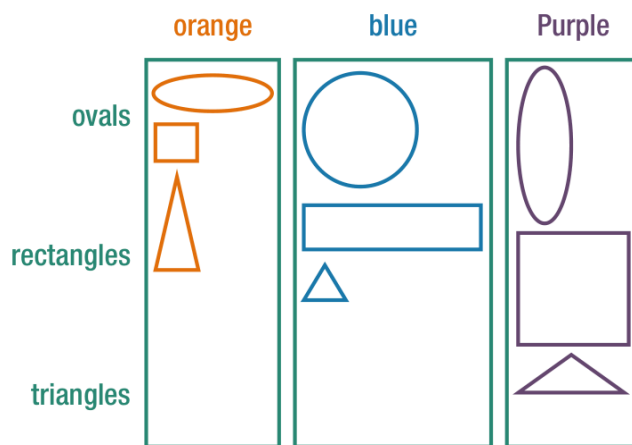
Widget Table

`GridView` rất tốt khi hiển thị widget theo hàng cột có xuống dòng. Phần "xuống dòng" nghĩa là bạn thực sự không quan tâm widget con nào rơi vào hàng nào, cột nào.

`Row` và `Column` tốt nhất khi bạn **có** quan tâm widget con nằm ở hàng nào, cột nào. Chúng cứng nhắc khi bạn muốn chúng cứng nhắc. Tiếc là các cột không "nói chuyện" được với nhau nên chúng thường bị lệch (Hình 6-30 và 6-31).



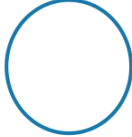
Hình 6-30. Các Row thì chạy đúng nhưng các cột bị lệch nhau



Hình 6-31. Các Column thì chạy đúng nhưng các hàng bị lệch nhau

✦ **Chú thích cho Hình 6-30 đến 6-32:** Nhãn trong hình là dữ liệu ví dụ — cột là *orange*, *blue*, *Purple* (màu), hàng là *ovals* (hình bầu dục), *rectangles* (hình chữ nhật), *triangles* (hình tam giác).

Widget `Table` khắc phục vấn đề đó. Nó cứng nhắc như `Row` và `Column` lồng nhau, nhưng mỗi hàng và cột đều biết tới các hàng cột khác và xếp thẳng hàng đẹp mắt như `GridView` (Hình 6-32).

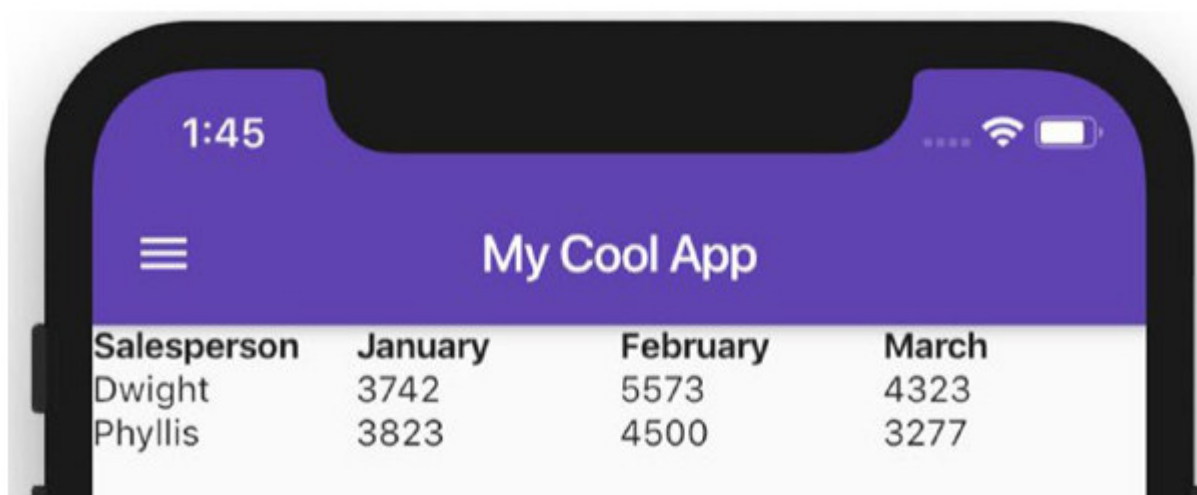
	orange	blue	Purple
ovals			
rectangles			
triangles			

Hình 6-32. *Table căn thẳng cả hàng lẫn cột*

Mọi widget `Table` đều có `children` là một `List` các widget `TableRow`. Và mỗi widget `TableRow` lại có `children` là một `List` các widget:

```
return Table(  
  children: <TableRow>[  
    TableRow(children: <Widget>[  
      Text('Salesperson', style: bold),  
      Text('January', style: bold),  
      Text('February', style: bold),  
      Text('March', style: bold),  
    ]),  
    TableRow(children: <Widget>[  
      Text('Dwight'),  
      Text('3742'),  
      Text('5573'),  
      Text('4323'),  
    ]),  
    TableRow(children: <Widget>[  
      Text('Phyllis'),  
      Text('3823'),  
      Text('4500'),  
      Text('3277'),  
    ]),  
  ],  
);
```

Đoạn code trên sẽ tạo ra Hình 6-33.



Hình 6-33. Widget `Table` căn thẳng hàng và cột cùng lúc

⚠️ Cẩn trọng

Ai đến từ nền tảng HTML đều biết rằng bố trí trang bằng thẻ `<table>` là chuyện làm được nhưng là ý tưởng tồi. `<table>` dành cho dữ liệu, không phải để bố cục. Trong Flutter cũng vậy. Bạn làm được, nhưng nói chung hãy tránh dùng `Table` để bố trí trang. Còn nếu bạn có dữ liệu thì `Table` là lựa chọn đúng.

Bất kể nội dung là gì, các cột của table đều được chia đều chiều rộng, trừ khi bạn ghi đè bằng property `columnWidths`. Đoạn sau sẽ cho cột đầu tiên 30% chiều rộng và chia đều 70% còn lại cho các cột khác:

```
return Table(
  columnWidths: {0: FractionColumnWidth(0.3)},
  children: <TableRow>[ /* ... */ ],
);
```

Còn gộp cột (span) thì sao? Ví dụ cho phần header của bảng. Tiếc là với `Table` của Flutter thì bạn chưa làm được — ít nhất là chưa. Nhưng hãy chờ xem, đã có feature request về việc gộp cột.

Ghi chú cập nhật: `Table` của Flutter đến nay **vẫn chưa hỗ trợ gộp cột (colspan)**. Nếu bạn cần bảng dữ liệu có tính năng phong phú hơn (sắp xếp, chọn hàng, header cố định), hãy dùng `DataTable` hoặc `PaginatedDataTable`.

Tổng kết

Tôi biết đây là một chương dài. Nhưng bố cục trong Flutter không chỉ cực kỳ quan trọng mà còn cực kỳ phức tạp, vì số lượng widget bố cục rất lớn và cách chúng tương tác với nhau. Nhưng vì hiểu được thuật toán sẽ giúp bạn tiết kiệm vô số công sức và đầu bút tại về sau, chúng tôi nghĩ nên trình bày nó thật kỹ. Chúng tôi hy vọng về lâu dài bạn sẽ đồng ý. Sau vài lần đọc lại chương này và thực hành với các widget, chúng tôi tin rằng bạn sẽ nắm chắc bố cục trong Flutter.

Tất nhiên, để có một app hoàn chỉnh, bạn cần tạo nhiều scene và có thể điều hướng giữa chúng. Làm thế nào? Chúng ta sẽ tìm hiểu ở chương tiếp theo.

-
1. Xem "The Paradox of Choice" tại https://wikipedia.org/wiki/The_Paradox_of_Choice ↩
 2. *Don't Make Me Think* của Steve Krug là một cuốn sách rất hay về tính khả dụng dựa trên lẽ thường. ↩
 3. Chi tiết hơn tại <http://bit.ly/flutterFittedBox> ↩

Chương 7: Navigation và Routing

Mọi app đều có khái niệm chuyển từ màn hình này sang màn hình khác. Người dùng bấm nút giỏ hàng và chúng ta chuyển tới màn hình giỏ hàng. Người dùng bấm nút "tiếp tục mua sắm" và chúng ta quay lại duyệt thêm sản phẩm. Một số developer gọi đó là **routing**. Số khác gọi là **navigation**. Dù bạn gọi thế nào, đây là một mảng mà Flutter làm rất dễ dàng, bởi chỉ có bốn cách điều hướng:

- **Stack** — Mỗi widget chiếm toàn màn hình. Người dùng chạm một nút để đi theo một quy trình định sẵn. Lịch sử được lưu lại, và họ có thể quay lại một cấp bằng nút back.
- **Drawer** — Phần lớn màn hình hiển thị một widget, nhưng ở cạnh trái có một ngăn kéo (drawer) ló ra. Khi họ bấm vào hoặc vuốt nó sang phải, nó trượt ra để lộ một menu lựa chọn. Bấm một lựa chọn sẽ đổi widget ở phần chính của màn hình.
- **Tab** — Một phần diện tích được dành cho một dãy tab ở trên hoặc dưới màn hình. Khi bạn bấm vào một tab, chúng ta hiển thị widget tương ứng với tab đó.
- **Dialog** — Tuy về mặt kỹ thuật không thuộc navigation, nhưng đây cũng là một cách để xem một widget khác, nên ta cứ tính vào. Dialog là các cửa sổ modal (còn gọi là pop-up) tồn tại cho tới khi người dùng đóng chúng.

Mỗi phương pháp này đều yêu cầu app của bạn có một `MaterialApp` làm widget tổ tiên. Hãy đi sâu vào từng cái, bắt đầu với stack navigation.

Stack navigation

Nếu bạn là developer có kinh nghiệm, bạn đã quen với queue và stack. Nếu chưa, để tôi giải thích ngắn gọn. Giả sử bạn làm việc trong bếp. Khi đĩa được rửa sạch, chúng được xếp chồng lên nhau, đúng không? Mỗi cái đĩa được đặt lên **đỉnh** chồng. Việc này gọi là **push** vào stack. Khi đến lúc phục vụ đồ ăn, bạn tự nhiên sẽ lấy cái đĩa được thêm vào sau cùng — cái nằm trên đỉnh chồng. Việc này gọi là **pop** ra khỏi đỉnh stack.

Navigation của Flutter hoạt động theo stack. Khi bạn muốn đưa người dùng tới một scene mới, bạn `push()` một widget lên đỉnh stack và người dùng nhìn thấy widget đó. Mỗi lần `push()`, bạn làm chồng scene cao thêm. Khi bạn muốn họ quay lại nơi họ vừa ở, bạn `pop()` scene cuối cùng ra khỏi đỉnh stack, và thứ lộ ra là gì? Chính là scene trước đó.

Với stack của Flutter, thường bạn sẽ định nghĩa trước các scene (còn gọi là **route**) và đặt tên cho từng cái. Việc này phải được làm ở cấp `MaterialApp` như sau:

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    title: 'Shopping App',  
    initialRoute: '/',  
    routes: {  
      '/': (BuildContext ctx) => LandingScene(),  
      '/browse': (BuildContext ctx) => Browse(),  
      '/product': (BuildContext ctx) => ViewProduct(),  
      '/checkout': (BuildContext ctx) => Checkout(),  
    },  
  );  
}
```

Lưu ý rằng khi dùng routing, chúng ta không dùng property `home` nữa. Thay vào đó, dùng property `initialRoute`.

Mẹo

Nếu `initialRoute` của bạn là `"/`, bạn có thể bỏ hẳn nó đi và Flutter sẽ mặc định hiểu là `"/`.

Điều hướng tiến và lùi

Để điều hướng người dùng tới một scene một cách thủ công, bạn dùng `Navigator.pushNamed(context, route)` và `Navigator.pop(context)`.

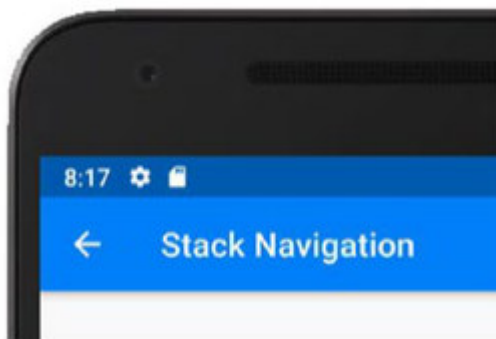
Để đẩy người dùng sang một route khác:

```
RaisedButton(  
  child: const Text('Check out'),  
  onPressed: () => Navigator.pushNamed(context, '/checkout'),  
),
```

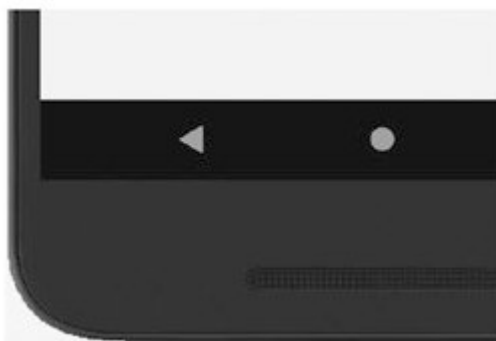
Khi họ xong việc và muốn quay lại:

```
RaisedButton(  
  child: const Text('Go back'),  
  onPressed: () => Navigator.pop(context),  
),
```

Nhưng khoan, còn nữa! Để ý rằng nếu bạn có một `Scaffold`, một mũi tên back sẽ tự động được thêm vào appbar (Hình 7-1). Khi chạm vào, nó hoạt động để quay lại. Và nếu người dùng của bạn dùng Android, nút back quen thuộc của Android cũng hoạt động (Hình 7-2).



Hình 7-1. Mũi tên back trong appbar



Hình 7-2. Nút back của Android hoạt động với stack

 Mẹo

Còn một kiểu routing khác không dùng bảng route định sẵn trong `MaterialApp`. Thay vào đó, bạn sinh route ngay tại chỗ:

```
Navigator.push<void>(
  context,
  MaterialPageRoute<void>(
    builder: (BuildContext context) => SecondRoute(),
  ),
);
```

Như bạn thấy, cách này phức tạp hơn kha khá. Nhưng nó phổ biến khi bạn muốn hiệu ứng chuyển cảnh tùy chỉnh, hoặc đơn giản là không muốn định nghĩa route trước.

 **Ghi chú cập nhật:** `RaisedButton` trong các ví dụ của chương này đã bị gỡ bỏ — hãy dùng `ElevatedButton`. Tương tự, `FlatButton` (xuất hiện ở phần Dialog bên dưới) đã được thay bằng `TextButton`.

Ngoài ra, cách định tuyến trình bày trong chương này (`Navigator 1.0`) vẫn hoạt động tốt và vẫn là cách nên học đầu tiên. Nhưng với app thực tế có deep link hoặc chạy trên web, cộng đồng hiện khuyến nghị dùng package `go_router` — nó xử lý URL, tham số đường dẫn và điều hướng lồng nhau gọn hơn nhiều.

Nhận kết quả sau khi một scene đóng lại

Với stack navigation, mỗi lần `pop()` đều trả về cho nơi đã gọi nó. Do đó, có thể trả về một giá trị từ mỗi scene. Việc này không quá phổ biến, nhưng cực kỳ hữu ích khi bạn dẫn người dùng qua một quy trình.

Giả sử bạn có một phần trong app quản lý một object user. Object user được định nghĩa trong `MyUserWidget`, và ta cung cấp một nút để sửa thông tin đăng nhập, một nút khác để sửa số điện thoại, và một nút nữa để sửa tài khoản Twitter. Khi người dùng chạm từng nút, ta có thể `push()` họ tới một route nơi họ thay đổi dữ liệu. Nếu vậy, ta cần trả dữ liệu đã đổi về cho `MyUserWidget`. Ta sẽ `push()` theo cách hơi khác, có một biến nhận giá trị trả về:

```
// Cần 'async' ở đây vì bên dưới chúng ta 'await'.
onPressed: () async {
  _user.twitterHandle =
    await Navigator.pushNamed(context, '/twitter');
},
```

📌 Lưu ý

Từ khóa `await` ngụ ý rằng `pushNamed()` trả về một `Future`. Cũng lưu ý rằng bất kỳ giá trị nào được trả về từ route này sẽ được gán cho `_user.twitterHandle`.

Vậy giá trị này được trả về bằng cách nào? Bằng `pop()` chứ sao!

```
Navigator.pop<String>(context, twitterHandle);
```

`Navigator.pop()` được nạp chồng. Nếu bạn thêm tham số thứ hai, nó sẽ được trả về cho widget đã gọi `push()` ban đầu. Trong ví dụ trên, `twitterHandle` sẽ được trả về.

App có cấu trúc nông thì rất hợp với `push()` và `pop()`. Nhưng app của bạn có thể có cây điều hướng sâu với rất nhiều lựa chọn. Những app như vậy thường không phù hợp với việc có cả tá nút để `push()` và `pop()`. Thay vào đó, chúng nên có một menu điều hướng. Flutter cung cấp cho chúng ta hai kiểu. App đơn giản hơn có thể dùng tab. App phức tạp hơn sẽ dùng drawer. Hãy xem drawer trước.

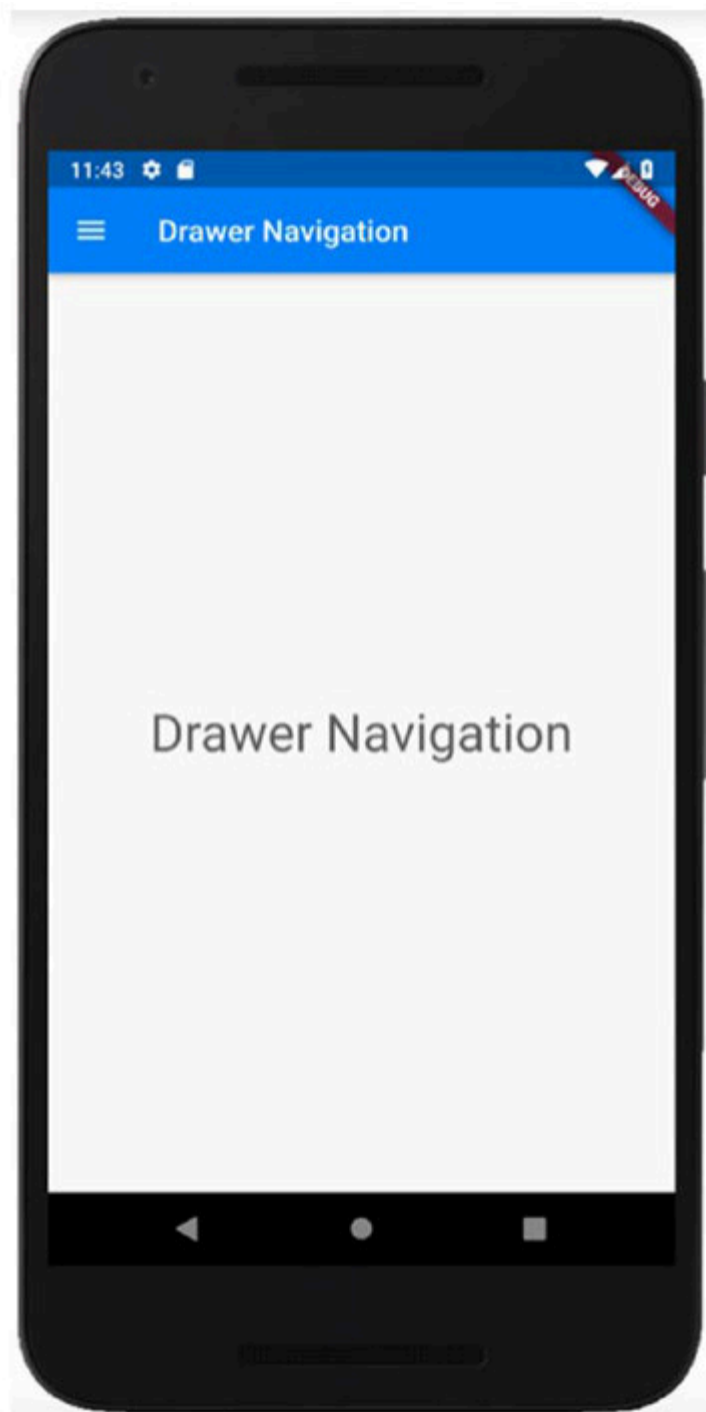
Drawer navigation

Drawer rất tốt khi chúng ta có nhiều lựa chọn điều hướng — nhiều đến mức không nhét vừa vào tab.

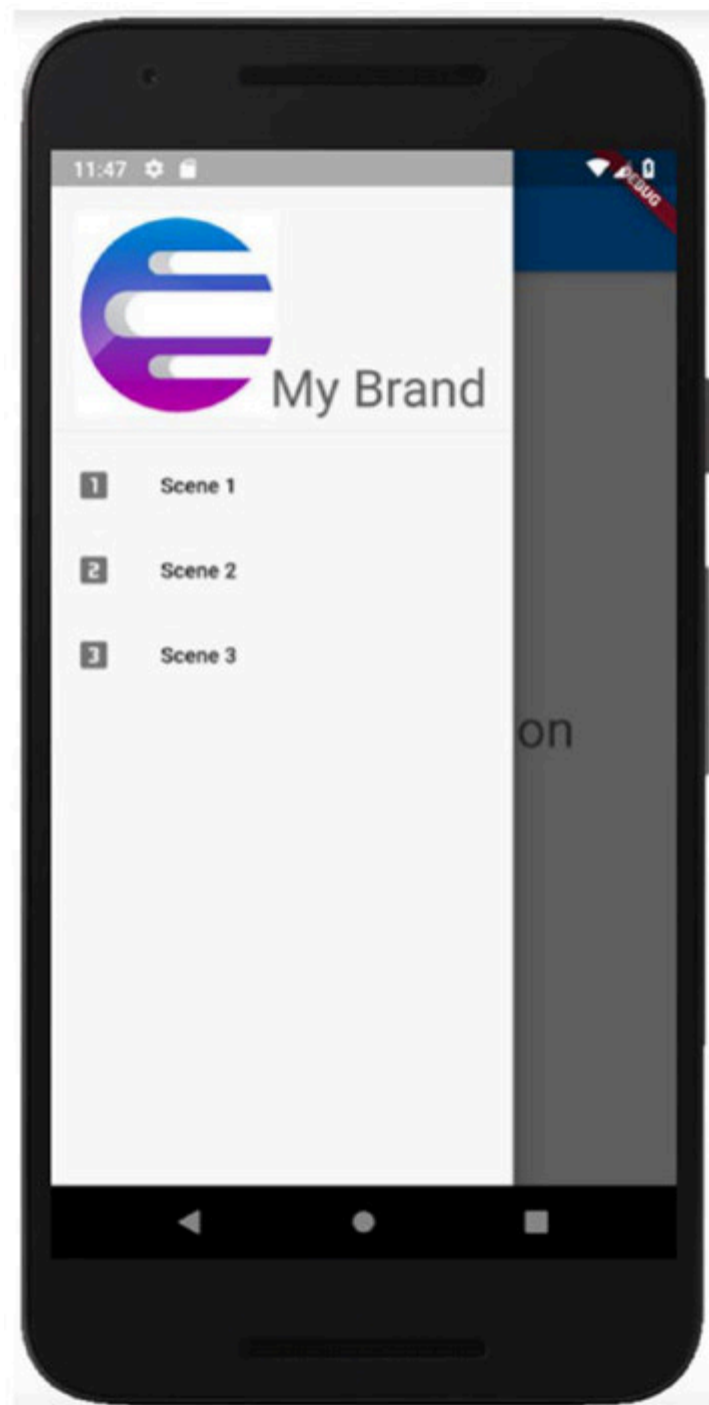
Ở rất nhiều website responsive, bạn thấy một menu chạy ngang đầu trang với các liên kết tới trang khác. Rồi khi site được xem trên thiết bị nhỏ hoặc trình duyệt hẹp, menu đó được thay bằng menu hamburger — khi bấm vào sẽ thả xuống một menu chứa đúng các lựa chọn đó. Về cơ bản, đây là cách site phản ứng với diện tích màn hình hạn chế: cung cấp các lựa chọn menu nhưng giấu đi cho tới khi người dùng cần.

Vì hầu hết điện thoại vốn đã có diện tích màn hình hạn chế, bạn có thể chọn đặt các lựa chọn menu vào một drawer để không ngốn mất diện tích quý giá đó cho tới khi người dùng sẵn sàng xem (Hình 7-3). Khi

họ sẵn sàng, họ bấm vào menu hamburger quen thuộc (biểu tượng ba gạch) và các lựa chọn trượt ra từ bên trái (Hình 7-4). Khi người dùng chọn một cái, chúng ta `Navigator.push()` họ tới route mới.



Hình 7-3. Một scene khi drawer đang đóng



Hình 7-4. Một scene khi drawer đang mở

Widget Drawer

Bạn cần một widget `Drawer` — widget dựng sẵn của Flutter có khả năng trượt ra, trượt vào và chứa các lựa chọn menu. Khi dùng drawer, bạn luôn đặt nó vào property `drawer` của `Scaffold`, như sau:

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Drawer Navigation'),  
    ),  
    body: const Text('DrawerNavigation'),  
    drawer: Drawer(  
      child: ListView(  
        children: <Widget>[  
          Text('Option 1'),  
          Text('Option 2'),  
          Text('Option 3'),  
        ],  
      ),  
    ),  
  );  
}
```

Để ý rằng khi `Scaffold` của bạn có drawer, icon hamburger của nó **thay thế** nút back của appBar. Bạn không thể thấy cả hai nút cùng lúc trừ khi bạn tự tạo nút thủ công. Vì vậy, tuy drawer navigation và stack navigation có thể phối hợp với nhau, việc đó có thể khá vụng nếu bạn không cẩn thận. Một ví dụ về sự phối hợp rất tốt là đặt `Drawer` ở cấp cao nhất, rồi dùng stack navigation ở mọi cấp bên dưới.

 Mẹo

Bạn muốn có một drawer nhất quán trên toàn bộ app? Nếu vậy, thường chúng ta đặt một `Scaffold` ở mỗi scene và đưa drawer vào đó. Do đó, tốt nhất là tách `Drawer` ra thành một widget riêng rồi đưa vào dùng:

```
return Scaffold(  
  appBar: AppBar(  
    title: const Text('Drawer Navigation'),  
  ),  
  body: const Text('DrawerNavigation'),  
  drawer: MyDrawer(),  
);
```

Đổ nội dung vào drawer

Thêm drawer thì dễ. Cái khó là đưa các mục vào drawer rồi làm cho chúng điều hướng sang widget khác. Lưu ý rằng `Drawer` có property `child` chỉ nhận **một** widget. Để có nhiều widget con trong drawer, bạn dùng một widget hỗ trợ điều đó, ví dụ `Column` (không cuộn) hoặc `ListView` (cuộn được).

Dù chọn cái nào, bạn cũng muốn đặt thứ gì đó có thể chạm được, vì để điều hướng bạn sẽ gọi `Navigator.push()` hoặc `Navigator.pushNamed()` giống như đã làm với stack navigation.

 Mẹo

Có một widget rất hay tên là `DrawerHeader`, được thiết kế để chiếm một vùng lớn ở đầu drawer. Nó rất hợp để đặt logo hoặc thông tin thương hiệu, kiểu như nhắc người dùng nhớ họ đang ở trong app nào. Nó chỉ mang tính thẩm mỹ nhưng trông thực sự đẹp.

```
return Drawer(  
  child: ListView(  
    children: <Widget>[  
      DrawerHeader(  
        child: Stack(  
          children: <Widget>[  
            Image.asset(  
              'lib/assets/images/BrandLogo.jpg',  
            ),  
            Container(  
              alignment: Alignment.bottomRight,  
              child: Text(  
                'My Brand',  
                style: Theme.of(context).textTheme.display1,  
              ),  
            ),  
          ],  
        ),  
      ),  
      ListTile(  
        leading: const Icon(Icons.looks_one),  
        title: const Text('Widget 1'),  
        onTap: () {  
          Navigator.pushNamed(context, '/widget1');  
        },  
      ),  
      ListTile(  
        leading: const Icon(Icons.looks_two),  
        title: const Text('Widget 2'),  
        onTap: () {  
          Navigator.pushNamed(context, '/widget2');  
        },  
      ),  
      ListTile(  
        leading: const Icon(Icons.looks_3),
```

```

        title: const Text('Widget 3'),
        onTap: () {
          Navigator.pushNamed(context, '/widget3');
        },
      ),
    ],
  ),
);

```

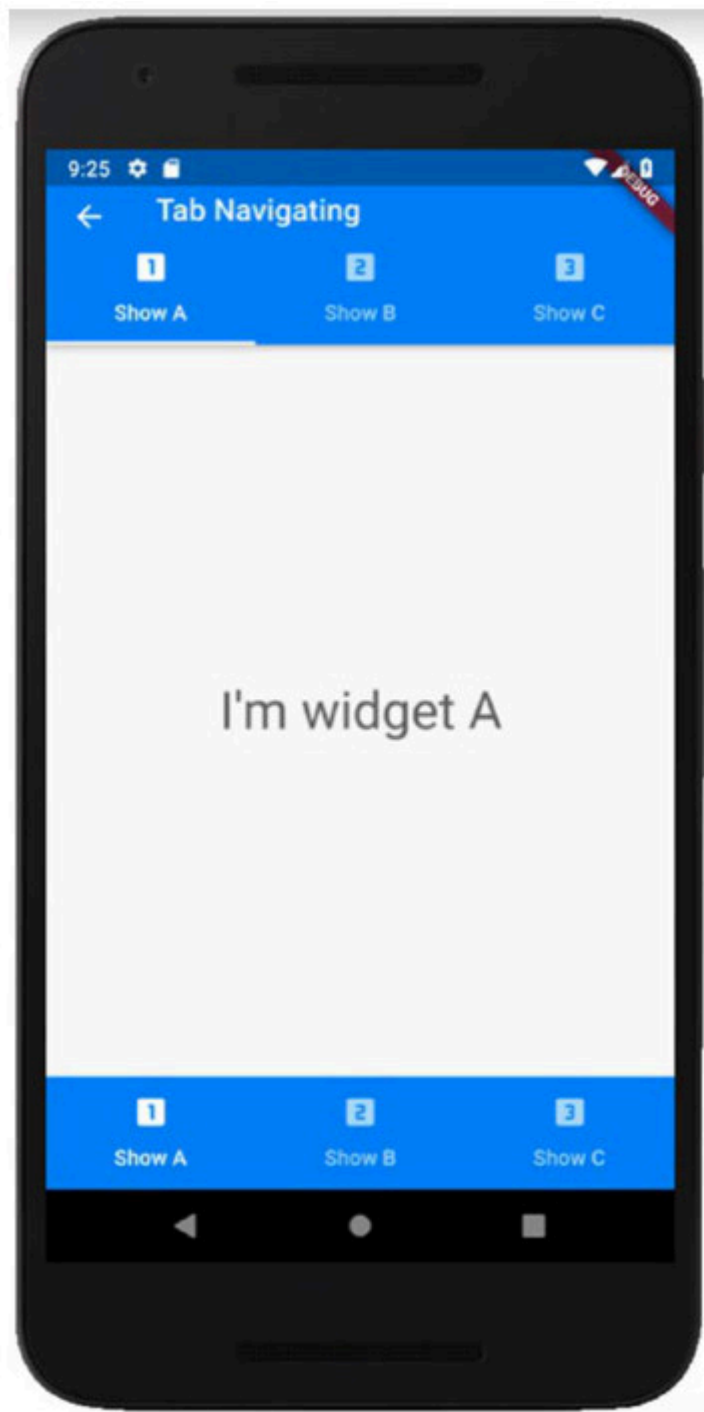
 **Ghi chú cập nhật:** Các tên style chữ cũ như `textTheme.display1` **đã bị gỡ bỏ**. Bộ tên mới theo Material 3 là `displayLarge`, `displayMedium`, `displaySmall`, `headlineLarge/Medium/Small`, `titleLarge/Medium/Small`, `bodyLarge/Medium/Small`, `labelLarge/Medium/Small`. Tương đương gần nhất của `display1` là `headlineMedium`:

```
style: Theme.of(context).textTheme.headlineMedium,
```

Drawer navigation cũng hay đấy, nhưng các chuyên gia UX có vài vấn đề với nó. Họ cho rằng¹ nó làm giảm mạnh tính khả dụng của app, khiến app khó khám phá hơn và khó dùng hơn. Họ nói vấn đề nằm ở chỗ các lựa chọn bị ẩn đi cho tới khi người dùng chủ động tìm. Phản đối của họ có thể được giải quyết bằng một thành phần luôn hiển thị. Nhân nói tới điều đó...

Tab navigation

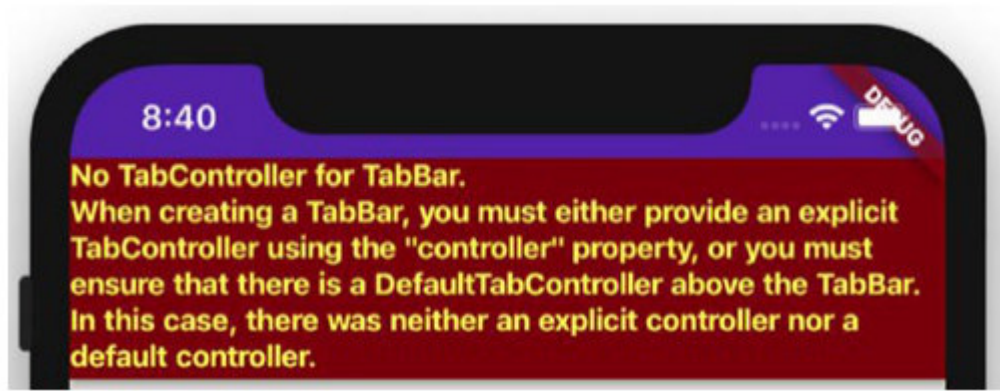
Như bạn hình dung, một hệ thống tab ghép N tab với N widget. Khi người dùng bấm tab 1, họ thấy widget 1, và cứ thế (Hình 7-5). Việc ghép này được thực hiện bằng widget `TabBar`, widget `TabBarView` và một `TabController`.



Hình 7-5. Một tabbar ở trên và một ở dưới

TabController

`TabController` là phần khó thấy nhất. Chỉ cần biết rằng bạn bắt buộc phải có một cái, nếu không bạn sẽ gặp lỗi như trong Hình 7-6.



Hình 7-6. *Khi bạn quên TabController*

Cách dễ nhất để tạo một cái là bọc mọi thứ trong một `DefaultTabController()` với property `length`. Xong vấn đề. Phần này khá đơn giản — đơn giản đến mức bạn có thể tự hỏi vì sao Flutter không tự tạo giúp bạn. Nếu bạn nghĩ vậy thì bạn cũng không sai:

```
Widget build(BuildContext context) {  
  return DefaultTabController(  
    length: 3,  
    child: Scaffold(  
      // ...  
    ),  
  );  
}
```

TabBarView

Tiếp theo bạn muốn thêm một widget `TabBarView`. Widget này chứa các widget sẽ được hiển thị khi người dùng bấm một tab, và xác định chúng sẽ hiển thị ở đâu. Thường thì đó là toàn bộ phần còn lại của màn hình, nhưng bạn có thể đặt widget phía trên `TabBarView`, phía dưới nó, hay thực ra là ở bất cứ đâu quanh nó:

```
child: Scaffold(  
  body: TabBarView(  
    children: <Widget>[  
      WidgetA(),  
      WidgetB(),  
      WidgetC(),  
    ],  
  ),  
)
```

TabBar và Tab

Cuối cùng chúng ta định nghĩa chính các tab. Tab có thể chứa chữ, hoặc icon, hoặc cả hai. Đây là một `TabBar` với ba tab, mỗi tab có cả icon lẫn chữ:

```
child: Scaffold(  
  appBar: AppBar(  
    title: const Text('Tab Navigating'),  
    bottom: TabBar(  
      tabs: const <Widget>[  
        Tab(icon: Icon(Icons.looks_one), child: Text('Show A'))),  
        Tab(icon: Icon(Icons.looks_two), child: Text('Show B'))),  
        Tab(icon: Icon(Icons.looks_3), child: Text('Show C'))),  
      ],  
    ),  
  ),  
  // ...  
)
```

Cẩn trọng

Có sự tương ứng một-một giữa mỗi tab và mỗi widget con của `TabBarView`; chúng được khớp theo **vị trí**. Bạn phải có số tab bằng đúng số widget bên trong `TabBarView`.

TabBar ở dưới cùng

Ở trên chúng ta chọn đặt `TabBar` vào `appBar`, và tất nhiên nó xuất hiện ở đầu màn hình. Nhưng đôi khi thiết kế của bạn cần các tab xuất hiện ở dưới cùng màn hình. Việc đó dễ, vì `Scaffold` có một property tên là `bottomNavigationBar` được tạo ra để chứa một `TabBar`:

```
child: Scaffold(  
  // ...  
  bottomNavigationBar: Material(  
    color: Theme.of(context).colorScheme.primary,  
    child: TabBar(tabs: const <Widget>[  
      Tab(icon: Icon(Icons.looks_one), child: Text('Show A')),  
      Tab(icon: Icon(Icons.looks_two), child: Text('Show B')),  
      Tab(icon: Icon(Icons.looks_3), child: Text('Show C')),  
    ]),  
  ),  
),
```

Lưu ý

`TabBar` mặc định có diện mạo chữ sáng trên nền tối. Vì vậy, khi bạn đặt `TabBar` lên trên nền sáng, chữ có thể khó nhìn (sáng trên sáng). Để khắc phục, hãy bọc `TabBar` trong một widget `Material` có màu nền tối hơn như chúng ta vừa làm ở trên.

Widget Dialog

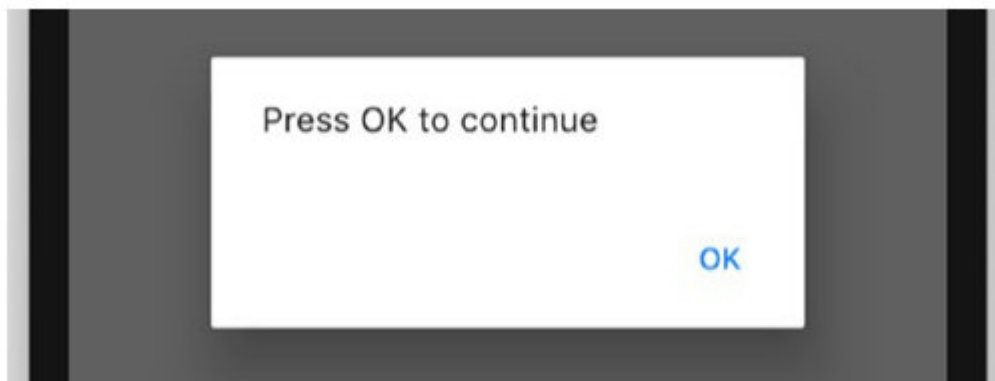
Nhóm navigation cuối cùng của chúng ta có lẽ không thực sự là navigation — đó là **dialog**. Theo một nghĩa nào đó, bạn đang hiển thị một widget khác nên... là navigation? Nhưng theo nghĩa khác, bạn chỉ đang hiển thị một pop-up nên... không phải navigation?

Dù thế nào, dialog là thứ rất phổ biến và chúng ta nên trình bày. Vì chúng không hợp với chỗ nào khác trong sách, hãy tạm coi chúng là một chủ đề navigation.

`showDialog()` và `AlertDialog`

`showDialog()` là một method dựng sẵn của Flutter. Bạn phải cung cấp một `context` và một method `builder` trả về một Widget, thường là `SimpleDialog` hoặc `AlertDialog`. `AlertDialog` có tham số

`actions` — một `List` gồm (thường là) các `FlatButton` cho phép người dùng đóng dialog (Hình 7-7).



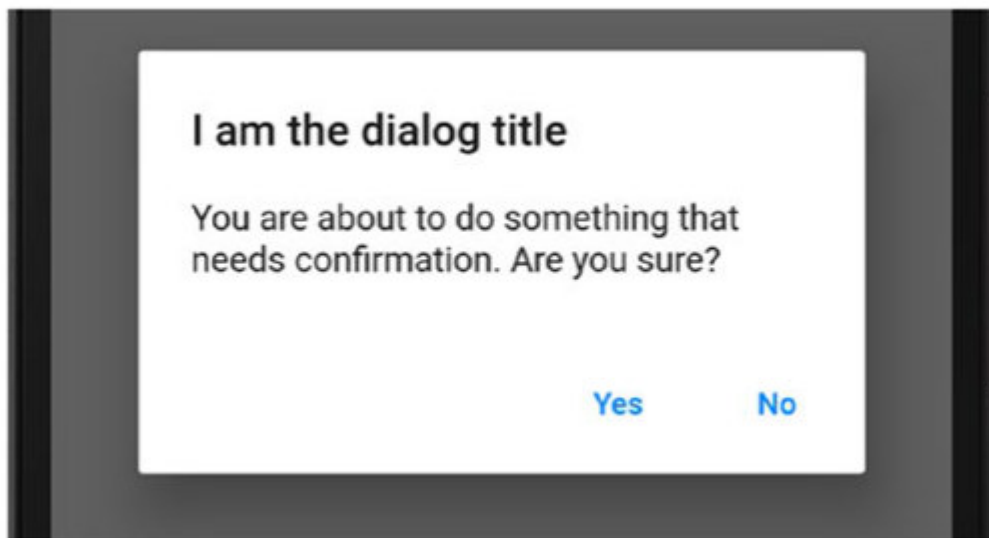
Hình 7-7. Một `AlertDialog` đơn giản

```
RaisedButton(  
  child: const Text('I am a button. Press me'),  
  onPressed: () => showDialog<void>(  
    context: context,  
    builder: (BuildContext context) {  
      return AlertDialog(  
        content: const Text('Press OK to continue'),  
        actions: <Widget>[  
          FlatButton(  
            child: const Text('OK'),  
            onPressed: () => Navigator.pop(context)),  
        ],  
      );  
    },  
  ),  
),
```

Cách này trông phức tạp hơn mức cần thiết. Và đây mới là dạng đơn giản nhất! Nó còn phức tạp hơn nếu bạn muốn cho người dùng các lựa chọn.

Nhận phản hồi từ Dialog

`showDialog()` trả về một `Future<T>`, nghĩa là bạn có thể để nó trả một giá trị về cho nơi gọi. Giả sử bạn muốn người dùng trả lời có hoặc không (Hình 7-8).



Hình 7-8. *AlertDialog trả về một giá trị*

Bạn có thể tạo dialog và xử lý phản hồi như sau:

```

RaisedButton(
  child: const Text('Get a response'),
  onPressed: () async {
    // builder trả về lựa chọn của người dùng ở đây.
    // Vì nó là Future<String>, ta 'await' để
    // chuyển nó thành String
    String response = await showDialog<String>(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          content: const Text('Are you sure?'),
          actions: <Widget>[
            FlatButton(
              child: const Text('Yes'),
              // Trả về "Yes" khi đóng.
              onPressed: () => Navigator.pop(context, 'Yes')),
            FlatButton(
              child: const Text('No'),
              // Trả về "No" khi đóng.
              onPressed: () => Navigator.pop(context, 'No')),
          ],
        );
      },
    );
    // Làm gì đó với phản hồi mà ta vừa 'await' ở trên.
    print(response);
  },
),

```

Mẹo

Đúng như tên gọi, widget `SimpleDialog` là phiên bản đơn giản hơn của `AlertDialog`. Nó không có `actions` và có ít tham số constructor hơn, ví dụ `titleTextStyle`, `contentTextStyle` và tương tự. Hãy dùng nó chủ yếu khi bạn không cần người dùng trả lời mà chỉ cần thông báo.

Có thể kết hợp các phương pháp navigation

Tuy bạn có thể dùng stack navigation để đi tới một widget có drawer, rồi từ đó tới một widget có tab, bạn nên cẩn thận. Các phương pháp không hề xung khắc, nhưng chúng có thể trở nên rất phức tạp khi trộn lẫn!

Ví dụ, nếu bạn dùng stack navigation qua `push()` để tới một widget có drawer, nút back trong appbar sẽ không còn nữa. Android có nút back mềm ở dưới cùng, nhưng iOS thì không. Vậy là người dùng bị kẹt lại, không có cách nào quay ra.

Một ví dụ khác: `TabBarView` có các widget, nhưng những widget này được "chứa" bên trong nên chúng không nên có `Scaffold`. Nếu bạn thử điều hướng tới chính widget đó bằng một trong hai phương pháp còn lại, bạn sẽ không có cách nào quay lại — không có drawer để mở và không có nút back để chạm. Một lần nữa, người dùng bị kẹt.

Chúng tôi khuyên bạn chỉ nên dùng hai kiểu và giữ các cấp cho nhất quán. Ví dụ, khá phổ biến là cho người dùng trải nghiệm điều hướng bằng tab, và trong mỗi tab thì dùng stack navigation. Nhưng phức tạp hơn thế nữa thì bạn sẽ khá vất vả đấy.

1. <http://bit.ly/HamburgerNav> ↗

Chương 8: Tạo kiểu cho Widget

Việc tạo kiểu (styling) cho widget không hoàn toàn xa lạ với bạn. Chúng ta đã chạm tới vài tính năng styling nhỏ ở các chương trước, và bạn cũng đã thấy các kỹ thuật styling trong những đoạn code mẫu. Nhưng đây mới là chương chúng ta đào sâu vào styling. Cuối cùng thì chúng ta cũng được làm cho widget của mình trông đẹp, chứ không chỉ chạy tốt.

Chúng ta đã bàn về bố cục ở Chương 6, nên đã biết cách đặt vị trí và kích thước widget. Đó không phải styling. Styling là những thứ còn lại ảnh hưởng tới diện mạo của widget. Những thứ như:

- Màu sắc
- Diện mạo chữ (font, độ đậm, gạch chân, v.v.)
- Đường viền (độ dày, kiểu nét, bo góc)
- Ảnh nền
- Áp dụng hình dạng cho một `Container`

Chúng ta sẽ bàn về những thứ này cùng với một vài widget Flutter nằm ở ranh giới giữa bố cục và styling: `Card` và `Stack`. Cuối cùng, chúng ta sẽ nói về best practice là áp dụng style mặc định hàng loạt bằng **Theme**.

Nhưng trước hết, vài lời về triết lý styling trong Flutter.

Tư duy theo kiểu Flutter

Có lẽ bạn đã thấy Flutter vay mượn những ý tưởng hay nhất từ Android, iOS và web development, đặc biệt là từ React và tính linh hoạt của JavaScript. Nhưng nó không sao chép y nguyên kỹ thuật của họ. Flutter làm theo cách riêng, và sẽ là sai lầm nếu đẩy phép so sánh với web đi quá xa. Hiểu được Flutter khác biệt ở đâu sẽ có lợi cho chúng ta.

Thứ nhất, Google đã làm cho Material Design¹ trở nên rất phổ biến trong phát triển Android và trên khắp Web. Và tuy diện mạo mặc định của Flutter tuân theo Material Design, đừng để ai nói với bạn rằng bạn **buộc** phải dùng nó. Đó là một hiểu lầm phổ biến nhưng không đúng.

Thứ hai, styling của Flutter **không phải** CSS. Trong khi CSS có một số property được truyền xuống cho widget con, style của Flutter **không được kế thừa**. Ví dụ, bạn không thể đặt một font family trên widget

tùy chỉnh của mình rồi kỳ vọng tất cả `Text`, `TextField` và `button` bên dưới nó đột nhiên hiển thị bằng font đó. Để làm được điều đó, bạn có thể dùng `Theme` — chúng tôi sẽ chỉ bạn cách ở cuối chương này.

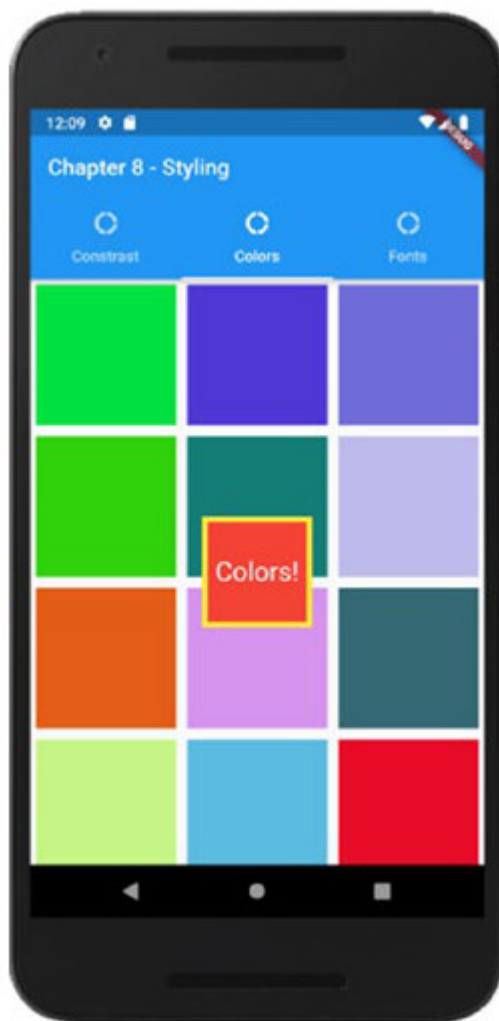
Cuối cùng, Flutter — cũng như Dart — rất dài dòng: cần khá nhiều code để diễn đạt một thứ mà bạn nghĩ lẽ ra phải đơn giản. Và không có gì ngạc nhiên, styling cũng vậy. Hãy yên tâm rằng những người rất giỏi đã viết ra Flutter và họ có lý do chính đáng cho sự dài dòng đó — an toàn và đầy đủ chỉ là hai trong số đó. Đây là điều không tránh khỏi. Chúng tôi chỉ muốn bạn biết trước và chuẩn bị tinh thần.

Vài lời về màu sắc

Hầu hết style của Flutter được áp dụng rất hẹp; chúng chỉ có nghĩa trong những tình huống được định nghĩa chặt chẽ. Ngược lại, màu sắc được áp dụng gần như ở mọi nơi (Hình 8-1). Đường viền, chữ, nền, icon, `button` và bóng đổ đều có màu. Và tất cả đều được chỉ định theo cùng một cách.

Ví dụ, đây là chữ `Text` màu trắng trong một container màu đỏ với viền vàng, và tất cả những widget đó đều được tô màu theo cùng cú pháp `color: Colors.somethingOrOther`:

```
child: Container(  
  child: Text(  
    'Colors!',  
    style: TextStyle(color: Colors.white),  
  ),  
  decoration: BoxDecoration(  
    color: Colors.red,  
    border: Border.all(  
      color: Colors.yellow,  
    ),  
  ),  
),
```



Hình 8-1. Màu sắc có mặt ở khắp nơi trong Flutter

Còn những khối màu ở nền kia? Chúng được tạo ra như thế này:

```
List<Widget> _randomColors() {  
  Random rnd = Random();  
  return List.generate(25,  
    (int i) => Container(  
      color: Color.fromRGBO(  
        rnd.nextInt(255), rnd.nextInt(255), rnd.nextInt(255), 1.0),  
    ));  
}
```

Vậy là bạn có thể tạo ra bất kỳ màu nào trong hơn 16 triệu màu bằng `Color.fromRGBO(red, green, blue, opacity)`, trong đó mỗi giá trị RGB là một số từ 0 đến 255, còn `opacity` là `1.0` cho đục hoàn toàn và

0.0 cho trong suốt hoàn toàn.

Nếu bạn đến từ nền tảng web, có lẽ bạn thấy thoải mái hơn khi tạo màu bằng số hex. Cách này cũng chạy:

```
color: Color(0xFFFF7F00),
```

⚠️ Cảnh trọng

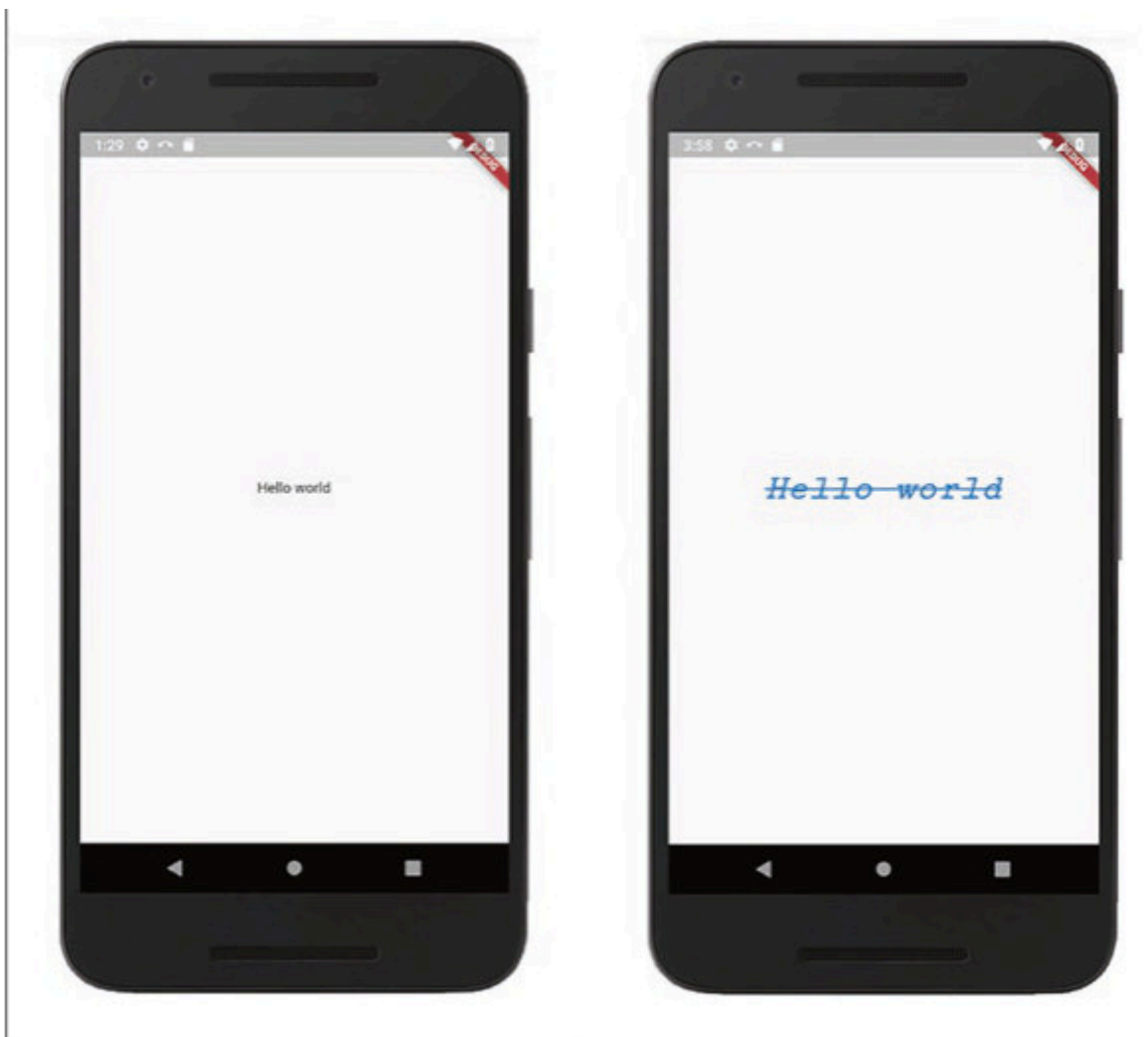
Hãy cẩn thận. Số hex đó thực chất là **"ARGB"**, trong đó hai ký tự hex đầu tiên là kênh alpha (độ trong suốt). Nếu bạn quên nó, ví dụ viết `Color(0xFFF700)`, bạn sẽ vẽ ra thứ trong suốt hoàn toàn và chẳng bao giờ nhìn thấy nó. Chỉ cần nhớ: nếu màu của bạn không hiện ra, hãy lấy số hex kiểu web thông thường và thêm `"FF"` vào đầu.

Tạo kiểu cho Text

Có hai chủ đề liên quan tới diện mạo của `Text`: **TextStyle** và **font tùy chỉnh**. Chúng ta sẽ xử lý chúng trong một hai trang tới, nhưng hãy chờ tới cuối chương, nơi chúng ta bàn về một cách tốt hơn để đặt những property này — **Theme**.

TextStyle

Widget `Text` có property `style` nhận vào một object `TextStyle` (Hình 8-2).



Hình 8-2. Có và không có style

Bạn chỉ cần đặt property `style` bằng một instance của widget `TextStyle` rồi đặt các property. `TextStyle` hỗ trợ khoảng 20 property. Đây là những cái hữu ích nhất:

- **color** — Bất kỳ màu nào trong hơn 16 triệu màu hợp lệ
- **decoration** — `TextDecoration.underline`, `overline`, `lineThrough`, `none`
- **fontSize** — Kiểu `double`. Chiều cao ký tự tính bằng pixel. Mặc định là `14.0` pixel
- **fontStyle** — `FontStyle.italic` hoặc `normal`
- **fontWeight** — `FontWeight.w100` đến `w900`. Hoặc `bold` (chính là `w700`) hoặc `normal` (chính là `w400`)
- **fontFamily** — Một chuỗi

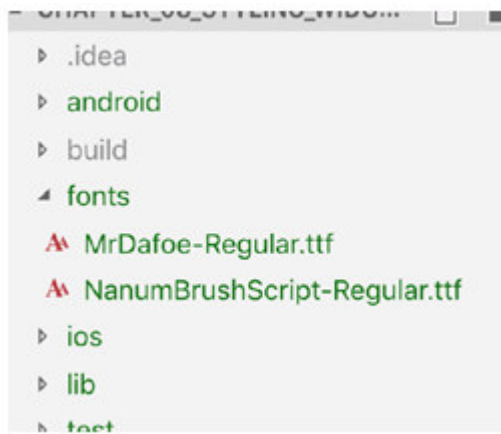
`fontFamily` là một chủ đề lớn hơn. Có một số font có sẵn như Courier, Times New Roman, serif và nhiều font khác. Bao nhiêu font nữa? Điều đó phụ thuộc vào loại thiết bị đang chạy app. Vì chúng ta không kiểm

soát được thiết bị của người dùng, best practice là bám vào font family mặc định trừ khi bạn cài và dùng font tùy chỉnh. Hãy nói về cách làm điều đó.

Font tùy chỉnh

Một số designer yêu cầu font tùy chỉnh khi họ thiết kế màn hình. Hóa ra với Flutter, việc dùng font tùy chỉnh khá dễ và chúng hoạt động trên mọi nền tảng. Có ba bước:

1. Tải về các file font ở định dạng `ttf`, `woff` hoặc `woff2`. Chúng thường được lưu trong một thư mục cấp gốc tên là `fonts`, nhưng tên là tùy bạn (Hình 8-3).



Hình 8-3. Font thường được lưu trong thư mục tên là `fonts`

Mẹo

Bạn có thể tìm nhiều font miễn phí và rất đẹp tại <http://fonts.google.com>. Tìm theo kiểu chữ, xem mẫu và tải về dễ dàng.

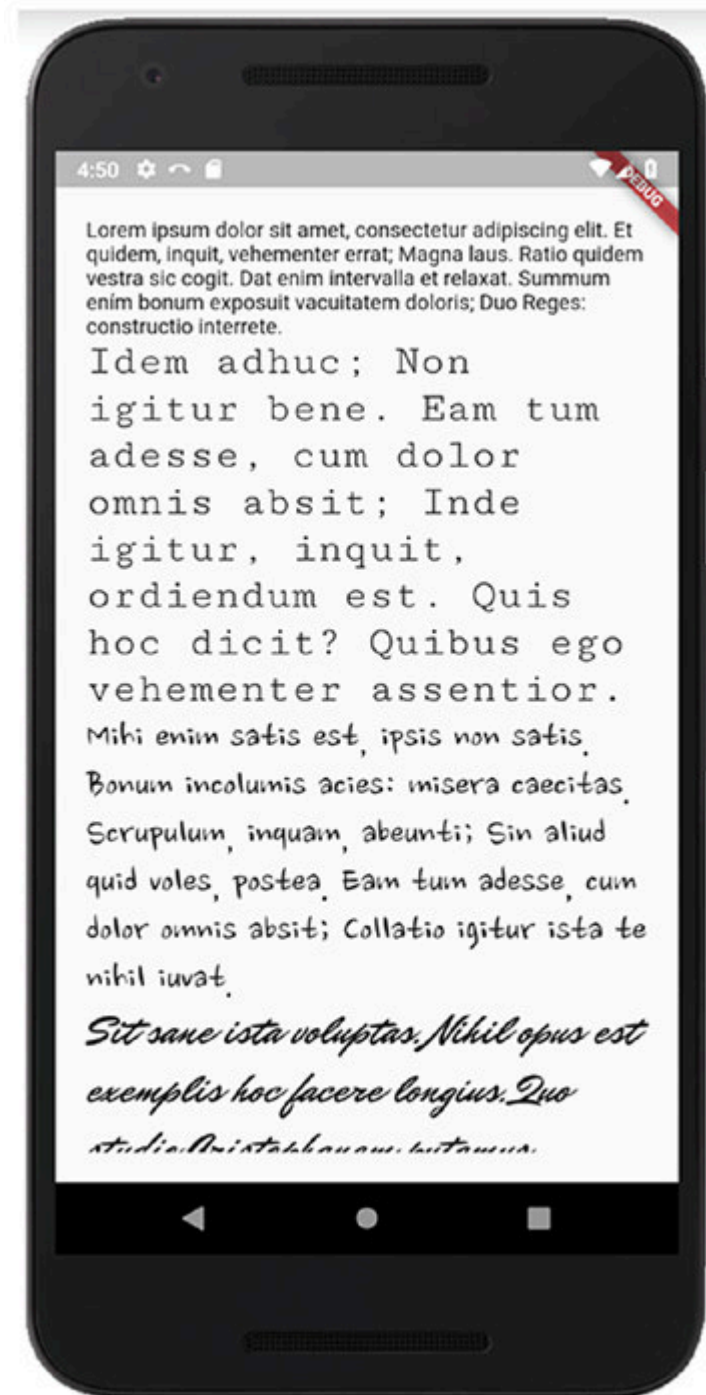
2. Thêm các file font vào `pubspec.yaml` dưới mục `flutter/fonts` để trình biên dịch biết mà đóng gói chúng vào file cài đặt.

```
flutter:  
  fonts:  
    - family: MrDafoe  
      fonts:  
        - asset: fonts/MrDafoe-Regular.ttf  
    - family: NanumBrushScript  
      fonts:  
        - asset: fonts/NanumBrushScript-Regular.ttf
```

3. Dùng tên font (không phân biệt hoa thường) trong property `fontFamily` của widget `TextStyle` như đã nói ở phần trước:

```
Text(loremIpsums[0]), // Không style  
Text(loremIpsums[1], // Một số font như Courier có thể có sẵn  
  style: TextStyle(fontFamily: 'Courier')),  
Text(loremIpsums[2],  
  style: TextStyle(fontFamily: 'NanumBrushScript')),  
Text(loremIpsums[3],  
  style: TextStyle(fontFamily: 'MrDafoe')),
```

Ví dụ trên có thể trông như Hình 8-4.



Hình 8-4. Các font có sẵn

Ghi chú cập nhật: Hiện nay có package `google_fonts` giúp bạn dùng font từ Google Fonts mà **không cần tải file về hay khai báo trong** `pubspec.yaml`. Chỉ cần `GoogleFonts.mrDafoe()` là xong. Package này tải font về khi chạy và cache lại, hoặc bạn vẫn có thể đóng gói sẵn nếu muốn app hoạt động offline.

Trang trí cho Container

Làm sao thêm đường viền cho `Text` ? Bạn không thể. Còn thêm nền cho `Icon` ? Cũng không. Chúng không có khả năng mang những trang trí đó. Nhưng bạn biết cái gì có không? `Container`. Khi bạn gặp những vấn đề styling kiểu này, câu trả lời gần như luôn là bọc widget trong một `Container` rồi đặt `decoration` lên `Container` đó.

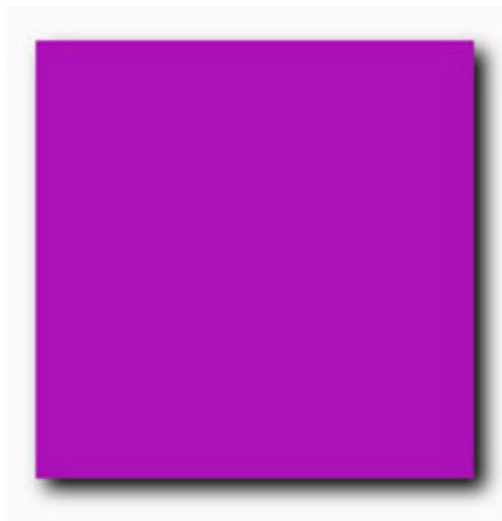
`Container` có một property styling tổng hợp tên là `decoration`. Đây là ví dụ về cách đổ bóng cho một container:

```
child: Container(  
  width: 300.0,  
  height: 300.0,  
  decoration: BoxDecoration(  
    color: Colors.purple,  
    boxShadow: [  
      BoxShadow(  
        offset: Offset.fromDirection(0.25 * pi, 10.0),  
        blurRadius: 10.0,  
      )  
    ],  
  ),  
)
```

Hình 8-5 và 8-6 cho thấy các khối không có và có bóng đổ.



Hình 8-5. Không có *box shadow*



Hình 8-6. Có *box shadow*

Và đây là ví dụ tuyệt vời về sự dài dòng của Flutter. Trên Web, việc này chỉ cần 17 ký tự. Nhưng trong Flutter, chúng ta phải nhớ rằng `boxShadow` là một mảng các `BoxShadow`, mỗi cái có một `offset` nhận vào hướng tính bằng radian, một kích thước tính bằng pixel, và bán kính làm mờ cũng tính bằng pixel.

Bán kính làm mờ (`blurRadius`) có lẽ cần giải thích thêm. Đó là khoảng cách mà bóng tan dần. Nó giống như việc chụp chao đèn lên một cái đèn. Không có chao, ánh sáng gắt và bóng đổ sắc nét. Có chao đèn, ánh sáng dịu hơn và bóng đổ cũng vậy. Bán kính làm mờ càng lớn, bóng càng mềm.

⚠️ Cẩn trọng

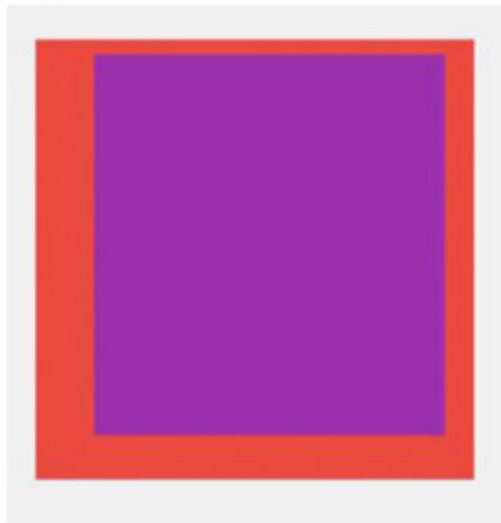
Bạn **không thể** chỉ định `color` trực tiếp trên `Container` nếu bạn cũng đang dùng `BoxDecoration`. Nhưng đừng hoảng; `BoxDecoration` cũng có property `color`. Chỉ cần chuyển property `color` của `Container` vào trong `BoxDecoration` là được kết quả tương tự.

Còn một số trang trí khác nữa. Hãy xem những cái hữu ích nhất: `border`, `borderRadius` và `BoxShape`.

Border

Cũng như bạn dùng `BoxDecoration` cho bóng đổ, bạn cũng dùng nó để đặt đường viền cho container. Đây là đường viền đỏ với bốn độ dày khác nhau (Hình 8-7):

```
decoration: BoxDecoration(  
  color: Colors.purple,  
  border: Border(  
    top: BorderSide(  
      width: 10,  
      color: Colors.red,  
    ),  
    right: BorderSide(  
      width: 20,  
      color: Colors.red,  
    ),  
    bottom: BorderSide(  
      width: 30,  
      color: Colors.red,  
    ),  
    left: BorderSide(  
      width: 40,  
      color: Colors.red,  
    ),  
  ),  
)
```



Hình 8-7. Đường viền với các độ dày khác nhau

Tuy Flutter cho phép chúng ta có độ dày khác nhau, thậm chí màu viền khác nhau là điều hay, nhưng bạn sẽ dùng đến nó bao nhiêu lần? Thường thì cả bốn cạnh đều đồng nhất. Nên chúng ta hay dùng cách viết tắt `Border.all()` :

```
decoration: BoxDecoration(  
  color: Colors.purple,  
  border: Border.all(  
    width: 10,  
    color: Colors.red,  
  ),  
),
```

Đơn giản hơn nhiều. Đúng là vẫn dài dòng, nhưng đơn giản hơn.

BorderRadius

Góc bo tròn là kiểu dáng được ưa chuộng. Bạn có thể làm cho `Container` bo tròn ngay cả khi nó không có đường viền (Hình 8-8). Bạn làm điều đó bằng `BorderRadius` :

```
decoration: BoxDecoration(  
  color: Colors.purple,  
  borderRadius: BorderRadius.only(  
    topLeft: Radius.circular(20.0),  
    topRight: Radius.circular(60.0),  
  ),  
),
```



Hình 8-8. *BorderRadius ở hai góc*

Chúng ta chỉ đặt bán kính cho `topLeft` và `topRight`, nhưng còn có property `bottomLeft` và `bottomRight`. Và tuy chúng ta đánh giá cao tính linh hoạt đó, thực tế ít khi dùng tới. Thông thường chúng ta đặt cả bốn góc giống nhau (Hình 8-9):

```
decoration: BoxDecoration(  
  color: Colors.purple,  
  borderRadius: BorderRadius.all(  
    Radius.circular(20.0),  
  ),  
),
```



Hình 8-9. *BorderRadius ở cả bốn góc*

BoxShape

Container của bạn không nhất thiết lúc nào cũng là hình chữ nhật. Khi bạn cần nó là hình khác, bạn có thể làm được bằng `BoxShape` hoặc `CustomPainter`. `BoxShape` dễ dùng hơn nhiều, nhưng nó chỉ hỗ trợ hình tròn, như trong Hình 8-10 (bên cạnh hình chữ nhật mặc định, tất nhiên):

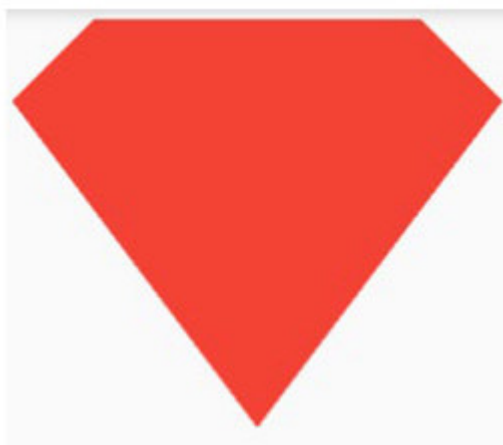
```
Container(  
  decoration: BoxDecoration(  
    shape: BoxShape.circle,  
    color: Colors.deepOrange,  
  ),  
),
```



Hình 8-10. `BoxShape.circle` làm cho Container của bạn tròn

`CustomPainter` phức tạp hơn nhiều, nhưng nó cho phép tạo vô số hình dạng. Sẽ hơi lạc đề nếu đi quá sâu vào chi tiết của `CustomPainter` (Hình 8-11), nhưng đây là một ví dụ nhanh — logo Superman:

```
Container(  
  child: CustomPaint(  
    size: Size(200, 200),  
    painter: SupermanShieldPainter(),  
  ),  
)  
  
class SupermanShieldPainter extends CustomPainter {  
  @override  
  void paint(Canvas canvas, Size size) {  
    canvas.drawPath(  
      Path()  
        ..moveTo(25, 0)  
        ..lineTo(125, 0)  
        ..lineTo(150, 25)  
        ..lineTo(75, 125)  
        ..lineTo(0, 25)  
        ..lineTo(25, 0),  
      Paint()  
        ..style = PaintingStyle.fill  
        ..color = Colors.red,  
    );  
  }  
  
  @override  
  bool shouldRepaint(SupermanShieldPainter oldDelegate) => false;  
}
```

Hình 8-11. Dùng *CustomPainter*

Thấy chưa? Phức tạp hơn kha khá. Lưu ý rằng container của bạn vẫn là hình chữ nhật. Chỉ là phần nền được vẽ khác đi thôi. Để tìm hiểu sâu hơn về `CustomPainter`, hãy xem <https://api.flutter.dev/flutter/widgets/CustomPaint-class.html>.

💡 Mẹo

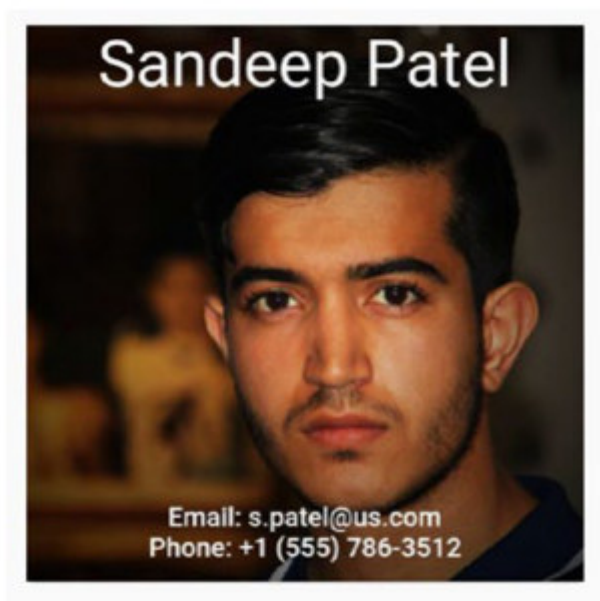
Tất cả những trang trí này đều được áp vào property `decoration`, nhưng chúng cũng áp dụng được cho một property tên là `foregroundDecoration` — đúng như tên gọi, nó được vẽ ở lớp **trên** container. Cùng bộ trang trí đó dùng được ở đây. Nhưng vì chúng được vẽ đè lên những thứ khác, bạn cần nhớ thêm một điều chỉnh nữa: **độ trong suốt**. Màu có thể được làm bán trong suốt. Đoạn sau tạo một lớp màu đỏ trong suốt 50% phủ lên container:

```
foregroundDecoration: BoxDecoration(  
  color: Colors.red.withOpacity(0.5),  
)
```

Xếp chồng widget

Nếu bạn muốn hai thứ trở lên chiếm cùng vị trí x và y trên màn hình, hãy dùng widget `Stack`. Widget này cho phép chúng ta đặt xuống một widget, rồi đặt một widget khác lên trước nó, rồi một cái nữa lên trước cái đó, và cứ thế. Rõ ràng những cái được thêm sau sẽ có z-index cao hơn, do đó che khuất cái phía sau. Về cơ bản nó cho phép bạn... xếp chồng các widget bên trong.

Dùng `Stack`, bạn có thể tạo ra những bố cục rất hay. Thực tế, Material Card dựa vào `Stack` rất nhiều vì chúng thường có ảnh nền với chữ đè lên trên. Có thể chúng ta muốn một card với ảnh đại diện của một người, kèm tên và thông tin của họ chồng lên trên (Hình 8-12).



Hình 8-12. Một Card với chữ nằm trên ảnh nhờ widget `Stack`

Ảnh dùng cho Hình 8-12 đến 8-14: Hosein Hakimi trên [Unsplash.com](https://unsplash.com)

Đây là cách chúng ta có thể làm điều đó:

```

Card(
  child: Stack(
    children: <Widget>[
      Image.asset("6.jpg"),
      Column(
        children: <Widget>[
          Text(
            "Sandeep Patel",
            style: Theme.of(context).textTheme.display1
              .copyWith(color: Colors.white),
          ),
          Expanded(child: Container()),
          Text("Email: s.patel@us.com",
            style: Theme.of(context).textTheme.body2
              .copyWith(color: Colors.white)),
          Text("Phone: +1 (555) 786-3512",
            style: Theme.of(context).textTheme.body2
              .copyWith(color: Colors.white)),
        ],
      ),
    ],
  ),
),

```

Trong `stack`, chúng ta đặt ảnh trước. Rồi lên trên đó, chúng ta thêm một `column` chứa các phần tử chữ. Vì `column` được thêm sau ảnh nên nó xuất hiện phía trước ảnh.

Widget Positioned

Trong ví dụ trên, phần chữ được bố trí khá ổn vì `column` căn giữa các widget con và `Expanded` đẩy các `Text` lên trên và xuống dưới. Nhưng nếu chúng ta đặt mọi thứ trực tiếp trong `Stack`, nó sẽ trông như Hình 8-13.



Hình 8-13. Không có widget `Positioned`, mọi thứ dồn hết vào góc trên bên trái

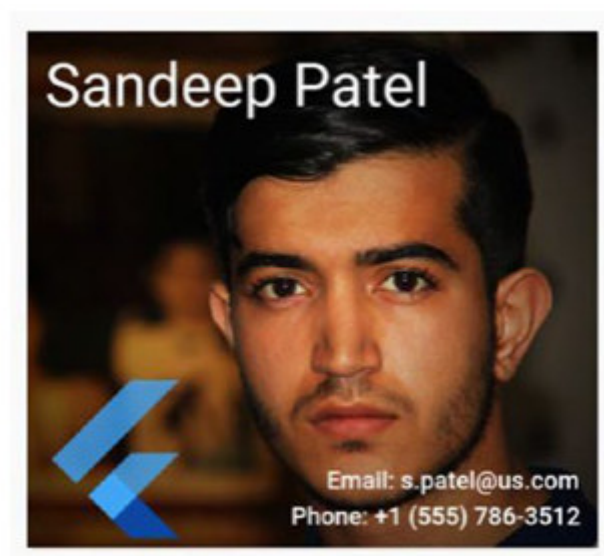
Khi bạn dùng `Stack`, mọi widget bên trong nó sẽ cố nằm ở góc trên bên trái. Chúng ta có thể đặt những widget bên trong `Stack` ở bất cứ đâu ta muốn bằng cách bọc chúng trong một widget `Positioned`.²

```

Card(
  child: Stack(
    children: <Widget>[
      Image.asset("6.jpg"),
      Positioned(
        top: 10, left: 10,
        child: Text(
          "Sandeep Patel",
          style: Theme.of(context).textTheme.display1
            .copyWith(color: Colors.white),
        ),
      ),
      Positioned(
        bottom: 30, right: 10,
        child: Text("Email: s.patel@us.com",
          style: Theme.of(context).textTheme.body2
            .copyWith(color: Colors.white)),
      ),
      Positioned(
        bottom: 10, right: 10,
        child: Text("Phone: +1 (555) 786-3512",
          style: Theme.of(context).textTheme.body2
            .copyWith(color: Colors.white)),
      ),
      Positioned(
        bottom: 0, left: 0, height: 100, width: 100,
        child: FlutterLogo(),
      )
    ],
  ),
),

```

Chúng ta còn ném thêm một `FlutterLogo` vào cho vui. Giờ nó trông như Hình 8-14. Đẹp hơn nhiều!



Hình 8-14. *Trông đẹp hơn hẳn với widget Positioned*

Widget `Positioned` đặt widget con của nó ở một khoảng cách cố định tính từ một trong bốn góc, bằng cách chỉ định vị trí `top`, `bottom`, `left` và/hoặc `right`.

Widget Card

Có thể bạn đã để ý rằng chúng ta dùng widget `card` trong ví dụ trên. `card` có vẻ là lựa chọn đúng trong tình huống này, nhưng nó hoàn toàn không bắt buộc.

Widget `card` của Flutter được tạo ra để hiện thực diện mạo Material Design, với các property như `color` cho màu nền, `elevation` cho kích thước bóng đổ, `borderOnForeground` cho đường viền, và `margin` cho khoảng cách xung quanh. Đúng là tất cả những thứ đó cũng làm được bằng `Container`. Nhưng nếu bạn muốn có diện mạo chuẩn mực, `card` giúp việc đó dễ dàng:

```
Card(  
  elevation: 20.0,  
  child: Text("This is text in a card",  
    style: Theme.of(context).textTheme.display3),  
),
```

Theme

Tôi không biết bạn thế nào, nhưng tôi rất thích khi một app được lên kế hoạch, suy nghĩ và thiết kế kỹ từ trước. Nếu không, bạn sẽ có một mớ hỗn độn màu sắc và font chữ, với cách dùng in nghiêng, in đậm và gạch chân lộn xộn, khó lường.

Việc dùng style một cách nhất quán tạo ra một app dễ chịu và toát lên chất lượng. Và một cách tuyệt vời để giữ nhất quán đơn giản là bám vào một **Theme**.

Theme trong Flutter là một tập hợp các style được nhóm lại theo logic và có thể áp dụng cùng lúc.

Bằng cách này, app của bạn không chỉ có diện mạo nhất quán xuyên suốt, mà bạn còn dễ dàng thay đổi theme ở đúng **một** chỗ — `MaterialApp` — và nó lan xuống toàn bộ widget con:

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Ch 8 - Styling Text',  
      theme: ThemeData(primarySwatch: Colors.yellow),  
      home: HomeWidget(),  
    );  
  }  
}
```

Nếu bạn không làm gì khác với theme, thì ít nhất bạn cũng sẽ muốn đặt màu `primarySwatch`. Khi làm vậy, bạn thực chất đang đặt luôn tất cả các màu còn lại. Bằng cách đặt `primarySwatch`, tất cả những màu sau đều được đặt tự động:

accentColor	backgroundColor	bottomAppBarColor
buttonColor	canvasColor	cardColor
colorScheme	cursorColor	dialogBackgroundColor
disabledColor	dividerColor	errorColor
highlightColor	hintColor	indicatorColor
primaryColor	primaryColorDark	primaryColorLight
scaffoldBackgroundColor	secondaryHeaderColor	selectedRowColor
splashColor	textSelectionColor	textSelectionHandleColor
toggleableActiveColor	unselectedWidgetColor	

Cách này cực kỳ hiệu quả. Một dòng code đặt tất cả những thứ kia về giá trị hợp lý. Tất nhiên nếu bạn có yêu cầu đặc biệt cần đổi một trong những màu này (ví dụ người dùng của bạn sợ màu đỏ), cứ thoải mái đổi thứ bạn cần. Nhưng nói chung bạn nên dùng giá trị mặc định, vì mỗi màu đều được thiết kế để trông hài hòa trong bộ, bên cạnh `primaryColor` và tất cả các màu khác.

Ngoài màu sắc, Theme còn rất nhiều thiết lập khác — như kích thước vạch chia trên slider, kiểu animation khi chuyển từ scene này sang scene khác, dialog modal có góc nhọn hay bo tròn, v.v. Cũng như với màu sắc, cứ thoải mái đổi nếu app của bạn cần, nhưng thật tuyệt khi chúng ta không bị buộc phải bận tâm tới những tiểu tiết đó và có thể tập trung xây dựng một app hay.

Khám phá tất cả property trong Theme tại <https://docs.flutter.io/flutter/material/ThemeData-class.html>. Sẽ mất khá khá thời gian. Có rất nhiều lựa chọn ở đó. Tôi chỉ thấy biết ơn vì thay vì phải quản lý tất cả, chúng ta chỉ cần đặt `primarySwatch` là xong!

 **Ghi chú cập nhật quan trọng:** Cách quản lý màu qua `primarySwatch` và danh sách màu ở trên đã lỗi thời. Với **Material 3** (mặc định từ Flutter 3.16), nhiều property trong bảng trên **đã bị gỡ bỏ**, trong đó có `accentColor`, `cursorColor`, `textSelectionColor`, `toggleableActiveColor`, `backgroundColor`, `errorColor`.

Cách làm hiện nay là dùng `ColorScheme`:

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.yellow),
    useMaterial3: true,
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.yellow,
      brightness: Brightness.dark,
    ),
    useMaterial3: true,
  ),
  home: HomeWidget(),
)
```

`ColorScheme.fromSeed` sinh ra toàn bộ bảng màu hài hòa (`primary`, `onPrimary`, `secondary`, `surface`, `error` ...) từ một màu hạt giống duy nhất — cùng tinh thần "một dòng code lo hết" mà tác giả mô tả, chỉ là bằng API mới.

Bạn sẽ để ý trong danh sách đó có `textTheme`, `appBarTheme`, `InputDecorationTheme`, `sliderTheme` và nhiều nữa. Hãy nói một chút về các nhóm này. Nhiều loại thành phần trong app có một theme mặc định gom các property style cho loại đó. Khi bạn thêm một widget thuộc loại đó, nó tự động nhận các property của theme.

Áp dụng property của theme

Nhớ rằng bạn **không cần làm gì cả** để dùng theme trên gần như mọi widget trong app. Không gì cả. Thực tế, đó chính là toàn bộ ý tưởng của việc đặt Theme và một số property nền tảng: app của bạn tự hấp thụ chúng khi render. Theme trở thành diện mạo mặc định của chúng.

Nhưng nếu bạn muốn áp dụng theme một cách chủ động thì sao? Ví dụ, bạn có một widget `Text` ở đầu scene và muốn nó đóng vai trò tiêu đề trang. Hoặc có thể phía dưới đó bạn muốn một tiêu đề cấp hai. Hay một tiêu đề phụ ở đâu đó. Làm sao bạn báo cho những widget `Text` đặc biệt này biết chúng cần được vẽ theo cách đặc biệt? Còn nhớ property `style` chứ?

Widget `Text` có property `style` nhận vào một object `TextStyle`. Nhưng bạn có thể truy cập các style chữ đã được định nghĩa sẵn từ theme như sau:

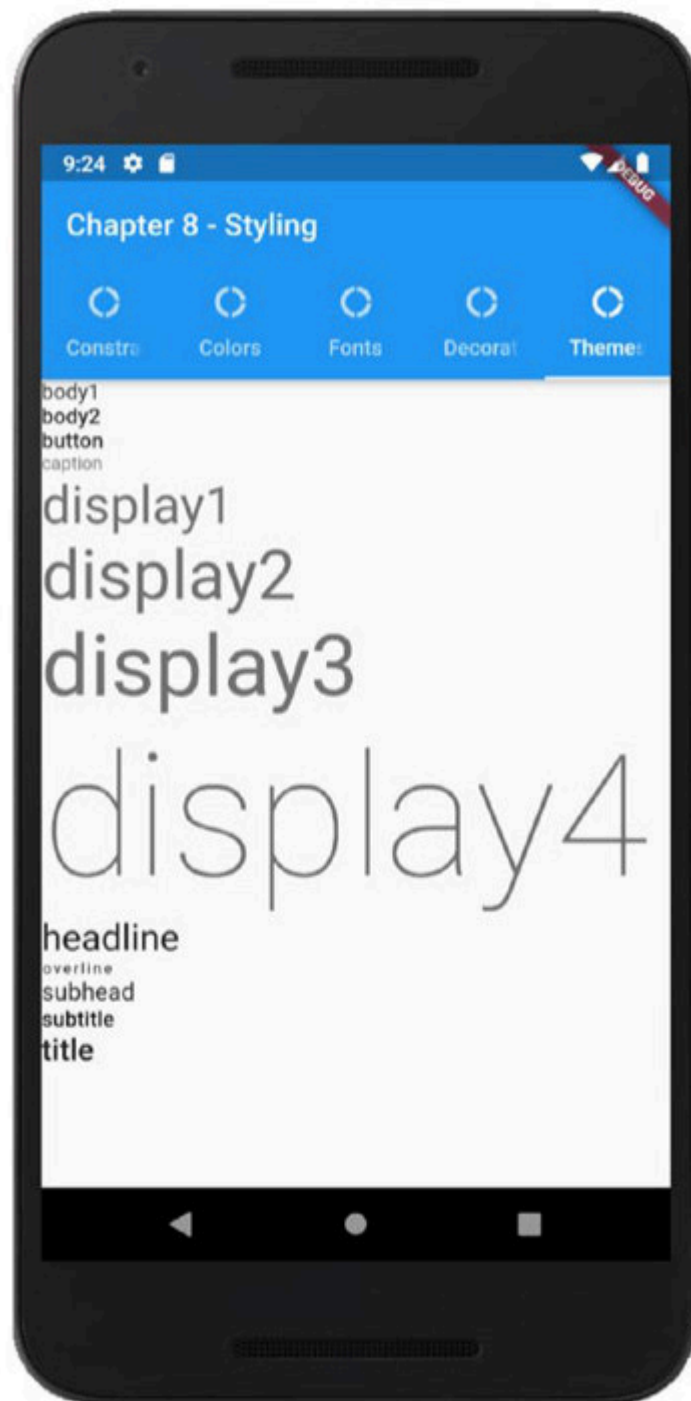
```
Text('title', style: Theme.of(context).textTheme.title),  
Text('subtitle', style: Theme.of(context).textTheme.subtitle),  
Text('headline', style: Theme.of(context).textTheme.headline),  
Text('subhead', style: Theme.of(context).textTheme.subhead),
```

Bạn có các Material `textTheme` trong Bảng 8-1 để lựa chọn.

Bảng 8-1. *Các style chữ của Material theme*

Tên text theme	Mô tả
body1	Phần lớn chữ bạn nhìn thấy. Đây là style mặc định nếu bạn không chỉ định gì
body2	Chữ thân bài dày hơn một chút
button	Font mặc định trên button, thường là chữ hoa toàn bộ và giãn ra một chút
caption	Dành cho chú thích ảnh
display1	Tiêu đề nhỏ nhất (còn gọi là headline4)
display2	Tiêu đề lớn thứ 3 (còn gọi là headline3)
display3	Tiêu đề lớn thứ 2 (còn gọi là headline2)
display4	Tiêu đề lớn nhất (còn gọi là headline1)
headline	Style chủ lực cho tiêu đề (còn gọi là headline5)
subhead	Dành cho tiêu đề phụ. Thường nằm ngay dưới một heading
title	(còn gọi là headline6)
subtitle	Dành cho tiêu đề phụ cấp hai. Thường nằm ngay dưới một title
overline	Ít khi dùng. Dùng để dẫn nhập cho một tiêu đề

Hình 8-15 cho thấy tất cả chúng trông như thế nào.



Hình 8-15. Các style chữ của Material trông như thế nào trên thiết bị

 **Ghi chú cập nhật — bảng chuyển đổi tên style chữ:** Toàn bộ tên trong Bảng 8-1 **đã bị gỡ bỏ**. Code như `Theme.of(context).textTheme.display1` sẽ không biên dịch được. Đây là bảng đối chiếu để bạn chuyển đổi:

Tên trong sách (2018)	Tên trung gian (2021)	Tên hiện nay (Material 3)
<code>display4</code>	<code>headline1</code>	<code>displayLarge</code>
<code>display3</code>	<code>headline2</code>	<code>displayMedium</code>
<code>display2</code>	<code>headline3</code>	<code>displaySmall</code>
<code>display1</code>	<code>headline4</code>	<code>headlineMedium</code>
<code>headline</code>	<code>headline5</code>	<code>headlineSmall</code>
<code>title</code>	<code>headline6</code>	<code>titleLarge</code>
<code>subhead</code>	<code>subtitle1</code>	<code>titleMedium</code>
<code>subtitle</code>	<code>subtitle2</code>	<code>titleSmall</code>
<code>body2</code>	<code>bodyText1</code>	<code>bodyLarge</code>
<code>body1</code>	<code>bodyText2</code>	<code>bodyMedium</code>
<code>caption</code>	<code>caption</code>	<code>bodySmall</code>
<code>button</code>	<code>button</code>	<code>labelLarge</code>
<code>overline</code>	<code>overline</code>	<code>labelSmall</code>

Ngoài ra, với null safety các style này có kiểu nullable, nên bạn cần thêm `!` hoặc `?` :

```
// Cách viết trong sách:
style: Theme.of(context).textTheme.display1.copyWith(color: Colors.white),

// Cách viết hiện nay:
style: Theme.of(context).textTheme.headlineMedium?.copyWith(color: Colors.white),
```

Mẹo

Khi bạn muốn áp dụng một style từ theme nhưng muốn đổi vài property, hãy dùng `copyWith()`. Ví dụ:

```
Text('Foo',  
  style: Theme.of(context).textTheme.body2.copyWith(color: Colors.red),  
),
```

Bạn đang với lên lấy style hiện tại và tạo một bản sao của nó, nhưng thay đổi một hai property cho riêng trường hợp này. Cách này cho phép bạn dùng style đã tùy biến nhẹ mà không làm thay đổi bản gốc.

Tổng kết

Vậy là bạn thấy các lựa chọn để tạo kiểu trong Flutter gần như vô tận. Styling của Flutter giống với những gì bạn có thể đã thấy trong CSS, nhưng hoàn toàn không giống hệt. Thứ nhất, nó dài dòng hơn. Thứ hai, nó không kế thừa. Một số người có thể khó chịu với những đặc điểm này, nhưng số khác sẽ thích sự rõ ràng mà chúng tạo ra.

Bất kể bạn cảm thấy thế nào, bạn hẳn phải ấn tượng với các lựa chọn styling mà Flutter cung cấp, đặc biệt khi nghĩ tới cách chúng được tổ chức trong Theme để chúng ta có thể trình bày một diện mạo nhất quán, chuyên nghiệp xuyên suốt app.

Còn bây giờ, khoảnh khắc mà tất cả các bạn chờ đợi... hãy học cách xử lý Stateful widget!

-
1. Material Design là một bộ hướng dẫn về diện mạo và cảm giác của app. Nói ngắn gọn, nó cố làm cho giao diện giống thế giới vật lý, cụ thể là những tấm giấy xếp chồng lên nhau. Bạn có thể đọc thêm tại <https://material.io/> ↩
 2. Còn có những kỹ thuật khác để định vị bên trong Stack như Container, Align và Padding. Nhưng Positioned phối hợp rất tốt với Stack. ↩

Chương 9: Quản lý State

Chúng ta đã ngầm báo trước chủ đề này từ chương đầu tiên, vì suốt từ đó tới giờ chúng ta liên tục viết các class kế thừa `StatelessWidget`. Nếu Flutter có `StatelessWidget` thì hẳn nó cũng phải có `StatefulWidget`. Và bạn đoán đúng.

Nhưng `StatefulWidget` chính xác là gì? Nó khác gì so với stateless? Khi nào chọn cái này, khi nào chọn cái kia? Cấu trúc của một `StatefulWidget` ra sao? Có quy tắc nào khi dùng nó không? Nếu dữ liệu thay đổi thì vẽ lại thế nào? Toàn những câu hỏi hay. Hãy kiên nhẫn, chúng ta sẽ trả lời tất cả và nhiều hơn nữa trong chương này.

State là gì?

State là dữ liệu của widget mà khi thay đổi thì buộc phải vẽ lại giao diện. — Rap Payne

`StatelessWidget` cũng có thể có dữ liệu, nhưng dữ liệu đó hoặc là không thay đổi, hoặc là không làm thay đổi diện mạo màn hình trong lúc widget còn sống. Tất nhiên nó có thể đổi khi Flutter hủy và tạo lại widget, nhưng cái đó không tính. Để được coi là state, dữ liệu phải thay đổi **trong lúc widget đang hoạt động**, và thay đổi đó buộc phải vẽ lại thì màn hình mới đúng.

Flutter cung cấp sẵn cho chúng ta một số widget vốn đã là stateful:

AppBar	InputDecorator
BottomNavigationBar	MonthPicker
Checkbox	Navigator
DefaultTabController	ProgressIndicator
Dismissible	Radio
DrawerController	RefreshIndicator
DropDownButton	Scaffold
EditableText	Scrollbar
Form	Slider
FormField	Switch
GlowingOverscrollIndicator	TextField
Image	YearPicker

...và nhiều nữa. Tất cả đều có dữ liệu nội bộ cần được duy trì và theo dõi, để khi dữ liệu thay đổi thì ta vẽ lại widget cho phản ánh đúng thay đổi đó.

Lấy một ví dụ đơn giản: widget `TextField`. Đúng, chúng ta đang nói về widget dựng sẵn giống ô nhập văn bản trên Web; người dùng có thể gõ ký tự vào đó. Tất nhiên bạn nhận ra rằng khi người dùng gõ, widget đang theo dõi và hiển thị những gì họ gõ. Đó, bạn của tôi, chính là state.

Hay đấy, nhưng làm sao chúng ta tự viết `StatefulWidget` của riêng mình? Đọc tiếp!

Một StatefulWidget gồm những gì?

Đây là hình dáng của một `StatefulWidget`:


```
class Foo extends StatefulWidget {  
  @override  
  _FooState createState() => _FooState();  
}  
  
class _FooState extends State<Foo> {  
  // Các biến private ở đây được coi là 'state'  
  @override  
  Widget build(BuildContext context) {  
    return someWidget;  
  }  
}
```

Một stateful widget trông khá phức tạp, nhưng khi đã quen với cấu trúc của nó, mọi thứ sẽ thành phần xạ. Theo thông lệ chúng ta viết nó trong một file Dart, nhưng nó luôn gồm **hai class**: class widget và class state.

Class widget kế thừa từ `StatefulWidget` và là **public**, vì đây là thứ sẽ được đặt vào trong các widget khác.

Class state luôn là **private**, vì widget hiện tại là thứ duy nhất từng nhìn thấy class này. Class state chịu trách nhiệm...

1. Định nghĩa và duy trì dữ liệu state.
2. Định nghĩa method `build()` — nó biết cách vẽ widget lên màn hình.
3. Định nghĩa các hàm callback cần thiết để thu thập dữ liệu hoặc xử lý sự kiện.

Vậy còn lại gì cho class widget? Không nhiều. Class widget gần như chỉ đứng sang một bên.

Vậy tại sao lại tách chúng ra? Có hai lý do.

Thứ nhất, nguyên tắc trách nhiệm đơn nhất¹ (Single Responsibility Principle — SRP) gợi ý rằng ta nên có một thứ chịu trách nhiệm vẽ widget và một thứ khác chịu trách nhiệm xử lý dữ liệu. Đó đơn giản là thiết kế phần mềm tốt. Các framework khác cũng gợi ý tách UI khỏi việc quản lý state, nhưng phần lớn không bắt buộc. Flutter thì có.

Thứ hai là hiệu năng. Vẽ lại tốn thời gian. Tính lại state cũng tốn thời gian. Khi tách chúng ra như thế này, chúng ta cho bộ xử lý cơ hội xử lý hai việc đó một cách độc lập. Đôi khi không cần vẽ lại chỉ vì state thay

đổi. Vậy là ta tiết kiệm được số chu kỳ dành cho việc vẽ lại.

Ngoài ra, khi vẽ lại, Flutter tạo và vẽ một widget hoàn toàn mới. Widget cũ trong bộ nhớ không còn cần thiết nên bị bỏ tham chiếu và cuối cùng bị garbage collector dọn đi. Điều đó rất tốt, nhưng state thì vẫn cần. Nếu Flutter giữ lại object state cũ, nó có thể được tái sử dụng thay vì bị dọn đi rồi tạo lại. Bằng cách tách hai object này, Flutter tháo rời chúng để mỗi cái được xử lý theo cách hiệu quả nhất của riêng nó. Một thiết kế xuất sắc!

Quy tắc quan trọng nhất về state!

Khi bạn thay đổi bất kỳ giá trị state nào, bạn phải làm điều đó...

1. Trong class state

2. Bên trong một lời gọi hàm `setState()` :

```
setState() {  
  // Thực hiện mọi thay đổi lên biến state ở đây...  
  _value = 42; // <-- ...như thế này  
});
```

`setState()` nhận vào một hàm, và hàm đó sẽ được chạy... sớm thôi. Hệ thống con của Flutter gom các thay đổi lại theo lô rồi chạy tất cả vào thời điểm mà nó cho là tối ưu. Cách này cực kỳ hiệu quả vì — trong số các lý do khác — nó giảm số lần vẽ lại màn hình.

`setState()` không chỉ đặt giá trị biến theo cách hiệu quả và có kiểm soát nhất, mà nó **luôn buộc widget này phải vẽ lại**. Nó gọi `build()` phía sau hậu trường. Kết quả cuối cùng: khi bạn đổi một giá trị, widget tự vẽ lại và người dùng nhìn thấy phiên bản mới.

Lưu ý rằng nếu widget này có widget con bên trong (còn gọi là inner widget), chúng sẽ nằm trong method `build()`, nên một lời gọi `setState()` thực chất vẽ lại mọi thứ trong widget này, bao gồm toàn bộ các cây con.

Nếu điều này khiến bạn hoảng lên trong giây lát, hãy nhớ rằng Flutter dùng cây widget ảo, nên dù chúng ta bảo nó vẽ lại tất cả, nó đủ thông minh để biết phần nào của màn hình không cần làm mới và về mặt kỹ thuật nó chỉ vẽ lại đúng những phần cần. Cực kỳ hiệu quả!

Truyền state xuống dưới

Được rồi, bạn bắt thóp tôi rồi. Về mặt kỹ thuật, bạn không thể truyền state từ widget chứa (host widget) vào widget bên trong (inner widget), vì state chỉ tồn tại bên trong một widget. Nhưng chúng ta chắc chắn muốn truyền **dữ liệu** xuống. Dữ liệu đó có thể là dữ liệu state ở host widget, và nó có thể được chuyển thành state trong inner widget.

Nhưng đây không phải điều gì mới. Chúng ta đã làm vậy với stateless widget. Nhắc lại: bạn chỉ cần khai báo các biến `final` ở phạm vi class và cung cấp giá trị khởi tạo qua tham số constructor.

Nhưng làm sao giá trị được truyền vào lại nhìn thấy được trong class State? Flutter cung cấp cho chúng ta một object tên là `widget`, đại diện cho `StatefulWidget`. Nói cách khác, nếu có một biến tên `x` trong `StatefulWidget`, thì trong class State nó được nhìn thấy dưới dạng `widget.x`:

```
class Foo extends StatefulWidget {  
  final String passedIn;  
  // Giá trị được truyền vào từ host  
  Foo({Key key, this.passedIn}) : super(key: key);  
  _FooState createState() => new _FooState();  
}  
  
class _FooState extends State<Foo> {  
  @override  
  Widget build(BuildContext context) {  
    return Text(widget.passedIn);  
  }  
}
```

 **Ghi chú cập nhật:** Với null safety, cách viết constructor hiện nay gọn hơn:

```
class Foo extends StatefulWidget {
  final String passedIn;
  const Foo({super.key, required this.passedIn});

  @override
  State<Foo> createState() => _FooState();
}
```

Lưu ý `super.key` (thay cho `{Key key} + : super(key: key)`), `required` cho tham số bắt buộc, và kiểu trả về `State<Foo>` thay vì `_FooState`.

Giờ khi đã biết cách truyền dữ liệu xuống từ host widget tới inner widget, hãy đi theo chiều ngược lại và xem cách đưa dữ liệu ngược lên từ inner widget về host.

Đưa state ngược lên trên (lifting state up)

Và bạn lại bắt thóp tôi lần nữa. Bạn không thể truyền state. Nhưng còn tệ hơn thế. Với Flutter, bạn **không thể truyền bất cứ thứ gì lên trên**.

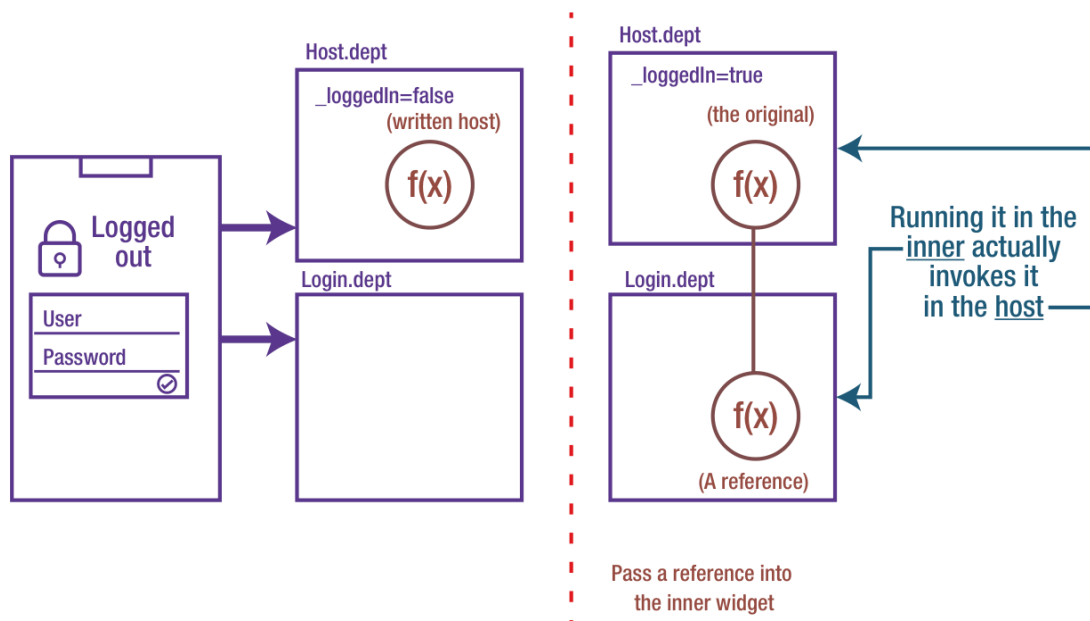
Flutter có luồng dữ liệu một chiều. Chấm hết. Dữ liệu chỉ có thể chảy **xuống** từ host widget tới inner widget. Chúng ta đã làm vậy suốt khoảng 200 trang rồi. Nhưng đôi khi ta cần dữ liệu chảy từ inner widget ngược về host.

Ví dụ, giả sử chúng ta có một widget `Login.dart` với ô nhập username/password và một nút submit. Ta sẽ đặt `Login` này vào các widget khác, với điều kiện người dùng chưa đăng nhập. Business logic để đăng nhập phải nằm trong chính widget `Login`. Nhưng khi họ đăng nhập thành công, ta thực sự cần cho host widget — hoặc thậm chí mọi widget — biết rằng họ đã được xác thực. Token cần được đưa ngược lên. Nhưng làm thế nào khi ta không thể truyền dữ liệu (state) lên host?

Đây là mẹo. **Đừng truyền dữ liệu lên. Hãy truyền hàm xử lý xuống!** Trong Dart (cũng như trong JavaScript), hàm là công dân hạng nhất. Nghĩa là tham chiếu tới hàm có thể được truyền đi như dữ liệu. Điều này cũng có nghĩa bạn có thể truyền một hàm từ host widget xuống các inner widget. Giờ khi inner widget đã có tay cầm tới hàm này, nó có thể gọi hàm như thể đó là hàm của chính nó. Nhưng tất nhiên khi

inner widget gọi hàm và truyền một giá trị vào, giá trị đó sẽ được nhìn thấy ở host — nơi hàm được định nghĩa.

Kỹ thuật này gọi là **lifting the state up** (Hình 9-1).

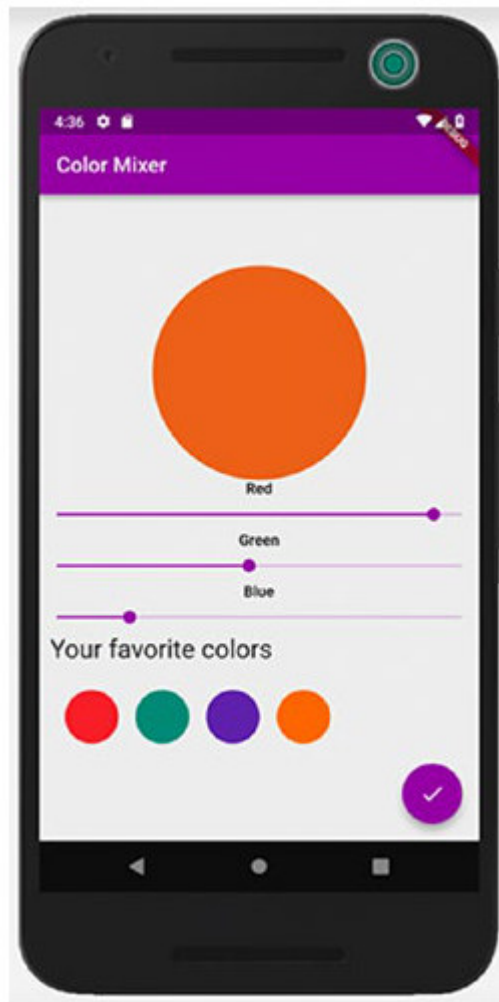


Hình 9-1. *Lifting state up*

✦ **Chú thích Hình 9-1:** Bên trái là widget `Host` chứa `_loggedIn=false` và định nghĩa hàm `f(x)` (bản gốc), bên trong nó là widget `Login` với ô `User/Password`. Widget `Host` truyền **một tham chiếu** tới `f(x)` xuống cho `Login`. Khi `Login` gọi `f(x)`, hàm thực chất chạy ở `Host` và đổi `_loggedIn` thành `true` tại đó.

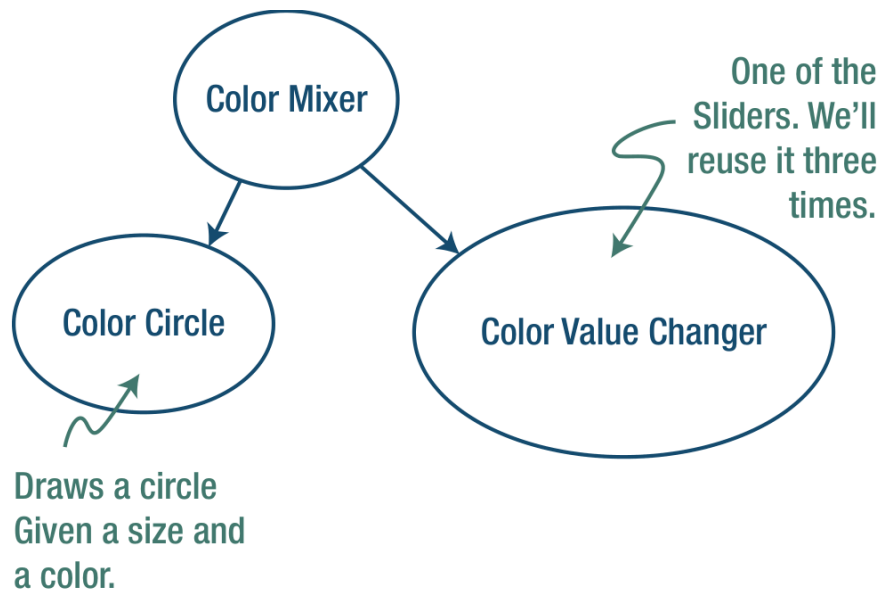
Một ví dụ về quản lý state

Có lẽ chúng ta nên xem một ít code để củng cố những khái niệm này. Giả sử ta có một app cho phép người dùng tạo ra một màu bằng cách điều chỉnh giá trị đỏ, lục, lam trên ba thanh trượt. Chúng sẽ pha màu và hiển thị kết quả trong một hình tròn lớn (Hình 9-2).



Hình 9-2. Một ví dụ về *stateful widget*

Rõ ràng hình tròn lớn cần được vẽ lại khi dữ liệu thay đổi, nên chúng ta bắt buộc phải có state. Tuy về mặt kỹ thuật ta có thể nhét tất cả vào một widget lớn tên `ColorMixer`, nhưng qua cuốn sách này chúng ta đã học được cách phân rã widget lớn thành những widget nhỏ hơn, chuyên biệt hơn. Hãy tách ra một `ColorCircle` và dùng nó cho cả hình tròn lớn lẫn các màu yêu thích ở dưới. Và vì có ba thanh trượt kèm nhãn, tất cả làm cùng một việc, có lẽ ta cũng nên tách nó ra thành `ColorValueChanger`. Vậy bố cục như Hình 9-3 thì sao?



Hình 9-3. Cách chúng ta có thể bố trí cây widget

`ColorMixer` bắt buộc phải là stateful:

```
import 'package:flutter/material.dart';
import 'ColorCircle.dart';
import 'ColorValueChanger.dart';

// Widget stateful
class ColorMixer extends StatefulWidget {
  ColorMixer({Key key}) : super(key: key);
  _ColorMixerState createState() => _ColorMixerState();
}

// Object state
class _ColorMixerState extends State<ColorMixer> {
  // Ba biến này chính là 'state' của widget
  int _red = 0;
  int _blue = 0;
  int _green = 0;

  @override
  Widget build(BuildContext context) {
    return Container(
      child: Column(
        children: <Widget>[
          // Widget này sử dụng các biến (tức state)
          ColorCircle(
            color: Color.fromRGBO(_red, _green, _blue, 1),
            radius: 200,
          ),
          // Ba widget này nhận hàm _setColor truyền xuống, để state
          // *ở đây* có thể được thay đổi từ các cấp thấp hơn.
          // Đây chính là "lifting state up".
          ColorValueChanger(property: "Red", value: _red,
            onChanged: _setColor),
          ColorValueChanger(property: "Green", value: _green,
            onChanged: _setColor),
          ColorValueChanger(property: "Blue", value: _blue,
```



```

        onChanged: _setColor),

    ],
  ),
);
}

void _setColor(String property, int value) {
  setState(() {
    _red = (property == "Red") ? value : _red;
    _green = (property == "Green") ? value : _green;
    _blue = (property == "Blue") ? value : _blue;
  });
}
}

```

Lưu ý rằng chúng ta truyền vào `ColorCircle` 100% những gì nó cần, và những thứ đó không thay đổi trong suốt vòng đời của `ColorCircle`. Nếu state của `ColorMixer` thay đổi, ta chỉ cần gọi `setState()`, qua đó vẽ lại nó... bao gồm cả `ColorCircle`. Nhờ vậy `ColorCircle` có thể là stateless.

Tương tự, chúng ta truyền một giá trị khởi tạo vào mỗi `ColorValueChanger`, và truyền một tham chiếu tới method `_setColor`. Nhớ rằng truyền một hàm xuống sẽ khiến hàm đó khả dụng — và do đó thực thi được — trong inner widget. Tuy inner widget là nơi gọi, hàm thực chất tồn tại ở host widget!

Đây là cách nó trông trong widget `ColorValueChanger` bên trong:

```
@override
Widget build(BuildContext context) {
  return Container(
    child: Column(
      children: <Widget>[
        Text(property),
        Slider(
          min: 0,
          max: 255,
          value: _value.toDouble(),
          label: property,
          onChanged: _onChanged,
        ),
      ],
    ),
  );
}

_onChanged(double value) {
  setState(() => _value = value.round());
  onChanged(property, value.round()); // Lifting state up
}
```

Khi nào nên dùng state?

Nhưng bạn biết không? Cách tốt nhất để tránh state phức tạp là **đừng có state ngay từ đầu**. Gần như mọi chuyên gia đều đồng ý rằng nếu bạn tránh được state hoàn toàn thì hãy tránh. Nhưng có thể sẽ khó phân biệt khi nào cần state và khi nào không.

Ví dụ, nhấn trên bộ chọn màu của chúng ta là dữ liệu bên trong component. Nó có nên là state không? Tất nhiên là không; nó không thay đổi. Còn biến đếm trong một vòng lặp `for` thì sao? Cũng không; nó không bao giờ ảnh hưởng tới bất cứ thứ gì trong method `build()`, nên không cần đưa vào `setState()`. Thấy chưa? State đôi khi có thể được đơn giản hóa hoặc loại bỏ.

Hình 9-4 tóm tắt cách quyết định.





Hình 9-4. Cách bạn có thể quyết định xem một widget có thực sự cần state hay không

Quản lý state nâng cao

Những gì chúng ta xem trong chương này vẫn hoạt động đúng như quảng cáo ngay cả khi cây widget sâu vô tận. Nhưng hãy nhận ra rằng khi app càng lớn, việc quản lý state càng phức tạp. Khi nó trở nên quá phức tạp, có lẽ bạn sẽ được phục vụ tốt hơn bởi một mô hình quản lý state nâng cao. Những mô hình này không phải lúc nào cũng dễ học, nhưng tới một giai đoạn nào đó trong quá trình app lớn lên, chúng xứng đáng với công sức bỏ ra để làm chủ.

Chúng tôi muốn giữ chương này ở mức dễ tiêu hóa nên không thể đi vào chi tiết mọi công nghệ, nhưng chúng tôi cũng không muốn giả vờ rằng vấn đề không tồn tại.

InheritedWidget

Đây là giải pháp tương đối đơn giản, có lẽ quá đơn giản cho phần lớn nhu cầu. `InheritedWidget`² là một widget dựng sẵn của Flutter. Về bản chất nó tạo ra một tập nhỏ các biến toàn cục được cung cấp một cách có kiểm soát cho toàn bộ widget con cháu trong cây của nó. Một số phương pháp khác (`ScopedModel`, `Provider`, `Redux`) là các lớp bọc quanh `InheritedWidget`.

Ưu: Không cần cài hay theo dõi thư viện nào. **Nhược:** Có sự trùng lặp giữa `InheritedWidget` và `StatefulWidget` bên dưới. Đáng tiếc. Ngoài ra toàn bộ cây con bị vẽ lại khi bất kỳ dữ liệu nào thay đổi.

BLoC

BLoC là viết tắt của **B**usiness **L**ogic **C**omponent, và nó ít mang tính giải pháp mà giống một design pattern hơn. BLoC do Google tạo ra nên tự nhiên nó được cộng đồng Flutter đón nhận.

Ưu: Rất nhiều người trong cộng đồng có thể và sẵn sàng giúp bạn. Đây là một pattern vững chắc, đã được kiểm chứng kỹ. **Nhược:** Bạn phải tự viết mọi thứ; nó không có sẵn cũng không phải thư viện. Có thể khó biết nên "tiêm" BLoC vào đâu.

ScopedModel

`ScopedModel`³ là thư viện được Brian Egan "mượn không ngại ngần" từ codebase Fuchsia⁴. (Này, đó là lời của Brian chứ không phải của tôi! Anh ấy là người khiêm tốn.) `ScopedModel` tạo ra các data model có khả

năng đăng ký listener. Mỗi model thông báo cho listener của nó khi dữ liệu thay đổi để chúng cập nhật. Thiết kế thông minh.

Ưu: Làm rất tốt việc tách presentation khỏi dữ liệu. **Nhược:** Có tin đồn rằng ScopedModel sẽ được gộp với `flutter_provide`⁵ của Google — thứ có vẻ là cách tiếp cận hiện đại và đơn giản hơn để quản lý state.

Hooks

Một bản hiện thực của React hooks do Rémi Rousselet ở Paris viết, tên là `flutter_hooks`.⁶ Bạn không dùng `StatefulWidget` nữa. Thay vào đó bạn kế thừa từ `HookWidget` — vốn là stateless (do đó đơn giản hơn) nhưng cho phép bạn tạo và truy cập các hàm tùy chỉnh để đọc ghi giá trị state. Nó thậm chí đi kèm sẵn một số hook mà bạn không cần tự viết.

Ưu: Đơn giản hóa đáng kể các widget vốn stateless của bạn. **Nhược:** Có đường cong học tập. Cách chúng hoạt động và quy tắc sử dụng không hiển nhiên, trừ khi bạn đã quen với React hooks.

Provider

Tại thời điểm viết cuốn sách này, có chút nhầm lẫn giữa `Provider`⁷ — cũng do Rémi Rousselet viết — và một package tên gần giống do Filip Hracek và các đồng nghiệp ở Google viết. Filip thừa nhận thẳng thắn rằng package của Rémi "nhiều tính năng hơn".⁸

Ưu: Một package rất vững chắc và mạnh mẽ, tương đối đơn giản để dùng. Trong tương lai gần, tôi kỳ vọng nó sẽ trở thành thư viện quản lý state chủ lực cho những developer chưa nghiêng về Redux và/hoặc hooks do kinh nghiệm trước đó với hệ sinh thái React. **Nhược:** (Chưa) phổ biến bằng một số cái khác.

Lưu ý rằng có nhiều nhầm lẫn giữa package này và `flutter-provide` do Google tạo ra, vì tên gọi giống nhau. Cái sau, lấy từ codebase Fuchsia và mở mã nguồn, có thể sẽ được gộp với ScopedModel và bị khai tử.

Redux

Cũng như vài cái khác trong danh sách này, Redux là thư viện mượn từ công nghệ khác rồi chuyển sang Dart. Redux có lịch sử lâu đời đến từ thế giới React qua Facebook. Có vài bản hiện thực, nhưng phổ biến nhất là `flutter_redux`⁹ — cũng do Brian Egan viết.

Nhược: Đường cong học tập rất dốc. **Ưu:** Hiệu năng rất tốt. Khả năng mở rộng rất tốt. Nhiều developer React đã biết Redux.js sẵn rồi. Với họ đường cong học tập phẳng đi đáng kể.

Chà! Nhiều package quá!

Rối chưa? Tôi không trách bạn. Những package này đều giải quyết cùng một vấn đề theo những cách khác nhau, có cái tương tự nhau, có cái dùng chiến lược hoàn toàn khác biệt. Không ai kỳ vọng bạn phải biết gì hơn ngoài việc **nhận thức được** rằng có những công cụ như vậy ngoài kia. Khi bạn nghe ai đó nói kiểu "State của bạn mình đang rối ròi. Có lẽ nên xem qua BLoC hoặc ScopedModel", ít nhất bạn sẽ biết họ đang nói về loại thứ gì. Rồi bạn có thể đào sâu vào các công nghệ đó để xem nên dùng cái nào.

 **Ghi chú cập nhật — bức tranh quản lý state hiện nay:** Phần này là chỗ lỗi thời nhiều nhất của cuốn sách. Bối cảnh năm 2026 đã khác hẳn 2019:

Công cụ trong sách	Tình trạng hiện nay
Provider (của Rémi Rousselet)	Dự đoán của tác giả đã đúng — nó đã trở thành lựa chọn chủ lực và là thứ Flutter chính thức khuyến nghị cho người mới. Tuy nhiên tác giả nay coi nó ở chế độ bảo trì và hướng người dùng sang Riverpod.
Riverpod	Không có trong sách vì chưa ra đời. Đây là bản viết lại Provider của cùng tác giả, khắc phục các hạn chế của Provider (không phụ thuộc <code>BuildContext</code> , kiểm tra lỗi ở thời điểm biên dịch). Là lựa chọn phổ biến nhất hiện nay cho dự án mới.
BLoC	Vẫn rất sống. Nay đã có thư viện chính thức <code>flutter_bloc</code> nên không còn phải tự viết mọi thứ như sách mô tả. Rất phổ biến trong các đội lớn và dự án doanh nghiệp.
ScopedModel	Đã lỗi thời , không nên dùng cho dự án mới.
flutter-provide (của Google)	Đã bị khai tử đúng như tác giả dự đoán.
flutter_hooks	Vẫn được duy trì, thường dùng kèm Riverpod.
Redux	Vẫn tồn tại nhưng ít phổ biến hơn nhiều trong cộng đồng Flutter.
GetX, signals, MobX	Các lựa chọn khác xuất hiện sau khi sách ra đời.

Lời khuyên cho bạn khi mới học: đừng vội chọn thư viện nào cả. Hãy nắm chắc `setState` và "lifting state up" như chương này dạy — chúng đủ dùng cho app nhỏ và là nền tảng để hiểu mọi thư viện khác. Khi thấy việc truyền hàm và dữ liệu qua quá nhiều tầng trở nên mệt mỏi, hãy học **Provider** trước (đơn giản, tài liệu chính thức đầy đủ), rồi mới cân nhắc Riverpod hoặc BLoC.

Tổng kết

Có những lúc rất rõ ràng là một widget cần tự duy trì trạng thái của nó qua dữ liệu chứa bên trong. Khi đó chúng ta gọi dữ liệu ấy là state và gọi widget đó là stateful widget. Stateful widget về bản chất phức tạp

hơn stateless widget nên chúng ta cố tránh chúng nếu có thể. Ngoài ra, càng nhiều stateful widget thì càng nhiều state phải truyền qua lại giữa các widget. Việc này có thể trở nên rất phức tạp rất nhanh, nên chúng ta tìm tới những công cụ và kỹ thuật như BLoC, Redux, ScopedModel và Provider để thuần hóa state.

May mắn thay, đây là phần phức tạp nhất của Flutter cơ bản. Cũng không tệ lắm nhỉ. Vì chúng ta đã bàn xong mọi thứ cần thiết để tạo một app Flutter, phần này kết thúc tại đây. Ở phần còn lại của cuốn sách, hãy chuyển sang những chủ đề bổ sung — có thể là tùy chọn — nhưng sẽ giúp bạn tạo ra những app Flutter thực tế!

-
1. https://en.wikipedia.org/wiki/Single_responsibility_principle ↩
 2. <https://docs.flutter.io/flutter/widgets/InheritedWidget-class.html> ↩
 3. https://pub.dartlang.org/packages/scoped_model ↩
 4. <https://fuchsia.googlesource.com/> ↩
 5. <https://pub.dartlang.org/packages/provider> ↩
 6. https://pub.dartlang.org/documentation/flutter_hooks ↩
 7. <https://pub.dartlang.org/packages/provider> ↩
 8. <https://github.com/google/flutter-provider/issues/3> ↩
 9. https://pub.dartlang.org/packages/flutter_redux ↩

PHẦN III — Đi xa hơn

Chương 10: App Flutter làm việc với file

Đến thời điểm này trong hành trình, bạn đã học được cách tạo một app Flutter và kiểm soát chính xác diện mạo cũng như bố cục của nó ở mọi hướng màn hình, trên mọi thiết bị. Khá tuyệt! Bạn cũng biết cách để nó lưu giữ dữ liệu bằng các trường trong `Form`. Nhưng làm sao để nó **lưu** dữ liệu đó lại? Và làm sao để **đọc** dữ liệu đó ngay từ đầu?

Dữ liệu của app chỉ có thể đến từ hai nơi: từ chính thiết bị, hoặc từ việc trao đổi dữ liệu với một server bên ngoài. Chúng ta sẽ xử lý server bên ngoài ở chương sau. Trong chương này, hãy học cách đọc và ghi dữ liệu từ các phương án lưu trữ ngay trên thiết bị. Nhưng để tới được đó vào cuối chương, chúng ta phải nắm một số kiến thức tiền đề như làm việc với JSON và xử lý các hoạt động bất đồng bộ. Vậy đây là kế hoạch cho chương này:

- Đưa thư viện vào app Flutter
- Đóng gói file kèm theo app
- Future, `async` và `await`
- Đọc/ghi một file cục bộ
- Chuyển đổi JSON
- Lưu tùy chọn của người dùng

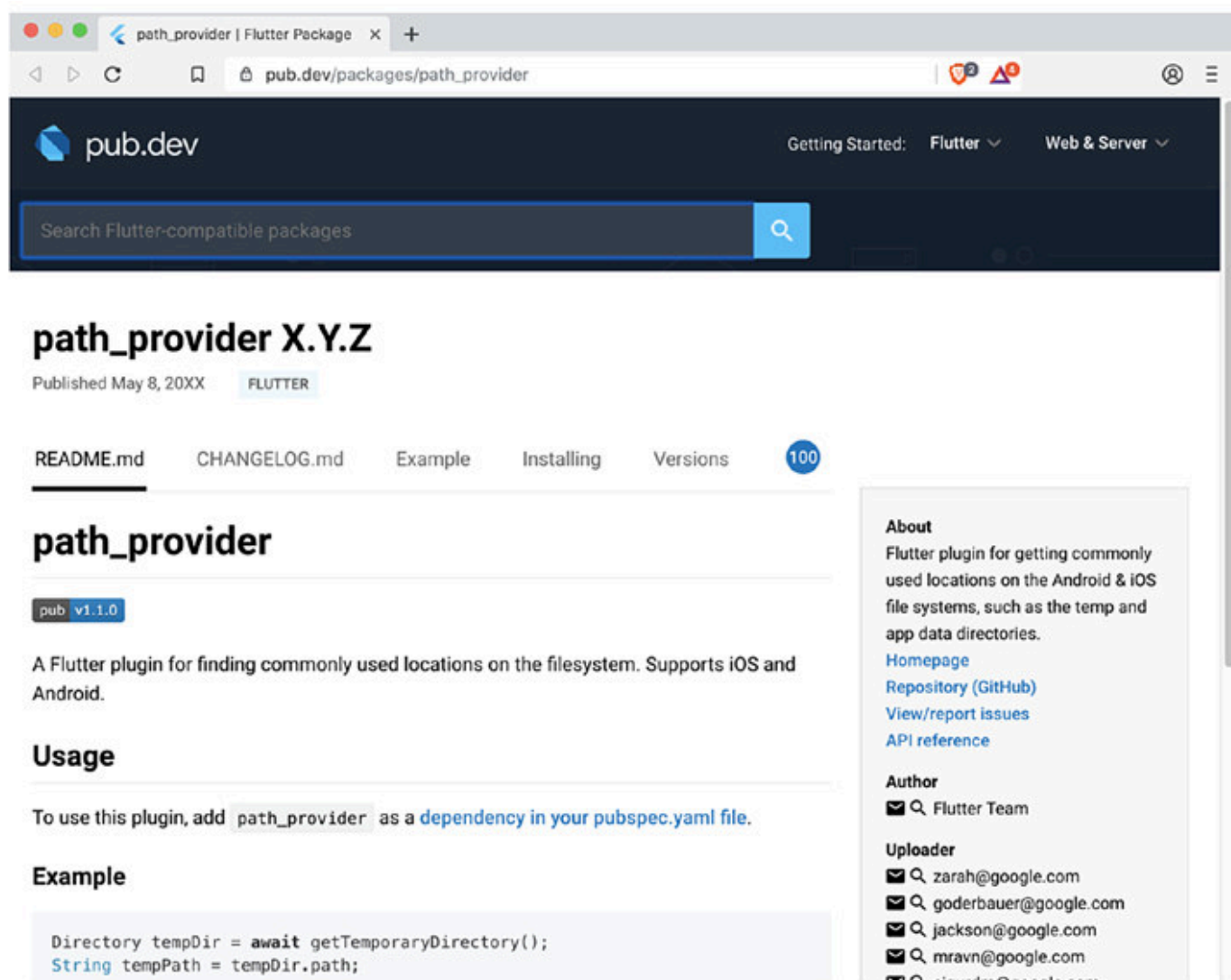
Đưa thư viện vào app Flutter

Rất nhiều developer tài năng đã viết những công cụ tuyệt vời mà bạn và tôi có thể dùng trong app Flutter của mình. Chúng thường được gọi là **thư viện** (library), và tất cả những gì ta phải làm là (1) tìm một cái ưng ý, (2) thêm nó vào file `pubspec.yaml`, (3) `import` nó trong file Dart, và (4) dùng nó trong code.

Tìm thư viện

Nói thật, đây là phần khó nhất vì không có một nơi duy nhất để tìm kiếm. Nghe có vẻ hơi buồn cười, nhưng cách tốt nhất là Google. Ở phần sau chương này, chúng ta sẽ đọc và ghi file, nên hãy lấy đó làm ví dụ. Nếu bạn Google "Flutter read file", cả năm kết quả đầu tiên đều khuyên dùng một thư viện tên là

`path_provider`. Vậy ta Google tiếp "Flutter `path_provider`" và đến được https://pub.dev/packages/path_provider, như trong Hình 10-1.



Hình 10-1. Trang chủ của `path_provider` phiên bản 1.3.0

Như bạn thấy, trang này cho ta số phiên bản (mà chúng tôi đã ẩn danh thành X.Y.Z trong ảnh chụp), cách dùng thư viện, các đoạn code mẫu và nhiều thứ khác.

Mẹo bổ sung: Hiện nay bạn nên tìm kiếm trên <https://pub.dev> — đây là kho package chính thức của Dart/Flutter, có bộ lọc và điểm đánh giá (likes, pub points, popularity) giúp bạn chọn thư viện đáng tin cậy. Package do chính đội Flutter/Google duy trì sẽ có nhãn **"Flutter Favorite"** hoặc huy hiệu **Verified publisher**.

Thêm vào pubspec.yaml

Mục **Usage** cho bạn biết những gì cần để sử dụng thư viện. Không có nút tải về và không có bước cài đặt. Thực tế, với gần như mọi thư viện Flutter/Dart, tất cả những gì bạn phải làm là sửa file `pubspec.yaml` và thêm một dòng vào mục `dependencies`:

```
dependencies:  
  flutter:  
    sdk: flutter  
  path_provider: ^X.Y.Z
```

Để ý dấu mũ (^) ở đầu. Nó nói với bộ công cụ phát triển rằng: "Bạn được dùng phiên bản mới nhất của thư viện, miễn là không vượt quá phiên bản major X." Một số developer Flutter có kinh nghiệm cho rằng để công cụ tự quyết định phiên bản thư viện là rủi ro nên họ bỏ dấu mũ đi. Tùy bạn chọn.

Ngay khi bạn lưu file `pubspec.yaml`, công cụ phát triển có thể phát hiện thư viện mới và tự tải về giúp bạn. Nếu không, chỉ cần chạy `flutter pub get`.

```
$ flutter pub get  
Running "flutter pub get" in myProject... 0.7s  
$
```

Giờ khi nó đã được tải về, bạn có thể bắt đầu sử dụng.

Import thư viện

Mở file mã nguồn bất kỳ nơi bạn cần dùng chức năng của thư viện này và thêm câu lệnh `import` vào đầu file:

```
import 'package:flutter/material.dart';  
import 'package:path_provider/path_provider.dart';
```

Bạn sẽ biết cần import gì khi đọc tài liệu của thư viện. Thấy chưa? Dễ mà.

Sử dụng thư viện

Cách dùng phụ thuộc rất nhiều vào từng thư viện và tất nhiên không thư viện nào giống thư viện nào. Hướng dẫn và ví dụ lại nằm trong tài liệu. Và tài liệu của `path_provider` bảo chúng ta gọi method

`getTemporaryDirectory()` hoặc `getApplicationDocumentsDirectory()` như sau:

```
Directory documents = await getApplicationDocumentsDirectory();  
File file = File('${documents.path}/${_filename}');
```

"Nhưng khoan," bạn nói, "cái `await` này là gì vậy?" Hãy nói về điều đó ngay bây giờ.

Future, async và await

Flutter được viết bằng Dart, và Dart là ngôn ngữ đơn luồng. Điều này nghĩa là một app Flutter chỉ làm được một việc tại một thời điểm, nhưng **không** có nghĩa là app Flutter buộc phải ngồi chờ các tiến trình chậm.

App Flutter dùng **event loop** (vòng lặp sự kiện). Điều này không có gì bất ngờ, vì Android có main looper và iOS có run loop (còn gọi là main loop). Thậm chí dân JavaScript cũng chẳng thấy lạ, vì bản thân JavaScript cũng có... event loop. Đúng vậy, thời nay ai cũng dùng event loop.

Event loop là một vòng lặp vô hạn chạy nền, định kỳ thức dậy và ngó vào hàng đợi sự kiện xem có tác vụ nào cần chạy không. Nếu có, event loop đưa chúng lên run stack — nhưng chỉ khi CPU đang rảnh.

Khi app chạy các câu lệnh, chúng chạy tuần tự — cái này nối tiếp cái kia. Nếu gặp một câu lệnh có khả năng chặn luồng chính vì phải chờ một tài nguyên nào đó, câu lệnh đó được khởi động và phần "chờ" được đưa sang một hàng đợi riêng.

Vì sao lại phải chờ?

Một số việc rất chậm so với CPU. Đọc file thì chậm. Ghi file còn chậm hơn. Còn giao tiếp qua mạng? Thôi khỏi bàn. Nếu chúng ta để hoạt động chờ đợi nằm trên luồng chính, nó sẽ chặn tất cả lệnh khác. Thật lãng phí!

Cách xử lý điều này trong JavaScript, iOS, Android và giờ là Dart như sau:

1. Một hoạt động vốn được biết là chậm sẽ được khởi động bình thường.
2. Ngay khoảnh khắc nó bắt đầu chờ thứ gì đó — ổ đĩa, HTTP request, bất cứ gì — nó được đưa ra khỏi CPU.
3. Một dạng listener được tạo ra. Nó theo dõi hoạt động đó và phát tín hiệu khi việc chờ kết thúc.
4. Tham chiếu tới listener đó được trả về cho luồng chính. Object tham chiếu này được gọi là **Future**.
5. Luồng chính tiếp tục chạy bình thường.

6. Khi hoạt động chờ cuối cùng cũng hoàn tất, event loop nhìn thấy và chạy một method liên quan (còn gọi là hàm callback) trên luồng chính để xử lý nốt phần cuối của sự kiện chậm đó.

Tất cả những gì bạn làm là viết code để tạo ra Future và để xử lý những Future được trả về từ các method khác:

```
// Giả sử goReadAFile() chậm và trả về một Future
Future myFuture = goReadAFile();
```

Trong Dart bạn có thể chỉ định kiểu dữ liệu mà Future sẽ trả về:

Kiểu Future	Khi sẵn sàng, tôi sẽ có...
<code>Future<String></code>	...một chuỗi
<code>Future<Foo></code>	...một <code>Foo</code>
<code>Future<Map<String, dynamic>></code>	...một <code>Map</code> mà key là <code>String</code> còn value là <code>dynamic</code>

Khi ta có object Future đó, có thể bạn chưa có dữ liệu, nhưng bạn chắc chắn có một **lời hứa** sẽ nhận được dữ liệu trong tương lai. (Đúng như tên gọi của nó.)

Làm sao lấy dữ liệu từ một Future?

Bạn bảo cho Future biết phải làm gì khi dữ liệu đã sẵn sàng. Về cơ bản, bạn đang phản hồi một sự kiện "Này, dữ liệu sẵn sàng rồi" và bảo Future phải làm gì bằng cách đăng ký một hàm.

```
myFuture.then(myCallback);
```

Hàm `.then()` chính là cách bạn đăng ký hàm callback đó. Callback nên được viết để xử lý dữ liệu đã được hứa hẹn. Ví dụ, nếu ta có một `Future<Foo>` thì callback của ta nên có chữ ký như sau:

```
void myCallback(Foo theIncomingData) {
  doSomethingWith(theIncomingData);
}
```

Vậy nếu Future sẽ trả về một `Person`, callback của bạn nên nhận vào một `Person`. Nếu Future hứa trả về một `String`, callback nên nhận vào một `String`. Và cứ thế.

Callback của bạn luôn nên trả về `void`, vì hàm `.then` không có cách nào nhận được giá trị trả về. Điều này cực kỳ hợp lý nếu bạn nghĩ kỹ, bởi hãy nhớ rằng nó không còn chạy trong luồng chính của app nữa nên nó không có cách nào nhập ngược trở lại. Vậy làm sao lấy được giá trị ra khỏi callback? Có vài cách, nhưng cách dễ hiểu nhất là dùng một biến được định nghĩa **bên ngoài** callback:

```
class FooState extends State<FooComponent> {  
  String _firstName; // <-- Biến mà cả class đều biết  
  
  Widget build(BuildContext context) {  
    // trả về một widget  
  }  
  
  void _myCallback(String someVar) {  
    _firstName = someVar; // <-- Lấy giá trị RA KHỎI callback bất đồng bộ  
  }  
}
```

Việc gắn thêm `.then()` vào object Future đôi khi làm code của bạn hơi rối. Nếu muốn, bạn có thể dọn dẹp một chút bằng `await`.

await

Có một cách khác để lấy dữ liệu, dễ đọc hơn. Thay vì dùng `.then()`, bạn có thể `await` dữ liệu.

```
Foo theIncomingData = await somethingThatReturnsAFuture();
```

`await` tạm dừng đoạn code đang chạy để... chờ Future hoàn tất trước khi chuyển sang dòng tiếp theo. Trong ví dụ trên, cái `Foo` mà bạn đang chờ sẽ được trả về và đưa vào `theIncomingData`. Đơn giản.

Hoặc có lẽ cũng không đơn giản đến thế...

async

Dù bạn có thích hay không, khi bạn dùng `await` bên trong một hàm, hàm đó giờ có nguy cơ chặn luồng chính nên nó **phải** được đánh dấu là `async`. Ví dụ, hàm này:

```
Bar someFunction() {
  Foo theIncomingData = somethingThatReturnsAFuture();
  return new Bar();
}
```

sẽ trở thành thế này khi ta dùng `await`:

```
Future<Bar> someFunction() async {
  Foo theIncomingData = await somethingThatReturnsAFuture();
  return new Bar();
}
```

Lưu ý rằng khi ta thêm `await` vào dòng thứ hai, ta buộc phải đánh dấu chính hàm đó là `async`. Điều tinh tế ở đây là khi hàm được đánh dấu `async`, mọi thứ được trả về từ hàm đó lập tức được bọc trong một `Future` — trừ khi nó vốn đã là một `Future`.

Bạn đang ngồi chứ? Xem này: mỗi khi bạn chọn `await` một `Future`, hàm chứa nó phải được đánh dấu `async`, và do đó mọi hàm gọi tới nó cũng phải được `await` và cũng phải đánh dấu `async`, cứ thế lan ra. Cuối cùng bạn sẽ leo đủ cao trong chuỗi gọi hàm tới chỗ không còn nằm trong hàm nào nữa, và ở đó bạn không phải đánh dấu `async`.

Có lẽ tôi đã nói hơi sớm khi bảo rằng cách này đơn giản hơn.

Gợi ý

Method `build()` của Flutter **không thể** là `async`, nhưng các sự kiện như `onPressed` thì có thể. Vậy hãy cố lái các hoạt động bất đồng bộ vào trong các sự kiện để giải quyết cái vòng luẩn quẩn `async-await-async-await` này.

Đây là những điều cần nhớ về `Future`:

1. `Future` cho phép code Dart của bạn chạy bất đồng bộ — nó có thể xử lý các tiến trình chậm ở một luồng riêng (đại khái là vậy).
2. Bạn xử lý callback của những thứ đó bằng `.then(callback)` hoặc bằng cách `await` chúng.
3. Nếu bạn `await` trong một hàm, hàm đó phải được đánh dấu `async`.

Nếu bạn muốn đọc thêm về Future, đây là tài liệu đầy đủ từ đội Dart: <https://www.dartlang.org/tutorials/language/futures>.

Ghi chú cập nhật: Đường dẫn nay là <https://dart.dev/libraries/async/async-await>. Ngoài ra, hiện nay Dart còn có `Stream` (luồng nhiều giá trị theo thời gian, xử lý bằng `await for` hoặc `StreamBuilder`) và `Isolate` (chạy code nặng trên luồng thực sự riêng biệt — dùng `Isolate.run()` cho các tác vụ tính toán nặng để không làm giạt giao diện).

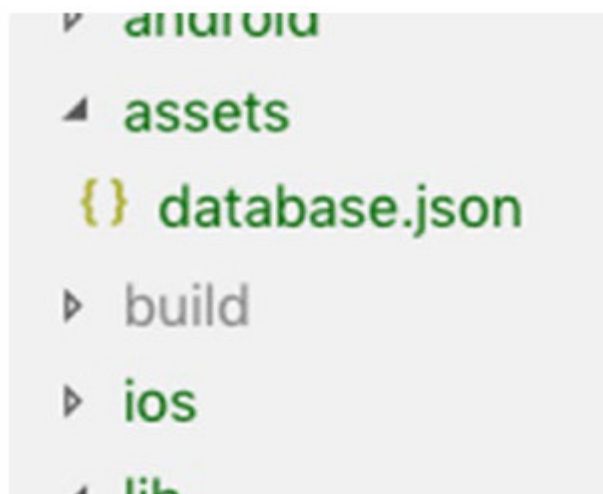
Đóng gói file kèm theo app

File mà bạn định đọc thì phải tồn tại (tất nhiên rồi). Có lẽ chúng ta nên tự tạo một cái.

Việc developer đóng gói một file phẳng để cài kèm theo app là chuyện rất bình thường. Nó rất hữu ích để khởi tạo một lượng dữ liệu lớn — kiểu như một cơ sở dữ liệu mini. Cách làm sẽ quen thuộc với bạn, vì đây chính là kỹ thuật ta đã dùng để đóng gói ảnh kèm app. Tất cả những gì bạn làm là tạo file trong IDE và khai báo nó trong `pubspec.yaml`.

Có cả tá cách để thêm file vào project. Dùng dòng lệnh, chuột phải chọn "new" trong IDE, kéo thả trong file explorer, v.v. Nhưng cuối cùng thì nó phải hiện ra trong IDE của bạn.

Thông thường (nhưng không bắt buộc) người ta tạo nó trong một thư mục tên là `assets` (Hình 10-2).



Hình 10-2. Thư mục `assets`

Nhưng dù file đã tồn tại, app vẫn không biết tới nó cho tới khi ta khai báo trong `pubspec.yaml`. Đặt nó vào mục `assets` của `pubspec.yaml` và nó sẽ được đóng gói cùng file `.ipa` / `.apk` để cài lên thiết bị:


```
# Để thêm asset vào app, hãy thêm mục assets như sau:  
assets:  
  - assets/database.json
```

Để đọc file đó, bạn dùng `rootBundle.loadString()` như sau:

```
try {  
  String data = await rootBundle.loadString('assets/db.json');  
  debugPrint(data);  
} catch (e) {  
  print('Error: $e');  
  rethrow;  
}
```

`rootBundle` thuộc `services.dart`, nên nhớ import nó:

```
import 'package:flutter/services.dart';
```

Mẹo

Nếu file asset có cấu trúc dạng key-value, method `rootBundle.loadStructuredData(key, function)` có thể là lựa chọn tốt hơn. Nó cho phép bạn truyền vào key cần đọc và một hàm để xử lý dữ liệu đọc được.

Vì dữ liệu này được ghi ở thời điểm biên dịch trên máy phát triển, nó không thể thay đổi. Nhưng chúng ta có thể tạo một file trong thư mục documents của app để đọc và ghi được. Hãy xem điều đó tiếp theo.

Ghi file

Đôi khi người dùng muốn lưu giá trị từ lần chạy này sang lần chạy khác. Và ta làm được điều đó bằng một file cục bộ nằm trên thiết bị của họ. Để tạo file, bạn chỉ cần ghi vào nó bằng `myFile.writeAsString(theString)`.

Nhưng app của chúng ta không thể ghi vào bất kỳ vị trí nào trên thiết bị. Ta phải lấy được tham chiếu tới một thư mục có quyền ghi — và đó chính xác là việc thư viện `path_provider` làm. Nhớ rằng nó có method `getApplicationDocumentsDirectory()` trả về một `Future<Directory>`. Vậy nếu ta `await` lời gọi đó, ta sẽ có được thư mục và tạo file trong đó:

```
// Lấy thư mục documents
Directory documents = await getApplicationDocumentsDirectory();
// Ghi file
try {
  File file = File('${documents.path}/${_filename}');
  await file.writeAsString(_someText);
} catch (e) {
  _message = 'Error: $e';
}
```

📌 Lưu ý

Kiểu `Directory` và `File` nằm trong thư viện `io` của Dart. Đừng quên import:

```
import 'dart:io';
```

Và đọc nó!

Đọc file thậm chí còn đơn giản hơn. Chúng ta chỉ cần dùng `File.readAsString()`:

```
File file = File('${documents.path}/${_filename}');
file.readAsString().then((String text) {
  setState(() {
    _text = text;
    _message = '_filename has this text inside it: "$_text"';
  });
}).catchError((e) {
  setState(() {
    _errorStatus = true;
    _message = 'Error: $e';
  });
});
```

✦ Lưu ý

Ở đây chúng tôi muốn dùng method `.then()` của `Future` thay vì `await`, bởi method `build()` không thể đánh dấu là `async`. `.then()` cho phép bạn xử lý `Future` mà không cần `async`. May mắn là object `File` cũng có method `readAsStringSync()` — đây là lời gọi chặn (blocking) và trả về thẳng nội dung chữ thay vì một `Future`.

```
try {
  File file = File('${documents.path}/${_filename}');
  _text = file.readAsStringSync();
  // Thành công
  _message = '_filename has this text inside it: "$_text"';
} catch (e) {
  _errorStatus = true;
  _message = 'Error: $e';
}
```

Sử dụng JSON

Khi ghi file, chúng ta đang lấy thứ gì đó từ bộ nhớ của app và lưu nó lại. Đúng là đôi khi thứ ta ghi chỉ là một giá trị đơn lẻ, nhưng rất thường xuyên nó là một object hoặc nhiều object.

Giả sử chúng ta có một danh sách người. Có thể người đầu tiên trong danh sách là Phoebe Buffay và người thứ hai là Rachel Green. Nếu định lưu danh sách này vào một file, ta sẽ phải chỉ rõ đó là một danh sách và ghi rõ các property cùng giá trị của từng người. Có vô số cách làm điều đó, nhưng cách phổ biến nhất là định dạng JSON:

```
{
  "people": [
    {
      "id": "7b5fa0b0-9760-11e9-805d-099f65ed4f55",
      "firstName": "Phoebe",
      "lastName": "Buffay",
      "occupation": "Massage Therapist"
    },
    {
      "id": "110ec58a-a0f2-4ac4-8393-c866d813b8d1",
      "firstName": "Rachel",
      "lastName": "Green",
      "occupation": "Coffee Waitress"
    }
  ]
}
```

Việc lấy dữ liệu trong bộ nhớ app và đưa nó về định dạng đó gọi là **serialize** (tuần tự hóa) dữ liệu. Đi theo chiều ngược lại — đọc dữ liệu ở định dạng JSON, mở nó ra và nạp vào bộ nhớ app — gọi là **deserialize** (giải tuần tự hóa).

Dart có một thư viện dựng sẵn tên là `dart:convert` với các method để serialize và deserialize, lần lượt là `json.encode()` và `json.decode()`.

Ghi bộ nhớ app ra JSON

Giả sử app của bạn có một object mà bạn muốn lưu trữ hoặc truyền đi. Để đưa dữ liệu đó về định dạng JSON, dùng `json.encode(someMap)` :

```
Map<String, dynamic> jsonMap = {  
  "id": _person.id,  
  "firstName": _person.firstName,  
  "lastName": _person.lastName,  
  "occupation": _person.occupation,  
};  
try {  
  String jsonString = json.encode(jsonMap);  
  await file.writeAsString(jsonString);  
} catch (e) {  
  print("Problem saving! Error: $e");  
}
```

Lưu ý

`Map` trong Dart hơi giống object trong JavaScript: một tập các cặp key/value, thường được định kiểu động. Để lấy một giá trị, bạn chỉ định key trong cặp ngoặc vuông như trong ví dụ dưới đây. Nếu bạn muốn serialize một object có kiểu chặt chẽ, bạn có thể hoặc chuyển nó thành `Map` (dễ hơn), hoặc hiện thực một method tên `toJson()` trả về một `Map` (sạch hơn). `toJson()` được tự động gọi mỗi khi `json.encode` được gọi trên một object.

Đọc JSON vào bộ nhớ app

Giờ giả sử bằng cách nào đó bạn có được một chuỗi ở định dạng JSON và muốn đọc dữ liệu đó vào app. Làm sao lấy dữ liệu ra? Dùng `json.decode()` :

```
// jsonString chứa dữ liệu JSON đã được serialize
Map<String, dynamic> personMap = json.decode(jsonString);
// "personMap" giờ là một Map với key là chuỗi
print(personMap["firstName"]);
Person p = Person(
  id: personMap["id"],
  firstName: personMap["firstName"],
  lastName: personMap["lastName"],
  occupation: personMap["occupation"],
);
```

📌 Lưu ý

`jsonEncode()` là cách viết tắt của `json.encode()`. Tương tự, `jsonDecode()` là cách viết tắt của `json.decode()`. Đây chỉ là sở thích về phong cách. Dùng cái nào bạn thích cũng được.

📖 **Ghi chú cập nhật:** Việc viết tay `toJson()` / `fromJson()` cho từng class như sách hướng dẫn vẫn đúng và vẫn nên hiểu. Nhưng với dự án thực tế có nhiều model, cộng đồng hiện dùng công cụ **sinh code tự động** để tránh viết tay và tránh lỗi chính tả tên field: `json_serializable` (kết hợp `build_runner`), hoặc `freezed` nếu bạn muốn cả immutable data class. Bạn chỉ cần khai báo class và chạy `dart run build_runner build`, phần `toJson` / `fromJson` sẽ được sinh ra.

Có thể bạn đang nghĩ tới việc dùng kỹ thuật này để lưu tùy chọn của người dùng. Và đúng, nó hoàn toàn chạy được. Nhưng nếu bạn muốn lưu giá trị giữa các lần chạy app, có một cách tốt hơn gọi là **shared preferences**.

Shared preferences

Hầu hết app đều lưu dữ liệu cục bộ giữa các lần chạy — những thiết lập như token xác thực, dữ liệu cá nhân, chế độ sáng/tối, âm thanh... nói chung là bất cứ thứ gì được coi là tùy chọn của người dùng. Trên iOS những thứ này gọi là `NSUserDefaults`. Trên Android, chúng gọi là `SharedPreferences`. Và đội Flutter đã cho chúng ta một thư viện rất hay tên là `shared_preferences`¹ để đọc ghi những giá trị này theo cách hoạt động trên mọi nền tảng. Giờ khi bạn đã biết cách thêm thư viện, việc thêm `shared_preferences` vào `pubspec.yaml` sẽ chỉ là chuyện nhỏ.

Để dùng nó, bạn cần khởi tạo một object `SharedPreferences`. Nhưng vì chúng ta đang làm việc với việc đọc từ hệ thống file, nó cần được xử lý như một hoạt động trì hoãn. May mắn là thư viện cung cấp một method static `getInstance()` trả về `Future<SharedPreferences>`. Tôi biết nghe có vẻ rối, nhưng chỉ cần nhớ rằng việc lấy tham chiếu tới bộ đọc/ghi là bất đồng bộ. Xử lý như sau:

```
SharedPreferences prefs = await SharedPreferences.getInstance();
```

Thấy chưa? Cũng không tệ. Nhưng lưu ý là nó phải được `await`.

Để ghi preferences

Để lưu vào shared preferences, dùng các method `set`:

```
prefs
  ..setString('organizationName', organizationName)
  ..setBool('isReady', true)
  ..setDouble('percentComplete', 12.5)
  ..setInt('numberOfTries', tries)
  ..setStringList('validValues', ['started', 'finished',
                                  'in process', 'approved']);
```

Mỗi lệnh này sẽ lưu vào đúng dạng tùy chọn người dùng tương ứng với từng nền tảng và trả về một `Future<bool>` — giá trị `bool` là `true` nếu thành công và `false` nếu không. Bỏ qua giá trị này cũng không sao nếu bạn muốn. Nhiều developer bỏ qua vì trường hợp lỗi cực kỳ hiếm.

Để đọc preferences

Nếu ghi là các method `set`, thì bạn hẳn đoán được đọc là các method `get`, và bạn đoán đúng:

```
String organizationName = prefs.getString('organizationName');
bool isReady = prefs.getBool('isReady');
double percentComplete = prefs.getDouble('percentComplete');
int numberOfTries = prefs.getInt('numberOfTries');
List<String> validValues = prefs.getStringList('validValues');
```

 **Ghi chú cập nhật:** Với null safety, các method `get` trả về kiểu **nullable** vì key có thể chưa tồn tại. Bạn cần xử lý null:

```
String organizationName = prefs.getString('organizationName') ?? 'Chưa đặt';  
bool isReady = prefs.getBool('isReady') ?? false;
```

Ngoài ra, `shared_preferences` phiên bản mới giới thiệu API `SharedPreferencesAsync` và `SharedPreferencesWithCache` để thay cho `getInstance()`. Cách cũ vẫn chạy nhưng đã được đánh dấu legacy.

Quan trọng về bảo mật: đừng lưu token xác thực hay dữ liệu nhạy cảm vào `shared_preferences` — nó **không được mã hóa**. Hãy dùng `package flutter_secure_storage` cho những dữ liệu đó (nó dùng Keychain trên iOS và EncryptedSharedPreferences trên Android).

Tổng kết

Như đã nói ở đầu chương, chúng ta đang xây nền để tiến tới việc trao đổi dữ liệu với server — một chủ đề rất phức tạp. Nhưng điều làm nó phức tạp là có quá nhiều công nghệ và kỹ thuật tham gia: thư viện của bên thứ ba, serialize và deserialize JSON, Future, và việc đọc ghi bất đồng bộ. Nhưng vì chúng ta đã xử lý xong những chủ đề đó trong chương này, phần còn lại sẽ không còn đáng sợ nữa.

Sẵn sàng học về gọi API chưa? Bắt đầu thôi!

1. Đọc thêm tại https://pub.dev/packages/shared_preferences ↩

Chương 11: Gọi RESTful API bằng HTTP

Giờ chúng ta đã biết cách tạo app Flutter, dùng thư viện bên ngoài và đọc ghi dữ liệu JSON một cách bất đồng bộ. Điều đó đặt chúng ta vào vị trí rất thuận lợi để tiến thêm một bước nhỏ nữa: trao đổi dữ liệu với một API server — thứ tuyệt đối cần thiết nếu chúng ta định viết app thực tế. Đây là những gì cần biết:

- API call rốt cuộc là gì?
- Thực hiện HTTP GET hoặc DELETE request.
- Thực hiện HTTP POST, PUT hoặc PATCH request.
- Xử lý response theo cách đơn giản nhất.
- Xử lý gọn hơn với `FutureBuilder` và `StreamBuilder`.
- Xử lý gọn hơn với object có kiểu chặt chẽ.

Đó là kế hoạch cho chương này. Cộng thêm chút thực hành trực tiếp với một website cho phép cập nhật qua HTTP. Và để chắc chắn mọi người cùng chung một xuất phát điểm, có lẽ chúng ta nên bắt đầu bằng việc API chính xác là gì. Cứ thoải mái đọc lướt hoặc bỏ qua nếu bạn đã quen.

API call là gì?

App của bạn đã có khả năng đọc từ một cơ sở dữ liệu nhỏ nằm ngay trên máy. Nhưng nó không thể đọc từ một cơ sở dữ liệu ở nơi khác. Nói cách khác, bạn không thể chỉ việc kết nối vào database của Google rồi đọc ghi bản ghi. Không làm được, kể cả khi bạn có thông tin đăng nhập database. Ý tôi là, hãy nghĩ tới hệ lưu bảo mật nếu ai cũng có thể kết nối từ bất cứ đâu và sửa trực tiếp dữ liệu của Google.

Vậy nên thay vào đó, developer tạo và chạy một chương trình phía server để đọc ghi một cách có kiểm soát, rồi công khai chương trình đó trên Internet tại một địa chỉ cụ thể với một giao thức cụ thể, thường là `https`.

Để đọc dữ liệu phía server này, người dùng có thể gửi HTTP request sau khi đã gửi thông tin xác thực dưới dạng username/password, hoặc tốt hơn là một khóa bí mật duy nhất gọi là **API key**.

Lại xuất hiện thuật ngữ API. Nó là viết tắt của **Application Programming Interface**. Nó mang nghĩa khác nhau trong những tình huống khác nhau, nhưng nghĩa mặc định của nó nay đã trở thành: bất kỳ địa chỉ Internet nào mà developer có thể gửi HTTP request tới nhằm mục đích đọc và ghi dữ liệu. Có rất nhiều API công khai sẵn có và nhiều lựa chọn để tự tạo API của riêng bạn.

Khi một API phản hồi, nó trả về một luồng dữ liệu gần như luôn ở định dạng JSON.¹

Các loại API request

Giao tiếp với API server chỉ được thực hiện qua vài kiểu (Bảng 11-1).

Bảng 11-1. Các HTTP method và mô tả

HTTP method	Mục đích	Ghi chú
GET	Đọc bản ghi	Giống đọc database, đơn giản là hỏi xin dữ liệu từ server
DELETE	Xóa bản ghi	Xóa bản ghi được trỏ tới bởi ID cung cấp. Không trả về dữ liệu
POST	Chèn bản ghi mới	Tạo bản ghi mới, kể cả khi đã có bản ghi giống hệt
PUT	Thay thế bản ghi có sẵn	Ghi đè hoàn toàn bản ghi cũ bằng bản ghi này. Xóa bản ghi cũ và đặt bản ghi này vào chỗ đó
PATCH	Cập nhật bản ghi có sẵn	Giữ nguyên bản ghi cũ nhưng cập nhật các trường bằng dữ liệu từ request này

Còn **HEAD, CONNECT, OPTIONS, TRACE** — dành cho các loại request khác. Ít khi được app thông thường dùng tới. Đọc thêm tại <http://bit.ly/HTTPMethods> nếu bạn muốn.

Developer hiếm khi dùng gì khác ngoài GET, POST, PUT, PATCH và DELETE. Tất cả đều được thực hiện trong Flutter bằng một thư viện Dart mà bạn có được khi import `http.dart`.

Trước tiên, bạn thêm package `http` vào mục `dependencies` trong `pubspec.yaml`. Khi bạn thêm xong và chạy `flutter pub get`, package sẽ được tải về từ <https://pub.dev/packages/http>.

```
import 'package:http/http.dart';
```

Lệnh này sẽ cung cấp class `http` với các method tương ứng từng HTTP method. Giờ hãy xem cách gửi request bằng thư viện này.

Thực hiện HTTP GET hoặc DELETE request

Chúng ta bắt đầu với GET và DELETE trước vì chúng đơn giản nhất; chúng không bao giờ có body.² Thực tế, điểm phức tạp duy nhất là HTTP request được thực hiện bất đồng bộ. Chúng trả về một `Future` mà bạn phải xử lý bằng `.then()` hoặc `await`. Vậy có thể viết request như sau:

```
String url = 'https://us.com/people/1234';
Response response = await get(url);
print(response.statusCode); // 200, hy vọng vậy
Map<String, dynamic> person = json.decode(response.body);
print(person['first']);
print(person['last']);
print(person['imageUrl']);
print(person['email']);
```

hoặc dùng `.then()` như sau:

```
get(url).then((Response res) {
  print(res.statusCode); // 200, hy vọng vậy
  Map<String, dynamic> person = jsonDecode(res.body);
  print(person['first']);
  print(person['last']);
  print(person['imageUrl']);
  print(person['email']);
});
```

DELETE request được thực hiện theo cùng cách. Thực tế nó thường còn đơn giản hơn vì thường không có giá trị trả về. DELETE hoặc là thành công và không trả về gì, hoặc thất bại với response mã 400 hoặc 500:

```
Response response = await delete(url);
```

⚠️ **Cẩn trọng**

Khi thực hiện HTTP request thuộc bất kỳ loại nào, bạn luôn nên mã hóa URL trước khi gửi. Việc này giúp đảm bảo URL hợp lệ và cũng giúp về mặt bảo mật, đặc biệt khi nhận input từ người dùng. Hãy gọi `Uri.encodeFull` như sau:

```
String url = Uri.encodeFull('http://us.com/api/ppl?query=Jo Ki');
```

Để cho đơn giản, chúng tôi sẽ bỏ qua việc mã hóa trong các ví dụ. Nhưng trong thực tế, hãy luôn nhớ làm điều này.

 **Ghi chú cập nhật:** Các hàm `get()`, `post()`, `put()`, `delete()` của package `http` **không còn nhận `String`** làm tham số URL. Chúng yêu cầu một object `Uri`. Cách viết hiện nay:

```
final uri = Uri.parse('https://us.com/people/1234');  
final response = await http.get(uri);
```

Cũng nên `import 'package:http/http.dart' as http;` (có tiền tố `http.`) thay vì import trần, để tránh xung đột tên hàm `get` / `delete` với những thứ khác trong file.

Nếu app của bạn cần chức năng nâng cao (interceptor, tự động retry, hủy request, upload file), cộng đồng thường dùng package `dio` thay cho `http`.

Thực hiện HTTP PUT, POST hoặc PATCH request

PUT, POST và PATCH rất giống GET và DELETE. Khác biệt lớn nhất là PUT, POST và PATCH đều cần một **body** cho request — thường là một chuỗi chứa key-value định dạng JSON:

```
String payload = '{"first":"Kamala", "last":"Khan", "id":374}';  
Response response = await post(url, body: payload);
```

Response này được mở ra y hệt như với GET và DELETE.

Lưu ý

Với POST, PUT và PATCH, chúng ta gửi dữ liệu từ client tới server. Việc báo cho server biết ta đã mã hóa dữ liệu đó thế nào là khôn ngoan và đôi khi là bắt buộc. Ta làm điều đó qua một HTTP header đính kèm request. Hãy cung cấp key "Content-Type" với giá trị "application/json", như sau:

```
Map<String, String> headers = {'Content-Type': 'application/json'};  
Response res = await post(url, headers: headers, body: payload);
```

Nhân tiện, có rất nhiều biến header mà bạn có thể thấy hữu ích như Accept, Accept-Encoding, Authorization, Content-MD5, Cookie, Date, Host, If-Modified-Since và nhiều nữa. Đọc thêm tại https://en.wikipedia.org/wiki/List_of_HTTP_header_fields#Request_fields.

Gọi HTTP request tới một API cũng không tệ lắm nhỉ? Rất nhanh chóng, chúng ta đã làm cho app Flutter có khả năng gửi request, deserialize response và in ra debug console. Nhưng Flutter là về việc hiển thị dữ liệu đó trong những widget đẹp mắt. Vậy làm sao tích hợp request vào widget?

Đưa HTTP response vào widget

Có vài cách để chờ Future hoàn tất rồi hiển thị. Chúng ta sẽ đơn giản hóa bằng cách chỉ trình bày ba cách: cách thô sơ, FutureBuilder và StreamBuilder. Cách thô sơ thì rõ ràng và dễ hiểu, nhưng tôi nghĩ bạn sẽ thích FutureBuilder / StreamBuilder hơn vì chúng gọn gàng và thanh lịch hơn.

Cách thô sơ — cách dễ

Bạn đã có sẵn mọi công cụ cần thiết để hiển thị dữ liệu: bạn hiểu Future và bạn biết cách bảo stateful widget tự vẽ lại với dữ liệu mới — setState(). Vậy nó có thể đơn giản như việc đặt một setState() bên trong .then() hoặc sau await:

```
String url = 'http://us.com/api/people/12345';
Response response = await get(url);
Map<String, dynamic> responseBody = json.decode(response.body);
String first = responseBody['first'];
String last = responseBody['last'];
String imageUrl = responseBody['profilePictureUrl'];
Widget card = Stack(
  children: <Widget>[
    Image.network(imageUrl,
      height: 300, width: 300, fit: BoxFit.cover),
    Text("$first $last"),
  ],
);
setState(() {
  _cardWidget = card;
});
```

Và tất nhiên, miễn là method `build` của bạn có hiển thị `_cardWidget` ở đâu đó, nó sẽ được render với dữ liệu đúng ngay khi Future hoàn tất — điều chỉ xảy ra khi HTTP GET request trả về dữ liệu. Dễ như ăn bánh! Nhưng đây chưa phải cách thanh lịch nhất.

FutureBuilder — cách gọn gàng

Giải pháp tốt hơn có thể là widget `FutureBuilder`. Nếu bạn từng ở trong tình huống có một Future mà khi hoàn tất sẽ mang dữ liệu cần render ra widget Flutter, hãy tìm tới `FutureBuilder`. Kịch bản này nghe quen chứ? Hẳn là quen, vì đó chính là lý do lớn nhất khiến Flutter có Future. Ví dụ code đơn giản ở trên có thể được viết đầy đủ hơn nhiều với `FutureBuilder`:

```
FutureBuilder(  
  future: get(url),  
  builder: (BuildContext ctx, AsyncSnapshot<dynamic> snapshot) {  
    if (snapshot.connectionState != ConnectionState.done) {  
      return const CircularProgressIndicator();  
    }  
    if (snapshot.hasError) {  
      return Text('Oh no! Error! ${snapshot.error}');  
    }  
    if (!snapshot.hasData) {  
      return const Text('Nothing to show');  
    }  
    final Map<String, dynamic> responseBody =  
      json.decode(snapshot.data.body);  
    final int statusCode = snapshot.data.statusCode;  
    if (statusCode > 299) {  
      return Text('Server error: $statusCode');  
    }  
    String first = responseBody['first'];  
    String last = responseBody['last'];  
    String imageUrl = responseBody['profilePictureUrl'];  
  
    return Stack(  
      children: <Widget>[  
        Image.network(imageUrl,  
          height: 300, width: 300, fit: BoxFit.cover),  
        Text("$first $last"),  
      ],  
    );  
  },  
);
```

Không cần `setState()` vì `FutureBuilder` có quyền truy cập vào chính cái `Future` nên nó biết khi nào và làm sao để tự vẽ lại. Trong ví dụ trên, bạn có thể thấy nó có khả năng hiển thị thứ khác nhau cho từng tình

huống: một `ProgressIndicator` trong khi chờ Future hoàn tất, một thông báo lỗi nếu có gì sai, một thông báo nếu Future không có gì bên trong, và tất nhiên là widget thật khi dữ liệu về thành công!

⚠️ Cẩn trọng

Hãy luôn kiểm tra `snapshot.hasData` và/hoặc `snapshot.hasError` **trước khi** truy cập `snapshot.data`. Tại thời điểm viết sách,³ Flutter không ném exception khi có lỗi. Thay vào đó nó nuốt lỗi và thất bại trong im lặng.

Cũng hãy cẩn thận với mã trạng thái HTTP nằm trong `response.statusCode`! Nếu con số đó nằm ở dải 400 hoặc 500, bạn đã nhận được response hợp lệ từ server, nhưng đó là một vấn đề và dữ liệu của bạn sẽ là null.

📌 **Ghi chú cập nhật:** Có một lỗi tinh vi trong ví dụ `FutureBuilder` ở trên mà bạn nên biết: đặt `future: get(url)` **trực tiếp** trong `build()` khiến request được gửi lại **mỗi lần widget vẽ lại** — có thể là hàng chục lần. Đây là lỗi cực kỳ phổ biến với người mới.

Cách đúng là tạo Future **một lần** trong `initState()` rồi lưu vào một biến:

```
late final Future<http.Response> _futurePerson;

@override
void initState() {
  super.initState();
  _futurePerson = http.get(Uri.parse(url)); // gọi đúng một lần
}

@override
Widget build(BuildContext context) {
  return FutureBuilder<http.Response>(
    future: _futurePerson, // dùng lại Future đã tạo
    builder: (ctx, snapshot) { /* ... */ },
  );
}
```


StreamBuilder

`FutureBuilder` làm gì với `Future` thì `StreamBuilder` làm điều đó với `Stream`. Hai class này gần như giống hệt nhau: cùng định dạng, dùng cùng kiểu `snapshot`, và cùng kiểm tra `snapshot.hasError` và `snapshot.hasData`. Nhưng đôi khi ta không làm việc với một lần trả dữ liệu duy nhất như `Future`, mà với một **luồng** dữ liệu có thể cập đến từng đợt. Khi ở tình huống đó, bạn sẽ muốn dùng `StreamBuilder`:

```
StreamBuilder(  
  stream: anythingThatReturnsAStream(),  
  builder: (BuildContext ctx, AsyncSnapshot<dynamic> snapshot) {  
    // Mọi thứ bên dưới gần như giống hệt FutureBuilder,  
    // chỉ khác là dữ liệu là một tập các document,  
    // mỗi document là một bản ghi  
    if (snapshot.connectionState != ConnectionState.done) {  
      return const CircularProgressIndicator();  
    }  
    if (snapshot.hasError) {  
      return Text('Oh no! Error! ${snapshot.error}');  
    }  
    if (!snapshot.hasData) {  
      return const Text('Nothing yet. Please wait ...');  
    }  
    return ListView.builder(  
      itemCount: snapshot.data.documents.length,  
      itemBuilder: (BuildContext context, int i) {  
        String first = snapshot.data.documents[i]['first'];  
        String last = snapshot.data.documents[i]['last'];  
        String imageUrl = snapshot.data.documents[i]['imageUrl'];  
        return Stack(  
          children: <Widget>[  
            Image.network(imageUrl,  
              height: 300, width: 300, fit: BoxFit.cover),  
            Text("$first $last"),  
          ],  
        );  
      },  
    );  
  },  
);
```

 Lưu ý

Việc viết code như thế này — code tự thức dậy và tự cập nhật dựa trên dữ liệu mới đến — có một tên gọi: **reactive programming** (lập trình phản ứng). Reactive programming xảy ra khi ta làm cho app nhận biết được các tác động từ bên ngoài và bảo nó phản ứng lại theo cách nào đó. Có thể bạn đã nghe tới các reactive extension như rxJava, rxJS và rx.NET — những thư viện chứa class và hàm dành cho phong cách này. Và không có gì bất ngờ, Flutter cũng có một cái tên là **rxDart**. Bạn tìm nó tại <https://github.com/ReactiveX/rxdart>.

Class có kiểu chặt chẽ

Đến đây, bạn đã biết cách gọi HTTP tới một API, và khi nhận được response, bạn biết cách mở dữ liệu ra để dùng. Điều này đặt chúng ta vào vị trí tốt để chuyển dữ liệu đó thành một class có kiểu chặt chẽ bằng **typed deserialization pattern**.⁴

 Lưu ý

Đây không bắt buộc để gọi HTTP. Nó chỉ là cách sạch sẽ hơn để xử lý lời gọi và đưa dữ liệu vào một cấu trúc có thể dự đoán được. Dữ liệu HTTP về bản chất là phi cấu trúc. Đây là best practice được nhiều developer Flutter dùng nhưng hoàn toàn không bắt buộc. Nếu bạn không thích, cứ thoải mái bỏ qua.

Typed deserialization diễn ra qua ba bước đơn giản:

1. Tạo class nghiệp vụ (business class).
2. Viết method `.fromJson()` và/hoặc `.fromJsonArray()`.
3. Khi đọc từ lời gọi HTTP, dùng `.fromJson()` để "hydrate" object.

Tạo class nghiệp vụ

Giả sử chúng ta đang đọc ghi dữ liệu về người. Ta nên tạo một class `Person` :

```
class Person {
  // Constructor chỉ cần vì property name là một map các chuỗi
  // và cần được khởi tạo
  Person() {
    name = <String, String>{};
  }
  String documentID;
  Map<String, String> name;
  String email;
  String imageUrl;
}
```

Viết method .fromJson()

Đây nên là một method static trả về một instance của class nghiệp vụ — trong trường hợp này là `Person` :

```
class Person {
  // Các phần code khác của class ở đây

  static Person fromJson(String jsonString) {
    final Map<String, dynamic> data = jsonDecode(jsonString);
    return Person()
      ..name = data['name']
      ..email = data['email']
      ..imageUrl = data['imageUrl'];
  }

  // và có thể thêm code khác của class ở đây
}
```

Lưu ý việc dùng **toán tử cascade** (`..`) của Dart và việc bỏ toán tử `new`. Cả hai đều là best practice.

Dùng .fromJson() để hydrate object

Từ *hydrate* nghĩa đen là "thêm nước". Trong ngữ cảnh này, dữ liệu chính là nước, và chúng ta đang tạo một object `Person` mới bằng cách đổ dữ liệu vào nó. Bạn đọc dữ liệu từ một HTTP service bằng method

`.get()` rồi truyền nó vào `.fromJson()` như sau:

```
// Gọi HTTP
final Response res = await get(url);
// Hydrate một object Person từ body của response - một chuỗi JSON
Person p = Person.fromJson(res.body);
```

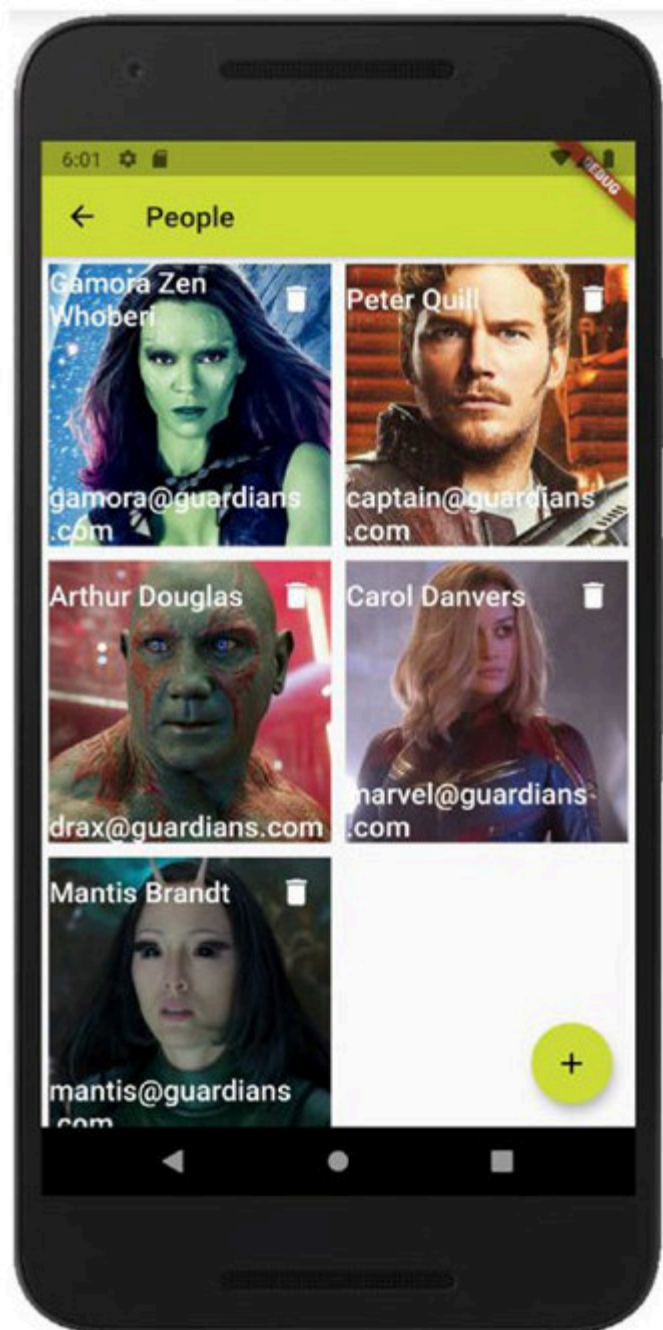
Thấy code sạch sẽ và rõ ràng chưa?

Tôi đoán rằng lúc này bạn muốn vận dụng toàn bộ kiến thức mới học. Hãy làm điều đó với một API service miễn phí ngay sau đây.

Một ví dụ lớn

Một API service thực thụ sẽ bao gồm một database với các endpoint GET, POST, DELETE, PUT và/hoặc PATCH được công khai — tất cả đều đòi hỏi khá nhiều thiết lập phía server. Sớm muộn bạn cũng sẽ tới đó, và chúng tôi sẽ giới thiệu một giải pháp lâu dài tuyệt vời ở chương sau. Nhưng bây giờ, hãy dùng một trang web chỉ để demo, không tốn phí và dễ thiết lập, để chúng ta có thể xây một app Flutter dùng dữ liệu API.

Hãy xây một app CRUD quản lý danh sách người (Hình 11-1).



Hình 11-1. Một app quản lý bản ghi, đọc và ghi dữ liệu từ một RESTful API

App này sẽ đọc danh sách người từ một HTTP web service tuân theo nguyên tắc REST — trong đó có việc hỗ trợ các HTTP method GET, POST, PUT và DELETE mà chúng ta vừa học. Ngoài ra, mọi bản ghi sẽ được gán một ID duy nhất trên server khi được tạo. Chúng ta sẽ tạo màn hình như trong hình để hiển thị danh sách người. Ta cũng cần một màn hình khác với các trường để nhập tên, email và địa chỉ ảnh của người đó.

Nghe vui chứ? Sẽ vui thôi sau khi chúng ta thiết lập xong server. Hãy bắt đầu từ đó.

Thiết lập

Rõ ràng chúng ta cần một server. Ta sẽ phải dựng một cái, cài và cấu hình database cùng một service nhận HTTP request ở cổng 80 và một loạt thứ khác. Hoặc là thế, hoặc chúng ta dùng của người khác.

Xin giới thiệu **Pipedream**!

Pipedream.com cung cấp các workflow trên cloud, nơi ngay cả developer chưa có kinh nghiệm cũng dễ dàng tạo ra những quy trình phức tạp. Dịch vụ tuyệt vời của họ cũng miễn phí với lượng dữ liệu và xử lý ở mức hợp lý. Những người tốt bụng ở Pipedream đã tạo sẵn một workflow công khai một giao diện RESTful đơn giản, cho phép chúng ta đọc ghi dữ liệu qua một RESTful API công khai với rất ít công sức và hoàn toàn miễn phí.

Bạn có thể làm tất cả một cách ẩn danh, nhưng chúng tôi khuyên bạn nên đăng nhập bằng tài khoản để có thể quay lại xem dữ liệu sau này. Vậy hãy mở <http://pipedream.com> trong trình duyệt. Tạo một tài khoản với mật khẩu.

Truy cập liên kết này: http://bit.ly/pipedream_api. Bấm nút **"Fork"** to ở góc trên bên phải. Việc này sẽ tạo một bản sao của riêng bạn mà bạn sở hữu và có thể chỉnh sửa nếu cần. Ở đầu trang bạn sẽ thấy một URL. Đây là URL bạn sẽ gửi request tới.



Để kiểm tra, hãy mở dòng lệnh. Dùng lệnh `curl` để chắc chắn mọi thứ đã sẵn sàng.

```
$ curl https://en7btxnrlhxnijq.m.pipedream.net
```

Nếu bạn nhận được phản hồi "Success" từ server thì nó đang hoạt động và bạn có thể đi tiếp.

 **Ghi chú cập nhật:** Liên kết `bit.ly/pipedream_api` và workflow mẫu này có từ năm 2019 nên **rất có thể đã không còn hoạt động**. Nếu bạn muốn làm theo ví dụ này, đây là vài lựa chọn thay thế còn sống:

- **`https://jsonplaceholder.typicode.com`** — API giả lập miễn phí, không cần đăng ký. Có sẵn endpoint `/users`, `/posts`. Chấp nhận POST/PUT/DELETE nhưng **giả vờ** lưu (dữ liệu không thực sự thay đổi) — vẫn đủ để học cách gửi request.
- **`https://crudcrud.com`** — cho bạn một endpoint REST thật, có lưu dữ liệu, miễn phí trong 24 giờ.
- **`https://mockapi.io`** — tạo API giả có lưu dữ liệu, miễn phí ở mức cơ bản.
- Hoặc bỏ qua phần này và đi thẳng tới **Chương 12 (Firebase)** — đó là giải pháp server thật mà chính tác giả giới thiệu ở cuối chương.

Trong các đoạn code phía dưới, `$pipedreamRESTURL` chỉ là một hằng chứa URL gốc của API. Bạn thay bằng URL của dịch vụ bạn chọn là được.

Tạo app Flutter

Tạo một app Flutter mới bằng `flutter create`. Mở `main.dart` và tìm widget `MaterialApp`. Bỏ property `home` đi và thêm vào:

```
initialRoute: '/peopleList',
routes: <String, WidgetBuilder>{
  '/peopleList': (BuildContext ctx) => PeopleList(),
  '/peopleUpsert': (BuildContext ctx) => PeopleUpsert(),
},
```

Sau đó tạo hai `StatelessWidget` mới, một cái tên `PeopleList.dart` và cái kia tên `PeopleUpsert.dart`. Chúng ta sẽ điền chi tiết ngay sau đây. Nhưng trước tiên, có lẽ nên tạo một class nghiệp vụ để đại diện cho object `Person`.

Tạo class nghiệp vụ có kiểu chặt chẽ

Vì chúng ta đang làm việc với `Person`, có lẽ nên tạo một class `Person` để chứa từng người. Best practice hoàn toàn tùy chọn này có thể giúp ta tránh lỗi khi serialize và deserialize dữ liệu server, đồng thời cho ta một nơi tập trung để quản lý mọi logic liên quan tới `Person`:


```
import 'dart:convert';

class Person {
  // Constructor chỉ cần vì name là một map các chuỗi
  // và cần được khởi tạo
  Person() {
    name = <String, String>{};
  }

  // Typed deserialization pattern cho một người
  static Person fromJson(String jsonString) {
    final Map<String, dynamic> data = jsonDecode(jsonString);
    return Person()
      ..id = data['id']
      ..name = data['name']
      ..email = data['email']
      ..imageUrl = data['imageUrl'];
  }

  // Typed deserialization pattern cho danh sách người
  static List<Person> fromJsonArray(String jsonString) {
    final Iterable<dynamic> data = jsonDecode(jsonString);
    return data.map<Person>((dynamic d) => Person()
      ..id = d['id']
      ..name = {'first': d['first'], 'last': d['last']}
      ..email = d['email']
      ..imageUrl = d['imageUrl']).toList();
  }

  // Các property thực sự của một người
  int id;
  Map<String, String> name;
  String email;
```

```
String imageUrl;  
}
```

PeopleList.dart

Cuối cùng chúng ta sẽ đọc danh sách người từ RESTful service và muốn hiển thị dữ liệu của họ. Widget `PeopleList` chịu trách nhiệm hiển thị danh sách đó:

```
import 'package:flutter/material.dart';
import 'package:http/http.dart';
import 'Person.dart';
import 'sensitiveConstants.dart';

class PeopleList extends StatefulWidget {
  @override
  _PeopleListState createState() => _PeopleListState();
}

class _PeopleListState extends State<PeopleList> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('People')),
      body: scaffoldBody,
      floatingActionButton: FloatingActionButton(
        // Nút Thêm. Khi người dùng chạm vào, ta đưa họ
        // tới PeopleUpsert mà KHÔNG kèm object person.
        child: Icon(Icons.add),
        onPressed: () {
          Navigator.pushNamed(context, '/peopleUpsert');
        },
      ),
    );
  }

  // Để ý cách chúng ta tách chi tiết ra để widget trừu tượng hơn.
  // Chúng ta cũng làm vậy với personWidget bên dưới.
  Widget get scaffoldBody {
    return FutureBuilder<dynamic>(
      future: fetchPeople(), // Cách chúng ta lấy danh sách người
      builder: (BuildContext context, AsyncSnapshot snapshot) {
        if (snapshot.hasError) {
          return Text('Oh no! Error! ${snapshot.error}');
        }
      },
    );
  }
}
```

```

    }
    if (!snapshot.hasData) {
      return const Text('No people found');
    }
    // Chuyển dữ liệu JSON thành mảng các Person
    final List<Person> people =
      Person.fromJsonArray(snapshot.data.body);
    // Chuyển danh sách person thành danh sách widget
    final List<Widget> personTiles = people
      .map<Widget>((Person person) => personWidget(person))
      .toList();
    // Hiển thị tất cả person tile trong một GridView cuộn được
    return GridView.extent(
      maxCrossAxisExtent: 300,
      children: personTiles,
    );
  },
);
}

// Hiển thị một person tile đơn lẻ.
Widget personWidget(Person person) {
  // Xem mã nguồn trên GitHub để biết chi tiết. Điều quan trọng là
  // khi người dùng chạm vào một person tile, ta điều hướng họ
  // tới PersonUpsert và truyền object person vào.
}
}

```

GET request trong Flutter

Hãy nhìn lại method `scaffoldBody`. Nó có một `FutureBuilder`. Property `future` trỏ tới một method tên `fetchPeople()` — method này chỉ cần gửi một GET request tới URL sẽ phản hồi bằng một mảng JSON các bản ghi `Person`:

```
Future<dynamic> fetchPeople() {  
  // pipedreamRESTUrl là URL bạn đã ghi lại trước đó  
  final String url =  
    '$pipedreamRESTUrl/people/?pipedream_response=1';  
  return get(url);  
}
```

GET request khá đơn giản khi bạn đã dựng xong hạ tầng Flutter, đúng không? Tất nhiên khi bạn gọi lần đầu, sẽ chẳng có gì hiện ra vì bạn chưa tạo người nào. Việc đó sẽ tới sớm thôi.

DELETE request trong Flutter

Mỗi person tile có một `IconButton` hình thùng rác ở góc trên bên phải. Chạm vào nó sẽ gọi `deletePerson()`, nhận vào người ta muốn xóa. Method `deletePerson()` này sẽ gửi một HTTP DELETE request, trở tới người đó theo ID:

```
void deletePerson(Person person) {  
  final String url =  
    '$pipedreamRESTUrl/people/${person.id}?pipedream_response=1';  
  delete(url).then((Response res) {  
    // Gọi setState() để vẽ lại SAU KHI người đó đã bị xóa  
    setState(() {  
      print('Status code: ${res.statusCode}');  
    });  
  });  
}
```

Lưu ý rằng sau khi Future của lệnh delete hoàn tất, chúng ta gọi `setState()` để buộc scene vẽ lại, qua đó làm mới danh sách người.

PeopleUpsert.dart

Chúng ta đã lo xong việc đọc danh sách người và xóa một người trong `PeopleList`. Nhưng thêm người mới sẽ cần một form để người dùng nhập thông tin. Bạn đọc tinh ý sẽ nhận ra rằng cần đúng cái form giống hệt để cập nhật người đã có. Để tuân theo nguyên tắc DRY,⁵ hãy tạo một form và dùng lại nó cho cả thêm lẫn sửa:

```
import 'package:flutter/material.dart';
import 'package:http/http.dart';
import 'Person.dart';
import 'sensitiveConstants.dart';

class PeopleUpsert extends StatefulWidget {
  @override
  _PeopleUpsertState createState() => _PeopleUpsertState();
}

class _PeopleUpsertState extends State<PeopleUpsert> {
  final GlobalKey<FormState> _key = GlobalKey<FormState>();
  Person person;

  @override
  Widget build(BuildContext context) {
    // Lấy person 'hiện tại' được đặt trong lúc điều hướng. Nếu
    // person này null, ta đang thêm người mới nên phải khởi tạo
    // một object. Nếu không null thì ta đang cập nhật người đó.
    final Person _person = ModalRoute.of(context).settings.arguments;
    person = (_person == null) ? Person() : _person;

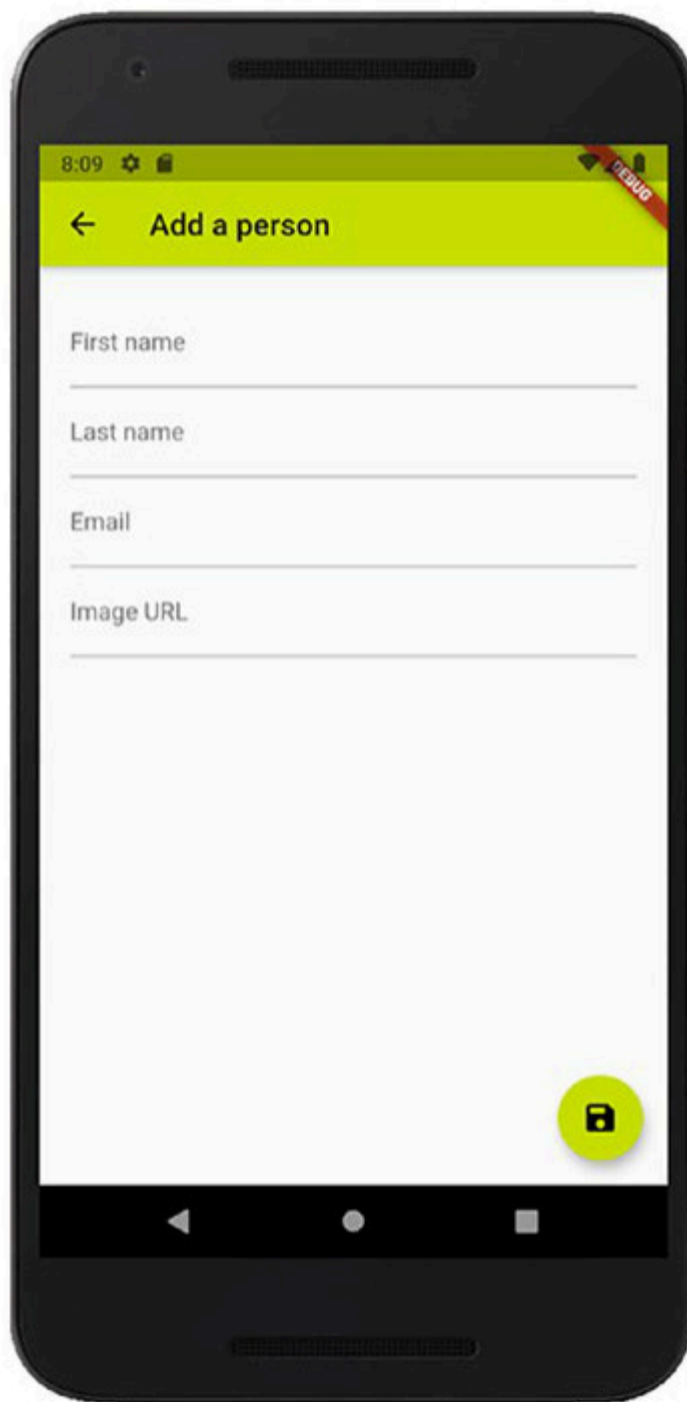
    return Scaffold(
      appBar: AppBar(
        title: Text(
          (_person == null) ? 'Add a person' : 'Update a person',
        ),
      ),
      body: _body,
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          // Đưa dữ liệu các trường vào form key
          _key.currentState.save();
          // Lưu người đó
          updatePersonToPipedream(person);
        },
      ),
    );
  }
}
```

```
        // Và quay lại nơi chúng ta đến từ đó
        Navigator.pop<Person>(context, person);
      },
      child: Icon(Icons.save),
    ),
  );
}

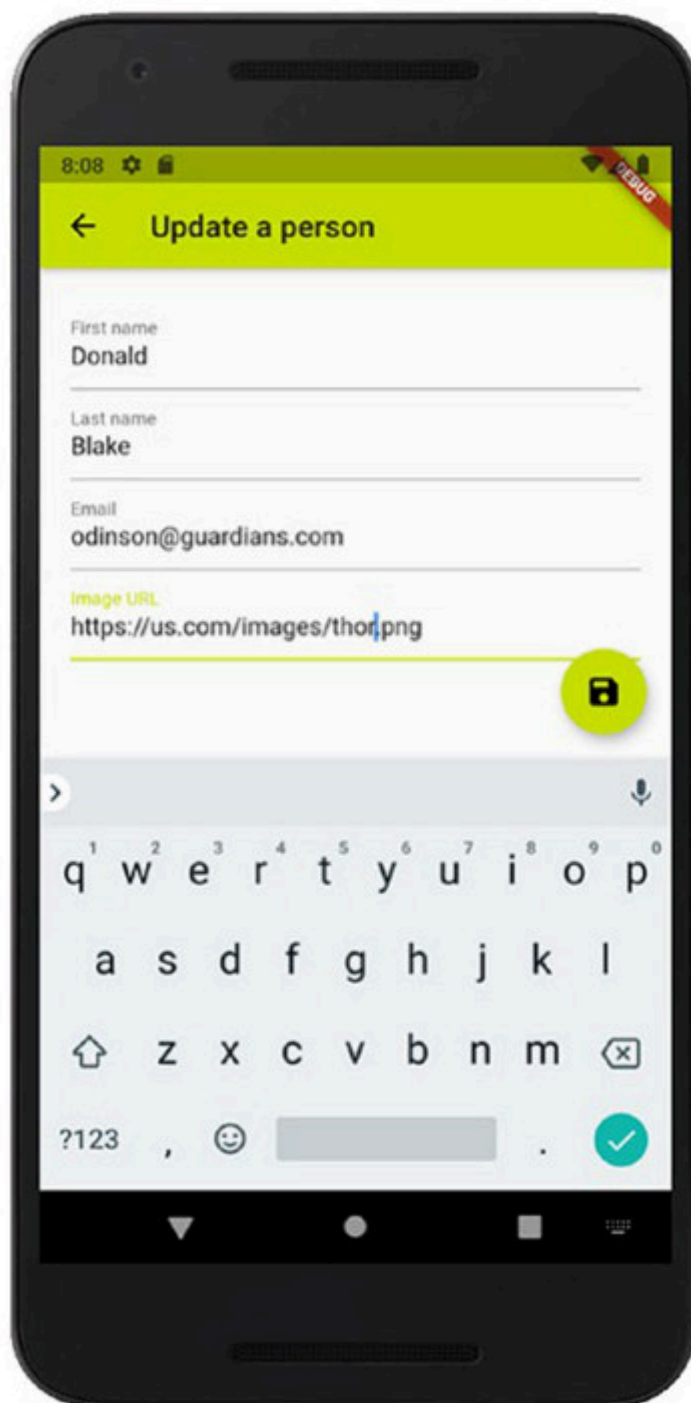
Widget get _body {
  return Form(
    key: _key,
    child: Container(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: <Widget>[
          TextFormField(
            initialValue: person.name['first'],
            decoration: InputDecoration(labelText: 'First name'),
            onSave: (String val) => person.name['first'] = val),
          TextFormField(
            initialValue: person.name['last'],
            decoration: InputDecoration(labelText: 'Last name'),
            onSave: (String val) => person.name['last'] = val),
          TextFormField(
            initialValue: person.email,
            decoration: InputDecoration(labelText: 'Email'),
            onSave: (String val) => person.email = val),
          TextFormField(
            initialValue: person.imageUrl,
            decoration: InputDecoration(labelText: 'Image URL'),
            onSave: (String val) => person.imageUrl = val),
        ],
      ),
    ),
  );
};
```

```
}  
}
```

Đoạn này sẽ tạo ra một trong hai màn hình ở Hình 11-2 hoặc 11-3.



Hình 11-2. Nếu người dùng bấm nút "+", chúng ta đang ở chế độ thêm mới



Hình 11-3. Nếu người dùng chạm vào một *person tile*, chúng ta đang cập nhật người đó

POST và PUT request trong Flutter

Khi người dùng bấm FAB trong màn hình `PeopleUpsert`, họ đang xác nhận dữ liệu vừa nhập và chúng ta gọi `updatePersonToPipedream()`. Nếu đó là thao tác Thêm, ta muốn gọi POST. Nếu là thao tác Cập nhật, ta muốn gọi PUT:

```
void updatePersonToPipedream(Person person) {
  Future<Response> response;
  final String payload = ""
    {
      "first": "${person.name['first']}",
      "last": "${person.name['last']}",
      "imageUrl": "${person.imageUrl}",
      "email": "${person.email}"
    }
    "";
  final Map<String, String> headers = <String, String>{
    'Content-type': 'application/json'
  };
  // Nếu id là null thì ta đang thêm. Nếu không thì đang cập nhật.
  if (person.id == null) {
    String url = '$pipedreamRESTUrl/people/?pipedream_response=1';
    response = post(url, headers: headers, body: payload);
  } else {
    String url =
      '$pipedreamRESTUrl/people/${person.id}?pipedream_response=1';
    response = put(url, headers: headers, body: payload);
  }
  response.then((Response res) {
    Navigator.pop(context, Person.fromJson(res.body));
  });
}
```

 **Ghi chú cập nhật cho toàn bộ ví dụ lớn này:** Để chạy được trên Flutter hiện nay, cần các thay đổi sau:

- Mọi lời gọi `get(url)`, `post(url, ...)`, `put(...)`, `delete(...)` phải truyền `Uri.parse(url)` thay vì chuỗi.
- `ModalRoute.of(context).settings.arguments` → `ModalRoute.of(context)?.settings.arguments as Person?`
- `_key.currentState.save()` → `_key.currentState!.save()`
- Các property của `Person` phải khai báo nullable (`int? id`, `String? email`; ...) hoặc dùng `late` / giá trị mặc định.
- `onSaved: (String val) → onSaved: (String? val)`
- Việc tự nối chuỗi JSON bằng `""...""` như trong `payload` **rất dễ sinh lỗi** khi dữ liệu chứa dấu nháy kép hoặc ký tự đặc biệt. Hãy dùng `jsonEncode(...)` thay thế:

```
final payload = jsonEncode({
  'first': person.name['first'],
  'last': person.name['last'],
  'imageUrl': person.imageUrl,
  'email': person.email,
});
```

Tổng kết

Cũng không tệ nhỉ? Chúng ta đã đi từ chỗ gần như không biết gì về việc đọc ghi dữ liệu qua HTTP tới một ví dụ đầy đủ, dùng những kỹ thuật khá nâng cao như typed serialization pattern và widget `FutureBuilder`.

Tuy nhiên, tiếc là các ví dụ của chúng ta đang dùng một giải pháp phía server tạm thời, không phù hợp cho ứng dụng chạy production. Bạn có hứng thú với một cơ sở dữ liệu phía server mạnh mẽ, lâu dài, hoạt động mượt mà với Flutter trên iOS, Android và Web không? Có chứ? Vậy hãy lật trang và cùng học về Google Firebase!

1. Đọc thêm về JSON tại <https://json.org> ↩

2. Đây là chủ đề gây tranh cãi. Trong khi đặc tả HTTP không nói rõ, RFC 2616 ngụ ý rằng body bị bỏ qua trong DELETE request nhưng không cấm hẳn. Một số server bỏ qua body. Một số server bỏ qua toàn bộ request. Số khác ném lỗi 400. Hoặc bạn tự test trên server của mình, hoặc chơi an toàn và bỏ body đi. ↩
3. Xem <https://github.com/flutter/flutter/issues/34545> để biết các thay đổi được đề xuất. ↩
4. Thuật ngữ "typed deserialization pattern" do chính tôi đặt ra dựa trên gợi ý từ cộng đồng Flutter. Có thể bạn sẽ không thấy ai khác dùng nó cho tới khi nó phổ biến hơn. ↩
5. https://en.wikipedia.org/wiki/Don%27t_repeat_yourself ↩

Chương 12: Sử dụng Firebase với Flutter

Với tư cách một doanh nhân kiêm người hướng dẫn, tôi đã được nghe vô số ý tưởng hay ho cho các app kinh doanh. Không có ngoại lệ nào: mọi ý tưởng có giá trị đều liên quan tới một server lưu trữ dữ liệu. Vậy nên nếu chúng ta quan tâm tới việc dùng Flutter trong thực tế, sẽ là thiếu sót lớn nếu bỏ qua việc giao tiếp với một server sẵn sàng cho production.

Có rất nhiều giải pháp server ngoài kia như AWS của Amazon, Microsoft Azure, IBM Cloud, Oracle Cloud Infrastructure, Alibaba Cloud và nhiều nữa. Tất cả đều tốt. Chúng tôi chọn tập trung vào Firebase, không phải vì nó vượt trội hẳn so với những cái khác, mà vì:

1. Firebase và Flutter đều là sản phẩm của Google, nên có một số điểm phối hợp ăn ý.
2. Firebase ít nhất cũng tốt ngang, và ở vài mặt còn tốt hơn, các lựa chọn khác.
3. Firebase (khá) dễ thiết lập và miễn phí với lượng dùng thấp — hoàn hảo để học và thử nghiệm.
4. Firebase đang được cộng đồng developer chú ý rất nhiều gần đây. Nó đang trở thành công nghệ mà nhà tuyển dụng muốn thấy trong CV của bạn.
5. Chúng tôi phải chọn một cái, vậy tại sao không phải Firebase?

Nếu định dùng Firebase làm server cho Flutter, chúng ta cần chắc chắn hiểu những điều sau:

- Firebase ở mức tổng quan
- Cách thiết lập một Firebase project
- Cách làm nó hoạt động với app iOS và Android sau này
- Tích hợp nó vào app Flutter kèm phân quyền
- Đọc dữ liệu Firestore vào app Flutter
- Truy vấn dữ liệu trong Flutter
- Thay đổi dữ liệu trong Flutter
- Xóa dữ liệu trong Flutter

Đó là kế hoạch cho chương này. Hãy bắt đầu bằng phần giới thiệu về chính Firebase.

Giới thiệu Firebase

Google Firebase là một tập hợp các dịch vụ và công cụ phía server. Nếu bạn dùng Firebase, bạn không cần mua hay thuê server riêng. Không phải vá lỗi bảo mật hay cập nhật phần mềm. Không phải tổ chức sao lưu. Không phải cấu hình firewall. Không có hệ thống phát hiện xâm nhập. Không phải bảo trì định nghĩa chống mã độc. Không phải trả tiền cho ISP để kết nối Internet. Về cơ bản, bạn tin tưởng Google xử lý tất cả những việc mà bạn phải làm với server của riêng mình. Tất nhiên vẫn có một server tồn tại, nhưng bạn và tôi không có quyền truy cập hệ điều hành nên chúng ta không có trách nhiệm mà cũng không có khả năng bảo trì nó. Nhẹ nhõm biết bao!

Danh sách những việc bạn **không** làm được với Firebase khá ngắn và ít khi cần tới. Còn danh sách những việc bạn **làm được** thì rộng và rất phổ biến. Firebase gồm hơn một chục công cụ.¹ Hãy lướt qua ba cái sau:

- **Cloud Firestore** — Một database kèm API để đọc và ghi dữ liệu
- **Cloud Functions** — Logic được kích hoạt bởi một lời gọi API
- **Authentication** — Đăng nhập một lần (single sign-on) cho phép người dùng đăng nhập an toàn vào app bằng tài khoản mạng xã hội hoặc cặp username/password

Hãy bàn nhanh về từng cái rồi mở rộng app Flutter ví dụ ở chương trước để đọc ghi từ Firestore.

Cloud Firestore

Chúng ta muốn một database mà mọi app đều đọc ghi được. Firestore cung cấp một database NoSQL cùng bộ lưu trữ và công cụ để truy cập dữ liệu đó. Là NoSQL nên nó cực kỳ linh hoạt — có lẽ linh hoạt hơn mức bạn thấy thoải mái. Nếu bạn từng làm việc với MongoDB, Cassandra hay CouchDB thì bạn hiểu chúng tôi đang nói gì. Khác biệt lớn nhất của Firestore là bản thân database và engine phía sau được Google bảo trì thay vì bạn và tôi. Tất cả những gì ta cần làm là truy cập dữ liệu.

Firestore công khai các API endpoint cho dữ liệu. Sau khi app tự định danh với Firestore, nó có thể đọc và ghi dữ liệu tại những endpoint đó. Nghe hơi giống giao diện RESTful chúng ta bàn ở chương trước nhỉ? Firestore có hỗ trợ một giao diện mang vài đặc điểm của REST, nhưng nó khác biệt đủ nhiều để tôi không xếp nó vào loại RESTful. Thay vào đó, chúng ta dùng một thư viện Dart lo phần nặng nhọc là xác thực app và thiết lập kênh giao tiếp riêng tư. Chúng ta sẽ gọi các method trong thư viện đó như `Firestore.get('people')` hay `Firestore.set('categories')`. Cách này hóa ra gọn gàng hơn nhiều khi đã thiết lập xong. (Nhưng phần thiết lập thì không đơn giản lắm.)

Cloud Functions

App của bạn gần như chắc chắn có liên quan tới việc xử lý dữ liệu. Một số thuật toán đó có thể rất, rất phức tạp. Nhưng thực tế là thiết bị của người dùng có lẽ thừa sức xử lý những việc đó.

"Vậy sao không xử lý mọi thứ ngay trong app?", bạn hỏi. Bởi vì nếu nó nằm trên thiết bị, thuật toán có thể bị dịch ngược bởi bất kỳ kẻ tấn công nào tải app Flutter của bạn về. Mọi quy trình kinh doanh bí mật sẽ bị lộ và logic có thể bị can thiệp. Mọi API key của bạn sẽ được lưu trên thiết bị và có thể bị đọc. Vậy nên chúng ta không muốn đặt dữ liệu hay xử lý nhạy cảm lên thiết bị. Hãy đặt nó lên server, nơi nằm ngoài tầm với.

Còn những quy trình hay dữ liệu cần chia sẻ giữa hai hay nhiều app thì sao? Bạn cần server cho những thứ đó.

Bạn cần Cloud Functions cho những việc như:

- Sử dụng API của bên thứ ba
- Xử lý file phía server như bảng tính
- Trích xuất, xử lý, biến đổi và nạp các tập dữ liệu lớn
- Chạy chatbot hoặc ứng dụng chat
- Phân tích ảnh như nhận diện khuôn mặt hoặc nhận dạng và trích xuất chữ
- Xử lý ảnh lớn như làm mờ ảnh phản cảm
- Phân tích văn bản như nhận diện ý định
- Machine learning và AI
- Đặt hàng từ một cửa hàng thương mại điện tử như Walmart hay Amazon

Cloud Functions được viết bằng JavaScript và chạy theo yêu cầu trong môi trường Node trên server của Google, khi một số trigger nhất định được kích hoạt — như khi một bản ghi được thêm hoặc cập nhật trong Firestore, khi người dùng đăng nhập, hoặc khi ai đó gửi request tới một URL cụ thể.

 **Ghi chú cập nhật:** Cloud Functions nay hỗ trợ cả **TypeScript** (được khuyến nghị) và **Python**, không chỉ JavaScript.

Authentication

Firebase Authentication giúp việc thêm xác thực vào app trở nên (tương đối) dễ dàng. Đúng là bạn có thể tự làm username và password cho app bằng sức mình, nhưng bạn sẽ phải lo thiết lập bảng người dùng,

viết logic xác thực, băm mật khẩu, xử lý quên mật khẩu và đủ thứ khác. Với Firebase Authentication, bạn có tất cả những chức năng đó cùng với xác thực qua Facebook, GitHub, Twitter và tất nhiên là chính Google. Người dùng có thể chọn dùng cặp username/password của riêng họ, hoặc thậm chí xác thực hai lớp bằng tin nhắn SMS trên thiết bị di động.

Thiết lập Firebase

Tất cả những tính năng này và nhiều hơn nữa đều có ở Firebase. Nếu bạn muốn thử Firebase, nó vui, miễn phí và là trải nghiệm học tập tuyệt vời. Ngoài ra, nó cho chúng ta cơ hội thử kiến thức vừa học về gọi API trong một môi trường đọc/ghi thật. Hãy lấy đó làm mục tiêu cho những trang tiếp theo.

Trước tiên, bạn phải có tài khoản Google. Hãy đăng nhập vào tài khoản Google của bạn và truy cập <http://firebase.google.com> để đăng ký một app với Firebase. Làm theo hướng dẫn trên màn hình. Bạn không cam kết gì cả và cũng không phải trả tiền cho tài khoản cơ bản.

Trong sự nghiệp, có lẽ bạn sẽ tham gia nhiều project — một số để học, một số cho việc làm thêm, và có thể cả cho công việc chính. Vì lý do đó, Firebase cho phép người dùng có nhiều project. Chúng ta sẽ tạo một cái để làm việc.

Cẩn trọng

Các bước dưới đây đúng tại thời điểm viết sách, nhưng chúng có thể thay đổi. Hãy xem tại <https://firebase.google.com/docs/flutter/setup> để có các bước mới nhất.

 **Ghi chú cập nhật cực kỳ quan trọng cho toàn bộ phần thiết lập:** Toàn bộ quy trình thủ công từ mục (3) đến (5) bên dưới — tải `GoogleService-Info.plist`, tạo Podfile, sửa hai file `build.gradle`, thêm `google-services.json` — **nay đã được thay thế bằng một công cụ dòng lệnh duy nhất**. Bạn vẫn nên đọc các mục đó để hiểu chuyện gì đang diễn ra bên dưới, nhưng khi làm thật hãy dùng cách sau:

```
# Cài Firebase CLI (cần Node.js) rồi đăng nhập
npm install -g firebase-tools
firebase login

# Cài FlutterFire CLI
dart pub global activate flutterfire_cli

# Chạy trong thư mục project Flutter của bạn
flutterfire configure
```

Lệnh `flutterfire configure` sẽ tự hỏi bạn chọn Firebase project và các nền tảng cần hỗ trợ, rồi **tự động** sinh ra file `lib/firebase_options.dart` và đặt mọi file cấu hình vào đúng chỗ cho iOS, Android, Web, macOS.

Sau đó, trong `main.dart` bạn khởi tạo:

```
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(MyApp());
}
```

Lưu ý `WidgetsFlutterBinding.ensureInitialized()` và `await Firebase.initializeApp(...)` là **bắt buộc** — bước này chưa tồn tại khi sách được viết.

(1) Tạo một Firebase project

Sau khi đăng nhập Google, truy cập `console.firebase.google.com` và bạn sẽ thấy console của mình — nơi sẽ hiển thị danh sách các project. Bấm nút tạo project mới. Đặt tên cho project, ví dụ "Learning Flutter" (Hình 12-1). Hình 12-2 cho thấy project đã sẵn sàng.

Add a project ✕

Project name

Learning Flutter ▼

 +  + 

Tip: Projects span apps across platforms ?

Project ID ?

learning-flutter-e3d0a 

Analytics location ?

United States 

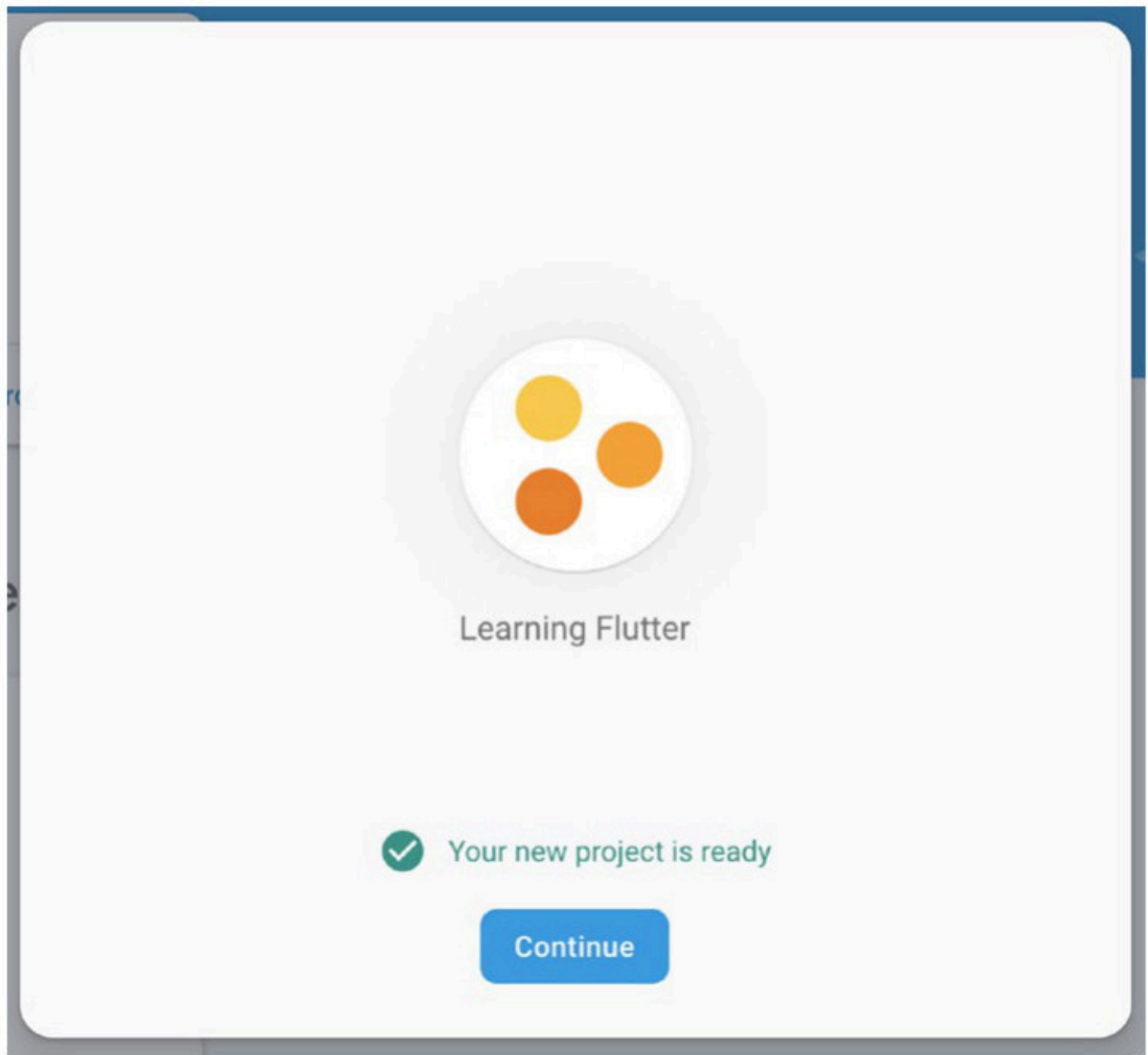
☒ Use the default settings for sharing Google Analytics for Firebase data

- ☒ Share your Analytics data with all Firebase features
- ☒ Share your Analytics data with Google to improve Google Products and Services
- ☒ Share your Analytics data with Google to enable technical support
- ☒ Share your Analytics data with Google to enable Benchmarking
- ☒ Share your Analytics data with Google Account Specialists

☒ I accept the [controller-controller terms](#). This is required when sharing Analytics data to improve Google Products and Services. [Learn more](#)

☒ I agree that I am using Firebase services in my app and I agree to the applicable [terms](#).

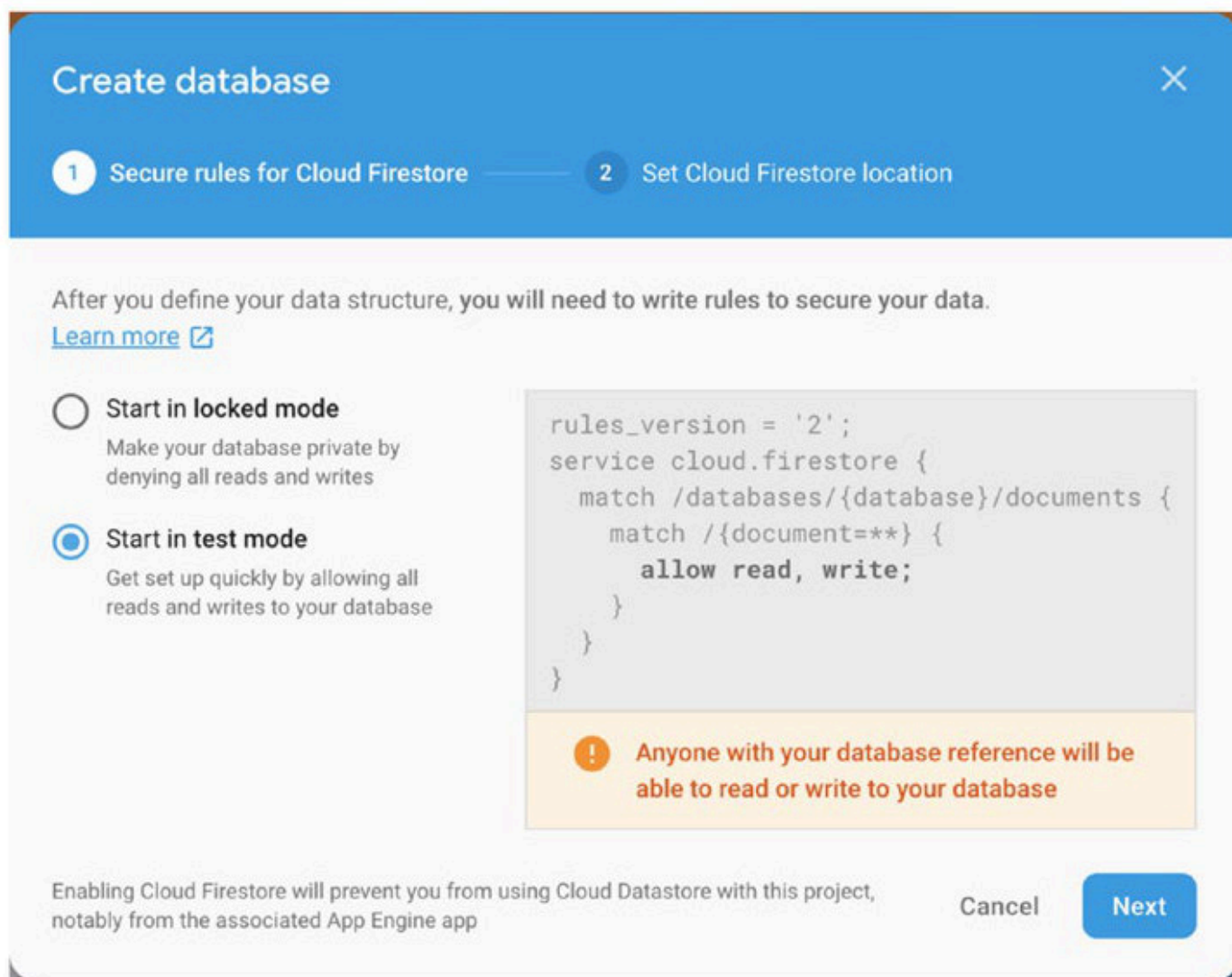
Cancel Create project

Hình 12-1. Thêm một project mới**Hình 12-2.** Thông báo xác nhận project của bạn đã sẵn sàng

Giờ hãy hình dung project này có một giao diện web, một database phía sau, và được truy cập bởi một app Flutter trên iPhone lẫn Android. Đây sẽ là **một project với nhiều app**. Cũng như với nhiều project, Google cho phép mỗi project có nhiều app. Mỗi app sẽ có thiết lập riêng vì các môi trường có yêu cầu khác nhau. Trước khi kết thúc, chúng ta sẽ thiết lập một app cho iOS và một cho Android. Nhưng trước hết, hãy tạo database và ít nhất một collection (tức là bảng).

(2) Tạo database

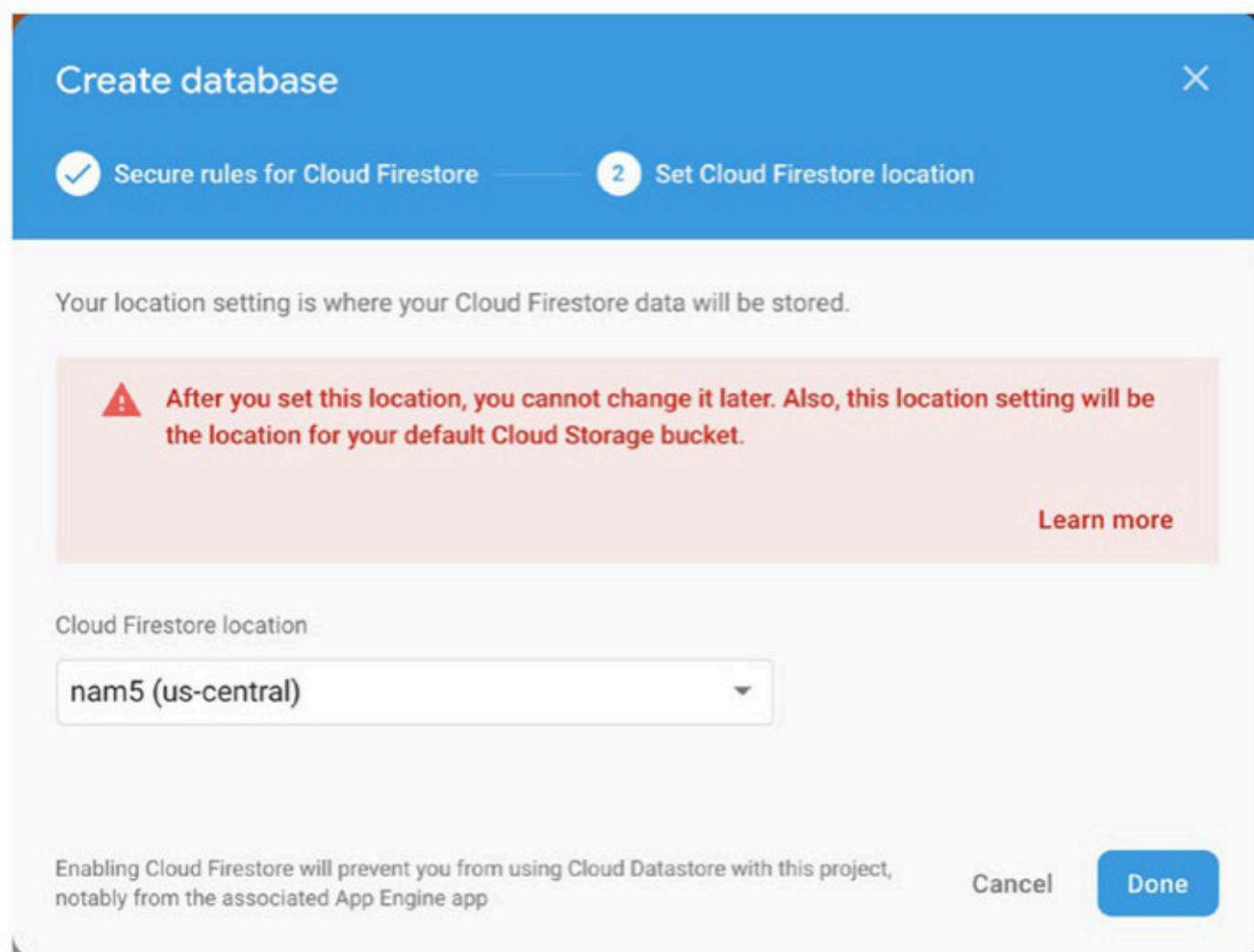
Quay lại dashboard của project. Bạn sẽ thấy một mục menu để phát triển với database. Hãy chọn tạo database mới (Hình 12-3).



Hình 12-3. Bước đầu tiên khi tạo database Cloud Firestore

Chọn khởi tạo ở chế độ **test mode** để chúng ta dễ dàng kiểm chứng code hoạt động. Với app thực tế bạn sẽ muốn thêm rule, và bạn có thể thêm bất cứ lúc nào. Bấm "Next".

Bước này hỏi phần lớn người dùng của bạn ở đâu về mặt địa lý (Hình 12-4). Nó đoán dựa trên vị trí hiện tại của bạn, và thành thật mà nói, vị trí nào cũng chạy tốt. Cứ để mặc định và bấm "Done". Việc này tạo ra database.



Hình 12-4. Chọn vị trí gần nhất với phần lớn người dùng của bạn

⚠ **Cẩn trọng khi dùng test mode:** Rule ở test mode cho phép **bất kỳ ai** đọc và ghi vào database của bạn, và Firebase sẽ **tự động chặn mọi truy cập sau 30 ngày**. Nếu app đột nhiên báo lỗi permission-denied sau khoảng một tháng, đây rất có thể là nguyên nhân. Trước khi phát hành thật, hãy viết Security Rules đàng hoàng.

Ngoài ra, vị trí database **không thể đổi sau khi đã tạo** — hãy chọn cẩn thận (với người dùng Việt Nam, asia-southeast1 Singapore là lựa chọn hợp lý).

💡 Mẹo

Đây là database NoSQL, khác với các database quan hệ truyền thống như MySQL, SQL Server, Oracle, Informix và tương tự. Trước hết, thuật ngữ khác nhau (Bảng 12-1).

Bảng 12-1. Cách gọi tên các thành phần ở những loại DB server khác nhau

Database quan hệ	Database NoSQL
Table (bảng)	Collection
Record / row (bản ghi / hàng)	Document
Column (cột)	Field

Database NoSQL cũng có key và value, nhưng chúng không có cấu trúc cố định. Nói cách khác, mỗi document trong một collection có thể có những field khác với các document khác trong cùng collection đó. Đây là khác biệt lớn nhất giữa database truyền thống và NoSQL, và cũng là điều khó quen nhất.

Giờ chúng ta sẽ tạo một collection. Bấm **Start Collection** và đặt id cho nó. Sau đó bạn sẽ thêm được một hoặc nhiều document (Hình 12-5).

Start a collection

✓ Give the collection an ID

2 Add its first document

Document parent path

/people

Document ID

Auto-ID

Field	Type	Value
email	string	chief@hawkins.c
name	map	
first	string	Jim
last	string	Hopper
imageUrl	string	chiefHopper.png

+ Add field

Cancel

Save

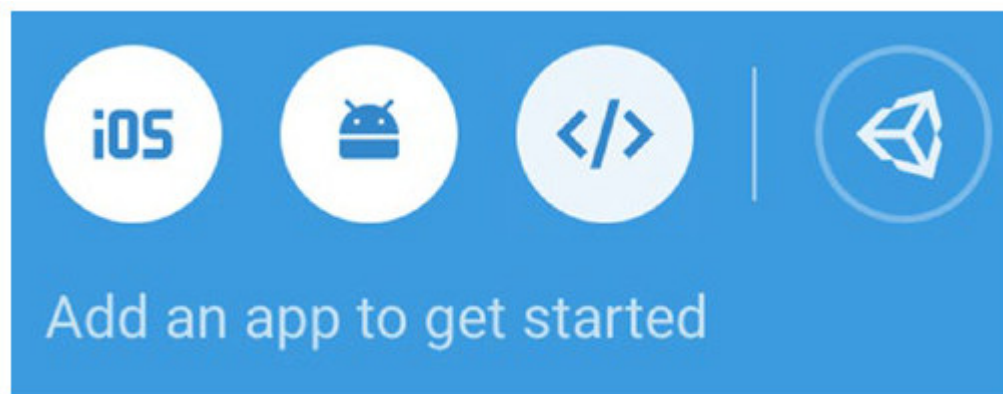
Hình 12-5. *Thêm document đầu tiên vào collection*

Giờ bạn sẽ thấy document duy nhất của mình trong Cloud Firestore viewer. Từ đây bạn có thể thêm, xóa và sửa document.

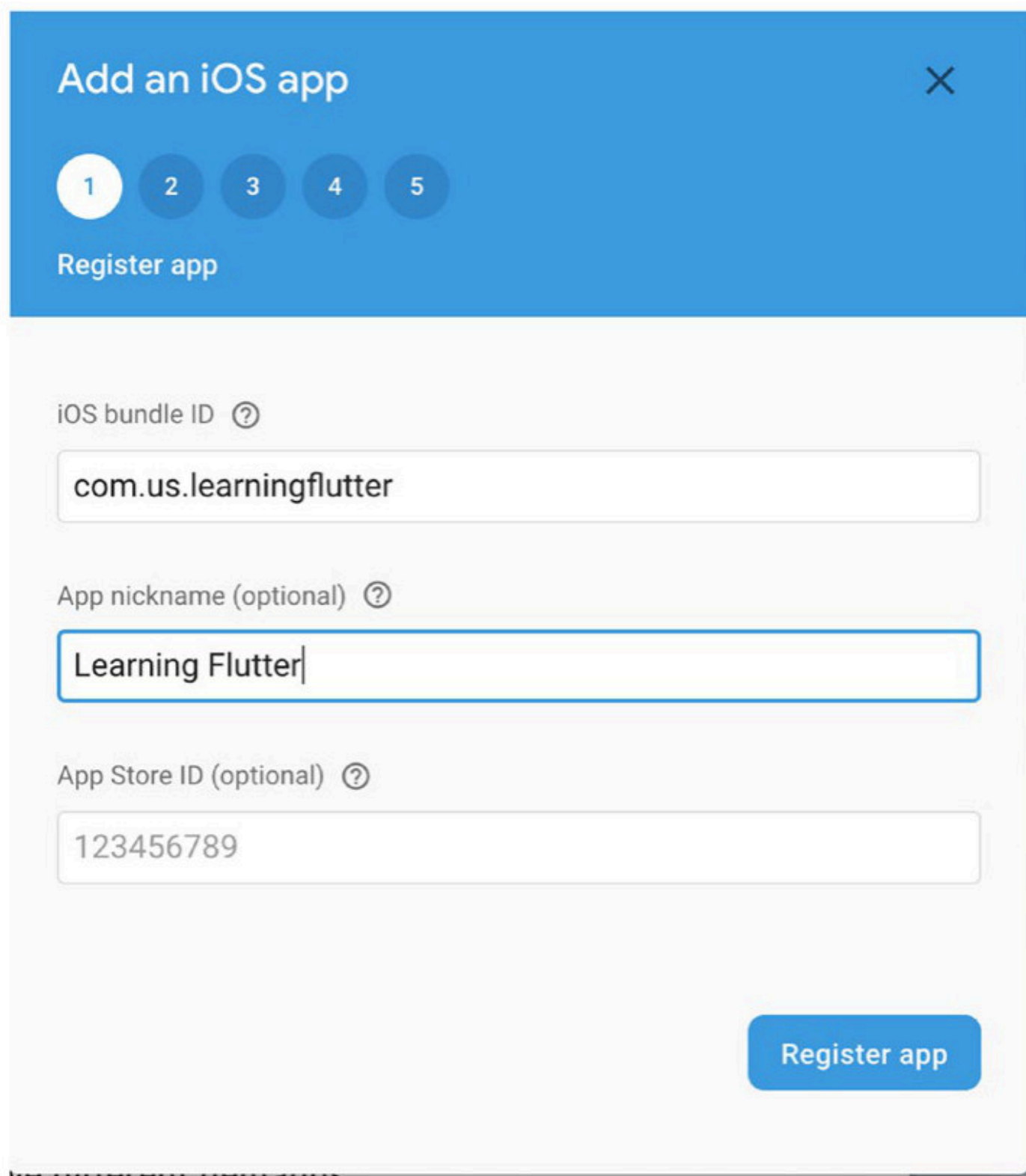
Việc quản lý database ngay trên website Firebase thì tiện đấy, nhưng mục tiêu của chúng ta rõ ràng là làm điều đó từ app. Vậy nên ta phải cấu hình app iOS và Android để đọc từ Cloud Firestore.

(3) Tạo app iOS

Trên trang tổng quan của project, bạn sẽ thấy các nút để tạo app trong project (Hình 12-6).

**Hình 12-6.** *Các lựa chọn tạo app Firestore của bạn*

Bấm nút iOS. Cung cấp tên cho app (Hình 12-7).



Add an iOS app ✕

1 2 3 4 5

Register app

iOS bundle ID ⓘ

com.us.learningflutter

App nickname (optional) ⓘ

Learning Flutter

App Store ID (optional) ⓘ

123456789

Register app

Hình 12-7. Đặt ID và tên gợi nhớ cho app iOS

Sau khi đăng ký app, bạn sẽ thấy màn hình như Hình 12-8.

Add an iOS app



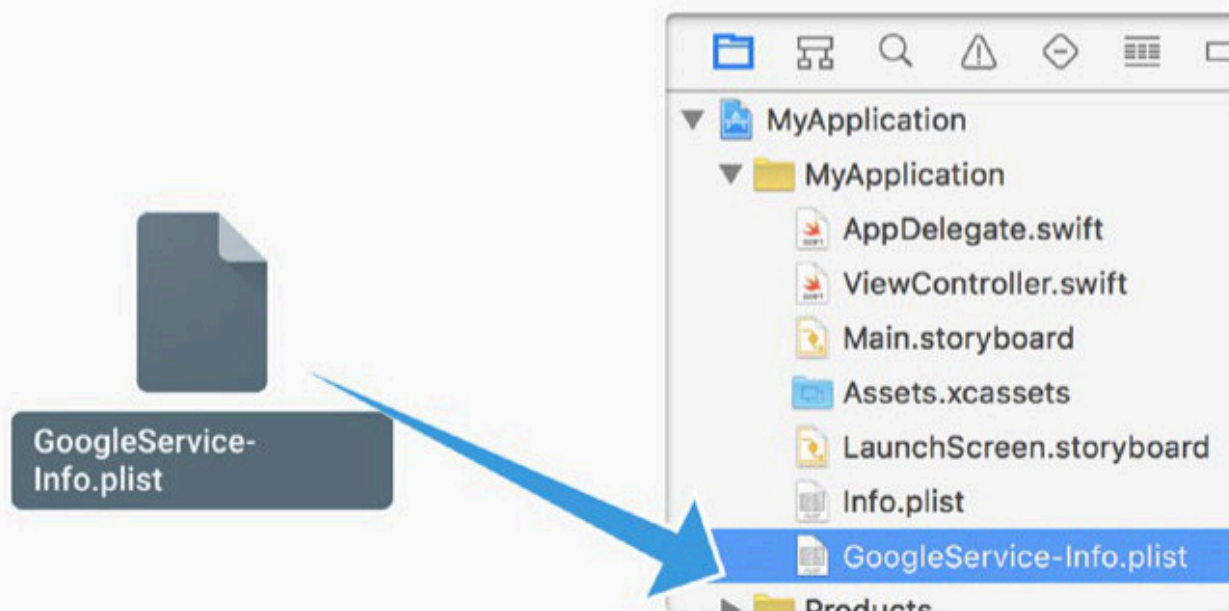
Download config file

Instructions for Xcode below | [C++](#)



Download GoogleService-Info.plist

Move the GoogleService-Info.plist file you just downloaded into the root of your Xcode project and add it to all targets.



Previous

Next

Hình 12-8. *Firestore cung cấp file cấu hình được tạo tự động để bạn tải về*

Tải `GoogleService-Info.plist` về và lưu nó trong thư mục `ios/Runner/Runner`. Bạn biết mình đang ở đúng thư mục khi thấy `Info.plist` ở đó. Đừng để sơ đồ họ hiển thị làm bạn phân tâm — nó trông sẽ khác vì họ đang minh họa một project kiểu Xcode, còn bạn đang làm việc trong một project Flutter.

Việc kết nối tới Firebase sẽ dễ hơn nếu ta dùng một số công cụ Google cung cấp. Nghĩa là chúng cần được tải về và cài vào project iOS/Xcode. iOS dùng CocoaPods để quản lý dependency.² Chúng ta nên tạo một `Podfile` nếu chưa có. Làm theo hướng dẫn tiếp theo để thêm `Podfile` và tạo file `.xcworkspace` (Hình 12-9).

Add an iOS app



Add Firebase SDK

Instructions for CocoaPods | [Download ZIP](#) [C++](#)

Google services use [CocoaPods](#) to install and manage dependencies. Open a terminal window and navigate to the location of the Xcode project for your app.

Create a Podfile if you don't have one:

```
$ pod init
```



Open your Podfile and add:

```
$ pod 'Firebase/Core'
```



Includes Analytics by default

Save the file and run:

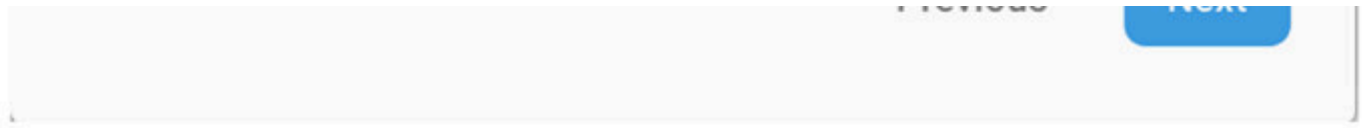
```
$ pod install
```



This creates an **.xcworkspace** file for your app. Use this file for all future development on your application.

[Previous](#)

[Next](#)



Hình 12-9. Google cung cấp các bước để tạo Podfile và .xcworkspace

Chép đoạn code trong Hình 12-10 để app kết nối tới Firebase khi khởi động. Đây chính là logic đăng nhập Firebase.

Add an iOS app



Add initialization code

To connect Firebase when your app starts up, add the initialization code below to your main **AppDelegate** class.

☒ Swift ☐ Objective-C

```
import UIKit
import Firebase

@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate {

    var window: UIWindow?

    func application(_ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions:
            [UIApplicationLaunchOptionsKey: Any]?) -> Bool {
        FirebaseApp.configure()
        return true
    }
}
```

[Previous](#)[Next](#)

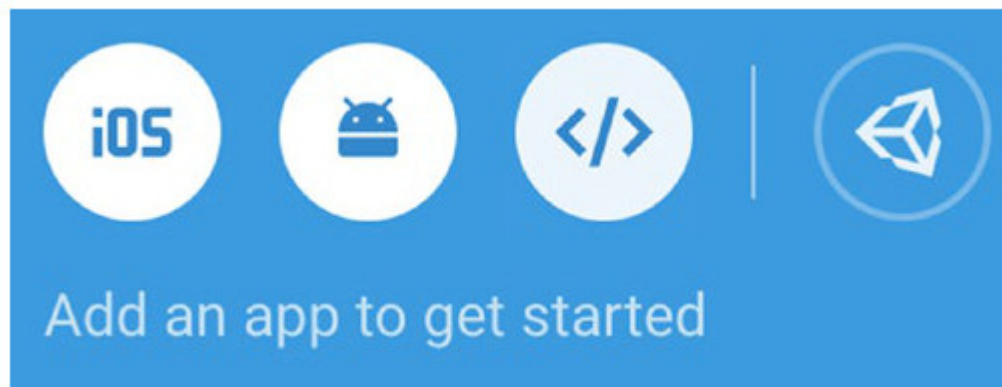
Hình 12-10. Bước cuối cùng để tạo app iOS — thêm code khởi tạo

Tới đây, khi app chạy, nó biết mình gắn với tài khoản Firebase và project nào. Chỉ những app có file `.plist` này mới kết nối được. Tất nhiên, khi bạn biên dịch app và phân phối tới thiết bị qua Apple App Store, tất cả sẽ kết nối tới một tài khoản Firebase duy nhất này. Đó là chuyện bình thường và đúng như dự kiến. Mỗi người dùng chạy app sẽ báo danh với Firebase. Bạn sẽ thấy hoạt động trong Firebase console. Firebase giờ đang lắng nghe và sẽ cung cấp cho bạn dữ liệu phân tích.

Vì chúng ta đang tạo app cross-platform, có lẽ ta cũng nên làm điều tương tự với Android. Nhớ rằng đó là hai môi trường hoàn toàn khác nhau nên các bước cũng khác.

(4) Tạo app Android

Còn nhớ chỗ chúng ta chọn iOS lúc nãy chứ? Giờ bấm nút Android như trong Hình 12-11.



Hình 12-11. Lần này chọn thêm app Android

Tuy các bước khác nhau, nhưng **application id** hay package name thì phải giống nhau. Nhập nó vào hộp thoại (Hình 12-12).

Add an Android app

×

1

2

3

4

Register app

Android package name ?

com.us.learningflutter

App nickname (optional) ?

Learning Flutter

Debug signing certificate SHA-1 (optional) ?

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00

Required for Dynamic Links, Invites, and Google Sign-In or phone number support in Auth. Edit SHA-1s in Settings.

Register app

Hình 12-12. Đặt package name và tên gợi nhớ cho Android

Chứng chỉ SHA-1 đó là tùy chọn với phần lớn ứng dụng. Bạn có thể để trống lúc này và quay lại tạo chứng chỉ nếu cần cho Google Sign-in hoặc xác thực bằng số điện thoại.

Cài file google-services.json

Ở bước tiếp theo, trình hướng dẫn sẽ tạo file `google-services.json` và cho bạn biết lưu nó ở đâu (Hình 12-13).

Add an Android app

1

Register app

2

Download config file

3

Add Firebase SDK

4

Run your app to verify installation

Instructions for Android Studio below | [C++](#)

Download google-services.json

Switch to the **Project** view in Android Studio to see your project root directory.

Move the google-services.json file you just downloaded into your Android app module root directory.

google-services.json

Project

MyApplication (~/.Desktop/MyApplication)

.gradle

.idea

app

build

libs

src

.gitignore

app.iml

build.gradle

google-services.json

proguard-rules.pro

gradle

Previous

Next

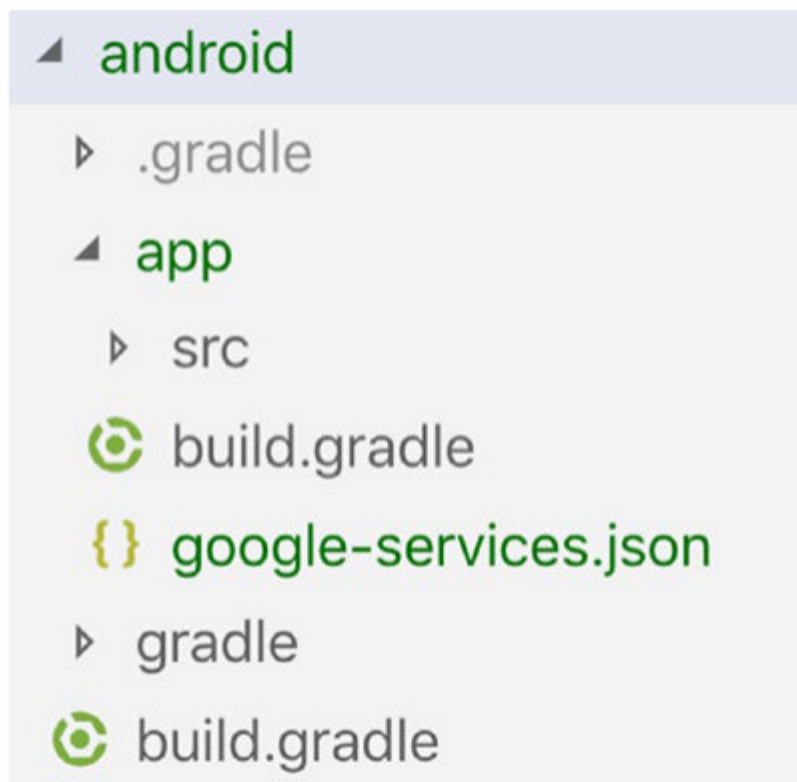
Hình 12-13. Sinh file `google-services.json` cho Android

File này chứa đủ loại thiết lập quan trọng, đặc biệt là số project và app id để báo cho app biết cần hỏi dữ liệu từ đâu trong Google Firestore. Với tất cả nằm trong một file cấu hình, bạn không phải hard-code chúng vào mã nguồn app.

Lưu ý rằng nó phải được lưu trong thư mục `app`, cùng cấp với file `build.gradle` cấp app. Nhân nói tới điều đó, bước tiếp theo của chúng ta là sửa file này.

Thêm vào các file Gradle

Project Android có các file Gradle để quản lý thư viện, giống như Podfile với iOS. Có một file Gradle cấp project và một file Gradle cấp app. Cả hai đều có tên là `build.gradle` một cách khó phân biệt (Hình 12-14).



Hình 12-14. Hãy chắc chắn bạn biết mình đang sửa file `build.gradle` nào

Thêm classpath này vào mục `dependencies` của file `build.gradle` **cấp project**, KHÔNG phải file nằm dưới `app`:

```
buildscript {  
    dependencies {  
        ( có thể đã có sẵn những thứ khác ở đây )  
        classpath 'com.google.gms:google-services:X.Y.Z'  
    }  
}
```

Tất nhiên, hãy dùng số phiên bản mới nhất thay cho X.Y.Z như đã nói ở trên. Sau đó thêm dòng `implementation` này vào mục `dependencies` của file `build.gradle` **cấp app**:

```
dependencies {  
    ( đã có sẵn những thứ khác ở đây )  
    implementation 'com.google.firebase:firebase-core:X.Y.Z'  
}
```

Và thêm dòng `apply plugin` này vào cuối cùng file `build.gradle` cấp app:

```
apply plugin: 'com.google.gms.google-services'
```

Bảo IDE đồng bộ lại các file Gradle. Trong Android Studio bạn sẽ thấy liên kết "Sync now". Cứ bấm vào đó.

(5) Thêm các plugin FlutterFire

Gần xong rồi! Tới đây chúng ta đã đặt xong nền móng để dùng Firebase với app. Ta chỉ cần thêm các plugin Firebase cho Flutter và bắt đầu code. Hóa ra Google cung cấp một thư viện chung cho bản thân Firebase và một plugin cho mỗi sản phẩm Firebase. Chúng ta cần plugin thư viện chung và plugin Cloud Firestore.

Plugin Firebase chính có tên là `firebase_core`. Plugin cho Firestore Cloud Storage có tên là `cloud_firestore`. Đặt những dòng này vào mục `dependencies` trong `pubspec.yaml` và chúng sẽ được cài cho bạn:

```
firebase_core: ^X.Y.Z # dependency chính cho Firebase Core  
cloud_firestore: ^X.Y.Z # dependency cho Firebase Cloud Storage
```

Tất nhiên nếu ta dùng thêm sản phẩm Firestore khác, ta sẽ cần thêm plugin tương ứng, nhưng không phải làm lại bất kỳ bước nào ở trên; làm một lần cho cả project là đủ.

Sử dụng Firestore

Đúng là thiết lập khá nhiều, nhưng cuối cùng chúng ta cũng sẵn sàng đọc và quản lý dữ liệu từ database. Để bạn dễ đặt các lời gọi Firestore này vào ngữ cảnh, chúng ta sẽ dùng lại ví dụ `Person` từ chương trước, thay các lời gọi tới server tạm thời bằng lời gọi tới Firestore. Hãy tham chiếu lại chúng và mã nguồn trên repository GitHub khi đọc những trang tiếp theo.

Ở đầu bất kỳ file Dart nào có gọi HTTP tới Firestore, hãy thêm import:

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

Lệnh này sẽ cung cấp một object tên `Firestore` mà bạn dùng để truy cập database. Cụ thể, `Firestore.instance` trở tới toàn bộ database của bạn. Và `Firestore.instance.collection('Foo')` trở tới toàn bộ collection `Foo`.

Lưu ý

Mặc dù các lời gọi Firestore thực chất là HTTP request ở phía sau, bạn không cần nhập API key, tạo thiết lập, hậu xử lý dữ liệu hay phần lớn những việc nặng nhọc cần thiết khi gọi API thông thường. Tất cả những việc đó được trừu tượng hóa đi nhờ các thư viện, vốn dựa vào file `google-services.json` và `GoogleService-Info.plist`. Dù phần thiết lập vừa rồi khá nhọc, giờ bạn đã thấy được phần thưởng.

Lấy một collection

Miễn là bạn nhớ rằng `Firestore.instance.collection('Foo')` trở tới collection `Foo`, việc lấy dữ liệu đó rất dễ. Bạn chỉ cần gọi method `.snapshots()` để kích hoạt request. Một hàm đơn giản như thế này có thể đóng gói request GET của bạn:

```
Stream<QuerySnapshot> fetchPeople() {  
  return Firestore.instance  
    .collection('people')  
    .limit(100) // Phòng khi có quá nhiều document  
    .snapshots();  
}
```

Lưu ý rằng `.snapshots()` đăng ký nhận một `Stream` kiểu `QuerySnapshot`. Và chúng ta biết từ chương trước rằng `Stream` có thể được hiển thị và giữ cập nhật liên tục bằng widget `StreamBuilder`. Đây gọi là reactive programming, nhớ chứ? Đoạn như sau có thể hiển thị dữ liệu đó trong một lưới:

```
Widget build(BuildContext context) {
  return StreamBuilder<QuerySnapshot>(
    stream: fetchPeople(),
    builder: (BuildContext ctx, AsyncSnapshot<dynamic> snapshot) {
      if (snapshot.hasError) {
        return Text('Oh no! Error! ${snapshot.error}');
      }
      if (!snapshot.hasData) {
        return const Text('No people found');
      }
      // Điều kỳ diệu! snapshot.data.documents chứa các bản ghi
      final List<Widget> widgets = snapshot.data.documents
        .map<Widget>((DocumentSnapshot p) => Stack(
          children: <Widget>[
            Image.network(p['imageUrl'],
              height: 300, width: 300, fit: BoxFit.cover),
            Text('${p['name']['first']} ${p['name']['last']}'),
          ],
        ))
        .toList();
      return GridView.extent(
        maxCrossAxisExtent: 300,
        children: widgets,
      );
    },
  );
}
```

Mẹo

Nếu bạn muốn đọc dữ liệu một lần duy nhất mà không đăng ký theo dõi, hãy bỏ `.snapshots()` đi và nó sẽ trả về một mảng đơn giản các `Map` (tức mảng object). Nếu làm vậy, bạn sẽ dùng `FutureBuilder` thay cho `StreamBuilder`.

Truy vấn

Firestore có hàm `.where()`, nhưng nó rất hạn chế so với mệnh đề `WHERE` chuẩn của SQL. Hàm `.where()` của Firestore cho phép bạn tìm những thứ kiểu:

```
.where('name.first', isEqualTo: someText)
.where('name.first', isGreaterThanOrEqualTo: someText)
.where('name.first', isLessThanOrEqualTo: someText)
```

Và tiếc là gần như chỉ có vậy. Nó không hỗ trợ bất kỳ logic mờ nào như ký tự đại diện, "contains" hay "like". Nếu bạn cần tìm kiếm toàn văn, đội Firebase khuyến nghị một dịch vụ bên thứ ba như Algolia. Xem <https://firebase.google.com/docs/firestore/solutions/search> để biết thêm chi tiết.

Upsert

Từ "upsert" nghĩa là nếu document đã tồn tại thì cập nhật, còn nếu chưa tồn tại thì thêm mới. Firestore thực hiện cả hai thao tác này bằng method `setData`.

Để cập nhật một document có sẵn, hãy đọc nó như đã làm ở trên rồi truyền `documentID` của nó vào `setData` như sau:

```
Firestore.instance
  .collection('people')
  .document(_person.documentID)
  .setData(<String, dynamic>{
    'name': person.name,
    'email': person.email,
    'imageUrl': person.imageUrl,
  }).then<void>((dynamic doc) {
    print('Document updated: $doc');
  }).catchError((dynamic error) {
    print('Error! $error');
  });
```

Nếu bạn bỏ `documentID` khi gọi `setData`, Firestore hiểu rằng bạn muốn thêm bản ghi mới:

```
Firestore.instance
  .collection('people')
  .document()
  .setData(<String, dynamic>{
    'name': person.name,
    'email': person.email,
    'imageUrl': person.imageUrl,
  }).then<void>((dynamic doc) {
    print('Document added: $doc');
  }).catchError((dynamic error) {
    print('Error! $error');
  });
```

⚠️ Cẩn trọng

Hãy cẩn thận. Rất dễ tạo ra bản ghi trùng lặp nếu quên `documentID` khi gọi `setData()` .

Xóa

Xóa cũng đơn giản tương tự:

```
Firestore.instance
  .collection('people')
  .document(personToDelete.documentID)
  .delete()
  .catchError((dynamic error) {
    print('Error! $error');
  });
```

Rõ ràng lệnh delete không trả về gì nên không cần `.then()` .

 **Ghi chú cập nhật — bảng chuyển đổi API Firestore:** Toàn bộ API Firestore trong chương này **đã đổi tên** ở phiên bản `cloud_firestore` 1.0 (đầu 2021). Code trong sách sẽ không biên dịch được. Đây là bảng đối chiếu:

Cách viết trong sách (2019)	Cách viết hiện nay
<code>Firestore.instance</code>	<code>FirebaseFirestore.instance</code>
<code>.document(id)</code>	<code>.doc(id)</code>
<code>.setData({...})</code>	<code>.set({...})</code>
<code>.updateData({...})</code>	<code>.update({...})</code>
<code>snapshot.data.documents</code>	<code>snapshot.data!.docs</code>
<code>document.documentID</code>	<code>document.id</code>
<code>Firestore.instance.collection('x').add(...)</code>	không đổi

Ngoài ra, để đọc field trong một document nên dùng `doc.data()` (trả về `Map<String, dynamic>`) thay vì `doc['field']`.

Ví dụ `fetchPeople()` viết lại theo API hiện nay:

```
Stream<QuerySnapshot<Map<String, dynamic>>> fetchPeople() {
  return FirebaseFirestore.instance
    .collection('people')
    .limit(100)
    .snapshots();
}
```

Và phần upsert:

```
// Cập nhật document có sẵn
await FirebaseFirestore.instance
  .collection('people')
  .doc(person.id)
  .set({
    'name': person.name,
    'email': person.email,
    'imageUrl': person.imageUrl,
  });

// Thêm mới (Firestore tự sinh id)
await FirebaseFirestore.instance.collection('people').add({
  'name': person.name,
  'email': person.email,
  'imageUrl': person.imageUrl,
});

// Xóa
await FirebaseFirestore.instance
  .collection('people')
  .doc(person.id)
  .delete();
```

Lưu ý cách viết hiện nay dùng `add()` để thêm mới thay vì `doc()` không tham số — rõ ràng hơn và tránh được đúng cái bẫy trùng lặp mà tác giả cảnh báo ở trên.

Đi tiếp từ đây

Hãy dành một chút nhìn lại hành trình chúng ta đã đi cùng nhau. Kể từ khi bắt đầu cuốn sách này...

- Bạn đã hiểu Flutter hoạt động và được kiến trúc như thế nào
- Bạn sử dụng thành thạo những widget dựng sẵn hữu ích nhất của Flutter
- Bạn tạo được widget tùy chỉnh, cả Stateless lẫn Stateful
- Trải nghiệm người dùng trong app Flutter của bạn có thể trực quan nhờ các layout widget

- Bạn làm chúng đẹp mắt được bằng style
- Bạn điều hướng được giữa các scene trong app Flutter
- Bạn xử lý được các hoạt động bất đồng bộ, bao gồm đọc dữ liệu cục bộ
- Bạn đọc và ghi được dữ liệu qua HTTP/RESTful API
- Bạn lưu được dữ liệu lâu dài trên một server mạnh mẽ, có khả năng mở rộng

Chà! Rất nhiều thứ! Nhưng còn nhiều hơn nữa để học. Ngay cả những người kỳ cựu cũng nên tiếp tục học. Để tôi giới thiệu vài nguồn tài liệu để bạn tiếp tục khám phá.

Trước hết, hãy tham gia cộng đồng Flutter (mà tôi cũng là thành viên). Bắt đầu với kênh Slack tại <http://flutterStudyGroup.slack.com>. Đọc các bài viết tại <https://medium.com/flutter-community>. Và tham gia cùng chúng tôi qua Zoom vào thứ Tư hằng tuần cho buổi Hump Day Q&A tại <https://tinyurl.com/humpdayqanda>, nơi bạn có thể trò chuyện trực tiếp với các developer Flutter trên khắp thế giới, đặt câu hỏi, thậm chí pair programming để giải quyết vấn đề. Những developer Flutter hàng đầu thế giới tụ tập ở đó và rất sẵn lòng giúp bạn.

Tôi cũng khuyên bạn đăng ký hai bản tin email miễn phí đầy ắp bài viết, video, hướng dẫn về Flutter. Mỗi bản tin được gửi tới hộp thư của bạn mỗi tuần một lần. **Flutter Weekly** có vài chục tài nguyên mỗi số. Đăng ký tại http://bit.ly/subscribe_to_flutter_weekly. **Flutter Press Weekly** mỗi tuần ngắn hơn vì nó chọn lọc kỹ hơn. Bạn đăng ký tại http://bit.ly/subscribe_to_flutter_press_weekly. Đọc chúng đều đặn sẽ giúp bạn nắm được nhịp đập của những phát triển mới nhất trong Flutter.

Đội Flutter của Google cũng có vài tài nguyên hay. Nơi tuyệt vời để bắt đầu là tài liệu Flutter tại <https://flutter.dev/docs>. Có những phần đọc khá khô khan nhưng đây là nguồn chuẩn xác nhất khi bạn tra cứu widget và API của Flutter. Ở thái cực còn lại là các video của họ — cực kỳ giải trí và dễ tiêu hóa. Tôi khuyên bạn đăng ký tại http://bit.ly/flutter_youtube_channel. Nếu bạn thấy video "Widget of the Week" ở đó, hãy bấm vào ngay! Chúng dài nhiều nhất một hai phút và sẽ cho bạn hiểu cách dùng widget đó nhanh hơn bất kỳ cách nào khác.

Tôi thực sự choáng ngợp trước sự nhiệt huyết của cộng đồng Flutter! Nếu ba nguồn lớn này chưa đủ với bạn, còn rất nhiều nguồn khác ngoài kia. Hãy tham gia cùng các đồng nghiệp Flutter, và nếu bạn thấy tôi ở đâu đó trong số họ, hãy dừng lại chào một tiếng nhé. Cảm ơn bạn rất nhiều vì đã đọc!

 **Ghi chú cập nhật — các nguồn học hiện nay:** Một số liên kết trên đã cũ hoặc không còn hoạt động (nhiều liên kết `bit.ly` từ 2019 đã hỏng). Đây là những nguồn còn sống và đáng theo dõi:

- **Tài liệu chính thức:** <https://docs.flutter.dev> — đã được viết lại rất tốt, dễ đọc hơn nhiều so với bản 2019.
- **Kho package:** <https://pub.dev>
- **YouTube Flutter chính thức:** vẫn còn, và loạt "Widget of the Week" mà tác giả khen vẫn tiếp tục ra tập mới.
- **Codelab chính thức:** <https://docs.flutter.dev/codelabs> — học qua thực hành, rất hợp cho người mới.
- **Discord của Flutter:** cộng đồng nay hoạt động chủ yếu trên Discord thay vì Slack.
- **r/FlutterDev** trên Reddit — nơi thảo luận sôi nổi nhất hiện nay.
- **Flutter Vietnam** — có các nhóm cộng đồng người Việt trên Facebook và Discord nếu bạn muốn trao đổi bằng tiếng Việt.

1. Đọc về tất cả tại <https://firebase.google.com/products> ↩

2. Giống như npm/ `package.json` cho app JavaScript, NuGet cho .NET, RubyGems cho project Ruby, v.v. ↩

Phụ lục A: Tổng quan ngôn ngữ Dart

Chúng ta dùng ngôn ngữ Dart khi viết Flutter, nhưng Dart (vẫn) chưa phổ biến lắm. Phần lớn developer nhảy thẳng vào Flutter mà không có kiến thức gì về ngôn ngữ này từ trước. Nếu bạn cũng vậy, chúng tôi muốn hỗ trợ bạn một chút.

Trong phụ lục này, chúng tôi không cố dạy bạn mọi thứ về Dart. Mục tiêu ở đây là cung cấp vừa đủ Dart để bạn làm việc hiệu quả khi viết Flutter. Vì vậy phụ lục này ngắn gọn và đi thẳng vào vấn đề. Chúng tôi chỉ đề cập những thứ mà nếu không biết sẽ làm bạn chậm lại khi viết Flutter. Ví dụ như kiểu dữ liệu `rune` — một tính năng rất hay và sáng tạo của Dart, nhưng hiếm khi dùng với Flutter nên chúng tôi bỏ qua.

 GHI CHÚ CẬP NHẬT QUAN TRỌNG NHẤT CỦA CẢ CUỐN SÁCH — Null Safety

Từ **Dart 2.12** (2021), Dart có **sound null safety** và từ **Dart 3.0** (2023) thì null safety là **bắt buộc**. Đây là thay đổi lớn nhất trong lịch sử ngôn ngữ, và nó ảnh hưởng tới gần như mọi ví dụ trong phụ lục này cũng như trong toàn bộ cuốn sách.

Nguyên tắc cốt lõi: mặc định, không biến nào được phép chứa `null`. Nếu bạn muốn một biến có thể null, bạn phải đánh dấu kiểu của nó bằng dấu `?`.

```
String ten = 'Nam';    // KHÔNG BAO GIỜ null. Bắt buộc phải khởi tạo.
String? biet danh;    // CÓ THỂ null. Mặc định là null.
```

Ba từ khóa mới bạn sẽ gặp liên tục:

Ký hiệu	Nghĩa	Ví dụ
<code>?</code> sau kiểu	Biến này có thể null	<code>String? email;</code>
<code>!</code> sau biến	"Tôi cam đoan cái này không null" (sẽ crash nếu sai)	<code>email!.length</code>
<code>late</code>	"Tôi sẽ gán giá trị trước khi dùng" — hoãn việc khởi tạo	<code>late final Future f;</code>
<code>required</code>	Tham số có tên này là bắt buộc	<code>Person({required this.name})</code>

Vì sao đáng công học? Vì nó **loại bỏ hoàn toàn** loại lỗi phổ biến nhất trong lập trình: `NullPointerException` / "The method was called on null". Trình biên dịch bắt lỗi trước khi bạn chạy chương trình.

Chúng tôi sẽ đánh dấu từng chỗ cụ thể bị ảnh hưởng ở bên dưới.

Dart là gì?

Dart là một ngôn ngữ lập trình được biên dịch, định kiểu tĩnh, hướng đối tượng và thủ tục. Nó có cấu trúc rất phổ thông, giống nhiều ngôn ngữ hướng đối tượng khác, khiến nó cực kỳ dễ nắm bắt với những người đã có kinh nghiệm Java, C#, C++ hoặc các ngôn ngữ OO kiểu C khác. Và nó bổ sung một số tính năng mà developer từ những ngôn ngữ kia sẽ không ngờ tới, nhưng rất hay và làm ngôn ngữ trở nên gọn gàng hơn nhiều.

Vì vậy, chúng tôi tổ chức phụ lục này thành hai phần:

- **Những tính năng quen thuộc** — Bản tra cứu nhanh ("cheatsheet") về các tính năng phổ thông, mức tối thiểu bạn cần biết cho Flutter. Bạn nên lướt qua phần này thật nhanh.
- **Những tính năng bất ngờ** — Đây là những thứ có thể gây bất ngờ cho developer làm việc với các ngôn ngữ OO truyền thống. Vì Dart đi chệch khỏi truyền thống ở những chỗ này, chúng tôi thấy tốt nhất nên giải thích ngắn gọn.

Những tính năng quen thuộc — Dart Cheatsheet

Bản tra cứu nhanh này giả định bạn là developer OO có kinh nghiệm và bỏ qua những thứ quá hiển nhiên. Để xem sâu và chi tiết hơn về Dart, hãy truy cập <https://dart.dev/language>.

Kiểu dữ liệu

```
int x = 10;           // Số nguyên
double y = 2.0;       // Số thực dấu phẩy động IEEE754
bool z = true;        // Kiểu luận lý
String s = "hello";   // Chuỗi
dynamic d;            // Biến dynamic có thể đổi kiểu
d = x;                // bất cứ lúc nào. Dùng tiết chế!
d = y;
d = z;
```

Mảng / danh sách

```
// Dấu ngoặc vuông nghĩa là một list/mảng
// Trong Dart, mảng và list là một.
List<dynamic> list = [1, "two", 3];
// Dấu ngoặc nhọn tùy chọn để chỉ kiểu - Dart hỗ trợ Generics

// Cách duyệt một list
for (var d in list) {
  print(d);
}
// Một cách khác để duyệt list
list.forEach((d) => print(d));
// Cả hai đều in ra "1", rồi "two", rồi "3"
```

Biểu thức điều kiện

```
// Câu lệnh if/else truyền thống
int x = 10;
if (x < 100) {
  print('Yes');
} else {
  print('No');
}
// Sẽ in ra "Yes"

// Dart cũng hỗ trợ toán tử ba ngôi
String response = (x < 100) ? 'Yes' : 'No';

// Nếu name đã được gán, dùng nó. Nếu không, dùng 'No name given'
String name;
String res = name ?? 'No name given';

// Toán tử "Elvis". Nếu object khác null thì lấy property.
// Ngăn không cho lỗi null bị ném ra.
print(name?.length);
```

 **Cập nhật:** Với null safety, dòng `String name;` **sẽ không biên dịch được** (biến non-nullable phải được khởi tạo). Viết đúng là `String? name;`. Khi đó `name ?? 'No name given'` và `name?.length` hoạt động đúng như mô tả.

Toán tử `??` gọi là **null-coalescing** và `?.` gọi là **null-aware access**. Còn có `??=` (gán nếu đang null):
`name ??= 'Khách';`

Vòng lặp

```
// Vòng lặp for
for (int i = 1; i < 10; i++) {
  print(i);
}
// Sẽ in từ 1 đến 9

// Vòng lặp while
int i = 1;
while (i < 10) {
  print(i++);
}
// Sẽ in từ 1 đến 9
```

Class

```
class Name {  
  String first;  
  String last;  
  String suffix;  
}  
  
class Person {  
  // Class có property  
  int id;  
  Name name;    // Một class khác có thể dùng làm kiểu  
  String email;  
  String phone;  
  
  // Class có method  
  void save() {  
    // Ghi vào database bằng cách nào đó.  
  }  
}
```

 **Cập nhật:** Với null safety, các property trên phải được xử lý theo một trong ba cách:

```
class Person {  
  int? id;           // 1. Cho phép null  
  String email = ''; // 2. Có giá trị mặc định  
  late Name name;    // 3. Hứa sẽ gán trước khi dùng  
}
```

Constructor của class

```
class Person {  
  Name name;  
  // Constructor thông thường  
  Person() {  
    name = Name();  
    name.first = "";  
    name.last = "";  
  }  
}
```

Những điều bất ngờ về Dart

Những tính năng Dart ở trên không có gì lạ với developer OO có kinh nghiệm, nhưng Dart có vài tính năng khá hay và độc đáo. Chúng ta sẽ xem chúng ngay sau đây. Vì chúng ít quen thuộc hơn, hãy dành một hai câu giải thích ngắn gọn cho từng cái trước khi xem code mẫu.

Suy luận kiểu (type inference)

Nếu tôi viết `x = 10.0`, bạn đoán `x` là kiểu gì? `double`? Và làm sao bạn biết? Vì bạn nhìn sang bên phải dấu bằng và **suy ra** kiểu dựa trên giá trị được gán. Dart cũng làm được điều đó. Nếu bạn dùng từ khóa `var` thay cho một kiểu dữ liệu, Dart sẽ suy ra kiểu và gán kiểu đó:

```
var i = 10;           // i giờ được định nghĩa là int.  
i = 12;              // Chạy được, vì 12 là int.  
i = "twelve";        // Không! "twelve" là String chứ không phải int.  
var str = "ten";      // str giờ được định nghĩa là String.  
str = "a million";    // Ừ, chạy tốt.  
str = 1000000.0;      // Không! 1000000.0 là double, không phải string.
```

Điều này thường bị nhầm với `dynamic`. `dynamic` chứa được mọi kiểu dữ liệu và có thể đổi kiểu lúc chạy. Còn `var` được định kiểu chặt chẽ và tĩnh.

final và const

`final` và `const` là các modifier cho biến trong Dart:

```
final int x = 10;  
const double y = 2.0;
```

Cả hai đều có nghĩa là một khi đã gán thì giá trị không đổi được nữa. Nhưng `const` đi xa hơn một chút — giá trị được đặt ở **thời điểm biên dịch** và do đó được nhúng thẳng vào gói cài đặt.

`final` nghĩa là biến không thể được **gán lại**. Nó **không** có nghĩa là nội dung không thể thay đổi. Ví dụ, đoạn này được phép:

```
final Employee e = Employee();  
e.employer = "The Bluth Company";
```

`e` đã thay đổi, nhưng nó không bị gán lại nên vẫn ổn. Còn đoạn này thì không được phép:

```
const Employee e = Employee();
```

`const` hoàn toàn không được phép ở đây, vì class này có những property có khả năng thay đổi lúc chạy. **`final` đánh dấu một biến là không thể gán lại, còn `const` đánh dấu một giá trị là không thể thay đổi.**

Tóm lại:

- `dynamic` — Chưa được mọi kiểu dữ liệu. Kiểu dữ liệu có thể đổi bất cứ lúc nào.
- `var` — Kiểu dữ liệu được suy ra từ giá trị bên phải dấu `=`. Kiểu dữ liệu không đổi.
- `final` — Biến, một khi đã gán, không thể gán lại.
- `const` — Giá trị được đặt lúc biên dịch, không phải lúc chạy.

Biến được khởi tạo bằng null

Kiểu dữ liệu mặc định của phần lớn biến là `null`. Giá trị trả về mặc định của một hàm là `null`:

```
int x;
double y;
bool z;
String s;
dynamic d;
```

Tất cả dữ liệu trên đều là `null` vì chúng chưa được gán giá trị.

 **Cập nhật — mục này nay đã SAI HOÀN TOÀN:** Đây chính là điều mà null safety đã thay đổi. Đoạn code trên **sẽ không biên dịch được** trên Dart hiện nay. Trình biên dịch sẽ báo lỗi kiểu: *"Non-nullable instance field 'x' must be initialized."*

Chỉ có `dynamic d;` là vẫn chạy được (và bằng `null`), vì `dynamic` vốn đã bao gồm cả `null`.

Cách viết đúng hiện nay:

```
int? x;           // null, và trình biên dịch biết điều đó
double y = 0.0;   // hoặc gán giá trị ngay
late bool z;      // hoặc hứa sẽ gán trước khi dùng
```

Tương tự, một hàm khai báo trả về `int` **bắt buộc** phải `return` một giá trị; không còn chuyện ngầm trả về `null`.

Nội suy chuỗi với \$

Nội suy (interpolation) giúp developer khỏi phải viết nối chuỗi. Đoạn này...

```
String fullName = '$first $last, $suffix';
```

...về cơ bản giống hệt đoạn này...

```
String fullName = first + " " + last + ", " + suffix;
```

Khi biến là một phần của map hoặc object, trình biên dịch có thể bị nhầm, nên bạn nên bọc phần nội suy trong dấu ngoặc nhọn:


```
String fullName = '${name['first']} ${name['last']}';
```

Chuỗi nhiều dòng

Bạn có thể tạo chuỗi nhiều dòng bằng ba dấu nhảy đơn hoặc nhảy kép:

```
String introduction = ""  
Now the story of a wealthy family  
who lost everything  
And the one son who had no choice  
but to keep them all together.  
"";
```

Toán tử spread

Toán tử `...` sẽ trải các phần tử của một mảng ra, làm phẳng chúng. Điều này sẽ rất quen thuộc với developer JavaScript:

```
List fiveTo10 = [5, 6, 7, 8, 9, 10];  
// Trải mảng bên trong bằng "...":  
List numbers = [1, 2, 3, 4, ...fiveTo10, 11, 12];  
// numbers giờ có [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

💡 **Bổ sung hữu ích cho Flutter:** Còn có toán tử `...?` (spread có kiểm tra null) và **collection if / collection for** — cực kỳ hay dùng khi dựng danh sách `children` :

```
Column(
  children: [
    Text('Luôn hiện'),
    if (daDangNhap) Text('Chào bạn!'),           // collection if
    for (var item in danhSach) Text(item),        // collection for
    ...?danhSachCoTheNull,                        // spread có kiểm tra null
  ],
)
```

Map<foo, bar>

Map giống như hash hoặc dictionary. Chúng chỉ đơn giản là một object với một tập các cặp key-value. Key và value có thể thuộc kiểu bất kỳ:

```
// Bạn đặt giá trị cho Map bằng dấu ngoặc nhọn:
Map<String, dynamic> person = {
  "first": "George",
  "last": "Bluth",
  "dob": DateTime.parse("1972-07-16"),
  "email": "amazingGob@gmail.com",
};

// Dấu ngoặc nhọn tam giác trên Map quy định kiểu của key và
// value. Chúng không bắt buộc nhưng là thói quen tốt.

// Bạn truy cập phần tử của map bằng dấu ngoặc vuông:
String introduction =
  person['first'] + " was born " + person['dob'].toString();
```

Hàm là object

Cũng như trong JavaScript, hàm là công dân hạng nhất. Chúng có thể được truyền đi như dữ liệu, được trả về từ một hàm, được truyền vào một hàm dưới dạng tham số, hoặc được gán cho một biến. Bạn có thể làm với hàm gần như mọi thứ mà bạn làm được với object trong Java hay C#:

```
Function sayHi = (String name) => print('Hello, ' + name);  
// Bạn có thể truyền sayHi đi như dữ liệu; nó là một object!  
Function meToo = sayHi;  
meToo("Tobias");
```

Mũi tên béo (fat arrow)

Trong ví dụ trên, chúng ta cũng đã thấy cú pháp mũi tên béo. Khi bạn có một hàm trả về giá trị chỉ trong một dòng code, bạn có thể đặt giá trị trả về đó ở bên phải dấu `=>` và danh sách tham số ở bên trái. Ba đoạn sau là như nhau:

```
int triple(int val) {  
    return val * 3;  
}  
  
Function triple = (int val) {  
    return val * 3;  
};  
  
Function triple = (int val) => val * 3;
```

Mũi tên béo chỉ là "đường cú pháp" (syntactic sugar), giúp developer diễn đạt nhiều hơn với ít code hơn.

Tham số hàm có tên

Tham số theo vị trí thì tốt, nhưng dùng tham số có tên sẽ ít gây lỗi hơn (dù phải gõ nhiều hơn). Thay vì gọi hàm thế này:

```
sendEmail('ceo@bluthcompany.com', 'Popcorn in the breakroom');
```

Bạn có thể gọi thế này:

```
sendEmail(subject: 'Popcorn in the breakroom',
  toAddress: 'ceo@bluthcompany.com');
```

Giờ thứ tự tham số không còn quan trọng. Đây là cách viết hàm để dùng tham số có tên. Chú ý dấu ngoặc nhọn:

```
void sendEmail({String toAddress, String subject}) {
  // gửi email ở đây
}
```

Tham số có tên cũng phối hợp rất tốt với constructor của class, nơi chúng được dùng cực kỳ phổ biến trong Flutter:

```
class Person {
  Name name;
  // Tham số có tên
  Person({String firstName, String lastName}) {
    name = Name()..first = firstName..last = lastName;
  }
}
```

 **Cập nhật:** Với null safety, tham số có tên mặc định là **tùy chọn**, nên chúng phải hoặc là nullable, hoặc có giá trị mặc định, hoặc được đánh dấu `required`:

```
void sendEmail({required String toAddress, String subject = '(không tiêu đề)'}) {
  // gửi email ở đây
}
```

Đây chính là lý do bạn thấy `required` xuất hiện khắp nơi trong code Flutter hiện đại.

Bỏ "new" và "this."

Trong Dart, việc tránh dùng từ khóa `new` khi khởi tạo class là hoàn toàn được phép — và được khuyến khích:

```
// Không. Tránh.  
Person p = new Person();  
  
// Đúng  
Person p = Person();
```

Tương tự, bên trong một class, việc dùng `this.` để tham chiếu tới thành viên của class không những không cần thiết (vì nó được ngầm hiểu) mà còn không được khuyến khích. Code sẽ ngắn và sạch hơn:

```
class Name {  
  String first;  
  String last;  
  String suffix;  
  
  String getFullName() {  
    // Không. Tránh dùng "this.":  
    String full = this.first + " " + this.last + ", " + this.suffix;  
  
    // Tốt hơn.  
    String full = first + " " + last + ", " + suffix;  
    return full;  
  }  
}
```

Cách viết tắt tham số constructor

Đây chỉ là cách viết ngắn hơn cho các class Dart nhận tham số. Khi bạn viết constructor nhận `this.something` và có một property cùng tên ở phạm vi class, trình biên dịch sẽ tự viết phần gán giá trị nên bạn không phải làm:

```
class Person {  
  String email;  
  String phone;  
  // Các tham số được gán vào property một cách tự động  
  // vì tham số có ghi "this."  
  Person(this.email, this.phone) {}  
}
```

Đoạn code trên tương đương với:

```
class Person {  
  String email;  
  String phone;  
  Person(String email, String phone) {  
    this.email = email;  
    this.phone = phone;  
  }  
}
```

Thành viên private của class

Dart không dùng các modifier phạm vi truy cập như `public`, `private`, `protected`, `package` hay `friend` như các ngôn ngữ OO khác. Mọi thành viên đều là `public` mặc định. Để làm một thành viên trở thành `private`, hãy đặt dấu gạch dưới trước tên:

```
class Person {  
  int id;  
  String email;  
  String phone;  
  String _password;  
  
  set password(String value) {  
    _password = value;  
  }  
  
  String get hashedPassword {  
    return sha512.convert(utf8.encode(_password)).toString();  
  }  
}
```

Trong ví dụ đó, `id`, `email` và `phone` là public. `_password` là private vì ký tự đầu tiên trong tên là dấu gạch dưới `_`.

✦ **Lưu ý quan trọng:** Trong Dart, "private" nghĩa là **private ở phạm vi file** (library-private), chứ không phải phạm vi class như Java/C#. Nghĩa là một class khác nằm **cùng file** vẫn truy cập được `_password`. Đây chính là lý do `_FooState` trong Flutter đọc được các biến private của widget cha khi cả hai ở cùng một file.

Mixin

Mixin là những "giò" chứa property và method có thể được thêm vào bất kỳ class nào. Chúng trông giống class nhưng không thể khởi tạo được:

```
mixin Employment {
  String employer;
  String businessPhone;

  void callBoss() {
    print('Calling my boss');
  }
}
```

Một mixin được thêm vào class khi class đó dùng từ khóa `with` :

```
class Employee extends Person with Employment {
  String position;
}
```

Class `Employee` này giờ có property `employer` , `businessPhone` và method `callBoss()` :

```
Employee e = Employee();
e.employer = "The Bluth Company";
e.callBoss();          // Một nhân viên có thể gọi cho sếp.
```

Dart, cũng như Java và C#, chỉ hỗ trợ đơn kế thừa. Một class chỉ `extends` được một thứ. Nhưng các thành viên của mixin được **thêm vào** class, nên bất kỳ class nào cũng có thể dùng nhiều mixin, và một mixin có thể được dùng trong nhiều class khác nhau.

Toán tử cascade (..)

Khi bạn thấy hai dấu chấm, nó có nghĩa là "trả về chính class này, nhưng trước đó hãy làm gì đó với một property hoặc method". Chúng ta có thể viết:

```
Person p = Person()..id = 100..email = 'gob@bluth.com'..save();
```

đây là cách viết súc tích hơn của:


```
Person p = Person();  
p.id = 100;  
p.email = 'gob@bluth.com';  
p.save();
```

Không có overloading

Dart không hỗ trợ nạp chồng (overloading) method. Điều này bao gồm cả constructor.

Constructor có tên

Vì không thể nạp chồng constructor, Dart hỗ trợ một cách khác để làm điều tương tự. Chúng được gọi là **named constructor** và được tạo bằng cách viết một constructor thông thường rồi thêm một dấu chấm và một tên khác:

```
class Person {  
  // Constructor thông thường  
  Person() {  
    name = Name()..first = ""..last = "";  
  }  
  
  // Một named constructor  
  Person.withName({String firstName, String lastName}) {  
    name = Name()  
      ..first = firstName  
      ..last = lastName;  
  }  
  
  // Một named constructor khác  
  Person.byId(int id) {  
    // Có thể đi lấy dữ liệu từ một service theo id được cung cấp  
  }  
}
```

Và để dùng những named constructor này:

```
Person p = Person();  
// p sẽ là một person với first name và last name rỗng  
Person p1 = Person.withName(firstName: "Lindsay", lastName: "Fünke");  
// p1 có first name là "Lindsay" và last name là "Funke"  
Person p3 = Person.byId(100);  
// p3 sẽ được lấy về dựa trên id 100
```

Bổ sung: những tính năng Dart ra đời sau cuốn sách

Phần này không có trong bản gốc. Đây là những tính năng bạn sẽ gặp thường xuyên trong code Flutter hiện nay nhưng chưa tồn tại khi sách được viết.

Từ khóa `late`

Dùng khi bạn biết chắc một biến sẽ có giá trị trước khi được dùng, nhưng không thể gán ngay lúc khai báo:

```
late final Database db; // sẽ được gán trong initState()
```

Nếu bạn dùng nó trước khi gán, chương trình sẽ báo lỗi lúc chạy — nên hãy chắc chắn.

super parameters

Giúp constructor của widget ngắn gọn hơn nhiều:

```
// Cách cũ  
const MyWidget({Key? key}) : super(key: key);  
  
// Cách hiện nay  
const MyWidget({super.key});
```

Records và Pattern matching (Dart 3)

Record cho phép trả về nhiều giá trị mà không cần tạo class:

```
(String, int) layThongTin() => ('Nam', 25);

var (ten, tuoi) = layThongTin(); // tách giá trị ra
print('$ten - $tuoi');
```

Pattern matching với `switch` biểu thức:

```
String moTa(Object obj) => switch (obj) {
  int i when i > 100 => 'Số lớn',
  int _ => 'Số nguyên',
  String s => 'Chuỗi: $s',
  _ => 'Không rõ',
};
```

sealed class

Cho phép trình biên dịch kiểm tra bạn đã xử lý hết mọi trường hợp trong `switch` — rất hay dùng để mô tả trạng thái màn hình (đang tải / thành công / lỗi):

```
sealed class TrangThai {}
class DangTai extends TrangThai {}
class ThanhCong extends TrangThai { final List<String> duLieu; ThanhCong(this.duLieu); }
class Loi extends TrangThai { final String thongBao; Loi(this.thongBao); }
```

Từ khóa mới cho class (Dart 3)

`base`, `interface`, `final`, `mixin class` — các modifier kiểm soát việc class của bạn được kế thừa hay hiện thực ra sao. Bạn sẽ thấy chúng trong code thư viện, nhưng chưa cần dùng khi mới học.

Bảng thuật ngữ Anh – Việt

Đây là danh sách những thuật ngữ được **giữ nguyên tiếng Anh** xuyên suốt bản dịch, kèm giải nghĩa. Lý do giữ nguyên: đây là những từ bạn sẽ gặp y hệt trong tài liệu chính thức, thông báo lỗi của trình biên dịch, câu trả lời trên StackOverflow và trong trao đổi hằng ngày của dân lập trình. Dịch chúng sang tiếng Việt sẽ khiến bạn khó tra cứu và khó trao đổi hơn.

Cột **Chương** cho biết thuật ngữ xuất hiện lần đầu ở chương nào.

A – C

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
API	Application Programming Interface — địa chỉ trên Internet mà app gửi request tới để đọc/ghi dữ liệu	11
API key	Khóa bí mật để định danh app khi gọi API	11
app store	Chợ ứng dụng (App Store của Apple, Google Play Store của Google)	1
assets	Tài nguyên tĩnh đóng gói kèm app: ảnh, font, file dữ liệu	4, 10
async / await	Từ khóa đánh dấu hàm bất đồng bộ và điểm chờ kết quả	10
authentication	Xác thực danh tính người dùng	12
behavior	Hành vi — phần logic xử lý của app, đối lập với presentation	3
BLoC	Business Logic Component — một pattern quản lý state	9
BoxConstraints	Ràng buộc kích thước (min/max width, height) mà widget cha áp lên widget con	6
BoxFit	Cách một ảnh được co giãn để vừa khung chứa	4
build (method)	Method trả về cây widget mô tả giao diện	3
BuildContext	Object định vị widget trong cây widget, do framework truyền vào <code>build()</code>	3
callback	Hàm được truyền đi để nơi khác gọi lại khi có sự kiện	5, 10
cascade operator (..)	Toán tử cho phép gọi liên tiếp nhiều property/method trên cùng object	Phụ lục A
CocoaPods	Trình quản lý thư viện cho project iOS	12
codebase	Toàn bộ mã nguồn của một dự án	1
collection	Trong Firestore: tương đương "bảng" của database quan hệ	12
compile-to-native	Nhóm giải pháp biên dịch ra mã máy gốc của nền tảng	1

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
componentization	Chia nhỏ giao diện thành các thành phần tái sử dụng được	3
composition	Ghép các thành phần nhỏ thành thành phần lớn hơn	3
const	Giá trị được xác định ngay lúc biên dịch, không đổi được	4, Phụ lục A
Container	Widget bọc ngoài để áp padding, margin, màu nền, đường viền	6
cross-platform	Đa nền tảng — một codebase chạy trên nhiều nền tảng	1
CRUD	Create – Read – Update – Delete, bốn thao tác cơ bản với dữ liệu	11

D – G

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
Dart	Ngôn ngữ lập trình dùng để viết Flutter	1
debug painting	Chế độ vẽ thêm khung viền màu để nhìn thấy bố cục	6
decomposition	Phân rã một thứ phức tạp thành nhiều phần đơn giản	3
decoration	Property của <code>Container</code> để trang trí: màu, viền, bóng đổ, bo góc	8
dependency	Thư viện mà project của bạn phụ thuộc vào	10
deploy	Đưa ứng dụng lên môi trường thật cho người dùng sử dụng	1
deserialization	Chuyển dữ liệu từ dạng lưu trữ/truyền (JSON) trở lại thành object	10
DevTools	Bộ công cụ debug và phân tích hiệu năng của Flutter	2
document	Trong Firestore: tương đương "bản ghi" của database quan hệ	12
drawer	Ngăn kéo menu trượt ra từ cạnh màn hình	7
DRY	Don't Repeat Yourself — nguyên tắc không lặp lại code	11
dynamic	Kiểu dữ liệu Dart cho phép đổi kiểu bất cứ lúc nào	Phụ lục A
element tree	Bản sao nhẹ của cây widget mà Flutter dùng để so sánh và vẽ lại tối ưu	3
emulator / simulator	Thiết bị ảo chạy trên máy tính để chạy thử app	2
endpoint	Một địa chỉ URL cụ thể của API	11
event loop	Vòng lặp nền xử lý các tác vụ bất đồng bộ	10
FAB	Floating Action Button — nút tròn nổi, thường ở góc dưới phải	5, 6
field	Trong Firestore: tương đương "cột" của database quan hệ	12
final	Biến chỉ gán được một lần, không gán lại được	Phụ lục A

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
Firestore	Database NoSQL trên cloud của Firebase	12
flex factor	Số nguyên quy định tỉ lệ chia khoảng trống giữa các <code>Expanded</code>	6
framework	Bộ khung có sẵn để xây ứng dụng trên đó	1
Future	Object đại diện cho một giá trị sẽ có trong tương lai	10
gesture	Cử chỉ của người dùng: chạm, vuốt, nhấn giữ, chụm ngón tay	5
GestureArena	"Đấu trường" nơi Flutter quyết định gesture nào thắng khi có nhiều gesture cùng lúc	5
Gradle	Hệ thống build và quản lý thư viện cho project Android	12

H – N

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
hot reload	Nạp lại code đã sửa vào app đang chạy mà vẫn giữ nguyên state	2
hybrid	Giải pháp lai: app đóng gói nội dung web chạy trong WebView	1
hydrate	"Đổ" dữ liệu vào một object rỗng để tạo thành object hoàn chỉnh	11
IDE	Integrated Development Environment — môi trường lập trình tích hợp	2
InheritedWidget	Widget cho phép widget con cháu truy cập trực tiếp dữ liệu từ tổ tiên	3, 9
JavaScript bridge	Cầu nối trung gian để JavaScript gọi sang mã native (làm chậm app)	1
JSON	Định dạng văn bản phổ biến để trao đổi dữ liệu	10
key	Định danh giúp Flutter khớp đúng widget khi vẽ lại	3
layout	Cách sắp xếp vị trí và kích thước các widget	6
lazy loading	Chỉ tạo/nạp phần tử khi nó sắp được nhìn thấy	6
library / package	Thư viện code do người khác viết mà bạn dùng lại	10
lifting state up	Kỹ thuật truyền hàm xử lý xuống dưới để widget con thay đổi state của widget cha	9
linting	Phân tích tĩnh mã nguồn để cảnh báo code không đúng chuẩn	2
main axis / cross axis	Trục chính / trục phụ của <code>Row</code> và <code>Column</code>	6
Material Design	Bộ hướng dẫn thiết kế giao diện của Google	6, 8
mixin	Tập hợp property và method có thể "trộn" vào bất kỳ class nào	Phụ lục A
native	Thuộc về nền tảng gốc — app biên dịch thẳng ra mã máy của iOS/Android	1
navigation / routing	Việc chuyển qua lại giữa các màn hình	7
NoSQL	Loại database không có cấu trúc bảng cố định	12

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
null safety	Cơ chế của Dart đảm bảo biến không thể chứa null trừ khi được khai báo rõ	Phụ lục A

O – S

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
open source	Mã nguồn mở	1
package	Xem <i>library</i>	10
padding / margin / border	Đệm trong / lề ngoài / đường viền — ba lớp của box model	6
pattern	Mẫu thiết kế lặp lại được dùng để giải quyết một loại bài toán	9
payload	Phần dữ liệu gửi kèm trong body của một HTTP request	11
plugin	Thư viện có kèm mã native cho từng nền tảng	12
presentation	Phần trình bày giao diện, đối lập với behavior	3
production	Môi trường chạy thật, phục vụ người dùng cuối	1
pubspec.yaml	File cấu hình của project Dart/Flutter: tên, phiên bản, dependency, assets	4
PWA	Progressive Web App — web app được cải tiến để hoạt động gần giống app thật	1
reactive programming	Phong cách lập trình trong đó giao diện tự cập nhật khi dữ liệu thay đổi	11
RenderBox	Kích thước thực tế (rộng × cao) mà một widget được vẽ ra	6
request / response	Yêu cầu gửi đi / phản hồi nhận về trong giao tiếp HTTP	11
REST / RESTful	Kiểu thiết kế API dựa trên các HTTP method GET/POST/PUT/DELETE	11
route	Một màn hình có tên trong hệ thống điều hướng	7
scene	Một màn hình của app (sách dùng từ này thay cho "screen"/"page")	6, 7
SDK	Software Development Kit — bộ công cụ phát triển phần mềm	2
serialization	Chuyển object trong bộ nhớ thành dạng có thể lưu/truyền (thường là JSON)	10

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
setState()	Method báo cho Flutter biết state đã đổi và cần vẽ lại widget	9
shared preferences	Kho lưu trữ cặp key-value đơn giản, tồn tại giữa các lần chạy app	10
snapshot	Ảnh chụp trạng thái dữ liệu tại một thời điểm (dùng trong FutureBuilder/StreamBuilder/Firestore)	11, 12
stack (navigation)	Chồng màn hình, vào bằng <code>push()</code> và ra bằng <code>pop()</code>	7
state	Dữ liệu của widget mà khi thay đổi thì phải vẽ lại giao diện	3, 9
stateless / stateful	Widget không giữ state / widget có giữ state	3, 9
Stream	Luồng dữ liệu phát ra nhiều giá trị theo thời gian	11

T – Z

Thuật ngữ	Nghĩa trong ngữ cảnh Flutter	Chương
theme	Tập hợp style được áp dụng thống nhất cho toàn app	8
toolchain	Chuỗi công cụ cần dùng để build ra được ứng dụng	2
type inference	Trình biên dịch tự suy ra kiểu dữ liệu từ giá trị được gán	Phụ lục A
typed deserialization	Chuyển JSON thành object có kiểu chặt chẽ thay vì Map	11
UI as code	Đặc điểm của Flutter: giao diện được viết bằng chính ngôn ngữ lập trình	3
unbounded height	Lỗi khi một widget muốn cao vô hạn mà không có widget cha giới hạn lại	6
upsert	Ghi dữ liệu: nếu bản ghi đã có thì cập nhật, chưa có thì thêm mới	12
validation / validator	Kiểm tra tính hợp lệ của dữ liệu người dùng nhập / hàm thực hiện việc đó	4
virtual DOM	Khái niệm từ React; trong Flutter tương ứng là element tree	3
WebView	Thành phần hiển thị nội dung web nhúng bên trong một app	1
widget	Thành phần dựng nên giao diện Flutter — "mọi thứ đều là widget"	3
widget tree	Cây phân cấp các widget tạo nên một màn hình	3, 6

Quy ước ký hiệu trong bản dịch

Xuyên suốt cuốn sách bạn sẽ gặp bốn loại hộp chú thích:

Ký hiệu	Ý nghĩa
 Mẹo	Mục <i>Tip</i> trong bản gốc
 Lưu ý	Mục <i>Note</i> trong bản gốc
 Cẩn trọng	Mục <i>Caution / Warning</i> trong bản gốc
 Ghi chú cập nhật	Không có trong bản gốc. Đây là phần người dịch bổ sung, chỉ ra chỗ nội dung năm 2019 đã lỗi thời và cách làm đúng hiện nay

Nếu bạn chỉ muốn đọc đúng nội dung nguyên bản của tác giả, hãy bỏ qua mọi hộp . Nhưng nếu bạn định gõ lại code trong sách và chạy thử, **hãy đọc kỹ chúng** — nhiều đoạn code trong bản gốc không còn biên dịch được trên Flutter hiện nay.